

weft 1.1.0

A Modern Literate Programming Tool

Cuauhtemoc Pacheco
Derived from nuweb 1.64 by Preston Briggs¹
with contributions by Keith Harwood, Simon Wright,
Marc W. Mengel, John D. Ramsdell, and Gregor Goldbach

August 1, 2026

¹Original nuweb work supported by ARPA, through ONR grant N00014-91-J-1989.

Contents

1	Introduction	1
1.1	weft	1
1.2	Writing weft	3
1.2.1	Source File Extension: <code>.weft</code>	3
1.2.2	The Major Commands	4
1.2.3	The Minor Commands	7
1.3	Sectioning commands	8
1.4	Running weft	9
1.5	Markdown Output	10
1.6	Section Markers and Language Auto-Detection	11
1.6.1	Overriding the Language with <code>@l</code>	11
1.6.2	Mid-Line Inline Expansion	12
1.6.3	The <code>@L</code> Command: Defining New Languages	13
1.6.4	The Language Table	13
1.6.5	Syntax Highlighting in Markdown Output	14
1.6.6	Output Format of Section Markers	14
1.6.7	Debugging Tangled Code	14
1.7	Restrictions	18
1.8	Project Introspection	19
1.8.1	Map (<code>-m</code>)	19
1.8.2	Extract (<code>-e</code>)	19
1.8.3	Reverse Map (<code>-R</code>)	19
1.9	Acknowledgements	20
2	The Overall Structure	21
2.1	Files	21
2.1.1	The Main Files	23
2.1.2	Support Files	25
2.2	The Main Routine	25
2.2.1	Command-Line Arguments	26
2.2.2	Help Message	30
2.2.3	Short Flags	34
2.2.4	File Names	38
2.3	Pass One	43
2.3.1	Accumulating Definitions	47
2.3.2	Block Comments	49
2.3.3	Fixing the Cross References	53
2.3.4	Dealing with fragment parameters	54
2.4	Writing the Latex File	57
2.4.1	Formatting Definitions	62
2.4.2	Generating the Indices	80
2.5	Writing the Markdown File	86

2.5.1	Formatting Definitions in Markdown	90
2.5.2	Generating the Indices in Markdown	100
2.5.3	Identifier Cross-References in Markdown	105
2.6	Map and Extract Output	107
2.6.1	JSON Map Output	107
2.6.2	Fragment Extraction	110
2.6.3	Callers (Upward Call Chain)	112
2.6.4	Lint (Static Analysis)	114
2.6.5	Reverse Map Output	116
2.6.6	Error Annotator	120
2.7	Writing the Output Files	122
3	The Support Routines	126
3.1	Source Files	126
3.1.1	Global Declarations	126
3.1.2	Local Declarations	127
3.1.3	Reading a File	127
3.1.4	Opening a File	131
3.2	Scraps	131
3.2.1	Collecting Page Numbers	155
3.3	Names	157
3.3.1	Language Auto-Detection	164
3.3.2	User-Defined Languages	170
3.3.3	Weave Format Declaration	172
3.3.4	Collecting File and Fragment Names	173
3.3.5	Comment Styles	176
3.4	Searching for Index Entries	184
3.4.1	Retrieving scrap uses	186
3.4.2	Building the Automata	189
3.4.3	Searching the Scraps	191
3.5	Labels	197
3.6	Memory Management	200
3.6.1	Allocating Memory	201
3.6.2	Freeing Memory	202
4	Man page	203
5	Indices	208
5.1	Files	208
5.2	Fragments	208
5.3	Identifiers	212

Chapter 1

Introduction

In 1984, Knuth introduced the idea of *literate programming* and described a pair of tools to support the practise [3]. His approach was to combine Pascal code with T_EX documentation to produce a new language, WEB, that offered programmers a superior approach to programming. He wrote several programs in WEB, including `weave` and `tangle`, the programs used to support literate programming. The idea was that a programmer wrote one document, the web file, that combined documentation (written in T_EX [4]) with code (written in Pascal).

Running `tangle` on the web file would produce a complete Pascal program, ready for compilation by an ordinary Pascal compiler. The primary function of `tangle` is to allow the programmer to present elements of the program in any desired order, regardless of the restrictions imposed by the programming language. Thus, the programmer is free to present his program in a top-down fashion, bottom-up fashion, or whatever seems best in terms of promoting understanding and maintenance.

Running `weave` on the web file would produce a T_EX file, ready to be processed by T_EX. The resulting document included a variety of automatically generated indices and cross-references that made it much easier to navigate the code. Additionally, all of the code sections were automatically prettyprinted, resulting in a quite impressive document.

Knuth also wrote the programs for T_EX and METAFONT entirely in WEB, eventually publishing them in book form [5, 6]. These are probably the largest programs ever published in a readable form.

Inspired by Knuth's example, many people have experimented with WEB. Some people have even built web-like tools for their own favorite combinations of programming language and typesetting language. For example, CWEB, Knuth's current system of choice, works with a combination of C (or C++) and T_EX [8]. Another system, FunnelWeb, is independent of any programming language and only mildly dependent on T_EX [10]. Inspired by the versatility of FunnelWeb and by the daunting size of its documentation, Preston Briggs decided to write his own, very simple, tool for literate programming.¹

`weft` is a fork of `nuweb`. The original `nuweb` code, written by Preston Briggs and maintained over the years by several contributors, provided an excellent foundation: a simple, language-independent literate programming tool. `weft` builds on that foundation with a modernized codebase, a modular source structure organized by concept, the removal of the unmaintained HTML output path, and the addition of a Markdown weave output format. The goal is to continue evolving the tool while preserving the simplicity and philosophy that made `nuweb` valuable in the first place.

1.1 weft

`weft` works with any programming language and produces documentation in L^AT_EX [7] or Markdown format. L^AT_EX was the original choice because it supports a multi-level sectioning scheme and has facilities for drawing figures. Markdown support was added to allow literate programs to produce lightweight documentation

¹There is another system similar to `nuweb`, written by Norman Ramsey, called `noweb` [9]. It perhaps suffers from being overly Unix-dependent and requiring several programs to use. On the other hand, its command syntax is very nice. In any case, `nuweb` certainly owes its name and a number of features to his inspiration.

that renders well on platforms like GitHub, GitLab, and any Markdown viewer, without requiring a $\text{\TeX}$ installation.

Support for arbitrary programming languages was a design goal because programmers work in many languages (and sometimes combinations of several languages), *e.g.*, C, Fortran, C++, yacc, lex, Scheme, assembly, Postscript, and so forth. The need to support arbitrary programming languages has many consequences:

No prettyprinting Both `WEB` and `CWEB` are able to prettyprint the code sections of their documents because they understand the language well enough to parse it. Since `weft` aims to support *any* language, this feature must be abandoned. However, `weft` does allow particular individual formulas or fragments of $\text{\LaTeX}$ code to be formatted and still be parts of output files. Also, keywords in scraps can be surrounded by `@_` to have them be bold in the output.

No index of identifiers Because `WEB` knows about Pascal, it is able to construct an index of all the identifiers occurring in the code sections (filtering out keywords and the standard type identifiers). Unfortunately, this isn't as easy for a language-independent tool. `weft` doesn't know what an identifier looks like in each language and certainly doesn't know all the keywords. (On the other hand, see the end of Section 1.2.3)

Of course, there must be some compensation for these losses or the whole idea would be a waste. Here are the advantages:

Simplicity The majority of the commands in `WEB` are concerned with control of the automatic prettyprinting. Since `weft` doesn't prettyprint, many commands are eliminated. A further set of commands is subsumed by $\text{\LaTeX}$ and may also be eliminated. As a result, the set of commands is reduced to only four members (explained in the next section). This simplicity is also reflected in the size of this tool, which is quite a bit smaller than the tools used with other approaches.

No prettyprinting Everyone disagrees about how their code should look, so automatic formatting annoys many people. One approach is to provide ways to control the formatting. `weft`'s approach is simpler—it performs no automatic formatting and therefore allows the programmer complete control of code layout. `weft` does allow individual scraps to be presented in either verbatim, math, or paragraph mode in the $\text{\TeX}$ output.

Control `weft` also offers the programmer complete control of the layout of his output files (the files generated during tangling). Of course, this is essential for languages that are sensitive to layout; but it is also important in many practical situations, *e.g.*, debugging.

Speed Since `weft` doesn't do too much, the `weft` tool runs quickly. By default it performs only tangling; weaving is activated with `-w`. When weaving is requested, both operations execute in a single pass.

Page numbers Inspired by the example of `noweb`, `weft` refers to all scraps by page number to simplify navigation. If there are multiple scraps on a page (say, page 17), they are distinguished by lower-case letters (*e.g.*, 17a, 17b, and so forth).

Multiple file output The programmer may specify more than one output file in a single `weft` file. This is required when constructing programs in a combination of languages (say, Fortran and C). It's also an advantage when constructing very large programs that would require a lot of compile time.

This last point is very important. By allowing the creation of multiple output files, the need for monolithic programs is avoided. This supports the creation of very large programs by groups of people.

A further reduction in compilation time is achieved by first writing each output file to a temporary location, then comparing the temporary file with the old version of the file. If there is no difference, the temporary file can be deleted. If the files differ, the old version is deleted and the temporary file renamed. This approach works well in combination with `make` (or similar tools), since `make` will avoid recompiling untouched output files.

1.2 Writing weft

The bulk of a weft file will be ordinary L^AT_EX (or Markdown, when using the `-w md` flag). In fact, any L^AT_EX file can serve as input to weft and will be simply copied through, unchanged, to the documentation file—unless a weft command is discovered. Similarly, any Markdown file can be processed with `-w md` and will be copied through to the `.md` output. All weft commands begin with an “at-sign” (`@`). Therefore, a file without at-signs will be copied unchanged. weft commands are used to specify *output files*, define *fragments*, and delimit *scraps*. These are the basic features of interest to the weft tool—all else is simply text to be copied to the documentation file.

1.2.1 Source File Extension: `.weft`

weft source files use the `.weft` extension. This single, format-agnostic extension replaces the earlier `.w` and `.wmd` conventions. The prose surrounding scraps may be written in L^AT_EX or Markdown; the at-sign commands (`@o`, `@d`, `@<`, etc.) are the same in both cases.

The weave output format is specified in one of two ways:

In the source file The `@W` directive declares the intended format: `@W tex` for L^AT_EX, `@W md` for Markdown. This should appear in the preamble, before any `@o` or `@d` directives.

On the command line `-w md` or `-w tex` overrides the `@W` directive. Using `-w` alone (without a format token) reads the format from `@W`.

If no extension is given on the command line, weft appends `.weft` automatically.

Default Behavior: Tangle Only

By default, `weft myfile.weft` performs only tangling (producing the output files). No documentation is generated unless `-w` is given:

<code>weft programa.weft</code>	tangle only (default)
<code>weft -w programa.weft</code>	weave per <code>@W</code> (error if missing)
<code>weft -w md programa.weft</code>	weave Markdown (override)
<code>weft -w tex programa.weft</code>	weave LaTeX (override)

The `-t` flag suppresses weaving even if `-w` is given. The `-o` flag suppresses tangling.

Cosmetic Fences

When writing `.weft` files with Markdown prose, it is often convenient to surround scrap definitions with Markdown fenced code blocks (triple backticks) so that the source file renders nicely in Markdown previewers such as GitHub or VS Code. For example:

Some prose in Markdown.

```
'''  
@d My fragment  
@{code here  
@}  
'''
```

More prose.

weft silently strips these fence lines (lines that consist of three or more backticks, optionally followed by a language identifier) during Markdown weaving. They do not appear in the generated `.md` file and do not affect tangling. This is purely a cosmetic convenience for making the source file readable in any Markdown viewer.

Note that fences are only stripped in Markdown mode (`-w md`). In L^AT_EX mode, backtick lines are treated as ordinary text.

1.2.2 The Major Commands

Files and fragments are defined with the following commands:

@o *file-name flags scrap* Output a file. The file name is terminated by whitespace.

@d *fragment-name scrap* Define a fragment. The fragment name is terminated by a return or the beginning of a scrap.

@q *fragment-name scrap* Define a fragment. The fragment name is terminated by a return or the beginning of a scrap. This a quoted fragment.

A specific file may be specified several times, with each definition being written out, one after the other, in the order they appear. The definitions of fragments may be similarly specified piecemeal.

Fragment Names with Embedded Parameters

Fragment names can carry *parameters*—placeholder values embedded directly in the name using the **@'value@'** syntax. Inside the fragment body, **@1**, **@2**, ... **@9** refer to the first, second, ... ninth parameter in order of appearance.

For example, a reusable validation fragment:

```
@d Validate field @'name@' type @'string@'
@{if (!data.@1) throw new Error("Missing @1");
if (typeof data.@1 !== '@2')
    throw new TypeError("@1 must be @2");
@}
```

Each invocation supplies its own arguments:

```
<@Validate field @'email@' type @'string@'>
<@Validate field @'age@' type @'number@'>
```

The first invocation expands to:

```
if (!data.email) throw new Error("Missing email");
if (typeof data.email !== 'string')
    throw new TypeError("email must be string");
```

Note three key points:

- The values in the *definition* (**@'name@'**, **@'string@'**) serve as default arguments and as documentation—they make the fragment name readable.
- At each *invocation*, different values can be substituted (**@'email@'** replaces **@1**, **@'string@'** replaces **@2**).
- If a fragment name is abbreviated with ..., any missing arguments fall back to the defaults given in the definition.

The difference between a quoted fragment (**@q**) and an ordinary one (**@d**) is that inside a quoted fragment fragments are not expanded on output. Rather, they are formatted as uses of fragments so that the output file can itself be **weft** source. This allows you to create files containing fragments which can undergo further processing before the fragments are expanded, while also describing and documenting them in one place.

You can have both quoted and unquoted fragments with the same name. They are written out in order as usual, with those introduced by **@q** being quoted and those with **@d** expanded as normal.

In quoted fragments, the **@f** filename is quoted as well, so that when it is expanded it refers to the finally produced file, not any of the intermediate ones.

Scraps

Scraps have specific begin markers and end markers to allow precise control over the contents and layout. Note that any amount of whitespace (including carriage returns) may appear between a name and the beginning of a scrap. Scraps may also appear in the running text where they are formatted (almost) identically to their use in definitions, but don't appear in any code output.

`@{anything@}` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in verbatim mode. Using the `-l` option will cause the program to typeset the scrap with the help of L^AT_EX's `listings` package.

`@[anything@]` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in paragraph mode, allowing sections of T_EX documents to be scraps, but still be prettyprinted in the document.

`@(anything@)` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in math mode. This allows this scrap to contain a formula which will be typeset nicely.

Inside a scrap, several special sequences are recognized.

Fragment invocation.

`<fragment-name>` Expands the named fragment inline. It is an error to specify recursive invocations. If the fragment name contains embedded parameters, the invocation supplies arguments at the same positions (see above).

When the reference appears *mid-line*—that is, after at least one visible character on the same line—section markers, `#line` directives, and fragment-name comments are automatically suppressed. The fragment content is spliced seamlessly into the surrounding text. At start-of-line positions (including indented lines), markers are emitted normally. See Section 1.6 for details.

Argument substitution.

`@1`, `@2`, ..., `@9` Inside a fragment body, substitutes the *n*th argument. If the argument was not supplied, the empty string is used.

At each argument position in an invocation, four forms are accepted:

`@'string@'` A literal string, copied verbatim (only `@@` is interpreted, producing a single `@`). This is the most common form.

`<fragment-name>` The fragment is expanded and its output becomes the argument value.

`@{text@}` The text is expanded as an anonymous fragment (inheriting the caller's arguments) and the result is passed as the argument. Useful for combining text and arguments into a single value.

`@1`, `@2`, ..., `@9` Forwards an argument from the calling fragment to the called one.

Other in-scrap commands.

`@xlabel@x` Marks a place inside a scrap and associates it to the label (which can be any text not containing a `@`). Expands to the reference number of the scrap followed by a numeric value. Outside scraps it expands to the same value. It is used so that text outside scraps can refer to particular places within scraps.

`@f` Inside a scrap this is replaced by the name of the current output file.

`@t` Inside a scrap this is replaced by the title of the fragment as it is at the point it is used, with all parameters replaced by actual arguments.

@# At the beginning of a line in a scrap this will suppress the normal indentation for that line. Use this, for example, when you have a `#ifdef` inside a nested scrap. Writing `@##ifdef` will cause it to be lined up on the left rather than indented with the rest of its code.

@s Inside a scrap suppresses indentation for the following fragment expansion.

Note that fragment names may be abbreviated, either during invocation or definition. For example, it would be very tedious to have to type, repeatedly, the fragment name

```
@d Check for terminating at-sequence and return name if found
```

Therefore, `weft` provides a mechanism (borrowed from Knuth) of indicating abbreviated names.

```
@d Check for terminating...
```

Basically, the programmer need only type enough characters to identify the fragment name uniquely, followed by three periods. An abbreviation may even occur before the full version; `weft` simply preserves the longest version of a fragment name. Note also that blanks and tabs are insignificant within a fragment name; each string of them is replaced by a single blank.

Sometimes, for instance during program testing, it is convenient to comment out a few lines of code. In C or Fortran placing `/* ... */` around the relevant code is not a robust solution, as the code itself may contain comments. `weft` provides the command

```
@%
```

only to be used inside scraps. It behaves exactly the same as `%` in the normal L^AT_EX text body.

When scraps are written to a program file or a documentation file, tabs are expanded into spaces by default. Currently, `weft` assumes tab stops are set every eight characters. Furthermore, when a fragment is expanded in a scrap, the body of the fragment is indented to match the indentation of the fragment invocation. Therefore, care must be taken with languages (*e.g.*, Fortran) that are sensitive to indentation. These default behaviors may be changed for each output file (see below).

Flags

When defining an output file, the programmer has the option of using flags to control output of a particular file. The flags are intended to make life a little easier for programmers using certain languages. They introduce little language dependences; however, they do so only for a particular file. Thus it is still easy to mix languages within a single document. There are five “per-file” flags:

- d** Forces the creation of `#line` directives in the output file. These are useful with C (and sometimes C++ and Fortran) on many Unix systems since they cause the compiler’s error messages to refer to the web file rather than to the output file. Similarly, they allow source debugging in terms of the web file.
- i** Suppresses the indentation of fragments. That is, when a fragment is expanded within a scrap, it will *not* be indented to match the indentation of the fragment invocation. This flag would seem most useful for Fortran programmers.
- t** Suppresses expansion of tabs in the output file. This feature seems important when generating `make` files.
- cx** Puts comments in generated code documenting the fragment that generated that code. The *x* selects the comment style for the language in use: **c** for C (`/* */`), **+** for C++ (`//`), and **p** for shell/Perl (`#`). If the global **-x** cross-reference flag is set the comment includes the page reference for the first scrap that generated the code.
- s** Suppresses section markers and `#line` directives in the tangled output for this file. The language is still detected (from the extension or a preceding `@1`), so the Markdown weaver can emit syntax-highlighting classes, but the tangled code is emitted without any tangle annotations. This is useful for bootstrap source files or generated output that should remain clean.

These flags are still available for explicit control, but in most cases the automatic language detection described in Section 1.6 is more convenient.

1.2.3 The Minor Commands

There are some very low-level utility commands that may appear anywhere in the web file.

`@@` Causes a single “at sign” to be copied into the output.

`@_` Causes the text between it and the next `@_` to be made bold (for keywords, etc.)

`@i file-name` Includes a file. Includes may be nested, though there is currently a limit of 10 levels. The file name should be complete (no extension will be appended) and should be terminated by a carriage return. Normally the current directory is searched for the file to be included, but this can be varied using the `-I` flag on the command line. Each such flag adds one directory to the search path and they are searched in the order given.

`@rx` Changes the escape character ‘@’ to ‘x’. This must appear before any scrap definitions.

`@v` Always replaced by the string established by the `-V` flag, or a default string if the flag isn’t given. This is intended to mark versions of the generated files.

`@l tag` Sets the language for the next `@o` or `@d` directive. The tag is looked up in the language table (Section 1.6) to determine the comment style for section markers and the `class` attribute for Markdown syntax highlighting. Placed on the line before the directive it applies to.

`@L name style [+d]` Defines a custom language mapping for the section marker system (see Section 1.6). This must appear in the document preamble, before any `@o` or `@d` directives that depend on it.

`@W format` Declares the weave output format for this source file. Valid formats are `md` (Markdown) and `tex` (L^AT_EX). Used by `-w` when no explicit format is given on the command line. A CLI format (`-w md` or `-w tex`) overrides this directive. Should appear in the document preamble.

In the text of the document, that is outside scraps, you may include scrap-like material.

`@{Anything@}` The included material, the *Anything*, is typeset as if it appeared inside a scrap. This is useful for referring to fragments in the text and for describing the literate programming process itself.

`@<Fragment name@>` The fragment named is expanded in place in the text. The expansion is presented verbatim, it is not interpreted for typesetting, so any special environment must be set up before and after this is used.

`@c` Outside of a scrap this is used to define a block comment using all the text until the next command. Inside a scrap the last block comment is expanded using the current indentation and breaking lines between words if 60 characters are exceeded.

Finally, there are three commands used to create indices to the fragment names, file definitions, and user-specified identifiers.

`@f` Create an index of file names.

`@m` Create an index of fragment names.

`@u` Create an index of user-specified identifiers.

These are usually placed in their own section in the L^AT_EX document; for example, see Chapter 5.

Identifiers must be explicitly specified for inclusion in the `@u` index. By convention, each identifier is marked at the point of its definition; all references to each identifier (inside scraps) will be discovered automatically. To “mark” an identifier for inclusion in the index, it must be mentioned at the end of a scrap. For example,

```
@d a scrap @{
  Suppose the variables FOO and BAR are being declared
  inside this scrap.
@| FOO BAR @}

```

Alphabetic identifiers have been used in this example, but any string of characters (not including whitespace or @ characters) will do. Therefore, it's possible to add index entries for things like <=< if desired. An identifier may be declared in more than one scrap.

In the generated index, each identifier appears with a list of all the scraps using and defining it, where the defining scraps are distinguished by underlining. Note that the identifier doesn't actually have to appear in the defining scrap; it just has to be in the list of definitions at the end of a scrap.

1.3 Sectioning commands

For larger documents the indexes and usage lists get rather unwieldy and problems arise in naming things so that different things in different parts of the document don't get confused. weft has a sectioning command which keeps the fragment names and user identifiers separate. Thus, you can give a fragment in one section the same name as a fragment in another and the two won't be confused or connected in any way. Nor will user identifiers defined in one section be referenced in another. Except for the fact that scraps in successive sections can go into the same output file, this is the same as if the sections came from separate input files.

However, occasionally you may really want fragments from one section to be used in another. More often, you will want to identify a user identifier in one section with the same identifier in another (as, for example, a header file defined in one section is included in code in another). Extra commands allow a fragment defined in one section to be accessible from all other sections. Similarly, you can have scraps which define user identifiers and export them so that they can be used in other sections.

@s Start a new section.

@S Close the current section and don't start another.

@d+ **fragment-name scrap** Define a fragment which is accessible in all sections, a global fragment.

@D+ Likewise

@q+ Likewise

@Q+ Likewise

@m+ Create an index of all such fragments.

@u+ Create an index of globally accessible user identifiers.

There are two kinds of section, the base section which is where normally everything goes, and local sections which are introduced with the @s command. A local section comprises everything from the command which starts it to the one which ends it. A @s command will start a new local section. A @S command closes the current local section, but doesn't open another, so what follows goes into the base section. Note that fragments defined in the base section aren't global; they are accessible only in the base section, but they are accessible regardless of any local sections between their definition and their use.

Within a scrap:

@<+*fragment-name*> Expand the globally accessible fragment with that name, rather than any local fragment.

@+ Like @| except that the identifiers defined are exported to the global realm and are not directly referenced in any scrap in any section (not even the one where they are defined).

@- Like @| except that the identifiers are imported to the local realm. The cross-references show where the global variables are defined and defines the same names as locally accesible. Uses of the names within the section will point to this scrap.

Note that the + signs above are part of the commands. They are not part of the fragment names. If you want a fragment whose name begins with a plus sign, leave a space between the command and the name.

1.4 Running weft

weft is invoked using the following command:

```
weft flags file-name...
```

One or more files may be processed at a time. If a file name has no extension, `.weft` will be appended. While a file name may specify a file in another directory, the resulting documentation file will always be created in the current directory. For example,

```
weft /foo/bar/quux
```

will take as input the file `/foo/bar/quux.weft` and will produce the tangled output files only. To also generate documentation, use `-w` (see below).

By default, weft performs only tangling—it generates the output files defined by `@o` directives and nothing else. Documentation (weaving) is produced only when the `-w` flag is given. When `-w` is present, both tangling and weaving execute in a single pass. Three flags control what weft produces:

- t Suppress weaving (useful to override `-w` in scripts).
- o Suppress generation of the output files (weave only).
- c Avoid testing output files for change before updating them.

Thus, the command

```
weft -to /foo/bar/quux
```

would simply scan the input and produce no output at all.

There are several additional command-line flags:

- v For “verbose”, causes weft to write information about its progress to `stderr`.
- n Forces scraps to be numbered sequentially from 1 (instead of using page numbers). This form is perhaps more desirable for small webs.
- s Doesn’t print list of scraps making up each file following each scrap.
- d Print “dangling” identifiers – user identifiers which are never referenced, in indices, etc.
- p *path* Prepend *path* to the filenames for all the output files.
- w [*format*] Activate weaving (documentation generation). Without `-w`, no documentation file is produced. The optional *format* argument selects the output format:
 - `-w md` — force Markdown output (`.md` file).
 - `-w tex` — force L^AT_EX output (`.tex` file).
 - `-w` alone — read the format from the `@W` directive in the source file. If `@W` is absent, weft reports an error.

A format given on the command line overrides any `@W` directive. When Markdown output is selected, sequential numbering (`-n`) is forced automatically, since Markdown has no concept of page numbers. The Markdown output uses HTML anchors (`<a id="...">`) for cross-references and `<pre><code>` blocks for scraps, allowing clickable navigation between definitions and references. Cosmetic fence lines (triple backticks) in the source are silently stripped.

- `-x` Include cross-reference numbers in scrap comments.
- `-h options` Provide options for the hyperref package.
- `-r` Turn on hyperlinks. You must include the `—usepackage—` options in the text.
- `-l` Use the `listings` package for formatting scraps. Use this if you want to have a pretty-printer for your scraps. In order to e.g. have pretty Perl scraps, include the following L^AT_EX commands in your document:

```
\usepackage{listings}
...
\lstset{extendedchars=true,keepspaces=true,language=perl}
```

See the `listings` documentation for a list of formatting options. Be sure to include a `\usepackage{listings}` in your document.

- `-V string` Provide *string* as the replacement for the `@v` operation.
- `-m` Emit a JSON map of the project to stdout and exit. The map describes every output file and every fragment, their definition locations, usage relationships, and transitive dependencies. Neither weaving nor tangling is performed. See Section 2.6 for details.
- `-e name` Extract a single fragment by name and emit its code and dependency information as Markdown to stdout. The match uses the same prefix logic as `@<name...@>`. Neither weaving nor tangling is performed. See Section 2.6 for details.
- `-R file[:line]` Read section markers from a tangled output file and emit JSON mapping to stdout. If a line number is given, emit the single innermost scrap covering that line. Without a line number, emit all scrap regions in the file. No `.weft` source files are needed—the reverse map operates entirely on the tangled output. See Section 2.6 for details.

1.5 Markdown Output

When invoked with `-w md`, `weft` generates a Markdown file (`.md`) instead of a L^AT_EX file (`.tex`). This allows literate programs to be read on any platform that renders Markdown, including GitHub, GitLab, and local editors like VS Code.

The Markdown output preserves all the essential features of the L^AT_EX output—scrap definitions, cross-references, and indices—but uses a different set of formatting conventions:

Scrap headers File definitions appear as `**"filename"** **N** =` and fragment definitions as `**<Name* N>** =`, where *N* is the sequential scrap number.

Scrap bodies Code is wrapped in `<pre><code>` HTML blocks (not fenced code blocks). This allows fragment references within scraps to be rendered as clickable HTML links with italic and bold formatting, which would not be possible inside Markdown fenced blocks.

HTML escaping Characters `<`, `>`, and `&` inside scraps are escaped as `<`, `>`, and `&` respectively.

Cross-references Each scrap has an HTML anchor (`<a id="weftN">`) and cross-reference links use `<a href="#weftN">`. Footer notes (“defined by”, “referenced in”) appear as Markdown blockquotes.

Indices The `@f`, `@m`, and `@u` commands produce Markdown lists with links. In the user identifier index, defining scraps appear in bold.

Sequential numbering The `-n` flag is forced automatically, since Markdown has no page numbers. Scraps are numbered 1, 2, 3, ... without letter suffixes.

Prose Text outside scraps passes through unmodified. The author writes Markdown prose directly in the `.weft` source file; `weft` does not interpret or transform it.

Syntax highlighting When the language of a scrap can be determined—from a preceding `@l` tag, the `@L` command, or the file extension of an `@o` name—`weft` emits `<code class="language-X">` on the opening tag. This allows client-side highlighters (`highlight.js`, `Prism`) and platforms like GitHub to apply syntax colouring automatically.

Tangling (output file generation) works identically regardless of the weave format. When `-w` is given, a single invocation produces both the documentation and the tangled output files in one pass.

1.6 Section Markers and Language Auto-Detection

When debugging tangled code, it is useful to correlate lines in the generated file back to the original web source. `weft` can auto-detect the programming language and generate *section markers*—comments that delimit the opening and closing of each scrap in the tangled output, including the source file name and line number. For C and C++, it also generates `#line` directives automatically.

The key idea is simple:

file extension → language → comment style → section markers in tangled output

For example, `@o server.js` has the extension `.js`. `weft` looks up `js` in its language table, finds that JavaScript uses `//` comments, and wraps each scrap in the tangled `server.js` with `//` markers:

```
// {1: web.weft:12}
console.log("hello");
// {1: }
```

Similarly, `@o main.c` matches `c`, which uses `/* */` comments and also enables `#line` directives:

```
#line 5 "core.weft"
/* {1: core.weft:5} */
#include <stdio.h>
/* {1: } */
```

And `@o config.py` matches `py`, using `#` comments:

```
# {3: settings.weft:40}
DATABASE_URL = "postgres://localhost/app"
# {3: }
```

This happens automatically—no flags are needed. The extension alone is enough.

1.6.1 Overriding the Language with `@l`

Sometimes the file extension is not enough, or you want a different behaviour. The `@l` command (lowercase) placed on the line before an `@o` or `@d` directive overrides the extension:

```
@l sql
@o queries.txt
@{SELECT * FROM users;
@}
```

Here the file is `queries.txt`, whose extension `.txt` has no comment style. But `@l sql` tells weft to use `--` comments, so the tangled output gets:

```
-- {1: db.weft:5}
SELECT * FROM users;
-- {:1}
```

`@l` also works on `@d` (fragment) definitions. This is useful for fragments that will be included in files of a specific language:

```
@l python
@d Validate email
@{import re
def validate(email):
    return bool(re.match(r'^[~@]+@[^@]+$', email))
@}
```

The language tag is stored on the fragment and used both for section markers in the tangled output and for syntax highlighting in the Markdown weave output.

`@l` is format-agnostic: it works identically in $\text{\LaTeX}$ and Markdown source files. Unlike the earlier approach of encoding the language in Markdown fences (`'''python`) or $\text{\LaTeX}$ markers (`%%python`), `@l` uses the same at-sign syntax as every other weft command.

Note that cosmetic fences (`'''`) may still be used around `@o/@d` definitions for readability in Markdown previewers—they are silently stripped during Markdown weaving (see Section 1.2.1). But fences no longer set the language; use `@l` for that.

Priority rules. When multiple sources of language information are available, weft resolves them in this order:

1. **Explicit per-file flags** (`-d`, `-s`, `-cx`, etc.) on the `@o` line always win for tangle behaviour.
2. **@l tag** immediately before the directive—overrides the extension.
3. **File extension** of the `@o` name—the default when no `@l` is present.

1.6.2 Mid-Line Inline Expansion

When a fragment reference `@<name>` appears in the middle of a line—after at least one visible character—weft automatically suppresses all tangle annotations for that expansion: section markers, `#line` directives, and fragment-name comments. The fragment content is spliced directly into the surrounding text, producing clean inline output.

This makes small constant fragments practical:

```
@d port @{4000@}
@d host @"localhost"@

@l c
@o config.c -s
@{const char *host = @<host>;
int port = @<port>;
@}
```

Tangling produces:

```
const char *host = "localhost";
int port = 4000;
```

If the same reference appears at the *start* of a line (even if indented), it is treated as a block expansion and markers are emitted normally. No special flags or directives are needed—the position on the line is the only thing that determines the behaviour.

This works with any language and any fragment size. When a multi-line fragment is expanded mid-line, its content is still spliced without markers—the rule is based on the *reference site*, not the fragment body.

1.6.3 The @L Command: Defining New Languages

The built-in table covers over 250 entries, but it cannot anticipate every language or every project’s preferences. The @L command defines new language mappings—or overrides existing ones—directly in the document preamble.

```
@L name style [+d]
```

- *name* is the tag that will be recognised in @l commands and file extensions (@o file.name).
- *style* is the comment syntax: //, #, --, /*, or <!--.
- +d (optional) enables #line directive generation.

Examples:

```
@L wgs1 //          % WebGPU shading language: // comments
@L cedar //         % Cedar policy language
@L c //            % override built-in: use // instead of /* */
```

How @L interacts with the rest of the system. User-defined entries are searched *before* the built-in table, so @L c // effectively changes the comment style for all subsequent @o main.c directives (and for any @l c tag) from /* */ to //. Up to 32 user-defined languages may be registered.

@L should appear in the document preamble, before any @o or @d that depends on it. It is processed during pass 1 and silently skipped during weaving.

1.6.4 The Language Table

weft includes a built-in table of over 250 entries (languages, file extensions, and common aliases), organised by comment style. Each entry maps a tag to a comment style and optionally enables #line directives. The tag is matched against @l tags and file extensions alike—there is no distinction between them.

Comment style	Tags (representative)
/* */ + #line	c, h, cpp, cc, cxx, hpp, hxx, objc, objcpp, m, mm, ino, pde, cu, cuh, cilk
/* */	css, scss, sass, less, stylus, cl (OpenCL)
//	js, ts, jsx, tsx, mjs, cjs, mts, svelte, vue, java, scala, kotlin, kt, groovy, gradle, clj, cs, fs, fsx, vb, go, rust, rs, zig, swift, dart, odin, mojo, carbon, php, graphql, prisma, glsl, hlsl, wgs1, metal, vert, frag, proto, thrift, fbs, capnp, sol, reason, res, ml, sml, fut, apex, bicep, jsonnet, qml, p4
#	python, py, pyx, pyi, sh, bash, zsh, fish, ksh, nu, perl, pl, ruby, rb, r, tcl, awk, sed, coffee, makefile, mk, cmake, just, meson, bazel, bzl, julia, jl, nim, crystal, cr, elixir, ex, yaml, yml, toml, dockerfile, terraform, tf, hcl, nix, conf, ini, env, editorconfig, gitignore, powershell, ps1, xdc, sdc
--	lua, luau, sql, psql, plsql, plpgsql, mysql, sqlite, hive, haskell, hs, elm, purescript, purs, ada, adb, ads, vhdl, vhd, lean, lean4, agda, idris, idr, erlang
<!-- -->	html, htm, xml, svg, xhtml, xsl, xsd, dtd, rss, xaml, csproj, jsp, aspx, cshtml, erb, ejs, hbs, njk, twig, blade, pug, liquid, mustache, md, mdx, rst, tex, latex
(none)	json, csv, tsv, txt, diff, patch

For the full list, see the `lang_table` array in Section 3.3.1. If a tag is not found in either the user-defined (@L) or built-in table, weft defaults to `//` comments with no `#line` directives. Tags mapped to style 0 (e.g. `json`, `txt`) produce no markers at all.

1.6.5 Syntax Highlighting in Markdown Output

When weaving to Markdown (`-w md`), weft emits the language tag on the `<code>` element:

```
<pre><code class="language-python">def hello():
    print("hello")
</code></pre>
```

This allows client-side highlighters (highlight.js, Prism) and platforms like GitHub to apply syntax colouring automatically. The language tag comes from the same chain described above: @l tag or file extension.

For @d fragments, the tag comes from a preceding @l command (there is no file extension to infer from). If no language is known, weft emits a plain `<pre><code>` without a class.

1.6.6 Output Format of Section Markers

Each scrap in the tangled file is wrapped with an opening marker at the top and a closing marker at the bottom, using the comment syntax of the detected language. The opening marker includes the scrap number and the source location (file name and line number). When fragments are nested, each expansion gets its own pair of markers, producing a clear hierarchy in the output.

1.6.7 Debugging Tangled Code

When you work with literate programming, the code you execute is *tangled output*—generated files assembled from scraps scattered across `.w` files. An error at `server.js:47` does not correspond to line 47 of any file you wrote; it is line 47 of a file that weft assembled from fragments defined in `server.w`, `routes.w`, and `database.w`. Without markers, debugging means mentally reconstructing which scrap produced which line. With markers, the tangled file tells you directly.

Reading the markers. Each marker encodes three pieces of information:

Part	Meaning
{ or {:	Opening or closing delimiter
Number	Scrap ID—unique across the entire document
file:line	Source .w file and line number (opening only)

Examples by language. **JavaScript** (`//` comments):

```
// {3: routes.w:42}
app.get("/api/users", async (req, res) => {
  // {7: database.w:18}
  const users = await db.query("SELECT * FROM users");
  // {7}
  res.json(users);
});
// {3}
```

Python (`#` comments):

```
# {5: models.w:31}
class User:
    def __init__(self, name, email):
        self.name = name
```

```

        self.email = email
    # {:5}

```

C (`/* */` comments with `#line` directives):

```

#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) {
    void *ptr = malloc(size);
    if (!ptr) {
        /* {4: errors.w:8} */
        fprintf(stderr, "out of memory\n");
        exit(1);
        /* {4:4} */
    }
    return ptr;
}
/* {2:2} */

```

Lua (`--` comments):

```

-- {9: config.w:55}
local defaults = {
    port = 8080,
    host = "localhost",
}
-- {9:9}

```

HTML (`<!-- -->` comments):

```

<!-- {6: layout.w:20} -->
<nav class="sidebar">
    <!-- {11: navigation.w:7} -->
    <ul>
        <li><a href="/">Home</a></li>
    </ul>
    <!-- {11:11} -->
</nav>
<!-- {6:6} -->

```

Debugging workflow. The following four-step procedure traces any error in tangled output back to the `literate` source.

Step 1: Locate the error in tangled output. You receive an error such as:

```

TypeError: Cannot read property 'id' of undefined
    at Object.<anonymous> (server.js:47)

```

Open `server.js` and go to line 47.

Step 2: Read the nearest opening marker. Scan *upward* from the error line until you find an opening marker:

```

// {3: routes.w:42}

```

This tells you: scrap 3 generated this code, defined in `routes.w` starting at line 42.

Step 3: Go to the `literate` source. Open `routes.w` at line 42. You will find the scrap with the bug, surrounded by prose that explains its intent:

We fetch the user by ID from the database. If the user doesn't exist, we should return a 404, but currently we assume the user always exists --- this is the bug.

```
@d Handle user fetch
@{const user = await getUser(req.params.id);
  res.json(user.id);
@}
```

The prose makes the bug obvious: there is no null check. Fix it in the `.w` file, re-tangle, and the error is gone.

Step 4: Nested markers = nested fragments. When fragments invoke other fragments, markers nest:

```
// {3: routes.w:42}
app.get("/api/users/:id", async (req, res) => {
  // {7: database.w:18}
  const user = await db.query(
    "SELECT * FROM users WHERE id=$1",
    [req.params.id]);
  // {7}
  if (!user) return res.status(404).json({error: "not found"});
  res.json(user);
});
// {3}
```

If the bug is inside the `{7: ... {7}}` block, go to `database.w:18`, not `routes.w:42`. The *innermost enclosing markers* point to the most specific source location. Always look at the tightest pair around the error line.

Interpreting nested hierarchies. Complex tangled output can have several nesting levels. Read them like parentheses—each opening matches its corresponding close:

```
/* {1: main.w:10} */           outermost: main file structure
#include <stdio.h>
/* {2: io.w:25} */             middle: I/O subsystem
void print_report() {
  /* {5: format.w:8} */         innermost: formatting detail
  printf("Report v%d\n", VERSION);
  /* {5} */
}
/* {2} */
int main() {
  /* {3: init.w:14} */
  init_subsystems();
  /* {3} */
  print_report();
  return 0;
}
/* {1} */
```

Scrap numbers are *document-global*—scrap 1 is the first `@{...@}` in the entire woven document, scrap 5 is the fifth. They are stable as long as you do not reorder scraps.

C/C++: #line directives. For C-family languages, weft also generates `#line` directives *above* the opening marker. These make the C compiler itself report errors against the `.w` source file:

```
#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) { ... }
/* {:2} */
```

Now gcc will say:

```
memory.w:14: error: implicit declaration of function 'malloc'
```

instead of:

```
memory.c:47: error: implicit declaration of function 'malloc'
```

The `#line` directive and the section marker are complementary: `#line` makes the compiler report errors against the `.w` file directly; the `{N: file:line}` markers are human-readable when browsing the tangled output.

Common debugging scenarios. Runtime error with stack trace.

```
Error: ENOENT: no such file or directory, open '/data/config.json'
    at readConfig (server.js:123)
```

Open `server.js:123`, find the nearest marker `// {12: config.w:35}`, open `config.w:35`—the scrap that reads the config—and fix the path or add an existence check in the `.w` file.

Compiler error in C. With `#line` directives active, the compiler already reports:

```
parser.w:84: error: 'token_type' undeclared
```

Go directly to `parser.w:84`. The prose around the scrap explains what `token_type` should be and where it is defined.

Wrong output, no error. The program runs but produces wrong results. Open the tangled file and search for the relevant code. The markers tell you which `.w` file and scrap to examine:

```
# {15: transform.w:67}
result = [x * 2 for x in data]    # should be x * 3?
# {:15}
```

Go to `transform.w:67`. The prose says “triple each value” but the code says `* 2`. Bug found.

Identifying which fragment owns a line. Sometimes you know the line of code but not which `@d` definition produced it (multiple fragments with the same name accumulate). The scrap number is unique:

```
// {14: handlers.w:89}
validateInput(data);
// {:14}
// {22: handlers.w:112}
validatePermissions(user);
// {:22}
```

Scraps 14 and 22 are two different definitions (or continuations) of the same fragment. The line numbers point to each one independently.

Tips.

- *Never edit tangled files.* The markers remind you these are generated. Always trace back to the `.w` source and edit there.
- *Scrap numbers are stable* as long as you do not add or remove `@{...@}` blocks before them. If numbering shifts after edits, re-tangle and the markers update automatically.
- *Use `grep` on markers* to find all scraps from a specific file:

```
grep -n '{[0-9]*: database.w:' server.js
```
- *Closing markers as verification.* If you see `{:7}` but no `{7:` above it in your viewport, scroll up—the scrap is larger than your screen. If you see `{7:` but no `{:7}`, the scrap extends below.
- *Markers in test output.* If your test framework shows code snippets on failure, the markers point straight to the literate source.
- *Disable markers selectively.* Use explicit flags on `@o` (e.g. `@o file -t`) with no comment flags to suppress markers for a specific output file when comments would interfere—for example, binary formats, data files, or files where comments are not valid syntax.

1.7 Restrictions

Because weft is intended to be a simple tool, a few restrictions have been established. Over time, some of these may be eliminated; others seem fundamental.

- The handling of errors is not completely ideal. In some cases, weft simply warns of a problem and continues; in other cases it halts immediately. This behavior should be regularized.
- weft warns about references to fragments that haven't been defined, but doesn't halt. The name of the fragment is included in the output file surrounded by `<>` signs. This seems most convenient for development, but may change in the future.
- File names and index entries should not contain any `@` signs.
- Fragment names may be (almost) any well-formed $\text{\TeX}$ string. It makes sense to change fonts or use math mode; however, care should be taken to ensure matching braces, brackets, and dollar signs.
- Anything is allowed in the body of a scrap; however, very long scraps (horizontally or vertically) may not typeset well.
- Temporary files (created for comparison to the eventual output files) are placed in the current directory. Since they may be renamed to an output file name, all the output files should be on the same file system as the current directory. Alternatively, you can use the `-p` flag to specify where the files go.
- Because page numbers cannot be determined until the document has been typeset, weft must be rerun after $\text{\LaTeX}$ to obtain a clean version of the document (very similar to the way it is sometimes necessary to rerun $\text{\LaTeX}$ to obtain an up-to-date table of contents after significant edits). weft will warn (in most cases) when this needs to be done; in the remaining cases, $\text{\LaTeX}$ will warn that labels may have changed. This restriction does not apply to Markdown output (`-w md`), which uses sequential numbering and requires no additional processing passes.

Very long scraps may be allowed to break across a page if declared with `@O` or `@D` (instead of `@o` and `@d`). This doesn't work very well as a default, since far too many short scraps will be broken across pages; however, as a user-controlled option, it seems very useful.

1.8 Project Introspection

The `-m` and `-e` flags support machine-readable introspection of a weft project. They run after pass 1 has built the name trees and do *not* invoke the weave or tangle phases.

1.8.1 Map (`-m`)

The command

```
weft -m project.weft
```

writes a JSON object to stdout with two arrays, `"files"` and `"fragments"`. For each entry the map records the name, language tag, definition scraps (with source file and line), and dependency relationships. File entries list the fragments they use; fragment entries list the scraps that invoke them and the fragments they reference in turn.

This gives an AI assistant or any external tool a complete dependency graph of the project in a single invocation, without having to parse the weft source files.

1.8.2 Extract (`-e`)

The command

```
weft -e "fragment name" project.weft
```

finds the named fragment (using prefix matching), then emits its scrap code and dependency information as Markdown to stdout. Referenced fragments are followed transitively, so the output contains the full closure of the requested fragment.

This allows an AI assistant to retrieve exactly the code it needs to understand or modify a specific fragment, together with all its dependencies, in one step.

1.8.3 Reverse Map (`-R`)

The `-m` and `-e` flags require the `.weft` source files. The reverse map provides the opposite direction: given a tangled output file, it parses the section markers already embedded in that file and reports which `.weft` source and line produced each region. No source files are needed and pass 1 is never invoked.

Single-line query.

```
weft -R server.js:47
```

emits a JSON object identifying the innermost scrap covering line 47:

```
{"scrap": 3, "file": "routes.weft", "line": 42}
```

If the line is outside any scrap region, an empty object `{}` is returned.

Full-file mapping.

```
weft -R server.js
```

emits a JSON object listing every scrap region in the file:

```
{"source": "server.js", "regions": [
  {"start": 2, "end": 15, "scrap": 3,
   "file": "routes.weft", "line": 42},
  {"start": 5, "end": 8, "scrap": 7,
   "file": "database.weft", "line": 18}
]}
```

Regions may overlap when fragments are nested.

This enables CI/CD pipelines to translate compiler errors back to the literate source automatically, and lets AI agents resolve tangled-output locations without loading the `.weft` files at all.

1.9 Acknowledgements

Several people have contributed their time, ideas, and debugging skills. In particular, the contributions of the following people are gratefully acknowledged: Osman Buyukisik, Manuel Carriba, Adrian Clarke, Tim Harvey, Michael Lewis, Walter Ravenek, Rob Shillingsburg, Kayvan Sylvan, Dominique de Waleffe, and Scott Warren. Of course, most of these people would never have heard of nuweb (the original tool) without the efforts of George Greenwade.

Since maintenance has been taken over by Marc Mengel, Simon Wright and Keith Harwood online contributions have been made by:

- Walter Brown <wb@fnal.gov>
- Nicky van Foreest <n.d.vanforeest@math.utwente.nl>
- Javier Goizueta <jgoizueta@jazzfree.com>
- Alan Karp <karp@hp.com>

Chapter 2

The Overall Structure

Processing a web requires three major steps:

1. Read the source, accumulating file names, fragment names, scraps, and lists of cross-references.
2. Reread the source, copying out to the documentation file, with protection and cross-reference information for all the scraps.
3. Traverse the list of files names. For each file name:
 - (a) Dump all the defining scraps into a temporary file.
 - (b) If the file already exists and is unchanged, delete the temporary file; otherwise, rename the temporary file.

2.1 Files

I have divided the program into several files for quicker recompilation during development.

"global.h" 21a≡

```
⟨ Include files 21b ⟩
⟨ Type declarations 23a, ... ⟩
⟨ Limits 23b ⟩
⟨ Global variable declarations 27, ... ⟩
⟨ Function prototypes 43a, ... ⟩
⟨ Operating System Dependencies 26 ⟩
```

◇

We'll need at least five of the standard system include files. We also require `_POSIX_C_SOURCE` at level 200809L so that POSIX functions like `strdup` are visible when compiling under strict C11 mode (`-std=c11`).

⟨ Include files 21b ⟩ ≡

```
#define _POSIX_C_SOURCE 200809L

#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <signal.h>
#include <locale.h>
```

◇

Fragment referenced in 21a.

Defines: **exit** 25f, 30, 37b, 38, 40b, 48cd, 70a, 112b, 113a, 117b, 123b, 128c, 129, 130ac, 137ab, 139b, 145d, 147b, 151d, 173a, 176b, 177a, 178cd, 180, 181, 186a, 192b, 202, **fclose** 58, 78b, 87a, 117b, 120, 123ab, 124bc, 130b, 143a, 155b, **FILE** 31, 57b, 58, 64a, 69a, 70b, 72bc, 73, 78b, 79, 80b, 82a, 84d, 86b, 87a, 91b, 94ab, 98ab, 100a, 101b, 103b, 104, 106a, 107, 108, 110b, 117b, 120, 123a, 124bc, 127ab, 132b, 133ab, 134b, 141d, 143a, 146b, 154a, 155b, 193cd, 195ab, 197bc, 202, **fopen** 58, 87a, 117b, 120, 123b, 124bc, 129, 130c, 155b, **fprintf** 34, 37c, 38, 40b, 43c, 45d, 48cd, 52a, 56b, 58, 60a, 62, 64b, 65d, 68bd, 70a, 74d, 76f, 77ab, 79, 80c, 85, 87a, 90a, 91c, 92d, 93c, 96b, 97ab, 98b, 100b, 103c, 105b, 106c, 107, 108, 110b, 112b, 113a, 114b, 115, 117b, 123ab, 125, 128c, 129, 130ac, 133a, 135c, 137ab, 139b, 141d, 143a, 145d, 147b, 149abc, 151cd, 153d, 154a, 160a, 170, 172, 173ab, 174, 176b, 177a, 178cd, 180, 181, 186a, 192b, 194b, 195d, 197de, 198bc, **fputs** 52c, 54b, 56b, 59ab, 61b, 62, 63a, 64ab, 65abcd, 67ae, 68abcd, 69a, 70b, 72bc, 73, 75a, 76a, 77b, 78ab, 79, 80acd, 81a, 82d, 83ab, 84c, 85, 89b, 90ab, 91bc, 92d, 93bc, 94ab, 95ac, 96ab, 97b, 98ab, 99, 100bcd, 101c, 102ab, 103ac, 105abc, 106bc, 107, 108, 109, 110b, 112a, 117b, 121, 124b, 133a, 143a, 147c, 151c, 153a, 194abc, 195cd, **getc** 124c, 127c, 128c, 129, 130abc, **getenv** 37c, **isgraph** 130a, 173a, 180, **islower** 161b, **isspace** 67a, 92a, 139b, 173ab, 176b, 178d, **isupper** Never used, **malloc** 82a, 101b, 186a, 201a, **putc** 52c, 53a, 60bc, 70b, 73, 74c, 77b, 78b, 79, 80cd, 82d, 83ab, 85, 87b, 88ab, 94b, 95abc, 97b, 98b, 100b, 101c, 102a, 133a, 147c, 150acd, 151d, 153abc, **remove** 124abc, **setlocale** 37c, **size_t** 78b, 116b, 121, 199d, 200b, 201a, **stderr** 34, 37c, 38, 40b, 43c, 45d, 48cd, 58, 68bd, 70a, 74d, 77b, 79, 87a, 92d, 93c, 97b, 98b, 112b, 113a, 114b, 115, 117b, 123ab, 125, 128c, 129, 130ac, 135c, 137ab, 139b, 145d, 147b, 151d, 160a, 170, 172, 173ab, 174, 176b, 177a, 178cd, 180, 181, 186a, 192b, 197e, 198bc, **strlen** 52c, 113b, 117b, 155c, 159ab, 188a, 196b, **toupper** 161b.

I like to use `TRUE` and `FALSE` in my code. I'd use an `enum` here, except that some systems seem to provide definitions of `TRUE` and `FALSE` by default. The following code seems to work on all the local systems.

< Type declarations 23a > ≡

```
#ifndef FALSE
#define FALSE 0
#endif
#ifndef TRUE
#define TRUE 1
#endif

#if defined(__GNUC__) && __GNUC__ >= 7
#define FALLTHROUGH __attribute__((fallthrough))
#elif defined(__clang__)
#define FALLTHROUGH __attribute__((fallthrough))
#else
#define FALLTHROUGH ((void)0)
#endif
```

◇
Fragment defined by 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b.
Fragment referenced in 21a.

Defines: `FALLTHROUGH` 45d, 60c, 75a, 88b, 127c, 159c, `FALSE` 27, 28, 34, 35, 36abcd, 37a, 40b, 60bc, 61c, 65c, 67d, 69a, 72c, 73, 78b, 83a, 84b, 85, 87b, 88ab, 89c, 92b, 94a, 98b, 102a, 103c, 105b, 106c, 111a, 128c, 130c, 151d, 155b, 160b, 162c, 163, 170, 173b, 175a, 192d, 194b, 195d, 196ab, `TRUE` 27, 28, 30, 34, 36c, 40b, 56b, 60c, 61c, 65ad, 69a, 74d, 78b, 83a, 84b, 85, 87b, 88ab, 89c, 91c, 92a, 94a, 98b, 102a, 103c, 105b, 106c, 110b, 112a, 128c, 133b, 135c, 151d, 160b, 162c, 163, 170, 173b, 192d, 194b, 195d, 196ab.

The `FALLTHROUGH` macro marks intentional switch case fall-throughs as a statement rather than a comment. GCC 7+ and Clang recognise `__attribute__((fallthrough))` and suppress `-Wimplicit-fallthrough` warnings; other compilers see a harmless `((void)0)`. The prose surrounding each switch already explains why the fall-through is intended, so no inline comment is needed.

Here we define the maximum length for names (of fragments etc), and the version string reported by `--version`. The version is a hard-coded literal — the single source of truth for weft's own version — so that *any* build from a fresh checkout carries the correct version, whether it is compiled with `make bootstrap`, re-tangled with plain `make`, or released with `make dist`. Cutting a release is therefore a one-line edit here. (The `-V` flag and the `@v` directive remain a separate, user-facing feature for stamping a version into *other* documents woven by weft; they no longer drive weft's own version.)

< Limits 23b > ≡

```
#ifndef MAX_NAME_LEN
#define MAX_NAME_LEN 1024
#endif

#define WEFT_VERSION "1.1.0"
```

◇
Fragment referenced in 21a.
Defines: `MAX_NAME_LEN` 139b, 141d, 145d, 173a, 176b, 180, 182a, 192b, 197a, `WEFT_VERSION` 30, 31.

2.1.1 The Main Files

The code is divided into four main files (introduced here) and five support files (introduced in the next section). The file `main.c` will contain the driver for the whole program (see Section 2.2).

"main.c" 23c≡

```
#include "global.h"
```

◇
File defined by [23c](#), [25f](#).

The first pass over the source file is contained in `pass1.c`. It handles collection of all the file names, fragments names, and scraps (see Section [2.3](#)).

```
"pass1.c" 24a≡  
    #include "global.h"
```

◇
File defined by [24a](#), [43bc](#).

The `.tex` file is created during a second pass over the source file. The file `latex.c` contains the code controlling the construction of the `.tex` file (see Section [2.4](#)).

```
"latex.c" 24b≡  
    #include "global.h"  
    static int scraps = 1;
```

◇
File defined by [24b](#), [57b](#), [58](#), [64a](#), [69ab](#), [70ab](#), [71a](#), [72abc](#), [73](#), [74a](#), [79](#), [80b](#), [81b](#), [82a](#), [84bd](#).
Uses: `scraps` [132a](#).

The Markdown output is created by the code in `markdown.c` (see Section [2.5](#)).

```
"markdown.c" 24c≡  
    #include "global.h"  
    static int scraps = 1;
```

◇
File defined by [24c](#), [86b](#), [87a](#), [91b](#), [94ab](#), [98ab](#), [100a](#), [101ab](#), [103b](#), [104](#), [106a](#).
Uses: `scraps` [132a](#).

The code controlling the creation of the output files is in `output.c` (see Section [2.7](#)).

```
"output.c" 24d≡  
    #include "global.h"
```

◇
File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).

2.1.2 Support Files

The support files contain a variety of support routines used to define and manipulate the major data abstractions. The file `input.c` holds all the routines used for referring to source files (see Section 3.1).

"input.c" 25a≡

```
#include "global.h"
```

◇

File defined by 25a, 127abc, 128b, 130c.

Creation and lookup of scraps is handled by routines in `scraps.c` (see Section 3.2).

"scraps.c" 25b≡

```
#include "global.h"
```

◇

File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

The handling of file names and fragment names is detailed in `names.c` (see Section 3.3).

"names.c" 25c≡

```
#include "global.h"
```

◇

File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.

Memory allocation and deallocation is handled by routines in `arena.c` (see Section 3.6).

"arena.c" 25d≡

```
#include "global.h"
```

◇

File defined by 25d, 199e, 200ab, 201b.

Finally, for best portability, I seem to need a file containing (useless!) definitions of all the global variables.

"global.c" 25e≡

```
#include "global.h"
< Operating System Dependencies 26 >
< Global variable definitions 28, ... >
```

◇

File defined by 25e, 199a.

2.2 The Main Routine

The main routine is quite simple in structure. It wades through the optional command-line arguments, then handles any files listed on the command line.

"main.c" 25f≡

```

#include <stdlib.h>
int main(int argc, char **argv)
{
    int arg = 1;
    < Interpret command-line arguments 29e, ... >
    < Set locale information 37c >
    < Handle reverse map mode 37b >
    initialise_delimit_scrap_array();
    < Process the remaining arguments (file names) 38 >
    exit(0);
}

```

◇
 File defined by [23c](#), [25f](#).
 Defines: `main` [202](#).
 Uses: `exit` [21b](#).

We only have two major operating system dependencies; the separators for file names, and how to set environment variables. For now we assume the latter can be accomplished via `putenv()` in `stdlib.h`.

< Operating System Dependencies 26 > ≡

```

#if defined(VMS)
#define PATH_SEP(c) (c=='|'||c==':')
#define PATH_SEP_CHAR ""
#define DEFAULT_PATH ""
#elif defined(MSDOS)
#define PATH_SEP(c) (c=='\\')
#define PATH_SEP_CHAR "\\"
#define DEFAULT_PATH "."
#else
#define PATH_SEP(c) (c=='/')
#define PATH_SEP_CHAR "/"
#define DEFAULT_PATH "."
#endif

```

◇
 Fragment referenced in [21a](#), [25e](#).

2.2.1 Command-Line Arguments

There are numerous possible command-line arguments:

- t Suppresses generation of the `.tex` file.
- o Suppresses generation of the output files.
- d list dangling identifier references in indexes.
- c Forces output files to overwrite old files of the same name without comparing for equality first.
- v The verbose flag. Forces output of progress reports.
- n Forces sequential numbering of scraps (instead of page numbers).
- s Doesn't print list of scraps making up file at end of each scrap.
- x Include cross-reference numbers in scrap comments.

- p *path* Prepend *path* to the filenames for all the output files.
- h *options* Provide options for the hyperref package.
- r Turn on hyperlinks. You must include the `—usepackage—` options in the text.
- l Deprecated no-op. Code scraps are always formatted with the `listings` package now, so this flag has no effect and is kept only for backward compatibility. Remember to `\usepackage{listings}` in your document preamble.

There are two ways to get hyper-links into your documentation. With one, the `-r` flag simply arranges for fragment cross-references to be hyperlinks. You have to provide all the rest of the `hyperref` material yourself. The `-h` flag does more work, but requires you provide the `hyperref` options on the command line. You would use the first if you know you will use the same tools all the time. You would use the second if you have to distribute the weft file and use the build system to provide the `hyperref` options.

The `-h` option is used as follows. If you want hyperlinks in your document give a string to this option which is a valid option for the `hyperref` package. For example, `"dvips,colorlinks=true"` sets it to use the `dvips` driver and mark the links in colour. For more information about these options, see the `hyperref` documentation.

Then in your document you put the command `\WEFTuseHyperlinks` immediately after your `usepackage` commands. If you do both of these you will get hyperlinks. If you miss out either then nothing interesting happens.

Global flags are declared for each of the arguments.

(*Global variable declarations 27*) $\equiv$

```
extern int tex_flag;      /* if FALSE, don't emit the documentation file */
extern int output_flag;  /* if FALSE, don't emit the output files */
extern int compare_flag; /* if FALSE, overwrite without comparison */
extern int verbose_flag; /* if TRUE, write progress information */
extern int number_flag;  /* if TRUE, use a sequential numbering scheme */
extern int scrap_flag;   /* if FALSE, don't print list of scraps */
extern int dangling_flag; /* if FALSE, don't print dangling identifiers */
extern int xref_flag; /* If TRUE, print cross-references in scrap comments */
extern int prepend_flag; /* If TRUE, prepend a path to the output file names */
extern char * dirpath;   /* The prepended directory path */
extern char * path_sep;  /* How to join path to filename */
extern int listings_flag; /* if TRUE, use listings package for scrap formatting */
extern int version_info_flag; /* If TRUE, set up version string */
extern char * version_string; /* What to print for @v */
extern int hyperref_flag; /* Are we preparing for hyperref
                           package. */
extern int hyperopt_flag; /* Are we preparing for hyperref options */
extern char * hyperoptions; /* The options to pass to the
                             hyperref package */
extern int includepath_flag; /* Do we have an include path? */
extern struct incl{char * name; struct incl * next;} * include_list;
                           /* The list of include paths */
extern int markdown_flag; /* if TRUE, generate Markdown instead of LaTeX */
extern int weave_flag;    /* if TRUE, weave was requested via -w */
extern int weave_format;  /* 0=none, 1=tex, 2=md */
extern int weave_format_from_cli; /* if TRUE, CLI override of @W */
extern int map_flag;      /* if TRUE, emit JSON map to stdout */
extern int extract_flag;  /* if TRUE, next arg is extract name */
extern char *extract_name; /* fragment name to extract */
extern int reverse_map_flag; /* if TRUE, reverse map mode */
extern char *reverse_map_arg; /* file[:line] argument for -R */
extern int bodies_flag;   /* if TRUE, include scrap bodies in -m JSON */
```

```

extern int prose_flag;      /* if TRUE, include prose context in -m JSON */
extern int diff_flag;      /* if TRUE, show diff instead of writing files */
extern int callers_flag;   /* if TRUE, show callers with -e */
extern int lint_flag;      /* if TRUE, run static analysis */
extern int errors_flag;    /* if TRUE, annotate compiler errors from stdin */

```

◇

Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.

Fragment referenced in 21a.

Defines: bodies_flag 28, 30, 108, callers_flag 28, 30, 40b, compare_flag 28, 34, 124a, dangling_flag 28, 34, 85, 103c, diff_flag 28, 30, 124a, errors_flag 28, 30, 37b, extract_flag 28, 34, 36d, extract_name 28, 36d, 40b, hyperoptions 28, 36c, 59a, 60a, hyperopt_flag 28, 34, 36c, hyperref_flag 28, 34, 36c, 59a, incl 28, 36a, 40a, 129, includepath_flag 28, 34, 36a, lint_flag 28, 30, 40b, map_flag 28, 34, 40b, markdown_flag 28, 34, 40b, number_flag 28, 34, 40b, 155b, output_flag 28, 34, 40b, prose_flag 28, 30, 43d, 44b, 108, reverse_map_arg 28, 37ab, reverse_map_flag 28, 34, 37a, scrap_flag 28, 34, 62, 92c, tex_flag 28, 34, 40b, 43c, 151d, verbose_flag 28, 34, 43c, 58, 87a, 123a, version_info_flag 28, 34, 36b, version_string 28, 36b, 61b, 76a, 89b, weave_flag 28, 34, 40b, weave_format 28, 34, 40b, 172, weave_format_from_cli 28, 34, 40b, 172, xref_flag 28, 34, 153ac.

Uses: FALSE 23a, next 131a, prose 131b, reverse 183e, scraps 132a, TRUE 23a.

The flags are all initialized for correct default behavior.

(*Global variable definitions 28*) ≡

```

int tex_flag = TRUE;
int output_flag = TRUE;
int compare_flag = TRUE;
int verbose_flag = FALSE;
int number_flag = FALSE;
int scrap_flag = TRUE;
int dangling_flag = FALSE;
int xref_flag = FALSE;
int prepend_flag = FALSE;
char * dirpath = DEFAULT_PATH; /* Default directory path */
char * path_sep = PATH_SEP_CHAR;
int listings_flag = FALSE;
int version_info_flag = FALSE;
char default_version_string[] = "no version";
char * version_string = default_version_string;
int hyperref_flag = FALSE;
int hyperopt_flag = FALSE;
char * hyperoptions = "";
int includepath_flag = FALSE; /* Do we have an include path? */
struct incl * include_list = NULL;
/* The list of include paths */

int markdown_flag = FALSE;
int weave_flag = FALSE;
int weave_format = 0;
int weave_format_from_cli = FALSE;
int map_flag = FALSE;
int extract_flag = FALSE;
char *extract_name = NULL;
int reverse_map_flag = FALSE;
char *reverse_map_arg = NULL;
int bodies_flag = FALSE;
int prose_flag = FALSE;
int diff_flag = FALSE;
int callers_flag = FALSE;
int lint_flag = FALSE;
int errors_flag = FALSE;

```

◇

Fragment defined by 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b.

Fragment referenced in 25e.

Uses: `bodies_flag` 27, `callers_flag` 27, `compare_flag` 27, `dangling_flag` 27, `diff_flag` 27, `errors_flag` 27, `extract_flag` 27, `extract_name` 27, `FALSE` 23a, `hyperoptions` 27, `hyperopt_flag` 27, `hyperref_flag` 27, `incl` 27, `includepath_flag` 27, `lint_flag` 27, `map_flag` 27, `markdown_flag` 27, `number_flag` 27, `output_flag` 27, `prose_flag` 27, `reverse_map_arg` 27, `reverse_map_flag` 27, `scrap_flag` 27, `tex_flag` 27, `TRUE` 23a, `verbose_flag` 27, `version_info_flag` 27, `version_string` 27, `weave_flag` 27, `weave_format` 27, `weave_format_from_cli` 27, `xref_flag` 27.

A global variable `nw_char` will be used for the weft meta-character, which by default will be @.

Global variable declarations 29a) $\equiv$

```
extern int nw_char;
```

◇
Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.

Fragment referenced in 21a.

Defines: `nw_char` 29b, 31, 40b, 43d, 45d, 48cd, 50a, 51c, 52b, 53c, 55, 56ab, 60bc, 62, 69b, 72b, 73, 74a, 75a, 76df, 80c, 85, 87b, 88b, 94b, 95c, 96b, 127c, 128c, 130c, 137ab, 138, 139b, 140cd, 141d, 145d, 147c, 150bd, 151cd, 153d, 154ac, 160a, 170, 172, 173a, 176b, 177ab, 178d, 180, 181, 182a, 192ab, 194b, 195d, 196a, 197a.

Global variable definitions 29b) $\equiv$

```
int nw_char='@';
```

◇
Fragment defined by 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b.

Fragment referenced in 25e.

Defines: `nw_char` 29a, 31, 40b, 43d, 45d, 48cd, 50a, 51c, 52b, 53c, 55, 56ab, 60bc, 62, 69b, 72b, 73, 74a, 75a, 76df, 80c, 85, 87b, 88b, 94b, 95c, 96b, 127c, 128c, 130c, 137ab, 138, 139b, 140cd, 141d, 145d, 147c, 150bd, 151cd, 153d, 154ac, 160a, 170, 172, 173a, 176b, 177ab, 178d, 180, 181, 182a, 192ab, 194b, 195d, 196a, 197a.

We save the invocation name of the command in a global variable `command_name` for use in error messages.

Global variable declarations 29c) $\equiv$

```
extern char *command_name;
```

◇
Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.

Fragment referenced in 21a.

Defines: `command_name` 29de, 31, 34, 38, 40b, 45d, 48cd, 58, 68d, 74d, 77b, 79, 87a, 93c, 97b, 98b, 112b, 113a, 123b, 125, 128c, 129, 130ac, 135c, 137ab, 139b, 145d, 147b, 151d, 160a, 170, 172, 173ab, 174, 176b, 177a, 178cd, 180, 181, 192b.

Global variable definitions 29d) $\equiv$

```
char *command_name = NULL;
```

◇
Fragment defined by 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b.

Fragment referenced in 25e.

Uses: `command_name` 29c.

The invocation name is conventionally passed in `argv[0]`.

Interpret command-line arguments 29e) $\equiv$

```
command_name = argv[0];
```

◇
Fragment defined by 29ef.

Fragment referenced in 25f.

Uses: `command_name` 29c.

We need to examine the remaining entries in `argv`, looking for command-line arguments.

⟨ Interpret command-line arguments 29f ⟩ ≡

```
while (arg < argc) {
    char *s = argv[arg];
    if (*s++ == '-') {
        ⟨ Check for long options 30 ⟩
        ⟨ Interpret the argument string s 34 ⟩
        arg++;
        ⟨ Perhaps get the prepend path 35 ⟩
        ⟨ Perhaps get the version info string 36b ⟩
        ⟨ Perhaps get the hyperref options 36c ⟩
        ⟨ Perhaps add an include path 36a ⟩
        ⟨ Perhaps get the extract name 36d ⟩
        ⟨ Perhaps get the reverse map argument 37a ⟩
    }
    else break;
}
```

◇

Fragment defined by [29ef](#).

Fragment referenced in [25f](#).

2.2.2 Help Message

When the user passes `--help`, we display a comprehensive reference that serves both human users and AI assistants. The help text is structured in sections—usage, options grouped by purpose, examples, and an AI workflow summary—so that a language model can parse it and use `weft` correctly without additional context.

We intercept long options before the character-by-character flag parsing. Currently `--help` and `--version` are the only long options.

⟨ Check for long options 30 ⟩ ≡

```
if (*s == '-') {
    if (strcmp(s + 1, "help") == 0) {
        ⟨ Print help message 31 ⟩
        exit(0);
    }
    if (strcmp(s + 1, "version") == 0) {
        printf("weft %s\n", WEFT_VERSION);
        exit(0);
    }
    if (strcmp(s + 1, "bodies") == 0) {
        bodies_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "prose") == 0) {
        prose_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "diff") == 0) {
        diff_flag = TRUE;
        arg++;
        continue;
    }
}
```

```

    if (strcmp(s + 1, "callers") == 0) {
        callers_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "lint") == 0) {
        lint_flag = TRUE;
        arg++;
        continue;
    }
    if (strcmp(s + 1, "errors") == 0) {
        errors_flag = TRUE;
        arg++;
        continue;
    }
}
}

```

◇

Fragment referenced in 29f.

Uses: bodies_flag 27, callers_flag 27, diff_flag 27, errors_flag 27, exit 21b, lint_flag 27, prose 131b, prose_flag 27, TRUE 23a, WEFT_VERSION 23b.

The help message itself. We write to `stdout` (not `stderr`) because `--help` is a deliberate request, not an error condition. The output is structured so that both humans and AI assistants can use `weft` effectively from `--help` alone: options grouped by function, per-file output flags, section marker format, a complete directives reference, and a concise AI workflow section.

⟨ *Print help message 31* ⟩ ≡

```

printf("weft %s -- A Literate Programming Tool\n\n", WEFT_VERSION);
printf("USAGE\n");
printf(" %s [options] file...\n\n", command_name);
printf("WHAT IS WEFT?\n");
printf(" weft is a literate programming tool: write .weft files where\n");
printf(" documentation and code live as one narrative. weft TANGLES them\n");
printf(" into executable output files and optionally WEAVES formatted\n");
printf(" documentation (LaTeX or Markdown). .weft files are the ONLY\n");
printf(" source of truth; tangled outputs are GENERATED -- never edit them.\n\n");
printf("HOW IT WORKS\n");
printf(" A .weft file contains prose with embedded code chunks:\n");
printf("  @o file.c @{ ... @}      Output file: tangle writes this to file.c\n");
printf("  @d Name   @{ ... @}      Fragment: reusable code block, invoked by name\n");
printf("  @<Name@>      Invocation: inserts the named fragment here\n");
printf("  @@                Literal @: escapes the meta-character\n");
printf("  @i other.weft      Include: pulls in another .weft file\n");
printf(" Fragments compose hierarchically. An @o chunk may invoke @d fragments,\n");
printf(" which invoke other fragments, forming a tree that weft flattens into\n");
printf(" the final output file.\n\n");
printf("OPTIONS\n");
printf(" Tangle control:\n");
printf("  -o                Suppress output file generation (parse only)\n");
printf("  -c                Overwrite files without comparing first\n");
printf("  -p path           Prepend path to all output file names\n");
printf("  -I path           Add directory to include search path (repeatable)\n");
printf("  -x                Include cross-reference numbers in section markers\n");
printf(" Weave control:\n");
printf("  -w [format]       Weave documentation (format: md or tex)\n");
printf("                    Without format, uses @W directive from source\n");
printf("  -t                Suppress weave output (even if -w given)\n");

```

```

printf("    -n                Sequential scrap numbering (auto for Markdown)\n");
printf("    -s                Omit scrap composition lists\n\n");
printf("Analysis (AI-friendly tools):\n");
printf("    -m                Emit JSON structure map to stdout (all fragments,\n");
printf("                    files, definitions, and dependency graph)\n");
printf("    -e name           Extract named fragment and all its transitive\n");
printf("                    dependencies as self-contained Markdown to stdout\n");
printf("    -R file[:line]    Reverse map: given a tangled output file (and\n");
printf("                    optional line number), prints the .weft source\n");
printf("                    location that generated it. Needs NO .weft files\n");
printf("                    -- reads section markers embedded in the output\n");
printf("    --bodies          Include raw scrap body text in -m JSON output\n");
printf("    --prose           Include preceding documentation in -m JSON output\n");
printf("    --diff            Show unified diff of what tangle would change\n");
printf("                    (no files written)\n");
printf("    --lint            Static analysis: unused fragments, undefined refs,\n");
printf("                    and fuzzy-match suggestions for likely typos\n");
printf("    --callers         With -e, show call chain from %co roots to fragment\n", nw_char);
printf("    --errors          Annotate compiler errors from stdin with .weft\n");
printf("                    source locations (reads section markers, no\n");
printf("                    .weft files needed)\n");
printf("                    Example: make 2>&1 | weft --errors\n\n");
printf("LaTeX:\n");
printf("    -r                Enable hyperref cross-reference links\n");
printf("    -h options        Pass options to hyperref package\n");
printf("    -l                Deprecated no-op (listings is always used)\n");
printf("    -d                List dangling identifier references\n\n");
printf("General:\n");
printf("    -V string         Set version string (expands @v in source)\n");
printf("    -v                Verbose progress output\n");
printf("    --help            Show this help message\n");
printf("    --version         Show version number\n\n");
printf("PER-FILE FLAGS (on @o)\n");
printf("    @o file [-d] [-i] [-t] [-s] [-cc|-c+|-cp] @{ ... @} \n");
printf("    -d                Emit #line N \"file\" directives (C/C++ debuggers)\n");
printf("    -i                Suppress auto-indentation of expanded fragments\n");
printf("    -t                Suppress tab expansion (write tabs literally)\n");
printf("    -cc              C block-comment markers: /* {N: file:line} */\n");
printf("    -c+              C++ line-comment markers: //{N: file:line}\n");
printf("    -cp              Hash-comment markers: # {N: file:line}\n");
printf("    -s                Suppress all section markers in output\n");
printf("    Without explicit flags, weft auto-detects from file extension\n");
printf("    (250+ extensions recognized). Use @l tag before @o to override.\n\n");
printf("SECTION MARKERS\n");
printf("    When comment style is known, tangled output includes markers:\n");
printf("    /* {5: source.weft:42} */ opening -- scrap 5, from line 42\n");
printf("    /* {5} */ closing\n");
printf("    Comment adapts to language: // for C++/Go, # for Python/shell, etc.\n");
printf("    Use weft -R to parse these markers and reverse-map to source.\n\n");
printf("DIRECTIVES REFERENCE\n");
printf("Core structure:\n");
printf("    @o file [flags] @{ code @} Output file (tangled to disk)\n");
printf("    @d Name @{ code @} Define reusable named fragment\n");
printf("    @d+ Name @{ code @} Global fragment (visible across sectors)\n");
printf("    @q Name @{ code @} Quoted fragment (no expansion on include)\n");
printf("    @O / @D / @Q Uppercase = big-format scrap in LaTeX\n");
printf("    @<Name@> Invoke (expand) a named fragment\n");
printf("    @<+Name@> Invoke the global version of a fragment\n");

```

```

printf("    @i file                Include another .weft file\n");
printf("    @@                Literal @ character\n\n");
printf("    Fragment parameters:\n");
printf("    @'text@"                Pass literal text argument to a fragment\n");
printf("    @1 .. @9                Substitute nth argument inside fragment\n");
printf("    @{ code @} as argument    Pass anonymous inline scrap as argument\n\n");
printf("    Inside scraps:\n");
printf("    @f                Current output file name\n");
printf("    @t                Current fragment title (as invoked)\n");
printf("    @v                Version string (set by -V flag)\n");
printf("    @c                Expand block comment at current indent\n");
printf("    @#                Suppress auto-indent for this line\n");
printf("    @%%                Comment to end of line (omitted)\n");
printf("    @| name            Mark identifier(s) as defined here\n");
printf("    @+ @- @*            Global-export / import / used markers\n");
printf("    @_ text @_          Bold (LaTeX weave); skipped in tangle\n");
printf("    @x label @x         Cross-reference label (anchor position)\n\n");
printf("    Scrap delimiters (LaTeX weave):\n");
printf("    @{ ... @}           Verbatim code (default)\n");
printf("    @[ ... @]           Paragraph mode\n");
printf("    @( ... @)           Math mode\n");
printf("    Sections and structure:\n");
printf("    @s                Start a new sector (section boundary)\n");
printf("    @S                Close current sector (return to base)\n");
printf("    @W fmt             Declare weave format: md or tex\n");
printf("    Language and formatting:\n");
printf("    @l tag             Set language for next @o or @d\n");
printf("    @L name style [+d] Define custom language (// # -- /* <!--)\n");
printf("    @r x               Change escape character from @ to x\n");
printf("    Indexes (weave output only):\n");
printf("    @f @m @u           File / fragment / identifier indexes\n");
printf("    @m+ @u+            Global fragment / identifier indexes\n");
printf("EXAMPLES\n");
printf("    %s project.weft                Tangle (default)\n", command_name);
printf("    %s -w md project.weft          Weave to Markdown\n", command_name);
printf("    %s -w tex project.weft         Weave to LaTeX\n", command_name);
printf("    %s -m project.weft | jq '.files' JSON structure map\n", command_name);
printf("    %s -e \"Validate\" project.weft    Extract fragment\n", command_name);
printf("    %s -R server.js:47             Reverse map line\n", command_name);
printf("    %s -R server.js               All regions in file\n", command_name);
printf("    %s -v -p build/ project.weft   Verbose + path prefix\n", command_name);
printf("    %s -o project.weft             Parse only, no output\n", command_name);
printf("AI ASSISTANT WORKFLOW\n");
printf("    GOLDEN RULE: .weft files are the ONLY source of truth.\n");
printf("    Tangled outputs (*.c, *.go, etc.) are GENERATED -- never edit them.\n");
printf("    If you edit a tangled file, your changes WILL be lost on next tangle.\n\n");
printf("    Step 1 -- Understand the project structure:\n");
printf("        %s -m project.weft > map.json\n", command_name);
printf("        The JSON contains: files (all @o output files), fragments (all @d\n");
printf("        named blocks), and a dependency graph (which fragments invoke which).\n");
printf("        Useful jq queries:\n");
printf("            jq '.files'                List output files\n");
printf("            jq '.fragments'            List all fragments\n");
printf("            jq '.fragments[] | select(.name | test(\"Auth\"))' Search\n");
printf("    Step 2 -- Read specific code without opening .weft files:\n");
printf("        %s -e \"fragment name\" project.weft\n", command_name);
printf("        Prints the fragment AND all its transitive dependencies as\n");
printf("        self-contained Markdown to stdout. Ideal for understanding\n");

```

```

printf("    one piece of the system without reading the whole source.\n\n");
printf(" Step 3 -- Edit the .weft source (NEVER tangled output):\n");
printf("    Locate the right .weft file (use -m to find it), then edit\n");
printf("    the @d or @o scrap that contains the code you need to change.\n");
printf("    Use standard text editing -- the prose around code is LaTeX or\n");
printf("    plain text, and code lives between @{ and @}.\n\n");
printf(" Step 4 -- Regenerate and verify:\n");
printf("    %s project.weft          Tangle (regenerate all output files)\n", command_name);
printf("    Then build and test as usual.\n\n");
printf(" Step 5 -- Debug errors with reverse map:\n");
printf("    %s -R output.c:42        Find .weft source for line 42\n", command_name);
printf("    %s -R output.c          Show ALL .weft regions in file\n", command_name);
printf("    Reads section markers in the tangled output -- needs NO .weft files.\n");
printf("    Fix errors in the .weft source, then re-tangle (step 4).\n");

```

◇

Fragment referenced in 30.

Uses: [command_name 29c](#), [FILE 21b](#), [first 200a](#), [Name 157c](#), [next 131a](#), [nw_char 29ab](#), [prose 131b](#), [reverse 183e](#), [scraps 132a](#), [search 188d](#), [sector 131b](#), [uses 131b](#), [WEFT_VERSION 23b](#).

2.2.3 Short Flags

Several flags can be stacked behind a single minus sign; therefore, we've got to loop through the string, handling them all. If this flag requires an argument we skip to getting its argument straight away. This allows arguments to butt up to their flags and also avoids ambiguity about which value goes with which flag.

⟨ Interpret the argument string s 34 ⟩ ≡

```

{
    char c = *s++;
    while (c) {
        switch (c) {
            case 'c': compare_flag = FALSE;
                       break;
            case 'd': dangling_flag = TRUE;
                       break;
            case 'h': hyperopt_flag = TRUE;
                       goto HasValue;
            case 'I': includepath_flag = TRUE;
                       goto HasValue;
            case 'l': listings_flag = TRUE;
                       break;
            case 'n': number_flag = TRUE;
                       break;
            case 'o': output_flag = FALSE;
                       break;
            case 'p': prepend_flag = TRUE;
                       goto HasValue;
            case 'r': hyperref_flag = TRUE;
                       break;
            case 's': scrap_flag = FALSE;
                       break;
            case 't': tex_flag = FALSE;
                       break;
            case 'e': extract_flag = TRUE;
                       goto HasValue;
            case 'R': reverse_map_flag = TRUE;
                       goto HasValue;
            case 'm': map_flag = TRUE;

```

```

        break;
    case 'v': verbose_flag = TRUE;
        break;
    case 'V': version_info_flag = TRUE;
        goto HasValue;
    case 'w': goto HasWeaveFormat;
    case 'x': xref_flag = TRUE;
        break;
    default: fprintf(stderr, "%s: unexpected argument ignored. "
        "Try '%s --help' for usage information.\n",
        command_name, command_name);
        break;
}
c = *s++;
}
goto HasValue;
HasWeaveFormat:
{
    char *fmt = NULL;
    weave_flag = TRUE;
    if (*s != '\0')
        fmt = s;
    else if (arg + 1 < argc && argv[arg + 1][0] != '-') {
        /* Peek: only consume next arg if it looks like a format token */
        char *candidate = argv[arg + 1];
        if (strcmp(candidate, "md") == 0 || strcmp(candidate, "tex") == 0) {
            fmt = candidate;
            ++arg;
        }
    }
    if (fmt != NULL) {
        if (strcmp(fmt, "md") == 0) {
            weave_format = 2;
            markdown_flag = TRUE;
            weave_format_from_cli = TRUE;
        } else if (strcmp(fmt, "tex") == 0) {
            weave_format = 1;
            weave_format_from_cli = TRUE;
        } else {
            fprintf(stderr, "%s: unknown weave format '%s'. Use 'md' or 'tex'.\n",
                command_name, fmt);
        }
    }
    /* If fmt is NULL, -w alone: format will come from @W */
    s = "";
}
HasValue:;
}

```

◇ Fragment referenced in 29f.

Uses: `command_name` 29c, `compare_flag` 27, `dangling_flag` 27, `extract_flag` 27, `FALSE` 23a, `fprintf` 21b, `hyperopt_flag` 27, `hyperref_flag` 27, `includepath_flag` 27, `map_flag` 27, `markdown_flag` 27, `next` 131a, `number_flag` 27, `output_flag` 27, `reverse_map_flag` 27, `scrap_flag` 27, `stderr` 21b, `tex_flag` 27, `TRUE` 23a, `verbose_flag` 27, `version_info_flag` 27, `weave_flag` 27, `weave_format` 27, `weave_format_from_cli` 27, `xref_flag` 27.

⟨ *Perhaps get the prepend path* 35 ⟩ ≡

```

if (prepend_flag)
{

```

```

    if (*s == '\0')
        s = argv[arg++];
    dirpath = s;
    prepend_flag = FALSE;
}

```

◇

Fragment referenced in [29f](#).

Uses: [FALSE 23a](#).

⟨ *Perhaps add an include path 36a* ⟩ ≡

```

if (includepath_flag)
{
    struct incl * le
        = (struct incl *)arena_getmem(sizeof(struct incl));
    struct incl ** p = &include_list;

    if (*s == '\0')
        s = argv[arg++];
    le->name = save_string(s);
    le->next = NULL;
    while (*p != NULL)
        p = &((*p)->next);
    *p = le;
    includepath_flag = FALSE;
}

```

◇

Fragment referenced in [29f](#).

Uses: [arena_getmem 200b](#), [FALSE 23a](#), [incl 27](#), [includepath_flag 27](#), [next 131a](#), [save_string 159b](#).

⟨ *Perhaps get the version info string 36b* ⟩ ≡

```

if (version_info_flag)
{
    if (*s == '\0')
        s = argv[arg++];
    version_string = s;
    version_info_flag = FALSE;
}

```

◇

Fragment referenced in [29f](#).

Uses: [FALSE 23a](#), [version_info_flag 27](#), [version_string 27](#).

⟨ *Perhaps get the hyperref options 36c* ⟩ ≡

```

if (hyperopt_flag)
{
    if (*s == '\0')
        s = argv[arg++];
    hyperoptions = s;
    hyperopt_flag = FALSE;
    hyperref_flag = TRUE;
}

```

◇

Fragment referenced in [29f](#).

Uses: [FALSE 23a](#), [hyperoptions 27](#), [hyperopt_flag 27](#), [hyperref_flag 27](#), [TRUE 23a](#).

⟨ *Perhaps get the extract name 36d* ⟩ ≡

```
if (extract_flag) {
    extract_name = (*s != '\0') ? s : argv[arg++];
    extract_flag = FALSE;
}
```

◇
Fragment referenced in 29f.

Uses: `extract_flag` 27, `extract_name` 27, `FALSE` 23a.

⟨ *Perhaps get the reverse map argument 37a* ⟩ ≡

```
if (reverse_map_flag) {
    reverse_map_arg = (*s != '\0') ? s : argv[arg++];
    reverse_map_flag = FALSE;
}
```

◇
Fragment referenced in 29f.

Uses: `FALSE` 23a, `reverse_map_arg` 27, `reverse_map_flag` 27.

If the user gave `-R file[:line]`, we handle it immediately and exit. No `.weft` source files are required—only the tangled output file.

⟨ *Handle reverse map mode 37b* ⟩ ≡

```
if (errors_flag) {
    write_error_annotator();
    exit(0);
}
if (reverse_map_arg) {
    write_reverse_map(reverse_map_arg);
    exit(0);
}
```

◇
Fragment referenced in 25f.

Uses: `errors_flag` 27, `exit` 21b, `reverse_map_arg` 27, `write_error_annotator` 121, `write_reverse_map` 117b.

In order to be able to process files in foreign languages we set the locale information. `isgraph()` and friends need this (see Section 3.3). Try to read `LC_CTYPE` from the environment. Use `LC_ALL` if that fails. Don't set the locale if reading `LC_ALL` fails, too. Print a warning if setting the program's locale fails.

⟨ *Set locale information 37c* ⟩ ≡

```
{
    /* try to get locale information */
    char *s=getenv("LC_CTYPE");
    if (s==NULL) s=getenv("LC_ALL");

    /* set it */
    if (s!=NULL)
        if(setlocale(LC_CTYPE, s)==NULL)
            fprintf(stderr, "Setting locale failed\n");
}
```

◇
Fragment referenced in 25f.

Uses: `fprintf` 21b, `getenv` 21b, `setlocale` 21b, `stderr` 21b.

2.2.4 File Names

We expect at least one file name. While a missing file name might be ignored without causing any problems, we take the opportunity to report the usage convention.

⟨ Process the remaining arguments (file names) 38 ⟩ ≡

```
{
  if (arg >= argc) {
    fprintf(stderr, "%s: expected a file name.\n"
               "Usage: %s [options] file...\n"
               "Try '%s --help' for more information.\n",
               command_name, command_name, command_name);
    exit(-1);
  }
  do {
    ⟨ Handle the file name in argv[arg] 39a ⟩
    arg++;
  } while (arg < argc);
}
```

◇

Fragment referenced in [25f](#).

Uses: `command_name` [29c](#), `exit` [21b](#), `fprintf` [21b](#), `stderr` [21b](#).

The code to handle a particular file name is rather more tedious than the actual processing of the file. A file name may be an arbitrarily complicated path name, with an optional extension. If no extension is present, we add `.w` as a default. The extended path name will be kept in a local variable `source_name`. The resulting documentation file will be written in the current directory; its name will be kept in the variable `tex_name`.

⟨ Handle the file name in `argv[arg]` 39a ⟩ ≡

```
{
    char source_name[FILENAME_MAX];
    char tex_name[FILENAME_MAX];
    char aux_name[FILENAME_MAX];
    char *trim;
    char *dot;
    ⟨ Build source_name and tex_name 39b ⟩
    ⟨ Process a file 40b ⟩
}
```

◇
Fragment referenced in 38.
Uses: `source_name` 126b.

I bump the pointer `p` through all the characters in `argv[arg]`, copying all the characters into `source_name` (via the pointer `q`).

At each slash, I update `trim` to point just past the slash in `source_name`. The effect is that `trim` will point at the file name without any leading directory specifications.

The pointer `dot` is made to point at the file name extension, if present. If there is no extension, we add `.weft` to the source name. The `tex_name` (weave output name) is computed later in “Process a file” once the weave format is known.

⟨ Build source_name and tex_name 39b ⟩ ≡

```
{
    char *p = argv[arg];
    char *q = source_name;
    char c;
    trim = q;
    dot = NULL;
    c = *p++;
    while (c) {
        *q++ = c;
        if (PATH_SEP(c)) {
            trim = q;
            dot = NULL;
        }
        else if (c == '.')
            dot = q - 1;
        c = *p++;
    }
    ⟨ Add the source path to the include path list 40a ⟩
    *q = '\0';
    if (!dot) {
        *q++ = '.';
        *q++ = 'w';
        *q++ = 'e';
        *q++ = 'f';
        *q++ = 't';
        *q = '\0';
    }
    /* Find trim again for tex_name computation later */
}
```

```

{
    char *t = source_name;
    trim = t;
    dot = NULL;
    while (*t) {
        if (PATH_SEP(*t)) { trim = t + 1; dot = NULL; }
        else if (*t == '.') dot = t;
        t++;
    }
}
}

```

◇
Fragment referenced in [39a](#).
Uses: [source_name 126b](#).

If the source file has a directory part we add that directory to the end of the search path so that the path of last resort is the same as the source path, not, as one person put it, something irrelevant like the directory weft was invoked from.

⟨ *Add the source path to the include path list 40a* ⟩ ≡

```

if (trim != source_name) {
    struct incl * le
        = (struct incl *)arena_getmem(sizeof(struct incl));
    struct incl ** p = &include_list;
    char sv = *trim;

    *trim = '\0';
    le->name = save_string(source_name);
    le->next = NULL;
    while (*p != NULL)
        p = &((*p)->next);
    *p = le;
    *trim = sv;
}

```

◇
Fragment referenced in [39b](#).
Uses: [arena_getmem 200b](#), [incl 27](#), [next 131a](#), [save_string 159b](#), [source_name 126b](#).

Now that we're finally ready to process a file, it's not really too complex. We bundle most of the work into three routines `pass1` (see [Section 2.3](#)), `write_tex` (see [Section 2.4](#)), and `write_files` (see [Section 2.7](#)). After we're finished with a particular file, we must remember to release its storage (see [Section 3.6](#)).

The default behavior is to tangle only. Weaving happens only when `-w` is given. The weave format is determined by (in priority order): a CLI argument (`-w md` or `-w tex`), or the `@W` directive in the source file. If `-w` is given without a format and no `@W` is present, weft reports an error and exits. The `-t` flag suppresses weaving even if `-w` was given.

⟨ *Process a file 40b* ⟩ ≡

```

{
    int do_weave = FALSE;
    int effective_format = 0;

    /* Reset weave_format for this file unless CLI locked it */
    if (!weave_format_from_cli)
        weave_format = 0;
}

```

```

pass1(source_name);
current_sector = 1;
prev_sector = 1;

if (lint_flag) {
    write_lint(file_names, macro_names);
    /* skip weave and tangle */
} else if (map_flag) {
    write_map(file_names, macro_names);
    /* skip weave and tangle */
} else if (extract_name) {
    if (callers_flag)
        write_callers(extract_name, file_names, macro_names);
    else
        write_extract(extract_name, file_names, macro_names);
    /* skip weave and tangle */
} else {
    /* Determine whether to weave and with which format */
    if (weave_flag && !tex_flag) {
        /* -t suppresses weave: tex_flag==FALSE means user passed -t */
    } else if (weave_flag) {
        effective_format = weave_format;
        if (effective_format == 0) {
            fprintf(stderr, "%s: -w requires a weave format.\n",
                "    Use -w md, -w tex, or add %cW md/%cW tex to the source file.\n",
                command_name, nw_char, nw_char);
            exit(-1);
        }
        do_weave = TRUE;
    }

    if (do_weave) {
        /* Compute weave output name now that format is known */
        const char *weave_ext = (effective_format == 2) ? ".md" : ".tex";
        if (dot) {
            *dot = '\\0';
            sprintf(tex_name, "%s%s%s%s", dirpath, path_sep, trim, weave_ext);
            sprintf(aux_name, "%s%s%s.aux", dirpath, path_sep, trim);
            *dot = '.';
        } else {
            sprintf(tex_name, "%s%s%s%s", dirpath, path_sep, trim, weave_ext);
            sprintf(aux_name, "%s%s%s.aux", dirpath, path_sep, trim);
        }

        if (effective_format == 2) {
            markdown_flag = TRUE;
            number_flag = TRUE;
            collect_numbers(aux_name);
            write_md(source_name, tex_name, 0);
        } else {
            collect_numbers(aux_name);
            write_tex(source_name, tex_name, 0);
        }
    }

    if (output_flag)
        write_files(file_names);
}

```

```
    arena_free();  
}
```

◇

Fragment referenced in 39a.

Uses: arena_free 201b, callers_flag 27, collect_numbers 155b, command_name 29c, current_sector 47b, exit 21b, extract_name 27, FALSE 23a, file_names 158a, fprintf 21b, lint_flag 27, macro_names 158a, map_flag 27, markdown_flag 27, number_flag 27, nw_char 29ab, output_flag 27, pass1 43c, prev_sector 47b, source_name 126b, stderr 21b, tex_flag 27, TRUE 23a, weave_flag 27, weave_format 27, weave_format_from_cli 27, write_callers 112b, write_extract 113a, write_files 122c, write_lint 115, write_map 109, write_md 87a, write_tex 58.

2.3 Pass One

During the first pass, we scan the file, recording the definitions of each fragment and file and accumulating all the scraps.

⟨ Function prototypes 43a ⟩ ≡

```
extern void pass1(char *file_name);
```

◇
Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: `file_name` 131b, `pass1` 43c.

The routine `pass1` takes a single argument, the name of the source file. It opens the file, then initializes the scrap structures (see Section 3.2) and the roots of the file-name tree, the fragment-name tree, and the tree of user-specified index entries (see Section 3.3). After completing all the necessary preparation, we make a pass over the file, filling in all our data structures. Next, we search all the scraps for references to the user-specified index entries. Finally, we must reverse all the cross-reference lists accumulated while scanning the scraps.

"pass1.c" 43b≡

```
static char prose_buf[4096];
static int prose_len = 0;
static char *pending_prose = NULL;
```

◇
File defined by 24a, 43bc.

Defines: `pending_prose` 43c, 44b, 47d, 48a, `prose_buf` 44ab, `prose_len` 43c, 44ab.

"pass1.c" 43c≡

```
void pass1(char *file_name)
{
    if (verbose_flag)
        fprintf(stderr, "reading %s\n", file_name);
    source_open(file_name);
    init_scraps();
    macro_names = NULL;
    file_names = NULL;
    user_names = NULL;
    prose_len = 0;
    pending_prose = NULL;
    ⟨ Scan the source file, looking for at-sequences 43d ⟩
    if (tex_flag)
        search();
    ⟨ Reverse cross-reference lists 53e ⟩
}
```

◇
File defined by 24a, 43bc.

Defines: `pass1` 40b, 43a, 75a, 95c.

Uses: `file_name` 131b, `file_names` 158a, `fprintf` 21b, `init_scraps` 132c, `macro_names` 158a, `pending_prose` 43b, `prose_len` 43b, `search` 188d, `source_open` 130c, `stderr` 21b, `tex_flag` 27, `user_names` 158a, `verbose_flag` 27.

The only thing we look for in the first pass are the command sequences. All ordinary text is skipped entirely.

⟨ Scan the source file, looking for at-sequences 43d ⟩ ≡

```

{
    int c = source_get();
    while (c != EOF) {
        if (c == nw_char)
            < Scan at-sequence 45d >
        else if (prose_flag) {
            < Accumulate prose character 44a >
        }
        c = source_get();
    }
}

```

◇ Fragment referenced in 43c.

Uses: `nw_char` 29ab, `prose_flag` 27, `source_get` 127c.

< Accumulate prose character 44a > ≡

```

if (prose_len < (int)sizeof(prose_buf) - 1)
    prose_buf[prose_len++] = c;

```

◇ Fragment referenced in 43d.

Uses: `prose_buf` 43b, `prose_len` 43b.

< Save pending prose 44b > ≡

```

if (prose_flag && prose_len > 0) {
    /* trim trailing whitespace */
    while (prose_len > 0 && (prose_buf[prose_len-1] == ' '
        || prose_buf[prose_len-1] == '\n'
        || prose_buf[prose_len-1] == '\t'
        || prose_buf[prose_len-1] == '\r'))
        prose_len--;
    prose_buf[prose_len] = '\0';
    pending_prose = save_string(prose_buf);
    prose_len = 0;
} else {
    pending_prose = NULL;
    prose_len = 0;
}

```

◇ Fragment referenced in 47d, 48a.

Uses: `pending_prose` 43b, `prose_buf` 43b, `prose_flag` 27, `prose_len` 43b, `save_string` 159b.

The `@l` command (lowercase) sets the language for the next `@o` or `@d` directive by storing the tag in `pending_lang`. It replaces the earlier mechanism of detecting Markdown fences (‘‘`lang`’) and L^AT_EX markers (`%%lang`) during pass1. A typical usage is:

```

@l python
@d Validate input
@{import re
...
@}

```

< Collect language apply in pass1 44c > ≡

```

{
    char lang[64];
    char *lp = lang;

```

```

int lc = source_get();
/* skip optional whitespace */
while (lc == ' ' || lc == '\t')
    lc = source_get();
/* read language tag */
while (lc != EOF && lc != '\n' && lc != ' ' && lc != '\t'
      && (lp - lang) < 62) {
    *lp++ = lc;
    lc = source_get();
}
*lp = '\0';
if (lang[0] != '\0') {
    strncpy(pending_lang, lang, 63);
    pending_lang[63] = '\0';
}
/* consume rest of line */
while (lc != EOF && lc != '\n')
    lc = source_get();
}

```

◇ Fragment referenced in 45d.
 Uses: `pending_lang` 158a, `source_get` 127c.

The `@L` command (Section 3.3.2) lets users register custom language mappings. During `pass1`, the actual parsing and registration is handled by `collect_lang_def()` in `names.c`. During the weave passes (L^AT_EX and Markdown), we simply consume the rest of the line so that the arguments do not leak into the typeset output.

⟨ *Collect user language definition in pass1* 45a ⟩ ≡

```
collect_lang_def();
```

◇ Fragment referenced in 45d.
 Uses: `collect_lang_def` 170.

⟨ *Collect weave format declaration in pass1* 45b ⟩ ≡

```
collect_weave_format();
```

◇ Fragment referenced in 45d.
 Uses: `collect_weave_format` 172.

⟨ *Skip at-L language definition line* 45c ⟩ ≡

```

{ int lc = source_get();
  while (lc != '\n' && lc != EOF)
    lc = source_get();
}
c = source_get();

```

◇ Fragment referenced in 60c, 88b.
 Uses: `source_get` 127c.

Six at-sequences are interesting during the first pass: `@o` and `@O` (file definitions), `@d`, `@D`, and `@q` (fragment definitions), `@c` (block comments), `@s`/`@S` (sectors), `@l` (language apply), and `@L` (language definitions). We skip past all others immediately, warning if unexpected sequences are discovered.

⟨ *Scan at-sequence* 45d ⟩ ≡

```

{
    char quoted = 0;

    c = source_get();
    switch (c) {
        case 'r':
            c = source_get();
            nw_char = c;
            update_delimit_scrap();
            break;
        case '0':
        case 'o': < Build output file definition 47d >
            break;
        case 'Q':
        case 'q': quoted = 1;
            FALLTHROUGH;
        case 'D':
        case 'd': < Build fragment definition 48a >
            break;
        case 's':
            < Step to next sector 46 >
            break;
        case 'S':
            < Close the current sector 47a >
            break;
        case '<':
        case '(':
        case '[':
        case '{': < Skip over an in-text scrap 48c >
            break;
        case 'c': < Collect a block comment 49a >
            break;
        case 'l': < Collect language apply in pass1 44c >
            break;
        case 'L': < Collect user language definition in pass1 45a >
            break;
        case 'W': < Collect weave format declaration in pass1 45b >
            break;
        case 'x':
        case 'v':
        case 'u':
        case 'm':
        case 'f': /* ignore during this pass */
            break;
        default: if (c==nw_char) /* ignore during this pass */
            break;
            fprintf(stderr,
                "%s: unexpected %c sequence ignored (%s, line %d)\n",
                command_name, nw_char, source_name, source_line);
            break;
    }
}

```

◇

Fragment referenced in 43d.

Uses: `command_name` 29c, `FALLTHROUGH` 23a, `fprintf` 21b, `nw_char` 29ab, `source_get` 127c, `source_line` 126b, `source_name` 126b, `stderr` 21b, `update_delimit_scrap` 74a.

< Step to next sector 46 > ≡

```

prev_sector += 1;
current_sector = prev_sector;
c = source_get();

```

◇
Fragment referenced in [45d](#), [60c](#), [88b](#).
Uses: [current_sector 47b](#), [prev_sector 47b](#), [source_get 127c](#).

⟨ *Close the current sector 47a* ⟩ ≡

```

current_sector = 1;
c = source_get();

```

◇
Fragment referenced in [45d](#), [60c](#), [88b](#).
Uses: [current_sector 47b](#), [source_get 127c](#).

⟨ *Global variable declarations 47b* ⟩ ≡

```

extern unsigned char current_sector;
extern unsigned char prev_sector;

```

◇
Fragment defined by [27](#), [29ac](#), [47b](#), [51d](#), [67b](#), [126b](#), [135d](#), [158a](#), [199c](#).
Fragment referenced in [21a](#).
Defines: [current_sector 40b](#), [46](#), [47ac](#), [81a](#), [84c](#), [100d](#), [103a](#), [136d](#), [139b](#), [176b](#), [180](#), [prev_sector 40b](#), [46](#), [47c](#).

⟨ *Global variable definitions 47c* ⟩ ≡

```

unsigned char current_sector = 1;
unsigned char prev_sector = 1;

```

◇
Fragment defined by [28](#), [29bd](#), [47c](#), [51e](#), [67c](#), [126c](#), [135e](#), [158b](#).
Fragment referenced in [25e](#).
Uses: [current_sector 47b](#), [prev_sector 47b](#).

2.3.1 Accumulating Definitions

There are three steps required to handle a definition:

1. Build an entry for the name so we can look it up later.
2. Collect the scrap and save it in the table of scraps.
3. Attach the scrap to the name.

We go through the same steps for both file names and fragment names.

⟨ *Build output file definition 47d* ⟩ ≡

```

{
    Name *name;
    int scrap;
    ⟨ Save pending prose 44b ⟩
    name = collect_file_name(); /* returns a pointer to the name entry */
    scrap = collect_scrap();    /* returns an index to the scrap */
    if (pending_prose) set_scrap_prose(scrap, pending_prose);
    pending_prose = NULL;
    ⟨ Add scrap to name's definition list 48b ⟩
}

```

◇

Fragment referenced in 45d.

Uses: collect_file_name 173a, collect_scrap 136c, Name 157c, pending_prose 43b, set_scrap_prose 143b.

⟨ Build fragment definition 48a ⟩ ≡

```
{
    Name *name;
    int scrap;
    ⟨ Save pending prose 44b ⟩
    name = collect_macro_name();
    scrap = collect_scrap();
    if (pending_prose) set_scrap_prose(scrap, pending_prose);
    pending_prose = NULL;
    ⟨ Add scrap to name's definition list 48b ⟩
}
```

◇
Fragment referenced in 45d.

Uses: collect_macro_name 176b, collect_scrap 136c, Name 157c, pending_prose 43b, set_scrap_prose 143b.

Since a file or fragment may be defined by many scraps, we maintain them in a simple linked list. The list is actually built in reverse order, with each new definition being added to the head of the list.

⟨ Add scrap to name's definition list 48b ⟩ ≡

```
{
    Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    def->scrap = scrap;
    def->quoted = quoted;
    def->next = name->defs;
    name->defs = def;
    set_scrap_owner(scrap, name);
}
```

◇
Fragment referenced in 47d, 48a.

Uses: arena_getmem 200b, defs 131b, next 131a, Scrap_Node 157b, set_scrap_owner 143c.

⟨ Skip over an in-text scrap 48c ⟩ ≡

```
{
    int c;
    int depth = 1;
    while ((c = source_get()) != EOF) {
        if (c == nw_char)
            ⟨ Skip over at-sign or go to skipped 48d ⟩
    }
    fprintf(stderr, "%s: unexpected EOF in text at (%s, %d)\n",
              command_name, source_name, source_line);
    exit(-1);

    skipped: ;
}
```

◇
Fragment referenced in 45d.

Uses: command_name 29c, exit 21b, fprintf 21b, nw_char 29ab, source_get 127c, source_line 126b, source_name 126b, stderr 21b.

⟨ *Skip over at-sign or go to skipped 48d* ⟩ ≡

```
{
    c = source_get();
    switch (c) {
        case '{': case '[': case '(': case '<':
            depth += 1;
            break;
        case '}': case ']': case ')': case '>':
            if (--depth == 0)
                goto skipped;
        case 'x': case '|': case ',':
        case '%': case '1': case '2':
        case '3': case '4': case '5': case '6':
        case '7': case '8': case '9': case '_':
        case 'f': case '#': case '+': case '-':
        case 'v': case '*': case 'c': case '\\':
        case 's':
            break;
        default:
            if (c != nw_char) {
                fprintf(stderr, "%s: unexpected %c%c in text at (%s, %d)\n",
                           command_name, nw_char, c, source_name, source_line);
                exit(-1);
            }
            break;
    }
}
```

◇ Fragment referenced in [48c](#).

Uses: [command_name 29c](#), [exit 21b](#), [fprintf 21b](#), [nw_char 29ab](#), [source_get 127c](#), [source_line 126b](#), [source_name 126b](#), [stderr 21b](#).

2.3.2 Block Comments

⟨ *Collect a block comment 49a* ⟩ ≡

```
{
    char * p = blockBuff;
    char * e = blockBuff + (sizeof(blockBuff)/sizeof(blockBuff[0])) - 1;

    ⟨ Skip whitespace 49b ⟩
    while (p < e)
    {
        ⟨ Add one char to the block buffer 50a ⟩
    }
    if (p == e)
    {
        ⟨ Skip to the next nw-char 51c ⟩
    }
    *p = '\000';
}
```

◇ Fragment referenced in [45d](#).

Uses: [blockBuff 51d](#).

⟨ *Skip whitespace 49b* ⟩ ≡

```

while (source_peek == ' '
      || source_peek == '\t'
      || source_peek == '\n')
    (void)source_get();

```

◇

Fragment referenced in [49a](#).

Uses: `source_get` [127c](#), `source_peek` [127c](#).

⟨ *Add one char to the block buffer 50a* ⟩ ≡

```

int c = source_get();

if (c == nw_char)
{
    ⟨ Add an at character to the block or break 50b ⟩
}
else if (c == EOF)
{
    source_ungetc(&c);
    break;
}
else
{
    ⟨ Add any other character to the block 51a ⟩
}

```

◇

Fragment referenced in [49a](#).

Uses: `nw_char` [29ab](#), `source_get` [127c](#), `source_ungetc` [128b](#).

⟨ *Add an at character to the block or break 50b* ⟩ ≡

```

int cc = source_peek;

if (cc == 'c')
{
    do
        c = source_get();
    while (c <= ' ');

    break;
}
else if (cc == 'd'
        || cc == 'D'
        || cc == 'q'
        || cc == 'Q'
        || cc == 'o'
        || cc == 'O'
        || cc == EOF)
{
    source_ungetc(&c);
    break;
}
else
{
    *p++ = c;
    *p++ = source_get();
}

```

◇

Fragment referenced in 50a.

Uses: `source_get` 127c, `source_peek` 127c, `source_ungetc` 128b.

⟨ *Add any other character to the block* 51a ⟩ ≡

```
⟨ Perhaps skip white-space 51b ⟩  
*p++ = c;
```

◇

Fragment referenced in 50a.

⟨ *Perhaps skip white-space* 51b ⟩ ≡

```
if (c == ' ')  
{  
    while (source_peek == ' ')  
        c = source_get();  
}  
if (c == '\\n')  
{  
    if (source_peek == '\\n')  
    {  
        do  
            c = source_get();  
        while (source_peek == '\\n');  
    }  
    else  
        c = ' ';  
}  
}
```

◇

Fragment referenced in 51a.

Uses: `source_get` 127c, `source_peek` 127c.

⟨ *Skip to the next nw-char* 51c ⟩ ≡

```
int c;  
  
while ((c = source_get()), c != nw_char && c != EOF) /* Skip */  
    source_ungetc(&c);
```

◇

Fragment referenced in 49a.

Uses: `nw_char` 29ab, `source_get` 127c, `source_ungetc` 128b.

⟨ *Global variable declarations* 51d ⟩ ≡

```
extern char blockBuff[6400];
```

◇

Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.

Fragment referenced in 21a.

Defines: `blockBuff` 49a, 51e, 52b.

⟨ *Global variable definitions* 51e ⟩ ≡

```
char blockBuff[6400];
```

◇

Fragment defined by 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b.

Fragment referenced in 25e.

Uses: `blockBuff` 51d.

We don't show block comments inside scraps in the T_EX file, but we do show where they go.

⟨ Show presence of a block comment 52a ⟩ ≡

```
{
  begin_tex_escape(file);
  fprintf(file, "\\hbox{\\sffamily\\slshape (Comment)}");
  end_tex_escape(file);
}
```

◇

Fragment referenced in [75a](#).

Uses: [begin_tex_escape 72c](#), [end_tex_escape 72c](#), [fprintf 21b](#).

The block comment is presently in the block buffer. Here we copy it into the scrap whence we shall copy it into the output file.

⟨ Include block comment in a scrap 52b ⟩ ≡

```
{
  char * p = blockBuff;

  push(nw_char, &writer);
  do
  {
    push(c, &writer);
    c = *p++;
  } while (c != '\\0');
}
```

◇

Fragment referenced in [137b](#).

Uses: [blockBuff 51d](#), [nw_char 29ab](#), [push 136a](#).

⟨ Copy block comment from scrap 52c ⟩ ≡

```
{
  int bgn = indent + global_indent;
  int posn = bgn + strlen(comment_begin[comment_flag]);
  int i;

  ⟨ Perhaps put a delayed indent 153b ⟩
  c = pop(&reader);
  fputs(comment_begin[comment_flag], file);
  while (c != '\\0')
  {
    ⟨ Move a word to the file 53a ⟩
    ⟨ If we break the line at this word 53b ⟩
    {
      putc('\\n', file);
      for (i = 0; i < bgn ; i++)
        putc(' ', file);
      c = pop(&reader);
      if (c != '\\0')
      {
        posn = bgn + strlen(comment_mid[comment_flag]);
        fputs(comment_mid[comment_flag], file);
      }
    }
  }
}
```

```

    }
}
fputs(comment_end[comment_flag], file);
}

```

◇ Fragment referenced in [150d](#).

Uses: [comment_begin 176a](#), [comment_end 176a](#), [comment_mid 176a](#), [fputs 21b](#), [pop 141c](#), [putc 21b](#), [strlen 21b](#).

⟨ *Move a word to the file 53a* ⟩ ≡

```

do
{
    putc(c, file);
    posn += 1;
    c = pop(&reader);
} while (c > ' ');

```

◇ Fragment referenced in [52c](#).

Uses: [pop 141c](#), [putc 21b](#).

⟨ *If we break the line at this word 53b* ⟩ ≡

```

if (c == '\n' || (c == ' ' && posn > 60))

```

◇ Fragment referenced in [52c](#).

⟨ *Skip over a block comment 53c* ⟩ ≡

```

if (last == nw_char && c == 'c')
    while ((c = pop(reader.m)) != '\0')
        /* Skip */;

```

◇ Fragment referenced in [191](#).

Uses: [nw_char 29ab](#), [pop 141c](#).

⟨ *Begin or end a block comment 53d* ⟩ ≡

```

if (inBlock)
{
    ⟨ End block 65c ⟩
}
else
{
    ⟨ Start block 65a ⟩
}

```

◇ Fragment referenced in [60c](#).

2.3.3 Fixing the Cross References

Since the definition and reference lists for each name are accumulated in reverse order, we take the time at the end of `pass1` to reverse them all so they'll be simpler to print out prettily. The code for `reverse_lists` appears in [Section 3.3](#).

⟨ *Reverse cross-reference lists 53e* ⟩ ≡

```

{
    reverse_lists(file_names);
    reverse_lists(macro_names);
    reverse_lists(user_names);
}

```

◇

Fragment referenced in [43c](#).

Uses: `file_names` [158a](#), `macro_names` [158a](#), `reverse_lists` [183d](#), `user_names` [158a](#).

2.3.4 Dealing with fragment parameters

Fragment parameters were added on later in nuweb's development. There still is not, for example, an index of fragment parameters. We need a data type to keep track of fragment parameters.

⟨ *Type declarations* 54a ⟩ ≡

```
typedef int *Parameters;
```

◇

Fragment defined by [23a](#), [54a](#), [122d](#), [157bc](#), [178a](#), [179c](#), [183a](#), [193b](#), [199b](#).

Fragment referenced in [21a](#).

Defines: `Parameters` [56a](#), [61c](#), [89c](#), [132b](#), [145d](#), [146b](#), [147c](#).

When we are copying a scrap to the output, we check for an embedded parameter. If we find it we copy it. Otherwise it's an old-style parameter and we can then pull the *n*th string from the `Parameters` list when we see an `@1 @2`, etc.

⟨ *Handle macro parameter substitution* 54b ⟩ ≡

```

case '1': case '2': case '3':
case '4': case '5': case '6':
case '7': case '8': case '9':
{
    Arglist * args;
    Name * name;

    lookup(c - '1', inArgs, inParams, &name, &args);

    if (name == (Name *)1) {
        Embed_Node * q = (Embed_Node *)args;
        indent = write_scraps(file, spelling, q->defs,
                                global_indent + indent,
                                indent_chars, debug_flag,
                                tab_flag, indent_flag,
                                0, 0,
                                q->args, inParams,
                                local_parameters, "");
    }
    else if (name != NULL) {
        int i, narg;
        char * p = name->spelling;
        Arglist *q = args;

        ⟨ Perhaps comment this macro 153a ⟩
        indent = write_scraps(file, spelling, name->defs,
                                global_indent + indent,
                                indent_chars, debug_flag,
                                tab_flag, indent_flag,

```

```

        comment_flag, location_flag,
        args, name->arg,
        local_parameters, p);
    }
    else if (args != NULL) {
        if (delayed_indent) {
            ⟨Put out the indent 150a⟩
        }
        fputs((char *)args, file);
    }
    else if ( parameters && parameters[c - '1'] ) {
        Scrap_Node param_defs;
        param_defs.scrap = parameters[c - '1'];
        param_defs.next = 0;
        write_scraps(file, spelling, &param_defs,
            global_indent + indent,
            indent_chars, debug_flag,
            tab_flag, indent_flag,
            comment_flag, location_flag,
            NULL, NULL, 0, "");
    } else if (delayed_indent) {
        ⟨Put out the indent 150a⟩
    }
}

```

◇ Fragment referenced in 150d.

Uses: Arglist 179c, defs 131b, delayed_indent 147a, Embed_Node 183a, fputs 21b, lookup 144a, Name 157c, next 131a, Scrap_Node 157b, write_scraps 146b.

Now onto actually parsing fragment parameters from a call. We start off checking for fragment parameters, an @ (sequence followed by parameters separated by @, sequences, and terminated by a @) sequence.

We collect separate scraps for each parameter, and write the scrap numbers down in the text. For example, if the file has:

```
@<foo @( param1 @, param2 @)@>
```

we actually make new scraps, say 10 and 11, for param1 and param2, and write in the collected scrap:

```
@<foo @(10@,11@)@>
```

⟨Save macro parameters 55⟩ ≡

```

{
    int param_scrap;
    char param_buf[10];

    push(nw_char, &writer);
    push('(', &writer);
    do {
        param_scrap = collect_scrap();
        sprintf(param_buf, "%d", param_scrap);
        pushs(param_buf, &writer);
        push(nw_char, &writer);
        push(scrap_ended_with, &writer);
        add_to_use(name, current_scrap);
    } while( scrap_ended_with == ',' );
    do
        c = source_get();
    while( ' ' == c );
}

```

```

    if (c == nw_char) {
        c = source_get();
    }
    if (c != '>') {
        /* ZZZ print error */;
    }
}

```

◇

Fragment referenced in 140c.

Uses: add_to_use 141a, collect_scrap 136c, nw_char 29ab, push 136a, pushes 136b, source_get 127c.

If we get inside, we have at least one parameter, which will be at the beginning of the parms buffer, and we prime the pump with the first character.

⟨ Check for macro parameters 56a ⟩ ≡

```

if (c == '(') {
    Parameters res = arena_getmem(10 * sizeof(int));
    int *p2 = res;
    int count = 0;
    int scrapnum;

    while( c && c != ')' ) {
        scrapnum = 0;
        c = pop(manager);
        while( '0' <= c && c <= '9' ) {
            scrapnum = scrapnum * 10 + c - '0';
            c = pop(manager);
        }
        if ( c == nw_char ) {
            c = pop(manager);
        }
        *p2++ = scrapnum;
    }
    while (count < 10) {
        *p2++ = 0;
        count++;
    }
    while( c && c != nw_char ) {
        c = pop(manager);
    }
    if ( c == nw_char ) {
        c = pop(manager);
    }
    *parameters = res;
}

```

◇

Fragment referenced in 146a.

Uses: arena_getmem 200b, nw_char 29ab, Parameters 54a, pop 141c.

These are used in write_tex to output the argument list for a fragment.

⟨ Format macro parameters 56b ⟩ ≡

```

char sep;

```

```

sep = '(';
do {
    fputc(sep,file);

    fputs("\\footnotesize ", file);
    write_single_scrap_ref(file, scraps + 1);
    fprintf(file, "\\label{scrap%d}", scraps + 1);
    fputs(" }", file);

    source_last = '{';
    {
        /* The parameter is rendered inside a fragment reference, i.e.
           inside a listings escape, so it must be single-line LaTeX with
           the special characters neutralised: switch copy_scrap into its
           inline mode for the duration. */
        int saved_inline = code_inline;
        code_inline = TRUE;
        copy_scrap(file, TRUE, NULL);
        code_inline = saved_inline;
    }

    ++scraps;

    sep = ',';
} while ( source_last != ')' && source_last != EOF );
fputs(" ) ",file);
do
    c = source_get();
while(c != nw_char && c != EOF);
if (c == nw_char) {
    c = source_get();
}

```

◇ Fragment referenced in 77b, 79.

Uses: `code_inline` 72c, `copy_scrap` 73, `fprintf` 21b, `fputs` 21b, `nw_char` 29ab, `scraps` 132a, `source_get` 127c, `source_last` 127c, `TRUE` 23a, `write_single_scrap_ref` 133b.

2.4 Writing the Latex File

The second pass (invoked via a call to `write_tex`) copies most of the text from the source file straight into a `.tex` file. Definitions are formatted slightly and cross-reference information is printed out.

Note that all the formatting is handled in this section. If you don't like the format of definitions or indices or whatever, it'll be in this section somewhere. Similarly, if someone wanted to modify `weft` to work with a different typesetting system, this would be the place to look.

⟨ *Function prototypes* 57a ⟩ ≡

```
extern void write_tex(char *file_name, char *tex_name, unsigned char sector);
```

◇ Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: `file_name` 131b, `sector` 131b, `write_tex` 58.

We need a few local function declarations before we get into the body of `write_tex`.

"`latex.c`" 57b ≡

```

static void copy_scrap(FILE *file, int prefix, Name *name);           /* formats the body of a
    scrap */
static void latex_code_putc(FILE *file, int c);                     /* escapes one code
    character for LaTeX */
static const char *latex_language(const char *lang);               /* maps a weft language
    tag to a listings name */
static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps); /* formats a list of
    scrap numbers */
static void format_entry(Name *name, FILE *tex_file, unsigned char sector); /*
    formats an index entry */
static void format_file_entry(Name *name, FILE *tex_file);          /* formats a file index entry
    */
static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector);
static void write_arg(FILE *tex_file, char *p);
static void write_literal(FILE *tex_file, char *p, int mode);
static void write_ArglistElement(FILE *file, Arglist *args, char **params);

```

◇

File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.

Uses: Arglist 179c, copy_scrap 73, FILE 21b, format_entry 82a, format_file_entry 80b, format_user_entry 84d, latex_code_putc 70b, latex_language 71a, Name 157c, print_scrap_numbers 69a, scraps 132a, Scrap_Node 157b, sector 131b, write_arg 64a, write_ArglistElement 79, write_literal 72b.

The routine `write_tex` takes two file names as parameters: the name of the web source file and the name of the `.tex` output file.

"latex.c" 58≡

```

void write_tex(char *file_name, char *tex_name, unsigned char sector)
{
    (void)sector;
    FILE *tex_file = fopen(tex_name, "w");
    if (tex_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", tex_name);
        source_open(file_name);
        { Write LaTeX limbo definitions 59a }
        { Copy source_file into tex_file 60b }
        fclose(tex_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, tex_name);
}

```

◇

File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.

Defines: `write_tex` 40b, 57a.

Uses: `command_name` 29c, `fclose` 21b, `FILE` 21b, `file_name` 131b, `fopen` 21b, `fprintf` 21b, `sector` 131b, `source_open` 130c, `stderr` 21b, `verbose_flag` 27.

Weft's woven output relies on a family of `\WEFT...` macros. We write default definitions for all of them into the document's limbo so that source files need not define anything new. If a user wants to change any of the macros (to use `hyperref`, to write in some language other than english, or to tune the code layout) he or she can redefine the commands *after* the point where weft's definitions are read, or (better) redefine them in the preamble with `\renewcommand`.

Two groups of macros are written. The first group are the textual and cross-reference macros (`\WEFTtarget`, `\WEFTlink`, the `\WEFTtxt...` labels, and so on). The second group, written by the scrap-formatting fragment below, are the layout macros that frame a scrap.

Code bodies are typeset with the `listings` package: every scrap becomes an `lstlisting` environment. This gives syntax highlighting (driven by the scrap’s language, see `latex_language`), automatic line wrapping (`breaklines`), and page breaking for free. The document *must* therefore load `\usepackage{listings}` in its preamble — `weft` cannot inject it, because the `limbo` is written before `\documentclass`. Fragment references that appear in the middle of code are typeset by briefly escaping out of the listing back to L^AT_EX; see §2.4 and the `escapeinside` option.

⟨ Write LaTeX limbo definitions 59a ⟩ ≡

```

if (hyperref_flag) {
  fputs("\\newcommand{\\WEFTtarget}[2]{\\hypertarget{#1}{#2}}\\n", tex_file);
  fputs("\\newcommand{\\WEFTlink}[2]{\\hyperlink{#1}{#2}}\\n", tex_file);
} else {
  fputs("\\newcommand{\\WEFTtarget}[2]{#2}\\n", tex_file);
  fputs("\\newcommand{\\WEFTlink}[2]{#2}\\n", tex_file);
}
fputs("\\newcommand{\\WEFTtxtMacroDefBy}{Fragment defined by}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtMacroRefIn}{Fragment referenced in}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtMacroNoRef}{Fragment never referenced}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtDefBy}{Defined by}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtRefIn}{Referenced in}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtNoRef}{Not referenced}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtFileDefBy}{File defined by}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsUsed}{Uses:}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsNotUsed}{Never used}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsDefed}{Defines:}\\n", tex_file);
fputs("\\newcommand{\\WEFTsep}{\\$\\diamond}\\n", tex_file);
fputs("\\newcommand{\\WEFTnotglobal}{(not defined globally)}\\n", tex_file);
⟨ Write the scrap formatting macros 59b ⟩
fputs("\\newcommand{\\WEFTuseHyperlinks}{", tex_file);
if (hyperoptions[0] != '\\0')
{
  ⟨ Write the hyperlink usage macro 60a ⟩
}
fputs("}\\n", tex_file);

```

◇

Fragment referenced in 58.

Uses: `fputs` 21b, `hyperoptions` 27, `hyperref_flag` 27, `Uses` 193b.

The layout macros. `\WEFTbegin`/`\WEFTend` open and close the vertical block that holds a scrap’s header line, its `lstlisting` body, and its cross-reference list. A fragment reference appearing in the middle of code is assembled inline and wrapped in an `\hbox` so that `listings`’ line-breaker never splits it while it sits inside an escape. (We use `\hbox` rather than `\mbox` because a fragment name may legitimately contain `\verb`, which is illegal in a macro argument such as `\mbox`’s but fine inside a primitive `\hbox`.) `\WEFTlstsetup` is a convenience the user may call once, after `\usepackage{listings}`, to apply `weft`’s recommended `\lstset` defaults (monospace, line breaking, keyword/comment styles). It is optional: the `escapeinside` and `language` options that make the output *correct* are written on every `lstlisting` itself, so references and highlighting work even without it.

⟨ Write the scrap formatting macros 59b ⟩ ≡

```

fputs("\\newcommand{\\WEFTbegin}{\\par\\addvspace{2.3ex plus .6ex}"
      "\\begingroup\\small\\raggedright}\\n", tex_file);
fputs("\\newcommand{\\WEFTend}{\\par\\endgroup"
      "\\addvspace{2.3ex plus .6ex}}\\n", tex_file);
fputs("\\providecommand{\\WEFTlstsetup}{\\lstset{basicstyle=\\ttfamily"
      "\\small,breaklines=true,breakatwhitespace=false,"
      "columns=fullflexible,keepspaces=true,showstringspaces=false,"

```

```
"keywordstyle=\bfseries,commentstyle=\itshape,tabsize=8,"
"xleftmargin=1.5em}}\n", tex_file);
```

◇
Fragment referenced in 59a.
Uses: `fputs` 21b.

⟨ *Write the hyperlink usage macro 60a* ⟩ ≡

```
fprintf(tex_file, "\\usepackage[%s]{hyperref}", hyperoptions);
```

◇
Fragment referenced in 59a.
Uses: `fprintf` 21b, `hyperoptions` 27.

We make our second (and final) pass through the source web, this time copying characters straight into the `.tex` file. However, we keep an eye peeled for `@` characters, which signal a command sequence.

⟨ *Copy source_file into tex_file 60b* ⟩ ≡

```
{
  int inBlock = FALSE;
  int c = source_get();
  while (c != EOF) {
    if (c == nw_char)
    {
      ⟨ Interpret at-sequence 60c ⟩
    }
    else {
      putc(c, tex_file);
      c = source_get();
    }
  }
}
```

◇
Fragment referenced in 58.
Uses: `FALSE` 23a, `nw_char` 29ab, `putc` 21b, `source_get` 127c.

⟨ *Interpret at-sequence 60c* ⟩ ≡

```
{
  int big_definition = FALSE;
  c = source_get();
  switch (c) {
    case 'r':
      c = source_get();
      nw_char = c;
      update_delimit_scrap();
      break;
    case '0': big_definition = TRUE;
              FALLTHROUGH;
    case 'o': ⟨ Write output file definition 62 ⟩
              break;
    case 'Q':
    case 'D': big_definition = TRUE;
              FALLTHROUGH;
    case 'q':
    case 'd': ⟨ Write macro definition 63a ⟩
              break;
    case 's':
```

```

        < Step to next sector 46>
        break;
case 'S':
    < Close the current sector 47a>
    break;
case '{':
case '[':
case '(': < Write in-text scrap 67d>
    break;
case '<': < Expand macro into tex_file 61c>
    break;
case 'x': < Copy label from source into (61a tex_file ) 76b>
    c = source_get();
    break;
case 'c': < Begin or end a block comment 53d>
    c = source_get();
    break;
case 'f': < Write index of file names 80a>
    break;
case 'm': < Write index of macro names 81a>
    break;
case 'u': < Write index of user-specified names 84c>
    break;
case 'v': < Copy version info into tex file 61b>
    break;
case 'l':
case 'L': < Skip at-L language definition line 45c>
    break;
case 'W': < Skip at-L language definition line 45c>
    break;
default:
    if (c==nw_char)
        putc(c, tex_file);
    c = source_get();
    break;
}
}

```

◇
Fragment referenced in 60b.

< Copy version info into tex file 61b> ≡

```

fputs(version_string, tex_file);
c = source_get();

```

◇
Fragment referenced in 60c.
Uses: fputs 21b, source_get 127c, version_string 27.

< Expand macro into file 61c> ≡

```

{
    Parameters local_parameters = 0;
    int changed;
    char indent_chars[MAX_INDENT];
    Arglist *a;
    Name *name;
    Arglist * args;
    a = collect_scrap_name(0);

```

◆
E

Uses: Arglist 179c, collect_scrap_name 180, defs 131b, FALSE 23a, instance 144b, MAX_INDENT 122d, Name 157c, Parameters 54a, source_get 127c, TRUE 23a, write_scrap 146b.

```

fprintf(tex_file, "\\verb%c\\\"%s\\\"%c\\nobreak\\ {\\footnotesize {", nw_char, name->spelling,
nw_char);
write_single_scrap_ref(tex_file, scraps);
fputs("}}$\\equiv$\n", tex_file);
< Fill in the middle of the scrap environment 65d >
< Begin the cross-reference environment 67e >
if ( scrap_flag ) {
    < Write file defs 68b >
}
format_defs_refs(tex_file, scraps);
format_uses_refs(tex_file, scraps++);
< Finish the cross-reference environment 68a >
< Finish the scrap environment 67a >
}

```

◇

Fragment referenced in 60c.

Uses: collect_file_name 173a, format_defs_refs 195b, format_uses_refs 193d, fprintf 21b, fputs 21b, Name 157c, nw_char 29ab, scraps 132a, scrap_flag 27, write_single_scrap_ref 133b.

I don't format a fragment name at all specially, figuring the programmer might want to use italics or bold face in the midst of the name.

< Write macro definition 63a > ≡

```

{
    Name *name = collect_macro_name();

    < Begin the scrap environment 64b >
    fputs("\\WEFTtarget{weft", tex_file);
    write_single_scrap_ref(tex_file, scraps);
    fputs("{} $\\langle\\,${\\itshape ", tex_file);
    < Write the macro's name 63b >
    fputs("\\nobreak\\ {\\footnotesize {", tex_file);
    write_single_scrap_ref(tex_file, scraps);
    fputs("}}$\\,\\rangle\\equiv$\n", tex_file);
    < Fill in the middle of the scrap environment 65d >
    < Begin the cross-reference environment 67e >
    < Write macro defs 68c >
    < Write macro refs 68d >
    format_defs_refs(tex_file, scraps);
    format_uses_refs(tex_file, scraps++);
    < Finish the cross-reference environment 68a >
    < Finish the scrap environment 67a >
}

```

◇

Fragment referenced in 60c.

Uses: collect_macro_name 176b, format_defs_refs 195b, format_uses_refs 193d, fputs 21b, Name 157c, scraps 132a, write_single_scrap_ref 133b.

< Write the macro's name 63b > ≡

```

{
    char * p = name->spelling;
    int i = 0;

    while (*p != '\\000') {
        if (*p == ARG_CHR) {
            write_arg(tex_file, name->arg[i++]);

```

```

        p++;
    }
    else
        fputc(*p++, tex_file);
}
}

```

◇
 Fragment referenced in [63a](#), [82d](#).
 Uses: ARG_CHR [178a](#), write_arg [64a](#).

"latex.c" 64a≡

```

static void write_arg(FILE * tex_file, char * p)
{
    fputs("\\hbox{\\slshape\\sffamily ", tex_file);
    while (*p)
    {
        switch (*p)
        {
            case '$':
            case '_':
            case '^':
            case '#':
                fputc('\\', tex_file);
                break;
            default:
                break;
        }
        fputc(*p, tex_file);
        p++;
    }

    fputs("\\/", tex_file);
}

```

◇
 File defined by [24b](#), [57b](#), [58](#), [64a](#), [69ab](#), [70ab](#), [71a](#), [72abc](#), [73](#), [74a](#), [79](#), [80b](#), [81b](#), [82a](#), [84bd](#).
 Defines: write_arg [57b](#), [63b](#), [75a](#), [79](#).
 Uses: FILE [21b](#), fputs [21b](#).

A scrap is opened with `\WEFTbegin` and closed with `\WEFTend` (both defined among the scrap-formatting macros above). Both big (`@0`, `@D`) and small (`@o`, `@d`) definitions use the same breakable block; the `inBlock` bookkeeping lets a run of small definitions share one open group, separated by a little vertical space.

⟨ *Begin the scrap environment* 64b ⟩ ≡

```

{
    if (big_definition)
    {
        if (inBlock)
        {
            ⟨ End block 65c ⟩
        }
        fputs("\\WEFTbegin", tex_file);
    }
    else
    {
        if (inBlock)
        {

```

```

        < Switch block 65b >
    }
    else
    {
        < Start block 65a >
    }
}
fprintf(tex_file, "\\label{scrap%d}\\n", scraps);
}

```

◇
 Fragment referenced in 62, 63a.
 Uses: `fprintf` 21b, `fputs` 21b, `scraps` 132a.

< Start block 65a > ≡

```

    fputs("\\WEFTbegin\\n", tex_file);
    inBlock = TRUE;

```

◇
 Fragment referenced in 53d, 64b.
 Uses: `fputs` 21b, `TRUE` 23a.

< Switch block 65b > ≡

```

    fputs("\\par\\vspace{\\baselineskip}\\n", tex_file);

```

◇
 Fragment referenced in 64b.
 Uses: `fputs` 21b.

< End block 65c > ≡

```

    fputs("\\WEFTend\\n", tex_file);
    inBlock = FALSE;

```

◇
 Fragment referenced in 53d, 64b, 67a.
 Uses: `FALSE` 23a, `fputs` 21b.

The body of a code scrap (`@{...@}`) is emitted as an `lstlisting`. We tag it with our fixed `escapeinside` pair and, when the scrap has a language listings understands, with a `language=` option so the code is highlighted. The `\end{lstlisting}` must start its own line, so we always emit a newline before it. The diamond (`\WEFTsep`) that marks the end of a scrap goes just after the listing. A non-code block scrap (the unusual `@d name @[...@]` or `@(...@)` forms) is emitted as before, without a listing.

< Fill in the middle of the scrap environment 65d > ≡

```

{
    int is_code = (source_last == '{');
    if (is_code) {
        const char *lstlang = latex_language(name->lang);
        /* the delimiter halves are split so this source line does not itself
           contain the escape sequence when weft typesets its own source */
        fputs("\\begin{lstlisting}[escapeinside={(*" "<{}>" "*)}]", tex_file);
        if (lstlang)
            fprintf(tex_file, ",language=%s", lstlang);
        fputs("\\n", tex_file);
    }
    extra_scraps = 0;
    copy_scrap(tex_file, TRUE, name);
}

```

```

if (is_code)
  /* the closing tag is split across two literals so that this very
     line does not end a listing when weft typesets its own source */
  fputs("\n\\end{ls" "tlisting}\n", tex_file);
  fputs("{\\WEFTsep}\n", tex_file);
}

```

◇

Fragment referenced in [62](#), [63a](#).

Uses: [copy_scrap 73](#), [extra_scraps 67b](#), [fprintf 21b](#), [fputs 21b](#), [latex_language 71a](#), [source_last 127c](#), [TRUE 23a](#).

We've got one last spacing command, controlling the amount of white space after a scrap.

Note also the whitespace eater. I use it to remove any blank lines that appear after a scrap in the source file. This way, text following a scrap will not be indented. Again, this is a matter of personal taste.

⟨ Finish the scrap environment 67a ⟩ ≡

```
{
  scraps += extra_scraps;
  if (big_definition)
    fputs("\\WEFTend\n", tex_file);
  else
  {
    ⟨ End block 65c ⟩
  }
  do
    c = source_get();
  while (isspace(c));
}
```

◇
Fragment referenced in 62, 63a.

Uses: `extra_scraps` 67b, `fputs` 21b, `isspace` 21b, `scraps` 132a, `source_get` 127c.

⟨ Global variable declarations 67b ⟩ ≡

```
extern int extra_scraps;
```

◇
Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.
Fragment referenced in 21a.

Defines: `extra_scraps` 65d, 67ac, 78b, 91c, 92a, 98b.

⟨ Global variable definitions 67c ⟩ ≡

```
int extra_scraps = 0;
```

◇
Fragment defined by 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b.
Fragment referenced in 25e.

Uses: `extra_scraps` 67b.

⟨ Write in-text scrap 67d ⟩ ≡

```
copy_scrap(tex_file, FALSE, NULL);
c = source_get();
```

◇
Fragment referenced in 60c.
Uses: `copy_scrap` 73, `FALSE` 23a, `source_get` 127c.

Formatting Cross References

⟨ Begin the cross-reference environment 67e ⟩ ≡

```
{
  fputs("\\vspace{-1.5ex}\n", tex_file);
  fputs("\\footnotesize\n", tex_file);
  fputs("\\begin{list}{\\setlength{\\itemsep}{-\\parsep}",
    tex_file);
  fputs("\\setlength{\\itemindent}{-\\leftmargin}}\n", tex_file);}
}
```

◇
Fragment referenced in 62, 63a.

Uses: [fputs 21b](#).

There may (unusually) be nothing to output in the cross-reference section, so output an empty list item anyway to avoid having an empty list.

⟨ Finish the cross-reference environment 68a ⟩ ≡

```
{
  fputs("\n\\item{}", tex_file);
  fputs("\n\\end{list}\n", tex_file);
}
```

◇
Fragment referenced in [62](#), [63a](#).

Uses: [fputs 21b](#).

⟨ Write file defs 68b ⟩ ≡

```
{
  if (name->defs) {
    if (name->defs->next) {
      fputs("\\item \\WEFTtxtFileDefBy\\ ", tex_file);
      print_scrap_numbers(tex_file, name->defs);
    }
  } else {
    fprintf(stderr,
      "would have crashed in 'Write file defs' for '%s'\n",
      name->spelling);
  }
}
```

◇
Fragment referenced in [62](#).

Uses: [defs 131b](#), [fprintf 21b](#), [fputs 21b](#), [next 131a](#), [print_scrap_numbers 69a](#), [stderr 21b](#).

⟨ Write macro defs 68c ⟩ ≡

```
{
  if (name->defs->next) {
    fputs("\\item \\WEFTtxtMacroDefBy\\ ", tex_file);
    print_scrap_numbers(tex_file, name->defs);
  }
}
```

◇
Fragment referenced in [63a](#).

Uses: [defs 131b](#), [fputs 21b](#), [next 131a](#), [print_scrap_numbers 69a](#).

⟨ Write macro refs 68d ⟩ ≡

```
{
  if (name->uses) {
    if (name->uses->next) {
      fputs("\\item \\WEFTtxtMacroRefIn\\ ", tex_file);
      print_scrap_numbers(tex_file, name->uses);
    }
  } else {
    fputs("\\item \\WEFTtxtMacroRefIn\\ ", tex_file);
    fputs("\\WEFTlink{weft", tex_file);
    write_single_scrap_ref(tex_file, name->uses->scrap);
  }
}
```

```

        fputs("}{", tex_file);
        write_single_scrap_ref(tex_file, name->uses->scrap);
        fputs("}", tex_file);
        fputs(".\n", tex_file);
    }
}
else {
    fputs("\\item {\WEFTtxtMacroNoRef}.\n", tex_file);
    fprintf(stderr, "%s: <%s> never referenced.\n",
            command_name, name->spelling);
}
}

```

◇ Fragment referenced in 63a.

Uses: `command_name` 29c, `fprintf` 21b, `fputs` 21b, `next` 131a, `print_scrap_numbers` 69a, `stderr` 21b, `uses` 131b, `write_single_scrap_ref` 133b.

"latex.c" 69a≡

```

static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps)
{
    int page;
    fputs("\\WEFTlink{weft", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, -1, &page);
    fputs("}{", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, TRUE, &page);
    fputs("}", tex_file);
    scraps = scraps->next;
    while (scraps) {
        fputs("\\WEFTlink{weft", tex_file);
        write_scrap_ref(tex_file, scraps->scrap, -1, &page);
        fputs("}{", tex_file);
        write_scrap_ref(tex_file, scraps->scrap, FALSE, &page);
        scraps = scraps->next;
        fputs("}", tex_file);
    }
    fputs(".\n", tex_file);
}

```

◇ File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.

Defines: `print_scrap_numbers` 57b, 68bcd, 80d, 83b.

Uses: `FALSE` 23a, `FILE` 21b, `fputs` 21b, `next` 131a, `page` 131b, `scraps` 132a, `Scrap_Node` 157b, `TRUE` 23a, `write_scrap_ref` 133a.

Formatting a Scrap

This is the only place we really care whether a scrap is delimited with `@{...@}`, `@[...@]`, or `@(...@)`, and we base our output sequences on that. The code (`@{...@}`) mode is now handled by `listings`, so its per-line strings are all empty: `copy_scrap` just copies the body out verbatim, one raw line per source line, and the surrounding `lstlisting` does the formatting. The `@[...@]` (verbatim `TEX`) and `@(...@)` (math) modes keep their old inline treatment.

We still keep the array `delimit_scrap`: it carries the small *begin/end* strings for the math mode (a pair of $\$$) and, for every mode, the single character that stands in for the weft character. Upon initialisation we copy the strings from `orig_delimit_scrap` into a writeable array; we might change the *insert nw_char* entry later, when the weft character changes.

"latex.c" 69b≡

```

static char *orig_delimit_scrap[3][5] = {
    /* {} mode (code): begin, end, insert nw_char, prefix, suffix.
       All empty: listings copies the body verbatim, one raw line per
       source line, so there is no per-line wrapper. */
    { "", "", "@", "", "" },
    /* [] mode: begin, end, insert nw_char, prefix, suffix */
    { "", "", "@", "", "" },
    /* () mode: begin, end, insert nw_char, prefix, suffix */
    { "$", "$", "@", "", "" },
};

static char *delimit_scrap[3][5];

```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Uses: `nw_char` 29ab.

The function `initialise_delimit_scrap_array` just copies the originals into a writeable array.

"`latex.c`" 70a≡

```

void initialise_delimit_scrap_array(void) {
    int i,j;
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 5; j++) {
            if((delimit_scrap[i][j] = strdup(orig_delimit_scrap[i][j])) == NULL) {
                fprintf(stderr, "Not enough memory for string allocation\n");
                exit(EXIT_FAILURE);
            }
        }
    }
}

```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Uses: `exit` 21b, `fprintf` 21b, `stderr` 21b.

Copying one code character to the `.tex` file. Because the code is no longer wrapped in `\verb`, every character that is special to L^AT_EX has to be turned into something harmless in `\ttfamily` text. A literal space becomes `\WEFTsp` (a breakable, fixed-width the code body itself needs none of this — `listings` copies it verbatim. `latex_code_putc` is used only for the little bits of text that are emitted *inside* an escape (a fragment reference's name and its substituted arguments), which land in ordinary L^AT_EX text mode and so must have the L^AT_EX-special characters neutralised.

"`latex.c`" 70b≡

```

static void latex_code_putc(FILE *file, int c)
{
    switch (c) {
        case '\\': fputs("\\textbackslash{}", file); break;
        case '{': fputs("\\{", file); break;
        case '}': fputs("\\}", file); break;
        case '$': fputs("\\$", file); break;
        case '&': fputs("\\&", file); break;
        case '#': fputs("\\#", file); break;
        case '%': fputs("\\%", file); break;
        case '_': fputs("\\_", file); break;
        case '^': fputs("\\textasciicircum{}", file); break;
        case '~': fputs("\\textasciitilde{}", file); break;
        default: putc(c, file); break;
    }
}

```

```

    }
}

```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: `latex_code_putc` 57b, 72b, 73, 78b.
Uses: FILE 21b, `fputs` 21b, `putc` 21b.

The language mapper turns a scrap's weft language tag ("c", "python", ...) into the exact name `listings` expects, or NULL when we have no safe mapping. Returning NULL (rather than guessing) matters: `listings` aborts the run on an unknown `language=`, so we only ever name languages it is known to ship.

"latex.c" 71a≡

```

static const char *latex_language(const char *lang)
{
    static const struct { const char *tag, *lst; } map[] = {
        {"c", "C"}, {"h", "C"},
        {"cpp", "C++"}, {"c++", "C++"}, {"cc", "C++"}, {"cxx", "C++"},
        {"hpp", "C++"}, {"hxx", "C++"},
        {"java", "Java"},
        {"python", "Python"}, {"py", "Python"},
        {"sql", "SQL"},
        {"html", "HTML"}, {"htm", "HTML"}, {"xml", "XML"},
        {"php", "PHP"},
        {"perl", "Perl"}, {"pl", "Perl"},
        {"ruby", "Ruby"}, {"rb", "Ruby"},
        {"pascal", "Pascal"}, {"pas", "Pascal"},
        {"lisp", "Lisp"}, {"el", "Lisp"}, {"scheme", "Lisp"},
        {"matlab", "Matlab"},
        {"tex", "[LaTeX]TeX"}, {"latex", "[LaTeX]TeX"},
        {"make", "make"}, {"makefile", "make"},
        {"awk", "Awk"},
        {"fortran", "Fortran"}, {"f", "Fortran"}, {"f90", "Fortran"},
        {"r", "R"}, {"lua", "Lua"}, {"ada", "Ada"},
        {"vhdl", "VHDL"}, {"verilog", "Verilog"},
        {"erlang", "erlang"}, {"cobol", "COBOL"}, {"eiffel", "Eiffel"},
        {"ocaml", "Caml"}, {"caml", "Caml"},
        {NULL, NULL}
    };
    int i;
    if (lang == NULL || lang[0] == '\0')
        return NULL;
    for (i = 0; map[i].tag; i++) {
        const char *a = lang, *b = map[i].tag;
        while (*a && *b && tolower((unsigned char)*a) == (unsigned char)*b) { a++; b++; }
        if (*a == '\0' && *b == '\0')
            return map[i].lst;
    }
    return NULL;
}

```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: `latex_language` 57b, 65d.

⟨ *Function prototypes* 71b ⟩ ≡

```

void initialise_delimit_scrap_array(void);

```

◇

Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
 Fragment referenced in 21a.

"latex.c" 72a≡

```
int scrap_type = 0;
```

◇

File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
 Defines: `scrap_type` 72c, 73, 74d, 75a, 77b.

"latex.c" 72b≡

```
static void write_literal(FILE * tex_file, char * p, int mode)
{
  fputs(delimit_scrap[mode][0], tex_file);
  while (*p!= '\000') {
    if (*p == nw_char)
      fputs(delimit_scrap[mode][2], tex_file);
    else if (mode == 0)
      latex_code_putc(tex_file, (unsigned char)*p);
    else
      fputc(*p, tex_file);
    p++;
  }
  fputs(delimit_scrap[mode][1], tex_file);
}
```

◇

File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
 Defines: `write_literal` 57b, 77b, 79.
 Uses: FILE 21b, `fputs` 21b, `latex_code_putc` 70b, `nw_char` 29ab.

Two little helpers bracket every place where we must step out of the verbatim body to run L^AT_EX (a fragment reference, a bold keyword, an italic title, a substituted parameter). In code mode this opens and closes a `listings` escape; in math mode it closes and reopens the math run; in the verbatim-T_EX (`@[...@]`) mode both are empty.

The escape delimiters are written with the two halves as separate string literals ("`(*`" "`<`"). The C compiler concatenates them, so the emitted text is exactly `(*<`; but the *source* never contains the delimiter as one contiguous token, which matters because weft typesets its own source — otherwise these very lines would trip the `listings` tokeniser when they appear in a scrap.

When `code_inline` is set we are already rendering inside an escape (a parameter of a fragment reference), so we must *not* open a nested escape — both helpers do nothing in code mode.

"latex.c" 72c≡

```
int code_inline = FALSE;

static void begin_tex_escape(FILE *file) /* leave the verbatim body */
{
  if (scrap_type == 0) { if (!code_inline) fputs("(" "<", file); }
  else fputs(delimit_scrap[scrap_type][1], file);
}
static void end_tex_escape(FILE *file) /* re-enter the verbatim body */
{
  if (scrap_type == 0) { if (!code_inline) fputs(">" ")", file); }
  else fputs(delimit_scrap[scrap_type][0], file);
}
```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: `begin_tex_escape` 52a, 75a, 76f, 77ab, `code_inline` 56b, 73, `end_tex_escape` 52a, 75a, 76f, 77ab.
Uses: `FALSE` 23a, `FILE` 21b, `fputs` 21b, `scrap_type` 72a, 73.

"latex.c" 73≡

```
static void copy_scrap(FILE *file, int prefix, Name *name)
{
    int indent = 0;
    int c;
    int coll_h1 = 0, coll_h2 = 0, coll_warned = FALSE;
    char ** params = name->arg;
    if (source_last == '{') scrap_type = 0;
    if (source_last == '[') scrap_type = 1;
    if (source_last == '(') scrap_type = 2;
    c = source_get();
    if (prefix) fputs(delimit_scrap[scrap_type][3], file);
    fputs(delimit_scrap[scrap_type][0], file);
    while (1) {
        switch (c) {
            case '\n': if (code_inline) {
                putc(' ', file); /* an inline parameter stays on one line */
                indent = 0;
                break;
            }
            fputs(delimit_scrap[scrap_type][1], file);
            if (prefix) fputs(delimit_scrap[scrap_type][4], file);
            fputs("\n", file);
            if (prefix) fputs(delimit_scrap[scrap_type][3], file);
            fputs(delimit_scrap[scrap_type][0], file);
            indent = 0;
            break;
            case '\t': { Expand tab into spaces 74c }
                break;
            default:
                if (c==nw_char)
                {
                    { Check at-sequence for end-of-scrap 75a }
                    break;
                }
                if (code_inline)
                    latex_code_putc(file, c); /* inside an escape: neutralise LaTeX */
                else {
                    { Note a delimiter collision 74d }
                    putc(c, file);
                }
                indent++;
                break;
        }
        c = source_get();
    }
}
```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: `copy_scrap` 56b, 57b, 65d, 67d, `scrap_type` 72ac, 74d, 75a, 77b.
Uses: `code_inline` 72c, `FALSE` 23a, `FILE` 21b, `fputs` 21b, `latex_code_putc` 70b, `Name` 157c, `nw_char` 29ab, `putc` 21b, `source_get` 127c, `source_last` 127c.

When we encounter the command to change the ‘weft character’ we call this function. It updates the scrap formatting directives accordingly.

"`latex.c`" 74a≡

```
void update_delimit_scrap(void)
{
    /* Every mode's "insert nw_char" string is the single character that
       stands in for the weft character inside a scrap; keep them in sync
       with the (possibly changed) weft character. */
    delimit_scrap[0][2][0] = nw_char;
    delimit_scrap[1][2][0] = nw_char;
    delimit_scrap[2][2][0] = nw_char;
}
```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: `update_delimit_scrap` 45d, 60c, 74b, 88b.
Uses: `nw_char` 29ab.

⟨ *Function prototypes* 74b ⟩ ≡

```
void update_delimit_scrap(void);
```

◇
Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
Fragment referenced in 21a.
Uses: `update_delimit_scrap` 74a.

⟨ *Expand tab into spaces* 74c ⟩ ≡

```
{
    int delta = 8 - (indent % 8);
    indent += delta;
    while (delta > 0) {
        putc(' ', file);
        delta--;
    }
}
```

◇
Fragment referenced in 73, 150c.
Uses: `putc` 21b.

Since the code body is copied verbatim, a scrap that literally contains our escape delimiter ((*< or >*)) would desynchronise listings’ tokeniser. This never happens in ordinary source, but we watch a two-character rolling window and warn once per scrap if it does, so a corrupted listing is never silent.

⟨ *Note a delimiter collision* 74d ⟩ ≡

```
if (scrap_type == 0) {
    if ((coll_h2 == '(' && coll_h1 == '*' && c == '<') ||
        (coll_h2 == '>' && coll_h1 == '*' && c == '>')) {
        if (!coll_warned) {
            fprintf(stderr,
                "%s: warning: a scrap body contains the listings escape "
                "delimiter; its typeset code may be corrupted.\n", command_name);
            coll_warned = TRUE;
        }
    }
    coll_h2 = coll_h1;
```

```

    coll_h1 = c;
}

```

◇

Fragment referenced in 73.

Uses: `command_name` 29c, `fprintf` 21b, `scrap_type` 72a, 73, `stderr` 21b, `TRUE` 23a.

⟨ *Check at-sequence for end-of-scrap 75a* ⟩ ≡

```

{
    c = source_get();
    switch (c) {
        case 'c': ⟨ Show presence of a block comment 52a ⟩
            break;
        case 'x': ⟨ Copy label from source into (75b file) 76b ⟩
            break;
        case 'v': ⟨ Copy version info into file 76a ⟩
        case 's':
            break;
        case '+':
        case '-':
        case '*':
        case '|': ⟨ Skip over index entries 76d ⟩
            FALLTHROUGH;
        case ',':
        case ')':
        case ']':
        case '': fputs(delimit_scrap[scrap_type][1], file);
            return;
        case '<': ⟨ Format macro name 77b ⟩
            break;
        case '%': ⟨ Skip commented-out code 76e ⟩
            break;
        case '_': ⟨ Bold Keyword 76f ⟩
            break;
        case 't': ⟨ Italic "fragment title" 77a ⟩
            break;
        case 'f': ⟨ Italic "file name" 77a ⟩
            break;
        case '1': case '2': case '3':
        case '4': case '5': case '6':
        case '7': case '8': case '9':
            if (name == NULL
                || name->arg[c - '1'] == NULL) {
                fputs(delimit_scrap[scrap_type][2], file);
                fputc(c, file);
            }
            else {
                begin_tex_escape(file);
                write_arg(file, name->arg[c - '1']);
                end_tex_escape(file);
            }
            break;
        default:
            if (c==nw_char)
            {
                fputs(delimit_scrap[scrap_type][2], file);
                break;
            }
    }
}

```

```

        /* ignore these since pass1 will have warned about them */
        break;
    }
}

```

◇
Fragment referenced in [73](#).

⟨ *Copy version info into file 76a* ⟩ ≡

```
fputs(version_string, file);
```

◇
Fragment referenced in [75a](#), [95c](#), [150d](#).
 Uses: `fputs` [21b](#), `version_string` [27](#).

⟨ *Copy label from source into 76b* ⟩ ≡

```
{
    ⟨ Get label from (76c source_get() ) 197a ⟩
    write_label(label_name, 01);
}
```

◇
Fragment referenced in [60c](#), [75a](#), [88b](#), [95c](#).
 Uses: `source_get` [127c](#).

There's no need to check for errors here, since we will have already pointed out any during the first pass.

⟨ *Skip over index entries 76d* ⟩ ≡

```
{
    do {
        do
            c = source_get();
            while (c != nw_char);
            c = source_get();
        } while (c != '}' && c != ']' && c != ')');
    }
```

◇
Fragment referenced in [75a](#), [95c](#).
 Uses: `nw_char` [29ab](#), `source_get` [127c](#).

⟨ *Skip commented-out code 76e* ⟩ ≡

```
{
    do
        c = source_get();
        while (c != '\n');
}
```

◇
Fragment referenced in [75a](#), [95c](#), [137b](#).
 Uses: `source_get` [127c](#).

This scrap helps deal with bold keywords:

⟨ *Bold Keyword 76f* ⟩ ≡

```

{
    begin_tex_escape(file);
    fprintf(file, "\\hbox{\\sffamily\\bfseries ");
    c = source_get();
    do {
        fputc(c, file);
        c = source_get();
    } while (c != nw_char);
    c = source_get();
    fprintf(file, "}");
    end_tex_escape(file);
}

```

◇

Fragment referenced in [75a](#).

Uses: [begin_tex_escape 72c](#), [end_tex_escape 72c](#), [fprintf 21b](#), [nw_char 29ab](#), [source_get 127c](#).

⟨ *Italic "whatever" 77a* ⟩ ≡

```

{
    begin_tex_escape(file);
    fprintf(file, "\\hbox{\\sffamily\\slshape whatever}");
    end_tex_escape(file);
}

```

◇

Fragment referenced in [75a](#).

Uses: [begin_tex_escape 72c](#), [end_tex_escape 72c](#), [fprintf 21b](#).

⟨ *Format macro name 77b* ⟩ ≡

```

{
    Arglist *args = collect_scrap_name(-1);
    Name *name = args->name;
    char *p = name->spelling;
    Arglist *q = args->args;
    int narg = 0;

    begin_tex_escape(file);
    if (prefix)
        /* \normalfont resets the listing's \ttfamily basicstyle, so the
           fragment name is set in the ordinary document font (roman italic)
           rather than as italic typewriter. */
        fputs("\\hbox{\\normalfont ", file);
    fputs("$\\langle\\,${\\itshape ", file);
    while (*p != '\\000') {
        if (*p == ARG_CHR) {
            if (q == NULL) {
                write_literal(file, name->arg[narg], scrap_type);
            }
            else {
                write_ArglistElement(file, q, params);
                q = q->next;
            }
            p++;
            narg++;
        }
        else
            fputc(*p++, file);
    }
}

```

```

fputs("}\\nobreak\\ ", file);
if (scrap_name_has_parameters) {
  < Format macro parameters 56b >
}
fprintf(file, "{\\footnotesize ");
if (name->defs)
  < Write abbreviated definition list 78a >
else {
  putc('?', file);
  fprintf(stderr, "%s: never defined <%s>\n",
           command_name, name->spelling);
}
fputs("}$\\,\\angle$", file);
if (prefix)
  fputs("}", file);
end_tex_escape(file);
}

```

◇

Fragment referenced in 75a.

Uses: Arglist 179c, ARG_CHR 178a, begin_tex_escape 72c, collect_scrap_name 180, command_name 29c, defs 131b, end_tex_escape 72c, fprintf 21b, fputs 21b, Name 157c, next 131a, putc 21b, scrap_type 72a, 73, stderr 21b, write_ArglistElement 79, write_literal 72b.

< Write abbreviated definition list 78a > ≡

```

{
  Scrap_Node *p = name->defs;
  fputs("\\WEFTlink{weft", file);
  write_single_scrap_ref(file, p->scrap);
  fputs("{", file);
  write_single_scrap_ref(file, p->scrap);
  fputs("}", file);
  p = p->next;
  if (p)
    fputs(", \\ldots\\ ", file);
}

```

◇

Fragment referenced in 77b, 79.

Uses: defs 131b, fputs 21b, next 131a, Scrap_Node 157b, write_single_scrap_ref 133b.

A quoted scrap (`@{...@}`) can appear as the argument of a parameterised fragment reference, e.g. `@<Name @<code@> eman@>`. In the old `\verb`-based output that code was simply set in a verbatim run in the middle of the reference. Under `listings` we cannot leave the escape here — the reference is being built inside an `\hbox`, and closing the escape mid-box would leave the braces straddling the escape boundary (a ! Extra } error). Instead we capture the scrap's expansion into a memory buffer and re-emit it *inside* the escape, as `LATEX`-escaped typewriter text on a single line, so the box stays balanced.

< Emit a quoted scrap argument inline 78b > ≡

```

{
  Scrap_Node * qq = (Scrap_Node *)q;
  char *buf = NULL;
  size_t sz = 0;
  FILE *mem = open_memstream(&buf, &sz);
  qq->quoted = TRUE;
  if (mem) {
    char *s;
    write_scrap(mem, "", qq, -1, "", 0, 0, 0, 0, 0, NULL, params, 0, "");
  }
}

```

```

fclose(mem);
fputs("\\hbox{\\ttfamily ", file);
for (s = buf; *s != '\\0'; s++) {
    if (*s == '\\n')
        putc(' ', file);
    else
        latex_code_putc(file, (unsigned char)*s);
}
fputs("}", file);
free(buf);
}
extra_scraps++;
qq->quoted = FALSE;
}

```

◇

Fragment referenced in 79.

Uses: extra_scraps 67b, FALSE 23a, fclose 21b, FILE 21b, fputs 21b, latex_code_putc 70b, putc 21b, Scrap_Node 157b, size_t 21b, TRUE 23a, write_scraps 146b.

"latex.c" 79≡

```

static void
write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        char * p = (char*)q;

        if (p[0] == ARG_CHR) {
            write_arg(file, params[p[1] - '1']);
        } else {
            write_literal(file, (char *)q, 0);
        }
    } else if (name == (Name *)1) {
        ⟨ Emit a quoted scrap argument inline 78b ⟩
    } else {
        char * p = name->spelling;
        fputs("$\\langle\\,${\\itshape ", file);
        while (*p != '\\000') {
            if (*p == ARG_CHR) {
                write_ArglistElement(file, q, params);
                q = q->next;
                p++;
            }
            else
                fputc(*p++, file);
        }
        fputs("\\nobreak\\ ", file);
        if (scrap_name_has_parameters) {
            int c;

            ⟨ Format macro parameters 56b ⟩
        }
        fprintf(file, "{\\footnotesize ");
        if (name->defs)
            ⟨ Write abbreviated definition list 78a ⟩
    }
}

```

```

    else {
        putc('?', file);
        fprintf(stderr, "%s: never defined <%s>\n",
            command_name, name->spelling);
    }
    fputs("}$\\,\\rangle$", file);
}
}

```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: write_ArglistElement 57b, 77b.
Uses: Arglist 179c, ARG_CHR 178a, command_name 29c, defs 131b, FILE 21b, fprintf 21b, fputs 21b, Name 157c, next 131a, putc 21b, stderr 21b, write_arg 64a, write_literal 72b.

2.4.2 Generating the Indices

⟨ Write index of file names 80a ⟩ ≡

```

{
    if (file_names) {
        fputs("\n{\small\\begin{list}{}{\setlength{\itemsep}{-\\parsep}",
            tex_file);
        fputs("\setlength{\itemindent}{-\\leftmargin}}\n", tex_file);
        format_file_entry(file_names, tex_file);
        fputs("\end{list}}", tex_file);
    }
    c = source_get();
}

```

◇
Fragment referenced in 60c.
Uses: file_names 158a, format_file_entry 80b, fputs 21b, source_get 127c.

"latex.c" 80b≡

```

static void format_file_entry(Name *name, FILE *tex_file)
{
    while (name) {
        format_file_entry(name->llink, tex_file);
        ⟨ Format a file index entry 80c ⟩
        name = name->rlink;
    }
}

```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: format_file_entry 57b, 80a.
Uses: FILE 21b, Name 157c.

⟨ Format a file index entry 80c ⟩ ≡

```

fputs("\\item ", tex_file);
fprintf(tex_file, "\\verb%c\\\"%s\\\"%c ", nw_char, name->spelling, nw_char);
⟨ Write file's defining scrap numbers 80d ⟩
putc('\n', tex_file);

```

◇
Fragment referenced in 80b.
Uses: fprintf 21b, fputs 21b, nw_char 29ab, putc 21b.

⟨ *Write file's defining scrap numbers 80d* ⟩ ≡

```
{
  Scrap_Node *p = name->defs;
  fputs("{\\footnotesize {\\WEFTtxtDefBy}", tex_file);
  if (p->next) {
    /* fputs("s ", tex_file); */
    putc(' ', tex_file);
    print_scrap_numbers(tex_file, p);
  }
  else {
    putc(' ', tex_file);
    fputs("\\WEFTlink{weft", tex_file);
    write_single_scrap_ref(tex_file, p->scrap);
    fputs("{}{", tex_file);
    write_single_scrap_ref(tex_file, p->scrap);
    fputs("}", tex_file);
    putc('.', tex_file);
  }
  putc('}', tex_file);
}
```

◇

Fragment referenced in 80c.

Uses: defs 131b, fputs 21b, next 131a, print_scrap_numbers 69a, putc 21b, Scrap_Node 157b, write_single_scrap_ref 133b.

⟨ *Write index of macro names 81a* ⟩ ≡

```
{
  unsigned char sector = current_sector;
  int c = source_get();
  if (c == '+')
    sector = 0;
  else
    source_ungetc(&c);
  if (has_sector(macro_names, sector)) {
    fputs("\\n{\\small\\begin{list}{}{\\setlength{\\itemsep}{-\\parsep}",
          tex_file);
    fputs("\\setlength{\\itemindent}{-\\leftmargin}}\\n", tex_file);
    format_entry(macro_names, tex_file, sector);
    fputs("\\end{list}}", tex_file);
  } else {
    fputs("None.\\n", tex_file);
  }
}
c = source_get();
```

◇

Fragment referenced in 60c.

Uses: current_sector 47b, format_entry 82a, fputs 21b, has_sector 84b, macro_names 158a, sector 131b, source_get 127c, source_ungetc 128b.

"latex.c" 81b≡

```
static int load_entry(Name * name, Name ** nms, int n)
{
  while (name) {
    n = load_entry(name->llink, nms, n);
    nms[n++] = name;
    name = name->rlink;
  }
}
```

```

    return n;
}

```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: `load_entry` 82a.
Uses: Name 157c.

"`latex.c`" 82a≡

```

static void format_entry(Name *name, FILE *tex_file, unsigned char sector)
{
    Name ** nms = malloc(num_scraps()*sizeof(Name *));
    int n = load_entry(name, nms, 0);
    int i;

    ⟨ Sort nms of size n for ⟨ Rob's ordering 82b ⟩ 82c ⟩
    for (i = 0; i < n; i++)
    {
        Name * name = nms[i];

        ⟨ Format an index entry 82d ⟩
    }
}

```

◇
File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.
Defines: `format_entry` 57b, 81a.
Uses: FILE 21b, `load_entry` 81b, `malloc` 21b, Name 157c, `num_scraps` 132a, `sector` 131b.

⟨ Rob's ordering 82b ⟩ ≡

```

    robs_strcmp(ki->spelling, kj->spelling) < 0

```

◇
Fragment referenced in 82a, 101b.
Uses: `robs_strcmp` 161b.

⟨ Sort key of size n for ordering 82c ⟩ ≡

```

int j;
for (j = 1; j < n; j++)
{
    int i = j - 1;
    Name * kj = key[j];

    do
    {
        Name * ki = key[i];

        if (ordering)
            break;
        key[i + 1] = ki;
        i -= 1;
    } while (i >= 0);
    key[i + 1] = kj;
}

```

◇
Fragment referenced in 82a, 101b.
Uses: Name 157c.

⟨ *Format an index entry 82d* ⟩ ≡

```

if (name->sector == sector){
    fputs("\\item ", tex_file);
    fputs("$\\langle\\,$", tex_file);
    ⟨ Write the macro's name 63b ⟩
    fputs("\\nobreak\\ {\\footnotesize ", tex_file);
    ⟨ Write defining scrap numbers 83a ⟩
    fputs("}$\\,\\rangle$", tex_file);
    ⟨ Write referencing scrap numbers 83b ⟩
    putc('\\n', tex_file);
}

```

◇

Fragment referenced in [82a](#).

Uses: [fputs 21b](#), [putc 21b](#), [sector 131b](#).

⟨ *Write defining scrap numbers 83a* ⟩ ≡

```

{
    Scrap_Node *p = name->defs;
    if (p) {
        int page;
        fputs("\\WEFTlink{weft", tex_file);
        write_scrap_ref(tex_file, p->scrap, -1, &page);
        fputs("{}{", tex_file);
        write_scrap_ref(tex_file, p->scrap, TRUE, &page);
        fputs("}", tex_file);
        p = p->next;
        while (p) {
            fputs("\\WEFTlink{weft", tex_file);
            write_scrap_ref(tex_file, p->scrap, -1, &page);
            fputs("{}{", tex_file);
            write_scrap_ref(tex_file, p->scrap, FALSE, &page);
            fputs("}", tex_file);
            p = p->next;
        }
    }
    else
        putc('?', tex_file);
}

```

◇

Fragment referenced in [82d](#).

Uses: [defs 131b](#), [FALSE 23a](#), [fputs 21b](#), [next 131a](#), [page 131b](#), [putc 21b](#), [Scrap_Node 157b](#), [TRUE 23a](#), [write_scrap_ref 133a](#).

⟨ *Write referencing scrap numbers 83b* ⟩ ≡

```

{
    Scrap_Node *p = name->uses;
    fputs("{\\footnotesize ", tex_file);
    if (p) {
        fputs("{\\WEFTtxtRefIn}", tex_file);
        if (p->next) {
            /* fputs("s ", tex_file); */
            putc(' ', tex_file);
            print_scrap_numbers(tex_file, p);
        }
        else {
            putc(' ', tex_file);
        }
    }
}

```

```

        fputs("\\WEFTlink{weft", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("{}{", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("}", tex_file);
        putc('.', tex_file);
    }
}
else
    fputs("{\\WEFTtxtNoRef}.", tex_file);
    putc('}', tex_file);
}

```

◇ Fragment referenced in 82d.

Uses: fputs 21b, next 131a, print_scrap_numbers 69a, putc 21b, Scrap_Node 157b, uses 131b, write_single_scrap_ref 133b.

⟨ *Function prototypes* 84a ⟩ ≡

```
extern int has_sector(Name *, unsigned char);
```

◇ Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: has_sector 84b, Name 157c.

"latex.c" 84b ≡

```

int has_sector(Name * name, unsigned char sector)
{
    while(name) {
        if (name->sector == sector)
            return TRUE;
        if (has_sector(name->llink, sector))
            return TRUE;
        name = name->rlink;
    }
    return FALSE;
}

```

◇ File defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.

Defines: has_sector 81a, 84ac, 100d, 103a.

Uses: FALSE 23a, Name 157c, sector 131b, TRUE 23a.

⟨ *Write index of user-specified names* 84c ⟩ ≡

```

{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    if (has_sector(user_names, sector)) {
        fputs("\n{\\small\\begin{list}{\\setlength{\\itemsep}{-\\parsep}",
            tex_file);
        fputs("\\setlength{\\itemindent}{-\\leftmargin}\\n", tex_file);
        format_user_entry(user_names, tex_file, sector);
        fputs("\\end{list}}", tex_file);
    }
}

```

```

    }
}

```

◇

Fragment referenced in 60c.

Uses: [current_sector 47b](#), [format_user_entry 84d](#), [fputs 21b](#), [has_sector 84b](#), [sector 131b](#), [source_get 127c](#), [user_names 158a](#).

"latex.c" 84d≡

```

static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector)
{
    while (name) {
        format_user_entry(name->llink, tex_file, sector);
        < Format a user index entry 85 >
        name = name->rlink;
    }
}

```

◇

File defined by [24b](#), [57b](#), [58](#), [64a](#), [69ab](#), [70ab](#), [71a](#), [72abc](#), [73](#), [74a](#), [79](#), [80b](#), [81b](#), [82a](#), [84bd](#).

Defines: [format_user_entry 57b](#), [84c](#).

Uses: [FILE 21b](#), [Name 157c](#), [sector 131b](#).

< Format a user index entry 85 > ≡

```

if (name->sector == sector){
    Scrap_Node *uses = name->uses;
    if ( uses || dangling_flag ) {
        int page;
        Scrap_Node *defs = name->defs;
        fprintf(tex_file, "\\item \\verb%c%s%c: ", nw_char,name->spelling,nw_char);
        if (!uses) {
            fputs("\\underline{", tex_file);
            fputs("\\WEFTlink{weft", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("{", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("})", tex_file);
            page = -2;
            defs = defs->next;
        }
        else
            if (!defs || uses->scrap < defs->scrap) {
                fputs("\\WEFTlink{weft", tex_file);
                write_scrap_ref(tex_file, uses->scrap, -1, &page);
                fputs("{", tex_file);
                write_scrap_ref(tex_file, uses->scrap, TRUE, &page);
                fputs("}", tex_file);
                uses = uses->next;
            }
        else {
            if (defs->scrap == uses->scrap)
                uses = uses->next;
            fputs("\\underline{", tex_file);

            fputs("\\WEFTlink{weft", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("{", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);

```

```

    fputs("}", tex_file);
    page = -2;
    defs = defs->next;
}
while (uses || defs) {
    if (uses && (!defs || uses->scrap < defs->scrap)) {
        fputs("\\WEFTlink{weft", tex_file);
        write_scrap_ref(tex_file, uses->scrap, -1, &page);
        fputs("{", tex_file);
        write_scrap_ref(tex_file, uses->scrap, FALSE, &page);
        fputs("}", tex_file);
        uses = uses->next;
    }
    else {
        if (uses && defs->scrap == uses->scrap)
            uses = uses->next;
        fputs(" ", tex_file);

        fputs("\\WEFTlink{weft", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("{", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("}", tex_file);

        putc('}', tex_file);
        page = -2;
        defs = defs->next;
    }
}
fputs(".\n", tex_file);
}
}

```

◇ Fragment referenced in 84d.

Uses: `dangling_flag` 27, `defs` 131b, `FALSE` 23a, `fprintf` 21b, `fputs` 21b, `next` 131a, `nw_char` 29ab, `page` 131b, `putc` 21b, `Scrap_Node` 157b, `sector` 131b, `TRUE` 23a, `uses` 131b, `write_scrap_ref` 133a, `write_single_scrap_ref` 133b.

2.5 Writing the Markdown File

This section implements the Markdown weave output, activated by the `-w md` command-line flag. It is structurally parallel to the L^AT_EX output in Section 2.4, but emits Markdown with embedded HTML for scraps (using `<pre><code>` blocks so that fragment references inside code can be clickable links).

⟨ *Function prototypes* 86a ⟩ ≡

```
extern void write_md(char *file_name, char *md_name, unsigned char sector);
```

◇ Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: `file_name` 131b, `sector` 131b, `write_md` 87a.

We need a few local function declarations before we get into the body of `write_md`.

"markdown.c" 86b ≡

```

static void md_copy_scrap(FILE *file, int prefix, Name *name);
static void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps);
static void md_format_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_format_file_entry(Name *name, FILE *md_file);
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_write_arg(FILE *md_file, char *p);
static void md_write_literal(FILE *md_file, char *p);
static void md_write_ArglistElement(FILE *file, Arglist *args, char **params);
static void md_format_uses_refs(FILE *md_file, int scrap);
static void md_format_defs_refs(FILE *md_file, int scrap);

```

◇

File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

Uses: Arglist 179c, FILE 21b, md_copy_scrap 94b, md_format_defs_refs 106a, md_format_entry 101b, md_format_file_entry 100a, md_format_user_entry 103b, md_format_uses_refs 104, md_print_scrap_numbers 94a, md_write_arg 91b, md_write_ArglistElement 98b, md_write_literal 98a, Name 157c, scraps 132a, Scrap_Node 157b, sector 131b.

The routine `write_md` takes two file names as parameters: the name of the web source file and the name of the .md output file.

"markdown.c" 87a≡

```

void write_md(char *file_name, char *md_name, unsigned char sector)
{
    (void)sector;
    FILE *md_file = fopen(md_name, "w");
    if (md_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", md_name);
        source_open(file_name);
        < Copy source_file into md_file 87b >
        fclose(md_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, md_name);
}

```

◇

File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

Defines: `write_md` 40b, 86a.

Uses: command_name 29c, fclose 21b, FILE 21b, file_name 131b, fopen 21b, fprintf 21b, sector 131b, source_open 130c, stderr 21b, verbose_flag 27.

We make our second pass through the source web, copying characters straight into the .md file. We watch for @ characters which signal a command sequence. We also detect and discard cosmetic fence lines (lines that are exactly ““ optionally followed by a language tag).

< Copy source_file into md_file 87b > ≡

```

{
    int c = source_get();
    int at_line_start = TRUE;
    while (c != EOF) {
        if (c == nw_char)
        {
            < Interpret md at-sequence 88b >
        }
        else if (c == ' ' && at_line_start) {
            < Check for cosmetic fence line 88a >

```

```

    }
    else {
        if (c == '\n') at_line_start = TRUE;
        else at_line_start = FALSE;
        putc(c, md_file);
        c = source_get();
    }
}
}

```

◇ Fragment referenced in 87a.
 Uses: FALSE 23a, nw_char 29ab, putc 21b, source_get 127c, TRUE 23a.

Cosmetic fence lines are lines that start with ‘‘’. We peek ahead to see if the line matches; if so, we skip to end of line. Otherwise we output what we already consumed and continue.

⟨ Check for cosmetic fence line 88a ⟩ ≡

```

{
    int c2 = source_get();
    int c3 = (c2 == '') ? source_get() : 0;
    if (c2 == '' && c3 == '') {
        /* Skip rest of fence line */
        int fc = source_get();
        while (fc != '\n' && fc != EOF)
            fc = source_get();
        c = source_get();
        at_line_start = TRUE;
    } else {
        /* Not a fence, output what we consumed */
        putc('', md_file);
        if (c2 == '') {
            putc('', md_file);
            c = c3;
        } else {
            c = c2;
        }
        at_line_start = FALSE;
    }
}

```

◇ Fragment referenced in 87b.
 Uses: FALSE 23a, putc 21b, source_get 127c, TRUE 23a.

⟨ Interpret md at-sequence 88b ⟩ ≡

```

{
    int big_definition = FALSE;
    c = source_get();
    switch (c) {
        case 'r':
            c = source_get();
            nw_char = c;
            update_delimit_scrap();
            break;
        case '0': big_definition = TRUE;
            FALLTHROUGH;
        case 'o': ⟨ Write md output file definition 90a ⟩
    }
}

```

```

        break;
    case 'Q':
    case 'D': big_definition = TRUE;
              FALLTHROUGH;
    case 'q':
    case 'd': < Write md macro definition 90b >
              break;
    case 's':
              < Step to next sector 46 >
              break;
    case 'S':
              < Close the current sector 47a >
              break;
    case '{':
    case '[':
    case '(': < Write md in-text scrap 92b >
              break;
    case '<': < Expand md macro into md_file 89c >
              break;
    case 'x': < Copy label from source into (89a md_file ) 76b >
              c = source_get();
              break;
    case 'c': /* Block comments: no special handling in Markdown */
              c = source_get();
              break;
    case 'f': < Write md index of file names 99 >
              break;
    case 'm': < Write md index of macro names 100d >
              break;
    case 'u': < Write md index of user-specified names 103a >
              break;
    case 'v': < Copy version info into md file 89b >
              break;
    case 'l':
    case 'L': < Skip at-L language definition line 45c >
              break;
    case 'W': < Skip at-L language definition line 45c >
              break;
    default:
        if (c==nw_char)
            putc(c, md_file);
        c = source_get();
        break;
    }
}

```

◇
Fragment referenced in [87b](#).

< Copy version info into md file 89b > ≡

```

    fputs(version_string, md_file);
    c = source_get();

```

◇
Fragment referenced in [88b](#).
Uses: `fputs` [21b](#), `source_get` [127c](#), `version_string` [27](#).

< Expand md macro into file 89c > ≡

```

{
    Parameters local_parameters = 0;
    int changed;
    char indent_chars[MAX_INDENT];
    Arglist *a;
    Name *name;
    Arglist * args;
    a = collect_scrap_name(0);
    name = a->name;
    args = instance(a->args, NULL, NULL, &changed);
    name->mark = TRUE;
    write_scraps(file, md_name, name->defs, 0, indent_chars, 0, 0, 1, 0, 0,
        args, name->arg, local_parameters, md_name);
    name->mark = FALSE;
    c = source_get();
}

```

◇

Fragment referenced in [88b](#).

Uses: Arglist [179c](#), collect_scrap_name [180](#), defs [131b](#), FALSE [23a](#), instance [144b](#), MAX_INDENT [122d](#), Name [157c](#), Parameters [54a](#), source_get [127c](#), TRUE [23a](#), write_scraps [146b](#).

2.5.1 Formatting Definitions in Markdown

File and fragment definitions use HTML anchors and formatting. A file definition header looks like:

```

<a id="weft1"></a>
**"filename"*** **1** =
<pre><code>...code...</code></pre>

```

A fragment definition header looks like:

```

<a id="weft1"></a>
**&lt;*Fragment Name* 1&gt;*** =
<pre><code>...code...</code></pre>

```

⟨ Write md output file definition 90a ⟩ ≡

```

{
    Name *name = collect_file_name();
    (void)big_definition;
    fputs("\n<a id=\"weft\", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("></a>\n", md_file);
    fprintf(md_file, "**\"%s\"***", name->spelling);
    fputs(" **", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("** =\n", md_file);
    ⟨ Fill in the md scrap 91c ⟩
    ⟨ Write md file cross-refs 92c ⟩
    ⟨ Finish md scrap 92a ⟩
}

```

◇

Fragment referenced in [88b](#).

Uses: collect_file_name [173a](#), fprintf [21b](#), fputs [21b](#), Name [157c](#), scraps [132a](#), write_single_scrap_ref [133b](#).

⟨ Write md macro definition 90b ⟩ ≡

```

{
    Name *name = collect_macro_name();
    (void)big_definition;
    fputs("\n<a id=\"weft\"", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("\n"></a>\n", md_file);
    fputs("**&lt;*", md_file);
    < Write the md macro's name 91a >
    fputs("* ", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("&gt;** =\n", md_file);
    < Fill in the md scrap 91c >
    < Write md macro cross-refs 93a >
    < Finish md scrap 92a >
}

```

◇

Fragment referenced in 88b.

Uses: collect_macro_name 176b, fputs 21b, Name 157c, scraps 132a, write_single_scrap_ref 133b.

< Write the md macro's name 91a > ≡

```

{
    char * p = name->spelling;
    int i = 0;

    while (*p != '\000') {
        if (*p == ARG_CHR) {
            md_write_arg(md_file, name->arg[i++]);
            p++;
        }
        else
            fputc(*p++, md_file);
    }
}

```

◇

Fragment referenced in 90b, 101c.

Uses: ARG_CHR 178a, md_write_arg 91b.

"markdown.c" 91b≡

```

static void md_write_arg(FILE * md_file, char * p)
{
    fputs("<i>", md_file);
    while (*p)
    {
        switch (*p)
        {
            case '<':
                fputs("&lt;", md_file);
                break;
            case '>':
                fputs("&gt;", md_file);
                break;
            case '&':
                fputs("&amp;", md_file);
                break;
            default:
                fputc(*p, md_file);
        }
    }
}

```

```

        break;
    }
    p++;
}
fputs("</i>", md_file);
}

```

◇
 File defined by [24c](#), [86b](#), [87a](#), [91b](#), [94ab](#), [98ab](#), [100a](#), [101ab](#), [103b](#), [104](#), [106a](#).
 Defines: `md_write_arg` [86b](#), [91a](#), [95c](#), [98b](#).
 Uses: FILE [21b](#), `fputs` [21b](#).

⟨ *Fill in the md scrap 91c* ⟩ ≡

```

{
    if (name->lang[0] != '\0')
        fprintf(md_file, "<pre><code class=\"%s\">", name->lang);
    else
        fputs("<pre><code>", md_file);
    extra_scraps = 0;
    md_copy_scrap(md_file, TRUE, name);
    fputs("</code></pre>\n", md_file);
}

```

◇
 Fragment referenced in [90ab](#).
 Uses: `extra_scraps` [67b](#), `fprintf` [21b](#), `fputs` [21b](#), `md_copy_scrap` [94b](#), `TRUE` [23a](#).

⟨ *Finish md scrap 92a* ⟩ ≡

```

{
    scraps += extra_scraps;
    do
        c = source_get();
        while (isspace(c));
        at_line_start = TRUE;
    }
}

```

◇
 Fragment referenced in [90ab](#).
 Uses: `extra_scraps` [67b](#), `isspace` [21b](#), `scraps` [132a](#), `source_get` [127c](#), `TRUE` [23a](#).

⟨ *Write md in-text scrap 92b* ⟩ ≡

```

md_copy_scrap(md_file, FALSE, NULL);
c = source_get();

```

◇
 Fragment referenced in [88b](#).
 Uses: `FALSE` [23a](#), `md_copy_scrap` [94b](#), `source_get` [127c](#).

Cross References in Markdown

Cross-references appear as blockquotes after each scrap.

⟨ *Write md file cross-refs 92c* ⟩ ≡

```

{
    if ( scrap_flag ) {
        ⟨ Write md file defs 92d ⟩
    }
    md_format_defs_refs(md_file, scraps);
}

```

```

    md_format_uses_refs(md_file, scraps++);
}

```

◇ Fragment referenced in 90a.

Uses: md_format_defs_refs 106a, md_format_uses_refs 104, scraps 132a, scrap_flag 27.

⟨ Write md file defs 92d ⟩ ≡

```

{
    if (name->defs) {
        if (name->defs->next) {
            fputs("> *File defined by* ", md_file);
            md_print_scrap_numbers(md_file, name->defs);
        }
    } else {
        fprintf(stderr,
            "would have crashed in 'Write md file defs' for '%s'\n",
            name->spelling);
    }
}

```

◇ Fragment referenced in 92c.

Uses: defs 131b, fprintf 21b, fputs 21b, md_print_scrap_numbers 94a, next 131a, stderr 21b.

⟨ Write md macro cross-refs 93a ⟩ ≡

```

{
    ⟨ Write md macro defs 93b ⟩
    ⟨ Write md macro refs 93c ⟩
    md_format_defs_refs(md_file, scraps);
    md_format_uses_refs(md_file, scraps++);
}

```

◇ Fragment referenced in 90b.

Uses: md_format_defs_refs 106a, md_format_uses_refs 104, scraps 132a.

⟨ Write md macro defs 93b ⟩ ≡

```

{
    if (name->defs->next) {
        fputs("> *Fragment defined by* ", md_file);
        md_print_scrap_numbers(md_file, name->defs);
    }
}

```

◇ Fragment referenced in 93a.

Uses: defs 131b, fputs 21b, md_print_scrap_numbers 94a, next 131a.

⟨ Write md macro refs 93c ⟩ ≡

```

{
    if (name->uses) {
        if (name->uses->next) {
            fputs("> *Fragment referenced in* ", md_file);
            md_print_scrap_numbers(md_file, name->uses);
        }
    } else {
        fputs("> *Fragment referenced in* ", md_file);
    }
}

```

```

        fputs("<a href=\"#weft\", md_file);
        write_single_scrap_ref(md_file, name->uses->scrap);
        fputs(">", md_file);
        write_single_scrap_ref(md_file, name->uses->scrap);
        fputs("</a>", md_file);
        fputs(".\n", md_file);
    }
}
else {
    fputs("> *Fragment never referenced*.\n", md_file);
    fprintf(stderr, "%s: <%s> never referenced.\n",
            command_name, name->spelling);
}
}

```

◇ Fragment referenced in 93a.

Uses: command_name 29c, fprintf 21b, fputs 21b, md_print_scrap_numbers 94a, next 131a, stderr 21b, uses 131b, write_single_scrap_ref 133b.

"markdown.c" 94a≡

```

static void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps)
{
    int page;
    fputs("<a href=\"#weft\", md_file);
    write_scrap_ref(md_file, scraps->scrap, -1, &page);
    fputs(">", md_file);
    write_scrap_ref(md_file, scraps->scrap, TRUE, &page);
    fputs("</a>", md_file);
    scraps = scraps->next;
    while (scraps) {
        fputs("<a href=\"#weft\", md_file);
        write_scrap_ref(md_file, scraps->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, scraps->scrap, FALSE, &page);
        fputs("</a>", md_file);
        scraps = scraps->next;
    }
    fputs(".\n", md_file);
}

```

◇ File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

Defines: md_print_scrap_numbers 86b, 92d, 93bc, 100c, 102b.

Uses: FALSE 23a, FILE 21b, fputs 21b, next 131a, page 131b, scraps 132a, Scrap_Node 157b, TRUE 23a, write_scrap_ref 133a.

Formatting a Scrap in Markdown

Scraps are output inside `<pre><code>` blocks. Characters `<`, `>`, and `&` are HTML-escaped. At-sequences within scraps (macro references, keywords, parameters, etc.) emit HTML markup directly.

"markdown.c" 94b≡

```

static int md_scrap_type = 0;

static void md_copy_scrap(FILE *file, int prefix, Name *name)
{
    int indent = 0;
    int c;

```

```

char ** params = name ? name->arg : NULL;
if (source_last == '{') md_scrap_type = 0;
if (source_last == '[') md_scrap_type = 1;
if (source_last == '(') md_scrap_type = 2;
c = source_get();
while (1) {
    switch (c) {
        case '\n': fputs("\n", file);
                    indent = 0;
                    break;
        case '\t': < Expand md tab into spaces 95b >
                    break;
        default:
            if (c==nw_char)
            {
                < Check md at-sequence for end-of-scrap 95c >
                break;
            }
            if (md_scrap_type == 0 && prefix) {
                < HTML-escape character c 95a >
            } else {
                putc(c, file);
            }
            indent++;
            break;
    }
    c = source_get();
}
}

```

◇ File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.
 Defines: md_copy_scrap 86b, 91c, 92b, md_scrap_type 95c.
 Uses: FILE 21b, fputs 21b, Name 157c, nw_char 29ab, putc 21b, source_get 127c, source_last 127c.

< HTML-escape character c 95a > ≡

```

switch (c) {
    case '<': fputs("&lt;", file); break;
    case '>': fputs("&gt;", file); break;
    case '&': fputs("&amp;", file); break;
    default: putc(c, file); break;
}

```

◇ Fragment referenced in 94b.
 Uses: fputs 21b, putc 21b.

< Expand md tab into spaces 95b > ≡

```

{
    int delta = 8 - (indent % 8);
    indent += delta;
    while (delta > 0) {
        putc(' ', file);
        delta--;
    }
}

```

◇ Fragment referenced in 94b.

Uses: `putc` [21b](#).

⟨ Check md at-sequence for end-of-scrap 95c ⟩ $\equiv$

```
{
  c = source_get();
  switch (c) {
    case 'c': ⟨ Show md presence of a block comment 96a ⟩
      break;
    case 'x': ⟨ Copy label from source into (95d file ) 76b ⟩
      break;
    case 'v': ⟨ Copy version info into file 76a ⟩
    case 's':
      break;
    case '+':
    case '-':
    case '*':
    case '|': ⟨ Skip over index entries 76d ⟩
    case ',':
    case ')':
    case ']':
    case '}': return;
    case '<': ⟨ Format md macro name 97b ⟩
      break;
    case '%': ⟨ Skip commented-out code 76e ⟩
      break;
    case '_': ⟨ Bold md Keyword 96b ⟩
      break;
    case 't': ⟨ Italic md "fragment title" 97a ⟩
      break;
    case 'f': ⟨ Italic md "file name" 97a ⟩
      break;
    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9':
      if (name == NULL
          || name->arg[c - '1'] == NULL) {
        if (md_scrap_type == 0 && prefix)
          fputs("&", file);
        else
          putc(nw_char, file);
        fputc(c, file);
      }
      else {
        md_write_arg(file, name->arg[c - '1']);
      }
      break;
    default:
      if (c==nw_char)
      {
        if (md_scrap_type == 0 && prefix)
          fputs("&", file);
        else
          putc(nw_char, file);
        break;
      }
      /* ignore these since pass1 will have warned about them */
      break;
  }
}
```

```

    }
}

```

◇ Fragment referenced in [94b](#).

⟨ *Show md presence of a block comment* 96a ⟩ ≡

```

{
    fputs("<i>(Comment)</i>", file);
}

```

◇ Fragment referenced in [95c](#).

Uses: [fputs](#) [21b](#).

⟨ *Bold md Keyword* 96b ⟩ ≡

```

{
    fprintf(file, "<b>");
    c = source_get();
    do {
        if (c == '<') fputs("&lt;", file);
        else if (c == '>') fputs("&gt;", file);
        else if (c == '&') fputs("&amp;", file);
        else fputc(c, file);
        c = source_get();
    } while (c != nw_char);
    c = source_get();
    fprintf(file, "</b>");
}

```

◇ Fragment referenced in [95c](#).

Uses: [fprintf](#) [21b](#), [fputs](#) [21b](#), [nw_char](#) [29ab](#), [source_get](#) [127c](#).

⟨ *Italic md "whatever"* 97a ⟩ ≡

```

{
    fprintf(file, "<i>whatever</i>");
}

```

◇ Fragment referenced in [95c](#).

Uses: [fprintf](#) [21b](#).

⟨ *Format md macro name* 97b ⟩ ≡

```

{
    Arglist *args = collect_scrap_name(-1);
    Name *name = args->name;
    char *p = name->spelling;
    Arglist *q = args->args;
    int narg = 0;

    fputs("&lt;<a href=\"\#weft", file);
    if (name->defs) {
        write_single_scrap_ref(file, name->defs->scrap);
    } else {
        putc('?', file);
    }
    fputs("><i>", file);
}

```

```

while (*p != '\000') {
    if (*p == ARG_CHR) {
        if (q == NULL) {
            md_write_literal(file, name->arg[narg]);
        }
        else {
            md_write_ArglistElement(file, q, params);
            q = q->next;
        }
        p++;
        narg++;
    }
    else {
        if (*p == '<') fputs("<", file);
        else if (*p == '>') fputs(">", file);
        else if (*p == '&') fputs("&", file);
        else fputc(*p, file);
        p++;
    }
}
fputs("</i> ", file);
if (name->defs) {
    Scrap_Node *dp = name->defs;
    write_single_scrap_ref(file, dp->scrap);
    dp = dp->next;
    if (dp)
        fputs(", ...", file);
}
else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\n",
            command_name, name->spelling);
}
fputs("</a>&gt;", file);
}

```

◇ Fragment referenced in 95c.

Uses: Arglist 179c, ARG_CHR 178a, collect_scrap_name 180, command_name 29c, defs 131b, fprintf 21b, fputs 21b, md_write_ArglistElement 98b, md_write_literal 98a, Name 157c, next 131a, putc 21b, Scrap_Node 157b, stderr 21b, write_single_scrap_ref 133b.

"markdown.c" 98a≡

```

static void md_write_literal(FILE * md_file, char * p)
{
    while (*p!= '\000') {
        if (*p == '<') fputs("<", md_file);
        else if (*p == '>') fputs(">", md_file);
        else if (*p == '&') fputs("&", md_file);
        else fputc(*p, md_file);
        p++;
    }
}

```

◇ File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.
 Defines: md_write_literal 86b, 97b, 98b.
 Uses: FILE 21b, fputs 21b.

"markdown.c" 98b≡

```
static void
md_write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        char * p = (char*)q;

        if (p[0] == ARG_CHR) {
            md_write_arg(file, params[p[1] - '1']);
        } else {
            md_write_literal(file, (char *)q);
        }
    } else if (name == (Name *)1) {
        Scrap_Node * qq = (Scrap_Node *)q;
        qq->quoted = TRUE;
        write_scraps(file, "", qq,
                     -1, "", 0, 0, 0, 0, 0,
                     NULL, params, 0, "");
        extra_scraps++;
        qq->quoted = FALSE;
    } else {
        char * p = name->spelling;
        fputs("<i>", file);
        while (*p != '\000') {
            if (*p == ARG_CHR) {
                md_write_ArglistElement(file, q, params);
                q = q->next;
                p++;
            }
            else {
                if (*p == '<') fputs("<", file);
                else if (*p == '>') fputs(">", file);
                else if (*p == '&') fputs("&", file);
                else fputc(*p, file);
                p++;
            }
        }
        fputs("</i> ", file);
        if (name->defs) {
            Scrap_Node *dp = name->defs;
            write_single_scrap_ref(file, dp->scrap);
            dp = dp->next;
            if (dp)
                fputs(", ...", file);
        }
        else {
            putc('?', file);
            fprintf(stderr, "%s: never defined <%s>\n",
                    command_name, name->spelling);
        }
        fputs(">", file);
    }
}
```

◇
File defined by [24c](#), [86b](#), [87a](#), [91b](#), [94ab](#), [98ab](#), [100a](#), [101ab](#), [103b](#), [104](#), [106a](#).

Defines: `md_write_ArglistElement` 86b, 97b.

Uses: `Arglist` 179c, `ARG_CHR` 178a, `command_name` 29c, `defs` 131b, `extra_scraps` 67b, `FALSE` 23a, `FILE` 21b, `fprintf` 21b, `fputs` 21b, `md_write_arg` 91b, `md_write_literal` 98a, `Name` 157c, `next` 131a, `putc` 21b, `Scrap_Node` 157b, `stderr` 21b, `TRUE` 23a, `write_scraps` 146b, `write_single_scrap_ref` 133b.

2.5.2 Generating the Indices in Markdown

< Write md index of file names 99 > ≡

```
{
  if (file_names) {
    fputs("\n", md_file);
    md_format_file_entry(file_names, md_file);
    fputs("\n", md_file);
  }
  c = source_get();
}
```

◇
Fragment referenced in 88b.

Uses: `file_names` 158a, `fputs` 21b, `md_format_file_entry` 100a, `source_get` 127c.

"markdown.c" 100a≡

```
static void md_format_file_entry(Name *name, FILE *md_file)
{
  while (name) {
    md_format_file_entry(name->llink, md_file);
    < Format a md file index entry 100b >
    name = name->rlink;
  }
}
```

◇
File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

Defines: `md_format_file_entry` 86b, 99.

Uses: `FILE` 21b, `Name` 157c.

< Format a md file index entry 100b > ≡

```
fputs("- ", md_file);
fprintf(md_file, "'\"%s\"' ", name->spelling);
< Write md file's defining scrap numbers 100c >
putc('\n', md_file);
```

◇
Fragment referenced in 100a.

Uses: `fprintf` 21b, `fputs` 21b, `putc` 21b.

< Write md file's defining scrap numbers 100c > ≡

```
{
  Scrap_Node *p = name->defs;
  fputs("*Defined by* ", md_file);
  if (p->next) {
    md_print_scrap_numbers(md_file, p);
  }
  else {
    fputs("<a href=\"#weft\"", md_file);
    write_single_scrap_ref(md_file, p->scrap);
    fputs(">", md_file);
  }
}
```

```

        write_single_scrap_ref(md_file, p->scrap);
        fputs("</a>.", md_file);
    }
}

```

◇ Fragment referenced in 100b.

Uses: defs 131b, fputs 21b, md_print_scrap_numbers 94a, next 131a, Scrap_Node 157b, write_single_scrap_ref 133b.

⟨ Write md index of macro names 100d ⟩ ≡

```

{
    unsigned char sector = current_sector;
    int c = source_get();
    if (c == '+')
        sector = 0;
    else
        source_ungetc(&c);
    if (has_sector(macro_names, sector)) {
        fputs("\n", md_file);
        md_format_entry(macro_names, md_file, sector);
        fputs("\n", md_file);
    } else {
        fputs("None.\n", md_file);
    }
}
c = source_get();

```

◇ Fragment referenced in 88b.

Uses: current_sector 47b, fputs 21b, has_sector 84b, macro_names 158a, md_format_entry 101b, sector 131b, source_get 127c, source_ungetc 128b.

"markdown.c" 101a≡

```

static int md_load_entry(Name * name, Name ** nms, int n)
{
    while (name) {
        n = md_load_entry(name->llink, nms, n);
        nms[n++] = name;
        name = name->rlink;
    }
    return n;
}

```

◇ File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

Defines: md_load_entry 101b.

Uses: Name 157c.

"markdown.c" 101b≡

```

static void md_format_entry(Name *name, FILE *md_file, unsigned char sector)
{
    Name ** nms = malloc(num_scrap()*sizeof(Name *));
    int n = md_load_entry(name, nms, 0);
    int i;

    ⟨ Sort nms of size n for ⟨ Rob's ordering 82b ⟩ 82c ⟩
    for (i = 0; i < n; i++)
    {

```

```

    Name * name = nms[i];

    ⟨ Format a md index entry 101c ⟩
}
free(nms);
}

```

◇

File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

Defines: md_format_entry 86b, 100d.

Uses: FILE 21b, malloc 21b, md_load_entry 101a, Name 157c, num_scrap 132a, sector 131b.

⟨ Format a md index entry 101c ⟩ ≡

```

if (name->sector == sector){
    fputs("- ", md_file);
    fputs("<*", md_file);
    ⟨ Write the md macro's name 91a ⟩
    fputs("* ", md_file);
    ⟨ Write md defining scrap numbers 102a ⟩
    fputs(">", md_file);
    ⟨ Write md referencing scrap numbers 102b ⟩
    putc('\n', md_file);
}

```

◇

Fragment referenced in 101b.

Uses: fputs 21b, putc 21b, sector 131b.

⟨ Write md defining scrap numbers 102a ⟩ ≡

```

{
    Scrap_Node *p = name->defs;
    if (p) {
        int page;
        fputs("<a href=\"#weft\", md_file);
        write_scrap_ref(md_file, p->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, p->scrap, TRUE, &page);
        fputs("</a>", md_file);
        p = p->next;
        while (p) {
            fputs("<a href=\"#weft\", md_file);
            write_scrap_ref(md_file, p->scrap, -1, &page);
            fputs(">", md_file);
            write_scrap_ref(md_file, p->scrap, FALSE, &page);
            fputs("</a>", md_file);
            p = p->next;
        }
    }
    else
        putc('?', md_file);
}

```

◇

Fragment referenced in 101c.

Uses: defs 131b, FALSE 23a, fputs 21b, next 131a, page 131b, putc 21b, Scrap_Node 157b, TRUE 23a, write_scrap_ref 133a.

⟨ Write md referencing scrap numbers 102b ⟩ ≡

```

{
    Scrap_Node *p = name->uses;
    if (p) {
        fputs("*Referenced in* ", md_file);
        if (p->next) {
            md_print_scrap_numbers(md_file, p);
        }
        else {
            fputs("<a href=\"#weft\", md_file);
            write_single_scrap_ref(md_file, p->scrap);
            fputs(">", md_file);
            write_single_scrap_ref(md_file, p->scrap);
            fputs("</a>.", md_file);
        }
    }
    else
        fputs("*Not referenced*.", md_file);
}

```

◇

Fragment referenced in 101c.

Uses: fputs 21b, md_print_scrap_numbers 94a, next 131a, Scrap_Node 157b, uses 131b, write_single_scrap_ref 133b.

⟨ Write md index of user-specified names 103a ⟩ ≡

```

{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    if (has_sector(user_names, sector)) {
        fputs("\n", md_file);
        md_format_user_entry(user_names, md_file, sector);
        fputs("\n", md_file);
    }
}

```

◇

Fragment referenced in 88b.

Uses: current_sector 47b, fputs 21b, has_sector 84b, md_format_user_entry 103b, sector 131b, source_get 127c, user_names 158a.

"markdown.c" 103b≡

```

static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector)
{
    while (name) {
        md_format_user_entry(name->llink, md_file, sector);
        ⟨ Format a md user index entry 103c ⟩
        name = name->rlink;
    }
}

```

◇

File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

Defines: md_format_user_entry 86b, 103a.

Uses: FILE 21b, Name 157c, sector 131b.

⟨ Format a md user index entry 103c ⟩ ≡

```

if (name->sector == sector){
    Scrap_Node *uses = name->uses;
    if ( uses || dangling_flag ) {
        int page;
        Scrap_Node *defs = name->defs;
        fprintf(md_file, "- '%s': ", name->spelling);
        if (!uses) {
            fputs("(**", md_file);
            fputs("<a href=\"#weft\", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs(">", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs("</a>(**", md_file);
            page = -2;
            defs = defs->next;
        }
        else
            if (!defs || uses->scrap < defs->scrap) {
                fputs("<a href=\"#weft\", md_file);
                write_scrap_ref(md_file, uses->scrap, -1, &page);
                fputs(">", md_file);
                write_scrap_ref(md_file, uses->scrap, TRUE, &page);
                fputs("</a>", md_file);
                uses = uses->next;
            }
        else {
            if (defs->scrap == uses->scrap)
                uses = uses->next;
            fputs("(**", md_file);

            fputs("<a href=\"#weft\", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs(">", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs("</a>(**", md_file);
            page = -2;
            defs = defs->next;
        }
    }
    while (uses || defs) {
        if (uses && (!defs || uses->scrap < defs->scrap)) {
            fputs("<a href=\"#weft\", md_file);
            write_scrap_ref(md_file, uses->scrap, -1, &page);
            fputs(">", md_file);
            write_scrap_ref(md_file, uses->scrap, FALSE, &page);
            fputs("</a>", md_file);
            uses = uses->next;
        }
        else {
            if (uses && defs->scrap == uses->scrap)
                uses = uses->next;
            fputs(", (**", md_file);

            fputs("<a href=\"#weft\", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs(">", md_file);
            write_single_scrap_ref(md_file, defs->scrap);
            fputs("</a>(**", md_file);
            page = -2;

```

```

        defs = defs->next;
    }
}
fputs(".\n", md_file);
}
}

```

◇

Fragment referenced in 103b.

Uses: `dangling_flag` 27, `defs` 131b, `FALSE` 23a, `fprintf` 21b, `fputs` 21b, `next` 131a, `page` 131b, `Scrap_Node` 157b, `sector` 131b, `TRUE` 23a, `uses` 131b, `write_scrap_ref` 133a, `write_single_scrap_ref` 133b.

2.5.3 Identifier Cross-References in Markdown

These functions are the Markdown equivalents of `format_uses_refs` and `format_defs_refs` from `scraps.c`, but emit Markdown blockquote format.

"markdown.c" 104≡

```

static void
md_format_uses_refs(FILE * md_file, int scrap)
{
    Uses * p = get_scrap_uses(scrap);
    if (p != NULL)
        ⟨ Write md uses references 105a ⟩
}

```

◇

File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

Defines: `md_format_uses_refs` 86b, 92c, 93a.

Uses: `FILE` 21b, `get_scrap_uses` 133c, `Uses` 193b.

⟨ Write md uses references 105a ⟩ ≡

```

{
    char join = ' ';
    fputs("> *Uses:* ", md_file);
    do {
        ⟨ Write one md use reference 105b ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".\n", md_file);
}

```

◇

Fragment referenced in 104.

Uses: `fputs` 21b, `next` 131a, `Uses` 193b.

⟨ Write one md use reference 105b ⟩ ≡

```

Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c '%s' ",
        join, name->spelling);
if (defs)
{
    do {

```

```

    < Write one md referenced scrap 105c >
    first = FALSE;
    defs = defs->next;
}while (defs!= NULL);
}
else
{
    fputs("(not defined globally)", md_file);
}

```

◇ Fragment referenced in 105a.
 Uses: defs 131b, FALSE 23a, first 200a, fprintf 21b, fputs 21b, Name 157c, next 131a, page 131b, Scrap_Node 157b, TRUE 23a.

```

< Write one md referenced scrap 105c > ≡

fputs("<a href=\"#weft\", md_file);
write_scrap_ref(md_file, defs->scrap, -1, &page);
fputs(\">\", md_file);
write_scrap_ref(md_file, defs->scrap, first, &page);
fputs("</a>\", md_file);

```

◇ Fragment referenced in 105b, 106c.
 Uses: defs 131b, first 200a, fputs 21b, page 131b, write_scrap_ref 133a.

"markdown.c" 106a≡

```

static void
md_format_defs_refs(FILE * md_file, int scrap)
{
    Uses * p = get_scrap_defs(scrap);
    if (p != NULL)
        < Write md defs references 106b >
}

```

◇ File defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.
 Defines: md_format_defs_refs 86b, 92c, 93a.
 Uses: FILE 21b, get_scrap_defs 134a, Uses 193b.

```

< Write md defs references 106b > ≡

{
    char join = ' ';
    fputs("> *Defines:", md_file);
    do {
        < Write one md def reference 106c >
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".\n", md_file);
}

```

◇ Fragment referenced in 106a.
 Uses: fputs 21b, next 131a.

< Write one md def reference 106c > ≡

```

Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c '%s' ",
        join, name->spelling);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
    fputs("*Never used*", md_file);
}
else {
    do {
        if (defs->scrap != scrap) {
            { Write one md referenced scrap 105c }
            first = FALSE;
        }
        defs = defs->next;
    }while (defs!= NULL);
}

```

◇

Fragment referenced in 106b.

Uses: defs 131b, FALSE 23a, first 200a, fprintf 21b, fputs 21b, Name 157c, next 131a, page 131b, Scrap_Node 157b, TRUE 23a, uses 131b.

2.6 Map and Extract Output

Two modes support AI-assisted navigation of weft projects:

- m (map) emits a JSON object to stdout describing every output file and every fragment, their definition locations, usage relationships, and transitive dependencies.
- e *name* (extract) emits the scrap code and dependency information for a single fragment (matched by prefix) as Markdown to stdout.

Both modes run after `pass1` has built the name trees and do *not* invoke the weave or tangle phases.

2.6.1 JSON Map Output

The `write_map` function performs an in-order traversal of the `file_names` and `macro_names` binary search trees, emitting a JSON object with two arrays: "files" and "fragments".

For each `Name` node, it records:

- The spelling (name of the file or fragment).
- The sector number.
- The language tag (if any).
- Each definition scrap with its source file and line.
- Each usage scrap (scraps that reference this name).
- For definition scraps, the list of fragments referenced inside (via `get_scrap_uses`).

A helper function escapes strings for JSON output.

"output.c" 107≡

```

static void json_puts(FILE *f, const char *s)
{
    fputc('\"', f);
    while (*s) {
        switch (*s) {
            case '\"': fputs("\\\"", f); break;
            case '\\': fputs("\\\\", f); break;
            case '\n': fputs("\\n", f); break;
            case '\r': fputs("\\r", f); break;
            case '\t': fputs("\\t", f); break;
            default:
                if ((unsigned char)*s < 0x20)
                    fprintf(f, "\\u%04x", (unsigned char)*s);
                else
                    fputc(*s, f);
        }
        s++;
    }
    fputc('\"', f);
}

```

◇
File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).
Defines: [json_puts](#) [108](#), [117b](#).
Uses: FILE [21b](#), [fprintf](#) [21b](#), [fputs](#) [21b](#).

The `emit_name_json` helper emits a single name entry as a JSON object. The `is_file` flag distinguishes file names (which use `"uses_fragments"` for fragments referenced inside their definition scraps) from macro fragments (which use `"used_by"` for the scraps that invoke them, and `"uses"` for fragments they invoke).

"output.c" 108≡

```

static void emit_name_json(FILE *f, Name *name, int is_file, int *first)
{
    Scrap_Node *sn;
    int first_inner;

    if (!name) return;
    emit_name_json(f, name->llink, is_file, first);

    if (!*first) fputs("\n", f);
    *first = 0;

    fputs("    {\n", f);
    fputs("        \"name\": ", f); json_puts(f, name->spelling);
    if (name->lang[0]) {
        fputs("\n        \"lang\": ", f); json_puts(f, name->lang);
    }
    fprintf(f, "\n        \"sector\": %d", name->sector);

    /* defs array */
    fputs("\n        \"defs\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
        if (!first_inner) fputs(" ", f);
        first_inner = 0;
        fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
        json_puts(f, scrap_file_name(sn->scrap));
        fprintf(f, ", \"line\": %d, \"end_line\": %d",

```

```

        scrap_file_line(sn->scrap), scrap_end_line(sn->scrap));
if (bodies_flag) {
    fputs(", \"body\": ", f);
    dump_scrap_json(f, sn->scrap);
}
if (prose_flag && scrap_prose(sn->scrap)) {
    fputs(", \"prose\": ", f);
    json_puts(f, scrap_prose(sn->scrap));
}
fputc('}', f);
}
fputs("]", f);

if (is_file) {
    /* uses_fragments: fragments referenced inside this file's scraps */
    fputs(",\n    \"uses_fragments\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
        Uses *u;
        for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
            if (!first_inner) fputs(", ", f);
            first_inner = 0;
            json_puts(f, u->defn->spelling);
        }
    }
    fputs("]", f);
} else {
    /* used_by: scraps that invoke this fragment */
    fputs(",\n    \"used_by\": [", f);
    first_inner = 1;
    for (sn = name->uses; sn; sn = sn->next) {
        if (!first_inner) fputs(", ", f);
        first_inner = 0;
        fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
        json_puts(f, scrap_file_name(sn->scrap));
        fprintf(f, ", \"line\": %d}", scrap_file_line(sn->scrap));
    }
    fputs("]", f);

    /* uses: fragments referenced inside this fragment's def scraps */
    fputs(",\n    \"uses\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
        Uses *u;
        for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
            if (!first_inner) fputs(", ", f);
            first_inner = 0;
            json_puts(f, u->defn->spelling);
        }
    }
    fputs("]", f);
}

fputs("\n    }", f);

emit_name_json(f, name->rlink, is_file, first);
}

```

◇

File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.

Defines: emit_name_json 109.

Uses: bodies_flag 27, defs 131b, dump_scrap_json 143a, end_line 131b, FILE 21b, first 200a, fprintf 21b, fputs 21b, get_scrap_uses 133c, json_puts 107, Name 157c, next 131a, prose 131b, prose_flag 27, scraps 132a, scrap_end_line 134e, scrap_file_line 134d, scrap_file_name 134c, Scrap_Node 157b, scrap_prose 135a, sector 131b, Uses 193b, uses 131b.

"output.c" 109≡

```
void write_map(Name *files, Name *macros)
{
    int first;

    fputs("{\n  \"files\": [\n", stdout);
    first = 1;
    emit_name_json(stdout, files, 1, &first);
    fputs("\n ],\n  \"fragments\": [\n", stdout);
    first = 1;
    emit_name_json(stdout, macros, 0, &first);
    fputs("\n ]\n}\n", stdout);
}
```

◇ File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.

Defines: write_map 40b, 122b.

Uses: emit_name_json 108, first 200a, fputs 21b, Name 157c.

2.6.2 Fragment Extraction

The `write_extract` function finds a fragment by name (using prefix matching, like `@<name...@>` in the source), then emits the scrap code and dependency information as Markdown. It recurses into referenced fragments to provide the full transitive closure of the requested fragment.

To find a name in the BST without creating a new entry, we use a read-only search function:

"output.c" 110a≡

```
static Name *find_name(Name *node, const char *spelling)
{
    Name *found;
    if (!node) return NULL;
    if (robs_strcmp(node->spelling, (char *)spelling) == 0)
        return node;
    found = find_name(node->llink, spelling);
    if (found) return found;
    return find_name(node->rlink, spelling);
}
```

◇ File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.

Defines: find_name 112b, 113a.

Uses: Name 157c, robs_strcmp 161b.

A recursive helper emits one fragment and all its transitive dependencies. A visited array (indexed by Name pointer cast to a simple mark) prevents infinite recursion on cyclic references.

"output.c" 110b≡

```
static void extract_fragment(FILE *f, Name *name, Name *files, Name *macros)
{
    Scrap_Node *sn;
```

```

if (!name || name->mark) return;
name->mark = TRUE;

for (sn = name->defs; sn; sn = sn->next) {
    fprintf(f, "\n## \"%s\" --- %s:%d\n\n",
            name->spelling,
            scrap_file_name(sn->scrap),
            scrap_file_line(sn->scrap));
    if (name->lang[0])
        fprintf(f, "'\"%s\n", name->lang);
    else
        fputs("'\"'\n", f);
    dump_scrap_text(f, sn->scrap);
    fputs("\n'\"'\n", f);

    /* Show dependencies */
    {
        Uses *u = get_scrap_uses(sn->scrap);
        if (u) {
            fputs("\n### Uses\n", f);
            for (; u; u = u->next)
                fprintf(f, "- %s\n", u->defn->spelling);
        }
    }

    /* Show used_by */
    if (name->uses) {
        Scrap_Node *usn;
        fputs("\n### Used by\n", f);
        for (usn = name->uses; usn; usn = usn->next) {
            fprintf(f, "- scrap %d (%s:%d)\n",
                    usn->scrap,
                    scrap_file_name(usn->scrap),
                    scrap_file_line(usn->scrap));
        }
    }

    /* Recurse into dependencies */
    for (sn = name->defs; sn; sn = sn->next) {
        Uses *u;
        for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
            extract_fragment(f, u->defn, files, macros);
        }
    }
}

```

◇ File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).

Defines: [extract_fragment 113a](#).

Uses: [defs 131b](#), [dump_scrap_text 141d](#), [FILE 21b](#), [fprintf 21b](#), [fputs 21b](#), [get_scrap_uses 133c](#), [Name 157c](#), [next 131a](#), [scrap_file_line 134d](#), [scrap_file_name 134c](#), [Scrap_Node 157b](#), [TRUE 23a](#), [Uses 193b](#), [uses 131b](#).

Clear marks after extraction so the name trees remain usable:

"output.c" 111a≡

```
static void clear_marks(Name *node)
```

```

{
    if (!node) return;
    clear_marks(node->llink);
    node->mark = FALSE;
    clear_marks(node->rlink);
}

```

◇
File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.
Defines: clear_marks 112b, 113a.
Uses: FALSE 23a, Name 157c.

2.6.3 Callers (Upward Call Chain)

The `--callers` feature, used with `-e`, traces the upward call chain from a given fragment back to its `@o` file roots. Each scrap records its owning Name (set in `pass1` when the scrap is attached to a definition list), so we can walk upward through the `uses` list to find all callers.

"output.c" 111b≡

```

static int is_file_name(Name *root, Name *target)
{
    if (!root) return 0;
    if (root == target) return 1;
    if (is_file_name(root->llink, target)) return 1;
    return is_file_name(root->rlink, target);
}

```

◇
File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.
Defines: is_file_name 112a.
Uses: Name 157c, root 185a.

"output.c" 112a≡

```

static void print_callers(Name *name, Name *file_names, int depth)
{
    Scrap_Node *usn;
    int i;

    if (!name || name->mark) return;
    name->mark = TRUE;

    for (i = 0; i < depth; i++) fputs(" ", stdout);
    if (is_file_name(file_names, name))
        printf("[file] %s\n", name->spelling);
    else
        printf("%s\n", name->spelling);

    for (usn = name->uses; usn; usn = usn->next) {
        Name *owner = scrap_owner(usn->scrap);
        if (owner && !owner->mark)
            print_callers(owner, file_names, depth + 1);
    }
}

```

◇
File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.
Defines: print_callers 112b.
Uses: file_names 158a, fputs 21b, is_file_name 111b, Name 157c, next 131a, owner 131b, Scrap_Node 157b, scrap_owner 135b, TRUE 23a, uses 131b.

"output.c" 112b≡

```
void write_callers(char *name, Name *files, Name *macros)
{
    Name *found = find_name(macros, name);
    if (!found)
        found = find_name(files, name);
    if (!found) {
        fprintf(stderr, "%s: fragment \"%s\" not found\n", command_name, name);
        exit(1);
    }
    printf("Callers of \"%s\":\n\n", found->spelling);
    printf("%s\n", found->spelling);
    {
        Scrap_Node *usn;
        for (usn = found->uses; usn; usn = usn->next) {
            Name *owner = scrap_owner(usn->scrap);
            if (owner)
                print_callers(owner, files, 1);
        }
    }
    clear_marks(files);
    clear_marks(macros);
}
```

◇ File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).

Defines: [write_callers](#) [40b](#), [122b](#).

Uses: [clear_marks](#) [111a](#), [command_name](#) [29c](#), [exit](#) [21b](#), [find_name](#) [110a](#), [fprintf](#) [21b](#), [Name](#) [157c](#), [next](#) [131a](#), [owner](#) [131b](#), [print_callers](#) [112a](#), [Scrap_Node](#) [157b](#), [scrap_owner](#) [135b](#), [stderr](#) [21b](#), [uses](#) [131b](#).

"output.c" 113a≡

```
void write_extract(char *name, Name *files, Name *macros)
{
    Name *found = NULL;

    /* Search in macro_names first, then file_names */
    found = find_name(macros, name);
    if (!found)
        found = find_name(files, name);

    if (!found) {
        fprintf(stderr, "%s: fragment \"%s\" not found\n", command_name, name);
        exit(1);
    }

    fprintf(stdout, "# Extract: \"%s\"\n", found->spelling);
    extract_fragment(stdout, found, files, macros);

    /* Clean up marks */
    clear_marks(files);
    clear_marks(macros);
}
```

◇ File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).

Defines: [write_extract](#) [40b](#), [122b](#).

Uses: [clear_marks](#) [111a](#), [command_name](#) [29c](#), [exit](#) [21b](#), [extract_fragment](#) [110b](#), [file_names](#) [158a](#), [find_name](#) [110a](#), [first](#) [200a](#), [fprintf](#) [21b](#), [macro_names](#) [158a](#), [Name](#) [157c](#), [stderr](#) [21b](#).

2.6.4 Lint (Static Analysis)

The `--lint` flag performs a post-pass1 static analysis of the fragment namespace. It reports three kinds of issues:

1. **Undefined references:** fragments that are used (`@<Name@>`) but never defined (`@d Name`).
2. **Unused fragments:** fragments defined but never referenced by any other scrap or output file.
3. **Fuzzy-match suggestions:** for undefined references, the closest defined name within Levenshtein distance 3.

"output.c" 113b≡

```
static int levenshtein(const char *a, const char *b)
{
    int la = (int)strlen(a), lb = (int)strlen(b);
    int prev[256], curr[256];
    int i, j;
    if (la > 255 || lb > 255) return 999;
    for (j = 0; j <= lb; j++) prev[j] = j;
    for (i = 1; i <= la; i++) {
        curr[0] = i;
        for (j = 1; j <= lb; j++) {
            int cost = (a[i-1] != b[j-1]) ? 1 : 0;
            int del = prev[j] + 1;
            int ins = curr[j-1] + 1;
            int sub = prev[j-1] + cost;
            curr[j] = del < ins ? (del < sub ? del : sub) : (ins < sub ? ins : sub);
        }
        for (j = 0; j <= lb; j++) prev[j] = curr[j];
    }
    return prev[lb];
}
```

◇
File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).
Defines: `levenshtein` [114a](#).
Uses: `strlen` [21b](#).

"output.c" 114a≡

```
#define MAX_LINT_NAMES 4096
static Name *lint_names[MAX_LINT_NAMES];
static int lint_name_count;

static void collect_lint_names(Name *node)
{
    if (!node) return;
    collect_lint_names(node->llink);
    if (node->defs && lint_name_count < MAX_LINT_NAMES)
        lint_names[lint_name_count++] = node;
    collect_lint_names(node->rlink);
}

static const char *find_closest(const char *target)
{

```

```

int i, best_dist = 4;
const char *best = NULL;
for (i = 0; i < lint_name_count; i++) {
    int d = levenshtein(target, lint_names[i]->spelling);
    if (d < best_dist) {
        best_dist = d;
        best = lint_names[i]->spelling;
    }
}
return best;
}

```

◇

File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).

Defines: [collect_lint_names](#) [115](#), [find_closest](#) [114b](#), [lint_names](#) Never used, [lint_name_count](#) [115](#), [MAX_LINT_NAMES](#) Never used.

Uses: [defs](#) [131b](#), [levenshtein](#) [113b](#), [Name](#) [157c](#).

"output.c" [114b](#)≡

```

static int lint_name(Name *node, int check_unused, int *issues)
{
    if (!node) return 0;
    lint_name(node->llink, check_unused, issues);

    if (node->defs == NULL && node->uses != NULL) {
        /* undefined reference */
        const char *suggestion = find_closest(node->spelling);
        fprintf(stderr, "lint: undefined fragment \"%s\"", node->spelling);
        if (node->uses) {
            fprintf(stderr, " (used at %s:%d)",
                    scrap_file_name(node->uses->scrap),
                    scrap_file_line(node->uses->scrap));
        }
        if (suggestion)
            fprintf(stderr, " -- did you mean \"%s\"?", suggestion);
        fputc('\n', stderr);
        (*issues)++;
    }
    else if (node->defs != NULL && node->uses == NULL && check_unused) {
        fprintf(stderr, "lint: unused fragment \"%s\" (defined at %s:%d)\n",
                node->spelling,
                scrap_file_name(node->defs->scrap),
                scrap_file_line(node->defs->scrap));
        (*issues)++;
    }

    lint_name(node->rlink, check_unused, issues);
    return 0;
}

```

◇

File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).

Defines: [lint_name](#) [115](#).

Uses: [defs](#) [131b](#), [find_closest](#) [114a](#), [fprintf](#) [21b](#), [Name](#) [157c](#), [scrap_file_line](#) [134d](#), [scrap_file_name](#) [134c](#), [stderr](#) [21b](#), [uses](#) [131b](#).

"output.c" [115](#)≡

```

void write_lint(Name *file_names, Name *macro_names)
{

```

```

int issues = 0;
lint_name_count = 0;
collect_lint_names(file_names);
collect_lint_names(macro_names);

/* check files for undefined refs (not unused -- files are roots) */
lint_name(file_names, 0, &issues);
/* check macros for both undefined and unused */
lint_name(macro_names, 1, &issues);

if (issues)
    fprintf(stderr, "lint: %d issue(s) found\n", issues);
else
    fprintf(stderr, "lint: no issues found\n");
}

```

◇

File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.

Defines: `write_lint` 40b, 122b.

Uses: `collect_lint_names` 114a, `file_names` 158a, `fprintf` 21b, `lint_name` 114b, `lint_name_count` 114a, `macro_names` 158a, `Name` 157c, `stderr` 21b.

2.6.5 Reverse Map Output

The reverse map feature provides programmatic navigation from tangled output back to the `.weft` source. Given a tangled output file (and optionally a line number), it parses the section markers embedded in that file and emits JSON mapping each region to its originating scrap, source file, and source line.

This operates entirely on the tangled output—no `.weft` files are needed and `pass1` is never invoked. The section markers have the form:

```

COMMENT_BEGIN{N: file:line}COMMENT_END      (opening)
COMMENT_BEGIN{:N}COMMENT_END                 (closing)

```

where `COMMENT_BEGIN` and `COMMENT_END` vary by language (`/* ... */`, `// ...`, `# ...`, etc.).

A `MarkerFrame` records the data extracted from one opening marker. We maintain a stack of these to handle nested fragment expansions.

"output.c" 116a≡

```

typedef struct {
    int scrap;
    char file[FILENAME_MAX];
    int line;
    int start_line; /* line in the tangled file where this marker opened */
} MarkerFrame;

```

◇

File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.

Defines: `MarkerFrame` 116b, 117b, 120, 121.

The `parse_open_marker` function scans a line for the pattern `{N: file:line}`. It returns 1 on success, filling in the supplied `MarkerFrame`. The scan is intentionally loose about what precedes the opening brace so that any comment style works.

"output.c" 116b≡

```

static int parse_open_marker(const char *line, MarkerFrame *mf)
{
    const char *p = line;

```

```

while (*p) {
    if (*p == '{') {
        const char *start = p + 1;
        /* scrap number */
        if (*start < '0' || *start > '9') { p++; continue; }
        int scrap = 0;
        while (*start >= '0' && *start <= '9')
            scrap = scrap * 10 + (*start++ - '0');
        if (*start != ':' || *(start+1) != ' ') { p++; continue; }
        start += 2; /* skip ": " */
        /* file:line -- find the closing brace, then work backwards for :line */
        const char *end = strchr(start, '}');
        if (!end) { p++; continue; }
        /* find last ':' between start and end */
        const char *colon = NULL;
        for (const char *q = end - 1; q >= start; q--) {
            if (*q == ':') { colon = q; break; }
        }
        if (!colon) { p++; continue; }
        /* digits after colon? */
        int src_line = 0;
        const char *d = colon + 1;
        if (d == end) { p++; continue; }
        while (d < end) {
            if (*d < '0' || *d > '9') break;
            src_line = src_line * 10 + (*d++ - '0');
        }
        if (d != end) { p++; continue; }
        /* success */
        mf->scrap = scrap;
        mf->line = src_line;
        {
            size_t len = (size_t)(colon - start);
            if (len >= FILENAME_MAX) len = FILENAME_MAX - 1;
            memcpy(mf->file, start, len);
            mf->file[len] = '\0';
        }
        return 1;
    }
    p++;
}
return 0;
}

```

◇
File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).
Defines: [parse_open_marker](#) [117b](#), [120](#).
Uses: [MarkerFrame](#) [116a](#), [size_t](#) [21b](#).

The `parse_close_marker` function scans for `{:N}` and returns the scrap number, or `-1` on failure.

"output.c" [117a](#)≡

```

static int parse_close_marker(const char *line)
{
    const char *p = line;
    while (*p) {
        if (*p == '{' && *(p+1) == ':') {
            const char *start = p + 2;

```

```

        if (*start < '0' || *start > '9') { p++; continue; }
        int scrap = 0;
        while (*start >= '0' && *start <= '9')
            scrap = scrap * 10 + (*start++ - '0');
        if (*start == '}')
            return scrap;
    }
    p++;
}
return -1;
}

```

◇ File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).
 Defines: `parse_close_marker` [117b](#), [120](#).

The main function `write_reverse_map` parses its argument to split an optional line number from the filename, opens the tangled file, and reads it line by line. A stack of `MarkerFrame` entries tracks nested scrap regions.

In *line mode* (a specific line was requested), as soon as `current_line` reaches the target the innermost stack frame is emitted and the function returns.

In *full mode* (no line number), every time a close marker is encountered the region from the matching open to the current line is emitted.

"output.c" 117b≡

```

void write_reverse_map(const char *arg)
{
    char filename[FILENAME_MAX];
    int target_line = -1;
    FILE *fp;
    char buf[8192];
    MarkerFrame stack[256];
    int depth = 0;
    int current_line = 0;
    int first_region = 1;

    /* Parse arg: find last ':' where suffix is all digits */
    {
        const char *colon = NULL;
        const char *p;
        for (p = arg + strlen(arg) - 1; p > arg; p--) {
            if (*p == ':') { colon = p; break; }
        }
        if (colon) {
            const char *d = colon + 1;
            int all_digits = (*d != '\0');
            while (*d) {
                if (*d < '0' || *d > '9') { all_digits = 0; break; }
                d++;
            }
            if (all_digits) {
                target_line = atoi(colon + 1);
                snprintf(filename, FILENAME_MAX, "%.s", (int)(colon - arg), arg);
            } else {
                snprintf(filename, FILENAME_MAX, "%s", arg);
            }
        } else {
            snprintf(filename, FILENAME_MAX, "%s", arg);
        }
    }
}

```

```

    }
}

fp = fopen(filename, "r");
if (!fp) {
    fprintf(stderr, "weft: cannot open %s: ", filename);
    perror(NULL);
    exit(1);
}

if (target_line < 0) {
    /* Full mode header */
    fputs("{ \"source\": ", stdout);
    json_puts(stdout, filename);
    fputs(", \"regions\": [\n", stdout);
}

while (fgets(buf, sizeof(buf), fp)) {
    MarkerFrame mf;
    int close_scrap;
    current_line++;

    if (parse_open_marker(buf, &mf)) {
        mf.start_line = current_line;
        if (depth < 256)
            stack[depth++] = mf;
        continue;
    }

    close_scrap = parse_close_marker(buf);
    if (close_scrap >= 0) {
        /* Find matching frame on stack */
        int i;
        for (i = depth - 1; i >= 0; i--) {
            if (stack[i].scrap == close_scrap) {
                if (target_line < 0) {
                    /* Full mode: emit region */
                    if (!first_region) fputs("\n", stdout);
                    first_region = 0;
                    fprintf(stdout,
                        "  { \"start\": %d, \"end\": %d, \"scrap\": %d, \"file\": ",
                        stack[i].start_line + 1, current_line - 1, stack[i].scrap);
                    json_puts(stdout, stack[i].file);
                    fprintf(stdout, ", \"line\": %d}", stack[i].line);
                }
                /* Remove frame by shifting */
                depth--;
                for (; i < depth; i++)
                    stack[i] = stack[i + 1];
                break;
            }
        }
        continue;
    }

    /* Regular line check for line mode match */
    if (target_line > 0 && current_line == target_line) {
        if (depth > 0) {

```

```

    MarkerFrame *inner = &stack[depth - 1];
    fprintf(stdout, "{\"scrap\": %d, \"file\": ", inner->scrap);
    json_puts(stdout, inner->file);
    fprintf(stdout, ", \"line\": %d}\n", inner->line);
} else {
    fputs("{}\n", stdout);
}
fclose(fp);
return;
}
}

fclose(fp);

if (target_line > 0) {
    /* Line was beyond end of file or not inside a scrap */
    fputs("{}\n", stdout);
} else {
    fputs("\n]\n", stdout);
}
}
}

```

◇ File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).
 Defines: [write_reverse_map](#) [37b](#), [122b](#).
 Uses: [exit](#) [21b](#), [fclose](#) [21b](#), [FILE](#) [21b](#), [fopen](#) [21b](#), [fprintf](#) [21b](#), [fputs](#) [21b](#), [json_puts](#) [107](#), [MarkerFrame](#) [116a](#),
[parse_close_marker](#) [117a](#), [parse_open_marker](#) [116b](#), [stack](#) [127b](#), [stderr](#) [21b](#), [strlen](#) [21b](#).

2.6.6 Error Annotator

The `--errors` flag reads compiler output from stdin and annotates lines matching `file:line:` patterns with the corresponding `.weft` source location. It works by opening each referenced file, parsing its section markers, and finding which scrap region contains the error line. Lines that don't match the pattern are passed through unchanged.

"output.c" 120≡

```

static int reverse_lookup_in_file(const char *filename, int target_line,
                                MarkerFrame *result)
{
    FILE *fp = fopen(filename, "r");
    char buf[8192];
    MarkerFrame stack[256];
    int depth = 0, current_line = 0;

    if (!fp) return 0;

    while (fgets(buf, sizeof(buf), fp)) {
        MarkerFrame mf;
        int close_scrap;
        current_line++;

        if (parse_open_marker(buf, &mf)) {
            mf.start_line = current_line;
            if (depth < 256)
                stack[depth++] = mf;
            continue;
        }
    }
}

```

```

close_scrap = parse_close_marker(buf);
if (close_scrap >= 0) {
    int i;
    for (i = depth - 1; i >= 0; i--) {
        if (stack[i].scrap == close_scrap) {
            depth--;
            for (; i < depth; i++)
                stack[i] = stack[i + 1];
            break;
        }
    }
    continue;
}

if (current_line == target_line && depth > 0) {
    *result = stack[depth - 1];
    fclose(fp);
    return 1;
}
}
fclose(fp);
return 0;
}

```

◇

File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).

Defines: [reverse_lookup_in_file](#) [121](#).

Uses: [fclose](#) [21b](#), [FILE](#) [21b](#), [fopen](#) [21b](#), [MarkerFrame](#) [116a](#), [parse_close_marker](#) [117a](#), [parse_open_marker](#) [116b](#), [stack](#) [127b](#).

"output.c" [121](#)≡

```

void write_error_annotator(void)
{
    char line[8192];

    while (fgets(line, sizeof(line), stdin)) {
        const char *p = line;
        char err_file[FILENAME_MAX];
        int err_line = -1;
        int parsed = 0;

        /* Try to parse file:digits: pattern */
        /* Skip optional drive letter on Windows (e.g. C:) */
        if (p[0] && p[1] == ':' && ((p[0] >= 'A' && p[0] <= 'Z') ||
            (p[0] >= 'a' && p[0] <= 'z'))) {
            p += 2;
        }
        {
            const char *colon1 = strchr(p, ':');
            if (colon1 && colon1 > p) {
                const char *d = colon1 + 1;
                int num = 0;
                int has_digits = 0;
                while (*d >= '0' && *d <= '9') {
                    num = num * 10 + (*d - '0');
                    has_digits = 1;
                    d++;
                }
                if (has_digits && *d == ':') {

```

```

        size_t flen = (size_t)(colon1 - line);
        if (flen < FILENAME_MAX) {
            memcpy(err_file, line, flen);
            err_file[flen] = '\0';
            err_line = num;
            parsed = 1;
        }
    }
}
}
}

fputs(line, stdout);

if (parsed) {
    MarkerFrame mf;
    if (reverse_lookup_in_file(err_file, err_line, &mf)) {
        int offset = err_line - mf.start_line;
        printf(" [weft: %s:%d, scrap %d",
            mf.file, mf.line + offset, mf.scrap);
        printf("]\n");
    }
}
}
}
}

```

◇
 File defined by [24d](#), [107](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113ab](#), [114ab](#), [115](#), [116ab](#), [117ab](#), [120](#), [121](#), [122c](#).
 Defines: [write_error_annotator 37b](#), [122b](#).
 Uses: [fputs 21b](#), [letter 131b](#), [MarkerFrame 116a](#), [reverse_lookup_in_file 120](#), [size_t 21b](#).

2.7 Writing the Output Files

⟨ *Function prototypes 122a* ⟩ ≡

```
extern void write_files(Name *files);
```

◇
 Fragment defined by [43a](#), [57a](#), [71b](#), [74b](#), [84a](#), [86a](#), [122ab](#), [126a](#), [128a](#), [132b](#), [134b](#), [141b](#), [145a](#), [155a](#), [158c](#), [161a](#), [179a](#), [188c](#), [193c](#), [195a](#), [197c](#), [199d](#).
 Fragment referenced in [21a](#).
 Uses: [Name 157c](#), [write_files 122c](#).

⟨ *Function prototypes 122b* ⟩ ≡

```

extern void write_map(Name *file_names, Name *macro_names);
extern void write_extract(char *name, Name *file_names, Name *macro_names);
extern void write_reverse_map(const char *arg);
extern void write_callers(char *name, Name *file_names, Name *macro_names);
extern void write_lint(Name *file_names, Name *macro_names);
extern void write_error_annotator(void);

```

◇
 Fragment defined by [43a](#), [57a](#), [71b](#), [74b](#), [84a](#), [86a](#), [122ab](#), [126a](#), [128a](#), [132b](#), [134b](#), [141b](#), [145a](#), [155a](#), [158c](#), [161a](#), [179a](#), [188c](#), [193c](#), [195a](#), [197c](#), [199d](#).
 Fragment referenced in [21a](#).
 Uses: [file_names 158a](#), [macro_names 158a](#), [Name 157c](#), [write_callers 112b](#), [write_error_annotator 121](#), [write_extract 113a](#), [write_lint 115](#), [write_map 109](#), [write_reverse_map 117b](#).

"output.c" 122c≡

```

void write_files(Name *files)
{
    while (files) {
        write_files(files->llink);
        < Write out files->spelling 123a >
        files = files->rlink;
    }
}

```

◇ File defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.
 Defines: `write_files` 40b, 122a.
 Uses: `Name` 157c.

`MAX_INDENT` defines the maximum number of characters tracked per line for indentation purposes. When a fragment is expanded mid-line, the characters preceding it are recorded so that subsequent lines of the fragment can reproduce the same indentation. Languages like HTML, XML, and SVG routinely contain lines longer than 500 characters, so we set this to 8192. If a line exceeds this limit, indentation beyond the recorded portion falls back to spaces instead of causing a fatal error.

< Type declarations 122d > ≡

```

#define MAX_INDENT 8192

```

◇ Fragment defined by 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b.
 Fragment referenced in 21a.
 Defines: `MAX_INDENT` 61c, 89c, 123a, 148, 150a.

< Write out files->spelling 123a > ≡

```

{
    static char temp_name[FILENAME_MAX];
    static char real_name[FILENAME_MAX];
    static int temp_name_count = 0;
    char indent_chars[MAX_INDENT];
    FILE *temp_file;

    < Find a free temporary file 123b >

    sprintf(real_name, "%s%s%s", dirpath, path_sep, files->spelling);
    if (verbose_flag)
        fprintf(stderr, "writing %s [%s]\n", files->spelling, temp_name);
    write_scraps(temp_file, files->spelling, files->defs, 0, indent_chars,
        files->debug_flag, files->tab_flag, files->indent_flag,
        files->comment_flag, files->location_flag,
        NULL, NULL, 0, files->spelling);
    fclose(temp_file);

    < Move the temporary file to the target, if required 124a >
}

```

◇ Fragment referenced in 122c.
 Uses: `defs` 131b, `fclose` 21b, `FILE` 21b, `fprintf` 21b, `MAX_INDENT` 122d, `stderr` 21b, `verbose_flag` 27, `write_scraps` 146b.

< Find a free temporary file 123b > ≡

```

for( temp_name_count = 0; temp_name_count < 10000; temp_name_count++) {
    sprintf(temp_name, "%s%snw%06d", dirpath, path_sep, temp_name_count);
}

```

```

#ifdef O_EXCL
    if (-1 != (temp_file_fd = open(temp_name, O_CREAT|O_WRONLY|O_EXCL))) {
        temp_file = fdopen(temp_file_fd, "w");
        break;
    }
#else
    if (0 != (temp_file = fopen(temp_name, "a"))) {
        if ( 0L == ftell(temp_file)) {
            break;
        } else {
            fclose(temp_file);
            temp_file = 0;
        }
    }
#endif
}
if (!temp_file) {
    fprintf(stderr, "%s: can't create %s for a temporary file\n",
        command_name, temp_name);
    exit(-1);
}

```

◇ Fragment referenced in [123a](#).
 Uses: [command_name 29c](#), [exit 21b](#), [fclose 21b](#), [fopen 21b](#), [fprintf 21b](#), [stderr 21b](#).

Note the call to **remove** before **rename** – the ANSI/ISO C standard does *not* guarantee that renaming a file to an existing filename will overwrite the file.

⟨ *Move the temporary file to the target, if required 124a* ⟩ ≡

```

if (diff_flag) {
    ⟨ Show diff of temp file vs real file 124b ⟩
} else if (compare_flag)
    ⟨ Compare the temp file and the old file 124c ⟩
else {
    remove(real_name);
    ⟨ Rename the temporary file to the target 125 ⟩
}

```

◇ Fragment referenced in [123a](#).
 Uses: [compare_flag 27](#), [diff_flag 27](#), [remove 21b](#).

⟨ *Show diff of temp file vs real file 124b* ⟩ ≡

```

{
    FILE *old_file = fopen(real_name, "r");
    if (!old_file) {
        /* New file: show all lines with + prefix */
        FILE *tf = fopen(temp_name, "r");
        if (tf) {
            char dbuf[8192];
            printf("new file: %s\n", real_name);
            while (fgets(dbuf, sizeof(dbuf), tf))
                printf("+%s", dbuf);
            fclose(tf);
        }
    } else {
        char cmd[FILENAME_MAX * 2 + 32];

```

```

FILE *dp;
fclose(old_file);
sprintf(cmd, "diff -u \"%s\" \"%s\"", real_name, temp_name);
dp = popen(cmd, "r");
if (dp) {
    char dbuf[8192];
    while (fgets(dbuf, sizeof(dbuf), dp))
        fputs(dbuf, stdout);
    pclose(dp);
}
}
remove(temp_name);
}

```

◇
Fragment referenced in [124a](#).
Uses: [fclose 21b](#), [FILE 21b](#), [fopen 21b](#), [fputs 21b](#), [remove 21b](#).

Again, we use a call to `remove` before `rename`.

⟨ Compare the temp file and the old file 124c ⟩ ≡

```

{
FILE *old_file = fopen(real_name, "r");
if (old_file) {
    int x, y;
    temp_file = fopen(temp_name, "r");
    do {
        x = getc(old_file);
        y = getc(temp_file);
    } while (x == y && x != EOF);
    fclose(old_file);
    fclose(temp_file);
    if (x == y)
        remove(temp_name);
    else {
        remove(real_name);
        ⟨ Rename the temporary file to the target 125 ⟩
    }
}
else
    ⟨ Rename the temporary file to the target 125 ⟩
}

```

◇
Fragment referenced in [124a](#).
Uses: [fclose 21b](#), [FILE 21b](#), [fopen 21b](#), [getc 21b](#), [remove 21b](#).

⟨ Rename the temporary file to the target 125 ⟩ ≡

```

if (0 != rename(temp_name, real_name)) {
    fprintf(stderr, "%s: can't rename output file to %s\n",
        command_name, real_name);
}

```

◇
Fragment referenced in [124ac](#).
Uses: [command_name 29c](#), [fprintf 21b](#), [stderr 21b](#).

Chapter 3

The Support Routines

3.1 Source Files

3.1.1 Global Declarations

We need two routines to handle reading the source files.

⟨ *Function prototypes* 126a ⟩ ≡

```
extern void source_open(char *name); /* pass in the name of the source file */
extern int source_get(void); /* no args; returns the next char or EOF */
extern int source_last; /* what last source_get() returned. */
extern int source_peek; /* The next character to get */
```

◇
Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: next 131a, source_get 127c, source_last 127c, source_open 130c, source_peek 127c.

There are also two global variables maintained for use in error messages and such.

⟨ *Global variable declarations* 126b ⟩ ≡

```
extern char *source_name; /* name of the current file */
extern int source_line; /* current line in the source file */
```

◇
Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.

Fragment referenced in 21a.

Defines: source_line 45d, 48cd, 126c, 127c, 128bc, 129, 130abc, 136d, 137b, 139b, 160a, 170, 172, 173ab, 174, 176b, 178c, 180, 181, source_name 39ab, 40ab, 45d, 48cd, 126c, 128c, 129, 130abc, 136d, 137b, 139b, 160a, 170, 172, 173ab, 174, 176b, 177a, 178cd, 180, 181.

⟨ *Global variable definitions* 126c ⟩ ≡

```
char *source_name = NULL;
int source_line = 0;
```

◇
Fragment defined by 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b.

Fragment referenced in 25e.

Uses: source_line 126b, source_name 126b.

3.1.2 Local Declarations

"input.c" 127a≡

```
static FILE *source_file;  /* the current input file */
static int double_at;
static int include_depth;
```

◇

File defined by 25a, 127abc, 128b, 130c.

Defines: `double_at` 128c, 130c, `include_depth` 129, 130bc, `source_file` 127c, 128bc, 129, 130abc.

Uses: FILE 21b.

"input.c" 127b≡

```
static struct {
    FILE *file;
    char *name;
    int line;
} stack[10];
```

◇

File defined by 25a, 127abc, 128b, 130c.

Defines: `stack` 117b, 120, 129, 130b.

Uses: FILE 21b.

3.1.3 Reading a File

The routine `source_get` returns the next character from the current source file. It notices newlines and keeps the line counter `source_line` up to date. It also catches EOF and watches for @ characters. All other characters are immediately returned. We define `source_last` to let us tell which type of scrap we are defining.

"input.c" 127c≡

```
int source_peek;
int source_last;
int source_get(void)
{
    int c;
    source_last = c = source_peek;
    switch (c) {
        case EOF:  ⟨ Handle EOF 130b ⟩
            return c;
        case '\n': source_line++;
            FALLTHROUGH;
        default:
            if (c==nw_char)
            {
                ⟨ Handle an "at" character 128c ⟩
                return c;
            }
            source_peek = getc(source_file);
            return c;
    }
}
```

◇

File defined by 25a, 127abc, 128b, 130c.

Defines: `source_get` 43d, 44c, 45cd, 46, 47a, 48cd, 49b, 50ab, 51bc, 55, 56b, 60bc, 61bc, 67ad, 73, 75a, 76cdef, 80a, 81a, 84c, 87b, 88ab, 89bc, 92ab, 94b, 95c, 96b, 99, 100d, 103a, 126a, 129, 130b, 137ab, 139ab, 140c, 170, 172, 173ab, 174, 176b, 177ab, 178d, 180, 181, 182a, `source_last` 56b, 65d, 73, 94b, 126a, `source_peek` 49b, 50b, 51b, 126a, 128bc, 129, 130bc.
 Uses: `FALLTHROUGH` 23a, `getc` 21b, `nw_char` 29ab, `source_file` 127a, `source_line` 126b.

`source_ungetc` pushes a read character back to the `source_file`.

⟨ *Function prototypes* 128a ⟩ ≡

```
extern void source_ungetc(int*);
```

◇
 Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: `source_ungetc` 128b.

"input.c" 128b≡

```
void source_ungetc(int *c)
{
    ungetc(source_peek, source_file);
    if(*c == '\n')
        source_line--;
    source_peek=*c;
}
```

◇
 File defined by 25a, 127abc, 128b, 130c.

Defines: `source_ungetc` 50ab, 51c, 81a, 100d, 128a.

Uses: `source_file` 127a, `source_line` 126b, `source_peek` 127c.

This whole @ character handling mess is pretty annoying. I want to recognize @i so I can handle include files correctly. At the same time, it makes sense to recognize illegal @ sequences and complain; this avoids ever having to check anywhere else. Unfortunately, I need to avoid tripping over the @@ sequence; hence this whole unsatisfactory `double_at` business.

⟨ *Handle an "at" character* 128c ⟩ ≡

```
{
    c = getc(source_file);
    if (double_at) {
        source_peek = c;
        double_at = FALSE;
        c = nw_char;
    }
    else
        switch (c) {
            case 'i': ⟨ Open an include file 129 ⟩
                break;
            case '#': case 'f': case 'm': case 'u': case 'v':
            case 'd': case 'o': case 'D': case 'O': case 's':
            case 'q': case 'Q': case 'S': case 'l': case 'L': case 't':
            case 'W':
            case '+':
            case '-':
            case '*':
            case '\\':
            case '{': case '}': case '<': case '>': case '|':
            case '(': case ')': case '[': case ']':
```

```

    case '%': case '_':
    case ':': case ',': case 'x': case 'c':
    case '1': case '2': case '3': case '4': case '5':
    case '6': case '7': case '8': case '9':
    case 'r':
        source_peek = c;
        c = nw_char;
        break;
    default:
        if (c==nw_char)
        {
            source_peek = c;
            double_at = TRUE;
            break;
        }
        fprintf(stderr, "%s: bad %c sequence %c[%d] (%s, line %d)\n",
            command_name, nw_char, c, c, source_name, source_line);
        exit(-1);
    }
}

```

◇ Fragment referenced in [127c](#).

Uses: [command_name 29c](#), [double_at 127a](#), [exit 21b](#), [FALSE 23a](#), [fprintf 21b](#), [getc 21b](#), [nw_char 29ab](#), [source_file 127a](#), [source_line 126b](#), [source_name 126b](#), [source_peek 127c](#), [stderr 21b](#), [TRUE 23a](#).

⟨ *Open an include file 129* ⟩ ≡

```

{
    char name[FILENAME_MAX];
    char fullname[FILENAME_MAX];
    struct incl * p = include_list;

    if (include_depth >= 10) {
        fprintf(stderr, "%s: include nesting too deep (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
    ⟨ Collect include-file name 130a ⟩
    stack[include_depth].file = source_file;
    fullname[0] = '\0';
    for (;;) {
        strcat(fullname, name);
        source_file = fopen(fullname, "r");
        if (source_file || !p)
            break;
        strcpy(fullname, p->name);
        strcat(fullname, "/");
        p = p->next;
    }
    if (!source_file) {
        fprintf(stderr, "%s: can't open include file %s\n",
            command_name, name);
        source_file = stack[include_depth].file;
    }
    else
    {
        stack[include_depth].name = source_name;
        stack[include_depth].line = source_line + 1;
    }
}

```

```

        include_depth++;
        source_line = 1;
        source_name = save_string(fullname);
    }
    source_peek = getc(source_file);
    c = source_get();
}

```

◇

Fragment referenced in 128c.

Uses: [command_name 29c](#), [exit 21b](#), [fopen 21b](#), [fprintf 21b](#), [getc 21b](#), [incl 27](#), [include_depth 127a](#), [next 131a](#), [save_string 159b](#), [source_file 127a](#), [source_get 127c](#), [source_line 126b](#), [source_name 126b](#), [source_peek 127c](#), [stack 127b](#), [stderr 21b](#).

⟨ *Collect include-file name 130a* ⟩ ≡

```

{
    char *p = name;
    do
        c = getc(source_file);
    while (c == ' ' || c == '\t');
    while (isgraph(c)) {
        *p++ = c;
        c = getc(source_file);
    }
    *p = '\0';
    if (c != '\n') {
        fprintf(stderr, "%s: unexpected characters after file name (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
}

```

◇

Fragment referenced in 129.

Uses: [command_name 29c](#), [exit 21b](#), [fprintf 21b](#), [getc 21b](#), [isgraph 21b](#), [source_file 127a](#), [source_line 126b](#), [source_name 126b](#), [stderr 21b](#).

If an EOF is discovered, the current file must be closed and input from the next stacked file must be resumed. If no more files are on the stack, the EOF is returned.

⟨ *Handle EOF 130b* ⟩ ≡

```

{
    fclose(source_file);
    if (include_depth) {
        include_depth--;
        source_file = stack[include_depth].file;
        source_line = stack[include_depth].line;
        source_name = stack[include_depth].name;
        source_peek = getc(source_file);
        c = source_get();
    }
}

```

◇

Fragment referenced in 127c.

Uses: [fclose 21b](#), [getc 21b](#), [include_depth 127a](#), [source_file 127a](#), [source_get 127c](#), [source_line 126b](#), [source_name 126b](#), [source_peek 127c](#), [stack 127b](#).

3.1.4 Opening a File

The routine `source_open` takes a file name and tries to open the file. If unsuccessful, it complains and halts. Otherwise, it sets `source_name`, `source_line`, and `double_at`.

"input.c" 130c≡

```
void source_open(char *name)
{
    source_file = fopen(name, "r");
    if (!source_file) {
        fprintf(stderr, "%s: couldn't open %s\n", command_name, name);
        exit(-1);
    }
    nw_char = '@';
    source_name = name;
    source_line = 1;
    source_peek = getc(source_file);
    double_at = FALSE;
    include_depth = 0;
}
```

◇ File defined by 25a, 127abc, 128b, 130c.

Defines: `source_open` 43c, 58, 87a, 126a.

Uses: `command_name` 29c, `double_at` 127a, `exit` 21b, `FALSE` 23a, `fopen` 21b, `fprintf` 21b, `getc` 21b, `include_depth` 127a, `nw_char` 29ab, `source_file` 127a, `source_line` 126b, `source_name` 126b, `source_peek` 127c, `stderr` 21b.

3.2 Scraps

"scraps.c" 131a≡

```
#define SLAB_SIZE 1024

typedef struct slab {
    struct slab *next;
    char chars[SLAB_SIZE];
} Slab;
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: `next` 27, 31, 34, 36a, 40a, 48b, 54b, 68bcd, 69a, 77b, 78a, 79, 80d, 83ab, 85, 92d, 93bc, 94a, 97b, 98b, 100c, 102ab, 103c, 105ab, 106bc, 108, 110b, 112ab, 126a, 129, 136a, 140ab, 141ac, 144ab, 145bc, 146b, 153d, 154c, 157b, 179cd, 182c, 183bce, 184abc, 185b, 187a, 189b, 190, 191, 192d, 193ab, 194ab, 195cd, 199e, 200c, 201a, 202, `Slab` 131b, 135f, 136ad, 141c, `SLAB_SIZE` 136a, 141c, 143d, 188b.

Uses: `slab` 131b.

"scraps.c" 131b≡

```
typedef struct {
    char *file_name;
    Slab *slab;
    struct uses *uses;
    struct uses *defs;
    int file_line;
    int end_line;
    int page;
    char letter;
    unsigned char sector;
```

```

    char *prose;
    struct name *owner;
} ScrapEntry;

```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: **defs** 48b, 54b, 61c, 68bc, 77b, 78a, 79, 80d, 83a, 85, 89c, 92d, 93b, 97b, 98b, 100c, 102a, 103c, 105bc, 106c, 108, 110b, 114ab, 123a, 132b, 134a, 136d, 140b, 145b, 146b, 147c, 149abc, 151cd, 153acd, 154b, 157c, 162c, 183ad, 191, 194bc, 195bd, **end_line** 108, 134e, 136d, 137b, **file_line** 134d, 136d, 137a, 147c, **file_name** 43ac, 57a, 58, 86a, 87a, 134c, 136d, 137a, 149ab, **letter** 121, 133a, 136d, 157a, **owner** 112ab, 135b, 136d, 143c, **page** 69a, 83a, 85, 94a, 102a, 103c, 105bc, 106c, 132b, 133ab, 136d, 155bc, 156b, 157a, 194bc, 195d, 202, **prose** 27, 30, 31, 108, 135a, 136d, 143b, 202, **ScrapEntry** 132ac, 136d, **sector** 31, 57ab, 58, 81a, 82ad, 84bcd, 85, 86ab, 87a, 100d, 101bc, 103abc, 108, 136d, 139b, 140abd, 141d, 151d, 154c, 157c, 158c, 159c, 160ab, 162bc, 176b, 178c, 180, 191, **slab** 131a, 136d, 141d, 147c, 191, **uses** 31, 68d, 83b, 85, 93c, 102b, 103c, 106c, 108, 110b, 112ab, 114b, 133c, 136d, 140a, 141a, 157c, 162c, 163, 183d, 191, 193bd, 195d.
Uses: **Slab** 131a.

There's some issue with reading the auxiliary file if it has more than 255 lines, so (as a workaround only) increase the size of the array of **ScrapEntry**s held in **SCRAP[n]**.

"scraps.c" 132a≡

```

#define SCRAP_BITS 10
#define SCRAP_SIZE (1<<SCRAP_BITS)
#define SCRAP_MASK (SCRAP_SIZE - 1)
#define SCRAP_SHIFT SCRAP_BITS
static ScrapEntry *SCRAP[SCRAP_SIZE];

#define scrap_array(i) SCRAP[(i) >> SCRAP_SHIFT][(i) & SCRAP_MASK]

static int scraps;
int num_scraps(void)
{
    return scraps;
}
< Forward declarations for scraps.c 147a, ... >

```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: **num_scraps** 82a, 101b, 132b, **SCRAP** 132c, 136d, **scraps** 24bc, 27, 31, 56b, 57b, 62, 63a, 64b, 67a, 69a, 86b, 90ab, 92ac, 93a, 94a, 108, 132c, 136d, 155bc, 156bc, 191, 202, **scrap_array** 133ac, 134acde, 135ab, 136d, 137ab, 140ab, 141d, 143bc, 147c, 149ab, 155bc, 156b, 157a, 191, 193d, 195b, **SCRAP_BITS** Never used, **SCRAP_MASK** 136d, **SCRAP_SHIFT** 136d, **SCRAP_SIZE** 132c, 136d.
Uses: **ScrapEntry** 131b.

< Function prototypes 132b >≡

```

extern void init_scraps(void);
extern int collect_scrap(void);
extern int write_scraps(FILE *file, char *spelling, Scrap_Node *defs, int global_indent, char *
    indent_chars, char debug_flag, char tab_flag, char indent_flag, unsigned char comment_flag,
    char location_flag, Arglist *inArgs, char *inParams[9], Parameters parameters, char *title);
extern void write_scrap_ref(FILE *file, int num, int first, int *page);
extern void write_single_scrap_ref(FILE *file, int num);
extern int num_scraps(void);
extern Uses * get_scrap_uses(int scrap);
extern Uses * get_scrap_defs(int scrap);

```

◇

Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: Arglist 179c, collect_scrap 136c, defs 131b, FILE 21b, first 200a, get_scrap_defs 134a, get_scrap_uses 133c, init_scraps 132c, num_scraps 132a, page 131b, Parameters 54a, Scrap_Node 157b, Uses 193b, write_scraps 146b, write_scrap_ref 133a, write_single_scrap_ref 133b.

"scraps.c" 132c≡

```
void init_scraps(void)
{
    scraps = 1;
    SCRAP[0] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(ScrapEntry));
}
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: init_scraps 43c, 132b.

Uses: arena_getmem 200b, SCRAP 132a, ScrapEntry 131b, scraps 132a, SCRAP_SIZE 132a.

"scraps.c" 133a≡

```
void write_scrap_ref(FILE *file, int num, int first, int *page)
{
    if (scrap_array(num).page >= 0) {
        if (first!=0)
            fprintf(file, "%d", scrap_array(num).page);
        else if (scrap_array(num).page != *page)
            fprintf(file, ", %d", scrap_array(num).page);
        if (scrap_array(num).letter > 0)
            fputc(scrap_array(num).letter, file);
    }
    else {
        if (first!=0)
            putc('?', file);
        else
            fputs(" ? ", file);
        ( Warn (only once) about needing to rerun after Latex 135c )
    }
    if (first>=0)
        *page = scrap_array(num).page;
}
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: write_scrap_ref 69a, 83a, 85, 94a, 102a, 103c, 105c, 132b, 133b, 194c.

Uses: FILE 21b, first 200a, fprintf 21b, fputs 21b, letter 131b, page 131b, putc 21b, scrap_array 132a.

"scraps.c" 133b≡

```
void write_single_scrap_ref(FILE *file, int num)
{
    int page;
    write_scrap_ref(file, num, TRUE, &page);
}
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: write_single_scrap_ref 56b, 62, 63a, 68d, 78a, 80d, 83b, 85, 90ab, 93c, 97b, 98b, 100c, 102b, 103c, 132b, 153ac, 197d.

Uses: FILE 21b, page 131b, TRUE 23a, write_scrap_ref 133a.

"scraps.c" 133c≡

```
Uses * get_scrap_uses(int scrap)
{
    return scrap_array(scrap).uses;
}
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: get_scrap_uses 104, 108, 110b, 132b.
Uses: scrap_array 132a, Uses 193b, uses 131b.

"scraps.c" 134a≡

```
Uses * get_scrap_defs(int scrap)
{
    return scrap_array(scrap).defs;
}
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: get_scrap_defs 106a, 132b.
Uses: defs 131b, scrap_array 132a, Uses 193b.

These accessors expose scrap metadata that is needed by the `--map` and `--extract` modes (Section 2.6) without leaking the internal `ScrapEntry` or `Slab` types.

⟨ *Function prototypes* 134b ⟩ ≡

```
extern const char *scrap_file_name(int i);
extern int scrap_file_line(int i);
extern int scrap_end_line(int i);
extern const char *scrap_prose(int i);
extern Name *scrap_owner(int i);
extern void dump_scrap_text(FILE *file, int scrap_idx);
extern void dump_scrap_json(FILE *f, int scrap_idx);
extern void set_scrap_prose(int i, char *p);
extern void set_scrap_owner(int i, Name *n);
```

◇ Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
Fragment referenced in 21a.
Uses: dump_scrap_json 143a, dump_scrap_text 141d, FILE 21b, Name 157c, scrap_end_line 134e, scrap_file_line 134d, scrap_file_name 134c, scrap_owner 135b, scrap_prose 135a, set_scrap_owner 143c, set_scrap_prose 143b.

"scraps.c" 134c≡

```
const char *scrap_file_name(int i) { return scrap_array(i).file_name; }
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: scrap_file_name 108, 110b, 114b, 134b.
Uses: file_name 131b, scrap_array 132a.

"scraps.c" 134d≡

```
int scrap_file_line(int i) { return scrap_array(i).file_line; }
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: `scrap_file_line` 108, 110b, 114b, 134b.
 Uses: `file_line` 131b, `scrap_array` 132a.

"scraps.c" 134e≡

```
int scrap_end_line(int i) { return scrap_array(i).end_line; }
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: `scrap_end_line` 108, 134b.
 Uses: `end_line` 131b, `scrap_array` 132a.

"scraps.c" 135a≡

```
const char *scrap_prose(int i) { return scrap_array(i).prose; }
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: `scrap_prose` 108, 134b.
 Uses: `prose` 131b, `scrap_array` 132a.

"scraps.c" 135b≡

```
Name *scrap_owner(int i) { return scrap_array(i).owner; }
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: `scrap_owner` 112ab, 134b.
 Uses: `Name` 157c, `owner` 131b, `scrap_array` 132a.

⟨ Warn (only once) about needing to rerun after Latex 135c ⟩ ≡

```
{
  if (!already_warned) {
    fprintf(stderr, "%s: you'll need to rerun weft after running latex\n",
             command_name);
    already_warned = TRUE;
  }
}
```

◇ Fragment referenced in 133a, 155c.
 Uses: `already_warned` 135d, `command_name` 29c, `fprintf` 21b, `stderr` 21b, `TRUE` 23a.

⟨ Global variable declarations 135d ⟩ ≡

```
extern int already_warned;
```

◇ Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.
 Fragment referenced in 21a.
 Defines: `already_warned` 135ce, 155b.

⟨ Global variable definitions 135e ⟩ ≡

```
int already_warned = 0;
```

◇ Fragment defined by 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b.

Fragment referenced in [25e](#).
Uses: [already_warned 135d](#).

"scraps.c" 135f≡

```
typedef struct {
    Slab *scrap;
    Slab *prev;
    int index;
} Manager;
```

◇ File defined by [25b](#), [131ab](#), [132ac](#), [133abc](#), [134acde](#), [135abf](#), [136abc](#), [141acd](#), [143abcd](#), [144ab](#), [145d](#), [146b](#), [155b](#), [184abc](#), [185abcd](#), [186ab](#), [187c](#), [188d](#), [189a](#), [192d](#), [193ad](#), [195b](#), [196ab](#), [197b](#).

Defines: [Manager 136abc](#), [141cd](#), [143d](#), [145d](#), [147c](#), [185d](#), [187c](#), [191](#).

Uses: [Slab 131a](#).

"scraps.c" 136a≡

```
static void push(char c, Manager *manager)
{
    Slab *scrap = manager->scrap;
    int index = manager->index;
    scrap->chars[index++] = c;
    if (index == SLAB_SIZE) {
        Slab *new_slab = (Slab *) arena_getmem(sizeof(Slab));
        scrap->next = new_slab;
        manager->scrap = new_slab;
        index = 0;
    }
    manager->index = index;
}
```

◇ File defined by [25b](#), [131ab](#), [132ac](#), [133abc](#), [134acde](#), [135abf](#), [136abc](#), [141acd](#), [143abcd](#), [144ab](#), [145d](#), [146b](#), [155b](#), [184abc](#), [185abcd](#), [186ab](#), [187c](#), [188d](#), [189a](#), [192d](#), [193ad](#), [195b](#), [196ab](#), [197b](#).

Defines: [push 52b](#), [55](#), [136b](#), [137ab](#), [138](#), [140cd](#).

Uses: [arena_getmem 200b](#), [Manager 135f](#), [next 131a](#), [Slab 131a](#), [SLAB_SIZE 131a](#).

"scraps.c" 136b≡

```
static void pushes(char *s, Manager *manager)
{
    while (*s)
        push(*s++, manager);
}
```

◇ File defined by [25b](#), [131ab](#), [132ac](#), [133abc](#), [134acde](#), [135abf](#), [136abc](#), [141acd](#), [143abcd](#), [144ab](#), [145d](#), [146b](#), [155b](#), [184abc](#), [185abcd](#), [186ab](#), [187c](#), [188d](#), [189a](#), [192d](#), [193ad](#), [195b](#), [196ab](#), [197b](#).

Defines: [pushs 55](#), [138](#), [140d](#).

Uses: [Manager 135f](#), [push 136a](#).

"scraps.c" 136c≡

```
int collect_scrap(void)
{
    int current_scrap, lblseq = 0;
    int depth = 1;
    Manager writer;
    < Create new scrap, managed by writer 136d >
    < Accumulate scrap and return scraps++ 137a >
```

```
}
```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: collect_scrap 47d, 48a, 55, 132b, 182c.
Uses: Manager 135f.

⟨ Create new scrap, managed by writer 136d ⟩ ≡

```
{
    Slab *scrap = (Slab *) arena_getmem(sizeof(Slab));
    if ((scraps & SCRAP_MASK) == 0)
        SCRAP[scraps >> SCRAP_SHIFT] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(ScrapEntry));
    scrap_array(scraps).slab = scrap;
    scrap_array(scraps).file_name = save_string(source_name);
    scrap_array(scraps).file_line = source_line;
    scrap_array(scraps).page = -1;
    scrap_array(scraps).letter = 0;
    scrap_array(scraps).uses = NULL;
    scrap_array(scraps).defs = NULL;
    scrap_array(scraps).sector = current_sector;
    scrap_array(scraps).end_line = source_line;
    scrap_array(scraps).prose = NULL;
    scrap_array(scraps).owner = NULL;
    writer.scrap = scrap;
    writer.index = 0;
    current_scrap = scraps++;
}
```

◇
Fragment referenced in 136c.
Uses: arena_getmem 200b, current_sector 47b, defs 131b, end_line 131b, file_line 131b, file_name 131b, letter 131b, owner 131b, page 131b, prose 131b, save_string 159b, SCRAP 132a, ScrapEntry 131b, scraps 132a, scrap_array 132a, SCRAP_MASK 132a, SCRAP_SHIFT 132a, SCRAP_SIZE 132a, sector 131b, Slab 131a, slab 131b, source_line 126b, source_name 126b, uses 131b.

⟨ Accumulate scrap and return scraps++ 137a ⟩ ≡

```
{
    int c = source_get();
    while (1) {
        switch (c) {
            case EOF: fprintf(stderr, "%s: unexpect EOF in (%s, %d)\n",
                                command_name, scrap_array(current_scrap).file_name,
                                scrap_array(current_scrap).file_line);
                        exit(-1);
            default:
                if (c==nw_char)
                {
                    ⟨ Handle at-sign during scrap accumulation 137b ⟩
                    break;
                }
                push(c, &writer);
                c = source_get();
                break;
        }
    }
}
```

◇
Fragment referenced in 136c.

Uses: `command_name` 29c, `exit` 21b, `file_line` 131b, `file_name` 131b, `fprintf` 21b, `nw_char` 29ab, `push` 136a, `scrap_array` 132a, `source_get` 127c, `stderr` 21b.

⟨ Handle at-sign during scrap accumulation 137b ⟩ ≡

```
{
  c = source_get();
  switch (c) {
    case '(':
    case '[':
    case '{': depth++;
              break;

    case '+':
    case '-':
    case '*':
    case '|': ⟨ Collect user-specified index entries 139b ⟩
              /* Fall through */

    case ')':
    case ']':
    case '}': if (--depth > 0)
              break;
              /* else fall through */

    case ',':
              push('\0', &writer);
              scrap_ended_with = c;
              scrap_array(current_scrap).end_line = source_line;
              return current_scrap;

    case '<': ⟨ Handle macro invocation in scrap 140c ⟩
              break;

    case '%': ⟨ Skip commented-out code 76e ⟩
              /* emit line break to the output file to keep #line in sync. */
              push('\n', &writer);
              c = source_get();
              break;

    case 'x': ⟨ Get label while collecting scrap 138 ⟩
              break;

    case 'c': ⟨ Include block comment in a scrap 52b ⟩
              break;

    case '1': case '2': case '3':
    case '4': case '5': case '6':
    case '7': case '8': case '9':
    case 'f': case '#': case 'v':
    case 't': case 's':
              push(nw_char, &writer);
              break;

    case '_': c = source_get();
              break;

    default :
              if (c==nw_char)
              {
                push(nw_char, &writer);
                push(nw_char, &writer);
                c = source_get();
                break;
              }

    fprintf(stderr, "%s: unexpected %c%c in scrap (%s, %d)\n",
              command_name, nw_char, c, source_name, source_line);
    exit(-1);
  }
}
```

```

    }
}

```

◇

Fragment referenced in [137a](#).

Uses: [command_name 29c](#), [end_line 131b](#), [exit 21b](#), [fprintf 21b](#), [nw_char 29ab](#), [push 136a](#), [scrap_array 132a](#), [source_get 127c](#), [source_line 126b](#), [source_name 126b](#), [stderr 21b](#).

⟨ *Get label while collecting scrap 138* ⟩ ≡

```

{
    ⟨ Get label from (139a source.get() ) 197a ⟩
    ⟨ Save label to label store 197f ⟩
    push(nw_char, &writer);
    push('x', &writer);
    pushs(label_name, &writer);
    push(nw_char, &writer);
}

```

◇

Fragment referenced in [137b](#).

Uses: [source_get 127c](#).

⟨ *Collect user-specified index entries 139b* ⟩ ≡

```

{
    do {
        int type = c;
        do {
            char new_name[MAX_NAME_LEN];
            char *p = new_name;
            unsigned int sector = 0;
            do
                c = source_get();
            while (isspace(c));
            if (c != nw_char) {
                Name *name;
                do {
                    *p++ = c;
                    c = source_get();
                } while (c != nw_char && !isspace(c));
                *p = '\0';
                switch (type) {
                    case '*':
                        sector = current_sector;
                        ⟨ Add user identifier use 140a ⟩
                        break;
                    case '-':
                        ⟨ Add user identifier use 140a ⟩
                        /* Fall through */
                    case '|':
                        sector = current_sector;
                        /* Fall through */
                    case '+':
                        ⟨ Add user identifier definition 140b ⟩
                        break;
                }
            }
        } while (c != nw_char);
        c = source_get();
    }
}

```

```

}while (c == '|' || c == '*' || c == '-' || c == '+');
if (c != '}' && c != ']' && c != ')') {
    fprintf(stderr, "%s: unexpected %c%c in index entry (%s, %d)\n",
        command_name, nw_char, c, source_name, source_line);
    exit(-1);
}
}
}

```

◇

Fragment referenced in 137b.

Uses: [command_name 29c](#), [current_sector 47b](#), [exit 21b](#), [fprintf 21b](#), [isspace 21b](#), [MAX_NAME_LEN 23b](#), [Name 157c](#), [nw_char 29ab](#), [sector 131b](#), [source_get 127c](#), [source_line 126b](#), [source_name 126b](#), [stderr 21b](#).

⟨ *Add user identifier use 140a* ⟩ ≡

```

name = name_add(&user_names, new_name, sector);
if (!name->uses || name->uses->scrap != current_scrap) {
    Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    use->scrap = current_scrap;
    use->next = name->uses;
    name->uses = use;
    add_uses(&(scrap_array(current_scrap).uses), name);
}

```

◇

Fragment referenced in 139b.

Uses: [add_uses 193a](#), [arena_getmem 200b](#), [name_add 162b](#), [next 131a](#), [scrap_array 132a](#), [Scrap_Node 157b](#), [sector 131b](#), [user_names 158a](#), [uses 131b](#).

⟨ *Add user identifier definition 140b* ⟩ ≡

```

name = name_add(&user_names, new_name, sector);
if (!name->defs || name->defs->scrap != current_scrap) {
    Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    def->scrap = current_scrap;
    def->next = name->defs;
    name->defs = def;
    add_uses(&(scrap_array(current_scrap).defs), name);
}

```

◇

Fragment referenced in 139b.

Uses: [add_uses 193a](#), [arena_getmem 200b](#), [defs 131b](#), [name_add 162b](#), [next 131a](#), [scrap_array 132a](#), [Scrap_Node 157b](#), [sector 131b](#), [user_names 158a](#).

⟨ *Handle macro invocation in scrap 140c* ⟩ ≡

```

{
    Arglist * args = collect_scrap_name(current_scrap);
    Name *name = args->name;
    ⟨ Save macro name 140d ⟩
    add_to_use(name, current_scrap);
    if (scrap_name_has_parameters) {
        ⟨ Save macro parameters 55 ⟩
    }
    push(nw_char, &writer);
    push('>', &writer);
    c = source_get();
}

```

◇

Fragment referenced in 137b.

Uses: [add_to_use 141a](#), [Arglist 179c](#), [collect_scrap_name 180](#), [Name 157c](#), [nw_char 29ab](#), [push 136a](#), [source_get 127c](#).

⟨ *Save macro name 140d* ⟩ ≡

```
{
    char buff[24];

    push(nw_char, &writer);
    push('<', &writer);
    push(name->sector, &writer);
    sprintf(buff, "%d", arglist_register(args));
    pushs(buff, &writer);
}
```

◇
Fragment referenced in [140c](#).

Uses: [arglist_register 147b](#), [nw_char 29ab](#), [push 136a](#), [pushs 136b](#), [sector 131b](#).

"scraps.c" 141a≡

```
void
add_to_use(Name * name, int current_scrap)
{
    if (!name->uses || name->uses->scrap != current_scrap) {
        Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
        use->scrap = current_scrap;
        use->next = name->uses;
        name->uses = use;
    }
}
```

◇
File defined by [25b](#), [131ab](#), [132ac](#), [133abc](#), [134acde](#), [135abf](#), [136abc](#), [141acd](#), [143abcd](#), [144ab](#), [145d](#), [146b](#), [155b](#), [184abc](#), [185abcd](#), [186ab](#), [187c](#), [188d](#), [189a](#), [192d](#), [193ad](#), [195b](#), [196ab](#), [197b](#).

Defines: [add_to_use 55](#), [140c](#), [141b](#), [183b](#).

Uses: [arena_getmem 200b](#), [Name 157c](#), [next 131a](#), [Scrap_Node 157b](#), [uses 131b](#).

⟨ *Function prototypes 141b* ⟩ ≡

```
extern void add_to_use(Name * name, int current_scrap);
```

◇
Fragment defined by [43a](#), [57a](#), [71b](#), [74b](#), [84a](#), [86a](#), [122ab](#), [126a](#), [128a](#), [132b](#), [134b](#), [141b](#), [145a](#), [155a](#), [158c](#), [161a](#), [179a](#), [188c](#), [193c](#), [195a](#), [197c](#), [199d](#).

Fragment referenced in [21a](#).

Uses: [add_to_use 141a](#), [Name 157c](#).

"scraps.c" 141c≡

```
static char pop(Manager *manager)
{
    Slab *scrap = manager->scrap;
    int index = manager->index;
    char c = scrap->chars[index++];
    if (index == SLAB_SIZE) {
        manager->prev = scrap;
        manager->scrap = scrap->next;
        index = 0;
    }
    manager->index = index;
    return c;
}
```

◊ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: pop 52c, 53ac, 56a, 141d, 145d, 146a, 147c, 149d, 150d, 151b, 186b.
 Uses: Manager 135f, next 131a, Slab 131a, SLAB_SIZE 131a.

The function `dump_scrap_text` walks the slab chain for a given scrap and writes every printable character (plus newlines and tabs) to the supplied file handle. The internal meta-character sequences that encode fragment references are decoded back to their `@<name@>` form; other control sequences are silently skipped.

"scraps.c" 141d≡

```
void dump_scrap_text(FILE *file, int scrap_idx)
{
    Manager reader;
    char c;
    reader.scrap = scrap_array(scrap_idx).slab;
    reader.prev = NULL;
    reader.index = 0;
    c = pop(&reader);
    while (c) {
        if (c == nw_char) {
            /* handle internal control sequences */
            c = pop(&reader);
            if (c == '<') {
                /* fragment reference encoded as: sector_byte + decimal_index
                 followed by nw_char + '>' */
                char idx_buf[MAX_NAME_LEN];
                char *p = idx_buf;
                int idx;
                (void)pop(&reader); /* sector byte, discard */
                c = pop(&reader);
                while (c != nw_char && c != '\0') {
                    *p++ = c;
                    c = pop(&reader);
                }
                *p = '\0';
                if (sscanf(idx_buf, "%d", &idx) == 1) {
                    Arglist *a = arglist_lookup(idx);
                    if (a && a->name) {
                        fprintf(file, "%c<%s%c>", nw_char, a->name->spelling, nw_char);
                    }
                }
                /* skip past closing '>' marker */
            }
            if (c == nw_char)
                c = pop(&reader);
            /* skip optional parameter block */
            if (c == '(') {
                while (c && c != nw_char)
                    c = pop(&reader);
                if (c == nw_char)
                    c = pop(&reader);
            }
        } else {
            /* other control seq: skip */
        }
        c = pop(&reader);
        continue;
    }
}
```

```

        fputc(c, file);
        c = pop(&reader);
    }
}

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: `dump_scrap_text` 110b, 134b, 143a.
 Uses: `Arglist` 179c, `arglist_lookup` 147b, `FILE` 21b, `fprintf` 21b, `Manager` 135f, `MAX_NAME_LEN` 23b, `nw_char` 29ab, `pop` 141c, `scrap_array` 132a, `sector` 131b, `slab` 131b.

The function `dump_scrap_json` is similar to `dump_scrap_text` but wraps the output in a JSON string, escaping special characters inline. It is used by `--bodies` to embed scrap code in the `-m` JSON output.

"scraps.c" 143a≡

```

void dump_scrap_json(FILE *f, int scrap_idx)
{
    FILE *tmp = tmpfile();
    int ch;
    if (!tmp) { fputs("\\""", f); return; }
    dump_scrap_text(tmp, scrap_idx);
    fflush(tmp);
    fseek(tmp, 0, SEEK_SET);
    fputc('\"', f);
    while ((ch = fgetc(tmp)) != EOF) {
        switch (ch) {
            case '\"': fputs("\\\"", f); break;
            case '\\': fputs("\\\\", f); break;
            case '\n': fputs("\\n", f); break;
            case '\r': fputs("\\r", f); break;
            case '\t': fputs("\\t", f); break;
            default:
                if (ch < 0x20)
                    fprintf(f, "\\u%04x", ch);
                else
                    fputc(ch, f);
        }
    }
    fputc('\"', f);
    fclose(tmp);
}

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: `dump_scrap_json` 108, 134b.
 Uses: `dump_scrap_text` 141d, `fclose` 21b, `FILE` 21b, `fprintf` 21b, `fputs` 21b.

"scraps.c" 143b≡

```

void set_scrap_prose(int i, char *p) { scrap_array(i).prose = p; }

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: `set_scrap_prose` 47d, 48a, 134b.
 Uses: `prose` 131b, `scrap_array` 132a.

"scraps.c" 143c≡

```
void set_scrap_owner(int i, Name *n) { scrap_array(i).owner = n; }
```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: set_scrap_owner 48b, 134b.
Uses: Name 157c, owner 131b, scrap_array 132a.

"scraps.c" 143d≡

```
static void backup(int n, Manager *manager)
{
    int index = manager->index;
    if (n > index
        && manager->prev != NULL)
    {
        manager->scrap = manager->prev;
        manager->prev = NULL;
        index += SLAB_SIZE;
    }
    manager->index = (n <= index ? index - n : 0);
}
```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: backup 149d.
Uses: Manager 135f, SLAB_SIZE 131a.

"scraps.c" 144a≡

```
void
lookup(int n, Arglist * par, char * arg[9], Name **name, Arglist ** args)
{
    int i;
    Arglist * p = par;

    for (i = 0; i < n && p != NULL; i++)
        p = p->next;
    if (p == NULL) {
        char * a = arg[n];

        *name = NULL;
        *args = (Arglist *)a;
    }
    else {
        *name = p->name;
        *args = p->args;
    }
}
```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: lookup 54b, 145b.
Uses: Arglist 179c, Name 157c, next 131a.

"scraps.c" 144b≡

```
Arglist * instance(Arglist * a, Arglist * par, char * arg[9], int * ch)
{
    if (a != NULL) {
```

```

    int changed = 0;
    Arglist *args, *next;
    Name* name;
    < Set up name, args and next 145b >
    if (changed){
        < Build a new arglist 145c >
        *ch = 1;
    }
}

return a;
}

```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: instance 61c, 89c, 145ab, 151d.
Uses: Arglist 179c, Name 157c, next 131a.

< Function prototypes 145a > ≡

```
Arglist *instance(Arglist *a, Arglist *par, char *arg[9], int *ch);
```

◇
Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
Fragment referenced in 21a.
Uses: Arglist 179c, instance 144b.

< Set up name, args and next 145b > ≡

```

next = instance(a->next, par, arg, &changed);
name = a->name;
if (name == (Name *)1) {
    Embed_Node * q = (Embed_Node *)arena_getmem(sizeof(Embed_Node));
    q->defs = (Scrap_Node *)a->args;
    q->args = par;
    args = (Arglist *)q;
    changed = 1;
} else if (name != NULL)
    args = instance(a->args, par, arg, &changed);
else {
    char * p = (char *)a->args;
    if (p[0] == ARG_CHR) {
        lookup(p[1] - '1', par, arg, &name, &args);
        changed = 1;
    }
    else {
        args = a->args;
    }
}
}

```

◇
Fragment referenced in 144b.
Uses: arena_getmem 200b, Arglist 179c, ARG_CHR 178a, defs 131b, Embed_Node 183a, instance 144b, lookup 144a, Name 157c, next 131a, Scrap_Node 157b.

< Build a new arglist 145c > ≡

```

a = (Arglist *)arena_getmem(sizeof(Arglist));
a->name = name;
a->args = args;

```

a->next = next;

◇

Fragment referenced in 144b.

Uses: arena_getmem 200b, Arglist 179c, next 131a.

"scraps.c" 145d≡

```
static Arglist *pop_scrap_name(Manager *manager, Parameters *parameters)
{
    char name[MAX_NAME_LEN];
    char *p = name;
    Arglist * args;
    int c;
    int idx;

    (void)pop(manager); /* not sure why we have to pop twice */
    c = pop(manager);

    while (c != nw_char) {
        *p++ = c;
        c = pop(manager);
    }
    *p = '\000';
    if (sscanf(name, "%d", &idx) != 1)
    {
        fprintf(stderr, "%s: found an internal problem (2)\n", command_name);
        exit(-1);
    }
    args = arglist_lookup(idx);
    ⟨ Check for end of scrap name 146a ⟩
    return args;
}
```

◇

File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: pop_scrap_name 151d.

Uses: Arglist 179c, arglist_lookup 147b, command_name 29c, exit 21b, fprintf 21b, Manager 135f, MAX_NAME_LEN 23b, nw_char 29ab, Parameters 54a, pop 141c, stderr 21b.

⟨ Check for end of scrap name 146a ⟩ ≡

```
{
    c = pop(manager);
    ⟨ Check for macro parameters 56a ⟩
}
```

◇

Fragment referenced in 145d.

Uses: pop 141c.

"scraps.c" 146b≡

```
int write_scraps(FILE *file, char *spelling, Scrap_Node *defs,
                int global_indent, char *indent_chars,
                char debug_flag, char tab_flag, char indent_flag,
                unsigned char comment_flag, char location_flag,
                Arglist *inArgs,
                char *inParams[9], Parameters parameters, char *title)
{
```

```

/* This is in file file name */
int indent = 0;
int newline = 1;
while (defs) {
    ⟨ Copy defs->scrap to file 147c ⟩
    defs = defs->next;
}
return indent + global_indent;
}

```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: write_scraps 54b, 61c, 78b, 89c, 98b, 123a, 132b, 151d, 154b.

Uses: Arglist 179c, defs 131b, FILE 21b, next 131a, Parameters 54a, Scrap_Node 157b.

⟨ Forward declarations for scraps.c 147a ⟩ ≡

```
int delayed_indent = 0;
```

◇
Fragment defined by 147ab, 154a, 176a, 192c.

Fragment referenced in 132a.

Defines: delayed_indent 54b, 147c, 150b, 151d, 153ab.

⟨ Forward declarations for scraps.c 147b ⟩ ≡

```

#define MAX_ARGLISTS 4096
static Arglist *arglist_table[MAX_ARGLISTS];
static int arglist_count = 0;

void arglist_reset(void)
{
    arglist_count = 0;
}

static int arglist_register(Arglist *a)
{
    if (arglist_count >= MAX_ARGLISTS) {
        fprintf(stderr, "%s: too many macro argument lists (max %d)\n",
            command_name, MAX_ARGLISTS);
        exit(EXIT_FAILURE);
    }
    arglist_table[arglist_count] = a;
    return arglist_count++;
}

static Arglist *arglist_lookup(int idx)
{
    if (idx < 0 || idx >= arglist_count) {
        fprintf(stderr, "%s: invalid arglist index %d\n", command_name, idx);
        exit(EXIT_FAILURE);
    }
    return arglist_table[idx];
}

```

◇
Fragment defined by 147ab, 154a, 176a, 192c.

Fragment referenced in 132a.

Defines: arglist_count Never used, arglist_lookup 141d, 145d, 192b, arglist_register 140d, arglist_reset 199d, 201b, arglist_table Never used, MAX_ARGLISTS Never used.

Uses: Arglist 179c, command_name 29c, exit 21b, fprintf 21b, stderr 21b.

⟨ Copy defs->scrap to file 147c ⟩ ≡

```
{
    char c;
    Manager reader;
    Parameters local_parameters = 0;
    int line_number = scrap_array(defs->scrap).file_line;
    reader.scrap = scrap_array(defs->scrap).slab;
    reader.prev = NULL;
    reader.index = 0;
    ⟨ Insert debugging information if required 149a ⟩
    ⟨ Insert section opening marker 149b ⟩
    if (delayed_indent)
    {
        ⟨ Insert appropriate indentation 149d ⟩
    }
    c = pop(&reader);
    while (c) {
        switch (c) {
            case '\n':
                if (global_indent >= 0) {
                    putc(c, file);
                    line_number++;
                    newline = 1;
                    delayed_indent = 0;
                    ⟨ Insert appropriate indentation 149d ⟩
                    break;
                } else {
                    /* Don't show newlines in embedded fragments */
                    fputs(". . .", file);
                    return 0;
                }
            case '\t': ⟨ Handle tab characters on output 150c ⟩
                delayed_indent = 0;
                break;
            default:
                if (c==nw_char)
                {
                    ⟨ Check for macro invocation in scrap 150d ⟩
                    break;
                }
                putc(c, file);
                if (global_indent >= 0) {
                    ⟨ Add more indentation ' ' 148 ⟩
                }
                indent++;
                if (c > ' ') newline = 0;
                delayed_indent = 0;
                break;
        }
        c = pop(&reader);
    }
    ⟨ Insert section closing marker 149c ⟩
}
```

◇
Fragment referenced in 146b.

Uses: [defs 131b](#), [delayed_indent 147a](#), [file_line 131b](#), [fputs 21b](#), [Manager 135f](#), [nw_char 29ab](#), [Parameters 54a](#), [pop 141c](#), [putc 21b](#), [scrap_array 132a](#), [slab 131b](#).

We need to make sure that we don't overflow `indent_chars[]`. If the line is longer than `MAX_INDENT`, we simply stop recording; the Put out the indent scrap will use spaces for any positions beyond the recorded range.

⟨ Add more indentation char 148 ⟩ ≡

```
{
    if (global_indent + indent < MAX_INDENT)
        indent_chars[global_indent + indent] = char;
}
```

◇
Fragment referenced in [147c](#), [150cd](#).
Uses: [MAX_INDENT 122d](#).

⟨ Insert debugging information if required 149a ⟩ ≡

```
if (debug_flag) {
    fprintf(file, "\n#line %d \"%s\"\n",
            line_number, scrap_array(defs->scrap).file_name);
    ⟨ Insert appropriate indentation 149d ⟩
}
```

◇
Fragment referenced in [147c](#), [150d](#).
Uses: [defs 131b](#), [file_name 131b](#), [fprintf 21b](#), [scrap_array 132a](#).

Section markers in the Knuth style delimit each scrap with opening and closing comments that include the scrap number and source location. For example, in C: `/* {5: weft.w:28} */` and `/* {:5} */`.

⟨ Insert section opening marker 149b ⟩ ≡

```
if (location_flag && comment_flag) {
    fprintf(file, "%s{:d: %s:%d}%s\n",
            comment_begin[comment_flag],
            defs->scrap,
            scrap_array(defs->scrap).file_name,
            line_number,
            comment_end[comment_flag]);
}
```

◇
Fragment referenced in [147c](#).
Uses: [comment_begin 176a](#), [comment_end 176a](#), [defs 131b](#), [file_name 131b](#), [fprintf 21b](#), [scrap_array 132a](#).

⟨ Insert section closing marker 149c ⟩ ≡

```
if (location_flag && comment_flag) {
    fprintf(file, "%s{:d}%s\n",
            comment_begin[comment_flag],
            defs->scrap,
            comment_end[comment_flag]);
}
```

◇
Fragment referenced in [147c](#).
Uses: [comment_begin 176a](#), [comment_end 176a](#), [defs 131b](#), [fprintf 21b](#).

⟨ *Insert appropriate indentation 149d* ⟩ ≡

```
{
    char c1 = pop(&reader);
    char c2 = pop(&reader);

    if (indent_flag && !(⟨ Indent suppressed 150b ⟩)) {
        ⟨ Put out the indent 150a ⟩
    }
    indent = 0;
    backup(2, &reader);
}
```

◇
Fragment referenced in [147c](#), [149a](#).
Uses: [backup 143d](#), [pop 141c](#).

⟨ *Put out the indent 150a* ⟩ ≡

```
if (tab_flag)
    for (indent=0; indent<global_indent; indent++)
        putc(' ', file);
else
    for (indent=0; indent<global_indent; indent++)
        putc(indent < MAX_INDENT ? indent_chars[indent] : ' ', file);
```

◇
Fragment referenced in [54b](#), [149d](#).
Uses: [MAX_INDENT 122d](#), [putc 21b](#).

Indent will be suppressed if the next character is a newline or if the next two characters are `@#`. If the next two characters are `@<`, we suppress the indent for now but mark that it may be needed when the next fragment is started.

⟨ *Indent suppressed 150b* ⟩ ≡

```
c1 == '\n'
|| (c1 == nw_char && (c2 == '#' || (delayed_indent != (c2 == '<'))))
```

◇
Fragment referenced in [149d](#).
Uses: [delayed_indent 147a](#), [nw_char 29ab](#).

⟨ *Handle tab characters on output 150c* ⟩ ≡

```
{
    if (tab_flag)
        ⟨ Expand tab into spaces 74c ⟩
    else {
        putc('\t', file);
        if (global_indent >= 0) {
            ⟨ Add more indentation '\t' 148 ⟩
        }
        indent++;
    }
}
```

◇
Fragment referenced in [147c](#).
Uses: [putc 21b](#).

⟨ *Check for macro invocation in scrap 150d* ⟩ ≡

```

{
    int oldin = indent;
    char oldcf = comment_flag;
    c = pop(&reader);
    switch (c) {
        case 't': < Copy fragment title into file 153c >
            break;
        case 'c': < Copy block comment from scrap 52c >
            break;
        case 'f': < Copy file name into file 151c >
            break;
        case 'x': < Copy label from scrap into file 151a >
        case '_': break;
        case 'v': < Copy version info into file 76a >
            break;
        case 's': indent = -global_indent;
            comment_flag = 0;
            break;
        case '<': < Copy macro into file 151d >
            < Insert debugging information if required 149a >
            indent = oldin;
            comment_flag = oldcf;
            break;
        < Handle macro parameter substitution 54b >
            indent = oldin;
            break;
        default:
            if(c==nw_char)
            {
                putc(c, file);
                if (global_indent >= 0) {
                    < Add more indentation ' ' 148 >
                }
                indent++;
                break;
            }
            /* ignore, since we should already have a warning */
            break;
    }
}

```

◇
Fragment referenced in 147c.
Uses: nw_char 29ab, pop 141c, putc 21b.

< Copy label from scrap into file 151a > ≡

```

{
    < Get label from (151b pop(&reader) ) 197a >
    write_label(label_name, file);
}

```

◇
Fragment referenced in 150d.
Uses: pop 141c.

< Copy file name into file 151c > ≡

```

if (defs->quoted)
    fprintf(file, "%cf", nw_char);

```

```

else
    fputs(spelling, file);

```

◇

Fragment referenced in [150d](#).

Uses: [defs 131b](#), [fprintf 21b](#), [fputs 21b](#), [nw_char 29ab](#).

⟨ *Copy macro into file 151d* ⟩ ≡

```

{
    Arglist *a = pop_scrap_name(&reader, &local_parameters);
    Name *name = a->name;
    int changed;
    Arglist *args = instance(a->args, inArgs, inParams, &changed);
    int i, narg;
    char * p = name->spelling;
    char * * inParams = name->arg;
    Arglist *q = args;

    if (name->mark) {
        fprintf(stderr, "%s: recursive macro discovered involving <%s>\n",
            command_name, name->spelling);
        exit(-1);
    }
    if (name->defs && !name->quoted) {
        unsigned char saved_comment = comment_flag;
        char saved_location = location_flag;
        char saved_debug = debug_flag;
        if (name->lang[0] != '\0') {
            comment_flag = name->comment_flag;
            location_flag = name->location_flag;
            debug_flag = name->debug_flag;
        }
        if (!newline) {
            comment_flag = 0;
            location_flag = FALSE;
            debug_flag = FALSE;
        }
        ⟨ Perhaps comment this macro 153a ⟩
        name->mark = TRUE;
        indent = write_scraps(file, spelling, name->defs, global_indent + indent,
            indent_chars, debug_flag, tab_flag, indent_flag,
            comment_flag, location_flag, args, name->arg,
            local_parameters, name->spelling);
        comment_flag = saved_comment;
        location_flag = saved_location;
        debug_flag = saved_debug;
        indent -= global_indent;
        name->mark = FALSE;
    }
    else
    {
        if (delayed_indent)
        {
            for (i = indent + global_indent; --i >= 0; )
                putc(' ', file);
        }

        fprintf(file, "%c<", nw_char);
    }
}

```

```

    if (name->sector == 0)
        fputc(' ', file);
    < Comment this macro use 153d >
    fprintf(file, "%c>", nw_char);
    if (!defs->quoted && !tex_flag)
        fprintf(stderr, "%s: macro never defined <%s>\n",
            command_name, name->spelling);
}
}

```

◇ Fragment referenced in 150d.

Uses: Arglist 179c, command_name 29c, defs 131b, delayed_indent 147a, exit 21b, FALSE 23a, fprintf 21b, instance 144b, Name 157c, nw_char 29ab, pop_scrap_name 145d, putc 21b, sector 131b, stderr 21b, tex_flag 27, TRUE 23a, write_scraps 146b.

< Perhaps comment this macro 153a > ≡

```

if (comment_flag && newline) {
    < Perhaps put a delayed indent 153b >
    fputs(comment_begin[comment_flag], file);
    < Comment this macro use 153d >
    if (xref_flag) {
        putc(' ', file);
        write_single_scrap_ref(file, name->defs->scrap);
    }
    fputs(comment_end[comment_flag], file);
    putc('\n', file);
    if (!delayed_indent)
        for (i = indent + global_indent; --i >= 0; )
            putc(' ', file);
}

```

◇ Fragment referenced in 54b, 151d.

Uses: comment_begin 176a, comment_end 176a, defs 131b, delayed_indent 147a, fputs 21b, putc 21b, write_single_scrap_ref 133b, xref_flag 27.

< Perhaps put a delayed indent 153b > ≡

```

if (delayed_indent)
    for (i = indent + global_indent; --i >= 0; )
        putc(' ', file);

```

◇ Fragment referenced in 52c, 153a.

Uses: delayed_indent 147a, putc 21b.

< Copy fragment title into file 153c > ≡

```

{
    char * p = title;
    Arglist *q = inArgs;
    int nargs;

    < Comment this macro use 153d >
    if (xref_flag) {
        putc(' ', file);
        write_single_scrap_ref(file, defs->scrap);
    }
}

```

◇
 Fragment referenced in 150d.
 Uses: Arglist 179c, defs 131b, putc 21b, write_single_scrap_ref 133b, xref_flag 27.

⟨ *Comment this macro use 153d* ⟩ ≡

```

narg = 0;
while (*p != '\000') {
  if (*p == ARG_CHR) {
    if (q == NULL) {
      if (defs->quoted)
        fprintf(file, "%c'%s%c'", nw_char, inParams[narg], nw_char);
      else
        fprintf(file, "'%s'", inParams[narg]);
    }
    else {
      comment_ArglistElement(file, q, defs->quoted);
      q = q->next;
    }
    p++;
    narg++;
  }
  else
    fputc(*p++, file);
}

```

◇
 Fragment referenced in 151d, 153ac.
 Uses: ARG_CHR 178a, comment_ArglistElement 154a, defs 131b, fprintf 21b, next 131a, nw_char 29ab.

⟨ *Forward declarations for scraps.c 154a* ⟩ ≡

```

static void
comment_ArglistElement(FILE * file, Arglist * args, int quote)
{
  Name *name = args->name;
  Arglist *q = args->args;

  if (name == NULL) {
    if (quote)
      fprintf(file, "%c'%s%c'", nw_char, (char *)q, nw_char);
    else
      fprintf(file, "'%s'", (char *)q);
  } else if (name == (Name *)1) {
    ⟨ Include an embedded scrap in comment 154b ⟩
  } else {
    ⟨ Include a fragment use in comment 154c ⟩
  }
}

```

◇
 Fragment defined by 147ab, 154a, 176a, 192c.
 Fragment referenced in 132a.
 Defines: comment_ArglistElement 153d, 154c.
 Uses: Arglist 179c, FILE 21b, fprintf 21b, Name 157c, nw_char 29ab.

⟨ *Include an embedded scrap in comment 154b* ⟩ ≡

```

Embed_Node * e = (Embed_Node *)q;
fputc('{', file);
write_scraps(file, "", e->defs, -1, "", 0, 0, 0, 0, 0, e->args, NULL, NULL, "");

```

```
fputc('}', file);
```

◇

Fragment referenced in [154a](#).

Uses: `defs` [131b](#), `Embed_Node` [183a](#), `write_scraps` [146b](#).

⟨ *Include a fragment use in comment 154c* ⟩ ≡

```
char * p = name->spelling;
if (quote)
    fputc(nw_char, file);
fputc('<', file);
if (quote && name->sector == 0)
    fputc('+', file);
while (*p != '\000') {
    if (*p == ARG_CHR) {
        comment_ArglistElement(file, q, quote);
        q = q->next;
        p++;
    }
    else
        fputc(*p++, file);
}
if (quote)
    fputc(nw_char, file);
fputc('>', file);
```

◇

Fragment referenced in [154a](#).

Uses: `ARG_CHR` [178a](#), `comment_ArglistElement` [154a](#), `next` [131a](#), `nw_char` [29ab](#), `sector` [131b](#).

3.2.1 Collecting Page Numbers

⟨ *Function prototypes 155a* ⟩ ≡

```
extern void collect_numbers(char *aux_name);
```

◇

Fragment defined by [43a](#), [57a](#), [71b](#), [74b](#), [84a](#), [86a](#), [122ab](#), [126a](#), [128a](#), [132b](#), [134b](#), [141b](#), [145a](#), [155a](#), [158c](#), [161a](#), [179a](#), [188c](#), [193c](#), [195a](#), [197c](#), [199d](#).

Fragment referenced in [21a](#).

Uses: `collect_numbers` [155b](#).

"scraps.c" 155b≡

```
void collect_numbers(char *aux_name)
{
    if (number_flag) {
        int i;
        for (i=1; i<scraps; i++)
            scrap_array(i).page = i;
    }
    else {
        FILE *aux_file = fopen(aux_name, "r");
        already_warned = FALSE;
        if (aux_file) {
            char aux_line[500];
            while (fgets(aux_line, 500, aux_file)) {
                ⟨ Read line in .aux file 155c ⟩
            }
            fclose(aux_file);
        }
    }
}
```

```

    }
  }
}

```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: collect_numbers 40b, 155a.
Uses: already_warned 135d, FALSE 23a, fclose 21b, FILE 21b, fopen 21b, number_flag 27, page 131b, scraps 132a, scrap_array 132a.

The *memoir* document class creates references in the .aux file with nested braces, so after each open brace we have to wait for the matching closing brace.

⟨ Read line in .aux file 155c ⟩ ≡

```

int scrap_number;
int page_number;
int i;
int dummy_idx;
int bracket_depth = 1;
if (1 == sscanf(aux_line,
                "\\newlabel{scrap%d}{%n",
                &scrap_number,
                &dummy_idx)) {
for (i = dummy_idx; i < (int)strlen(aux_line) && bracket_depth > 0; i++) {
  if (aux_line[i] == '{') bracket_depth++;
  else if (aux_line[i] == '}') bracket_depth--;
}
if (i > dummy_idx
    && i < (int)strlen(aux_line)
    && 1 == sscanf(aux_line+i, "{%d}" ,&page_number)) {
  if (scrap_number < scraps)
    scrap_array(scrap_number).page = page_number;
  else
    ⟨ Warn (only once) about needing to rerun after Latex 135c ⟩
}
}
}

```

◇
Fragment referenced in 155b.
Uses: page 131b, scraps 132a, scrap_array 132a, strlen 21b.

⟨ Add letters to scraps with duplicate page numbers 156a ⟩ ≡

```

{
  int i = 0;

  ⟨ Step i to the next valid scrap 156b ⟩
  ⟨ For all remaining scraps 156c ⟩ {
    int j = i;
    ⟨ Step j to the next valid scrap 156b ⟩
    ⟨ Perhaps add letters to the page numbers 157a ⟩
    i = j;
  }
}

```

◇
Fragment referenced in 155b.

< Step i to the next valid scrap 156b > ≡

```
do
    i++;
    while (i < scraps && scrap_array(i).page == -1);
```

◇
Fragment referenced in [156a](#).
Uses: [page 131b](#), [scraps 132a](#), [scrap_array 132a](#).

< For all remaining scraps 156c > ≡

```
while (i < scraps)
```

◇
Fragment referenced in [156a](#).
Uses: [scraps 132a](#).

< Perhaps add letters to the page numbers 157a > ≡

```
if (scrap_array(i).page == scrap_array(j).page) {
    if (scrap_array(i).letter == 0)
        scrap_array(i).letter = 'a';
    scrap_array(j).letter = scrap_array(i).letter + 1;
}
```

◇
Fragment referenced in [156a](#).
Uses: [letter 131b](#), [page 131b](#), [scrap_array 132a](#).

3.3 Names

Each scrap is linked through a simple node:

< Type declarations 157b > ≡

```
typedef struct scrap_node {
    struct scrap_node *next;
    int scrap;
    char quoted;
} Scrap_Node;
```

◇
Fragment defined by [23a](#), [54a](#), [122d](#), [157bc](#), [178a](#), [179c](#), [183a](#), [193b](#), [199b](#).
Fragment referenced in [21a](#).
Defines: [Scrap_Node 48b](#), [54b](#), [57b](#), [69a](#), [78ab](#), [80d](#), [83ab](#), [85](#), [86b](#), [94a](#), [97b](#), [98b](#), [100c](#), [102ab](#), [103c](#), [105b](#), [106c](#), [108](#), [110b](#), [112ab](#), [132b](#), [140ab](#), [141a](#), [145b](#), [146b](#), [157c](#), [182c](#), [183ade](#), [191](#), [192cd](#), [194b](#), [195d](#).
Uses: [next 131a](#).

A **Name** represents a file or fragment definition. Beyond the spelling and the binary tree links, several flags control how the tangled output is formatted:

- **debug_flag** — if **TRUE**, emit **#line** directives (set by **-d** or auto-detected for C/C++).
- **comment_flag** — selects the comment style for section markers and block comments: 0=none, 1=C, 2=C++, 3=shell, 4=Lua/SQL, 5=HTML/XML (see [Section 3.3.5](#)).
- **location_flag** — if **TRUE**, emit Knuth-style opening and closing section markers around each scrap in the tangled output. Activated automatically by **@1** auto-detection or the **@L** command.

- **lang** — the language tag (e.g. "c", "python") used for syntax highlighting in the Markdown weave output. Set from a preceding @l command or inferred from the file extension of an @o name. Empty if unknown.

⟨ *Type declarations* 157c ⟩ ≡

```
typedef struct name {
    char *spelling;
    struct name *llink;
    struct name *rlink;
    Scrap_Node *defs;
    Scrap_Node *uses;
    char * arg[9];
    int mark;
    char tab_flag;
    char indent_flag;
    char debug_flag;
    char location_flag;
    char suppress_markers;
    unsigned char comment_flag;
    char lang[64];
    unsigned char sector;
} Name;
```

◇

Fragment defined by 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b.

Fragment referenced in 21a.

Defines: Name 31, 47d, 48a, 54b, 57b, 61c, 62, 63a, 73, 77b, 79, 80b, 81b, 82ac, 84abd, 86b, 89c, 90ab, 94b, 97b, 98b, 100a, 101ab, 103b, 105b, 106c, 108, 109, 110ab, 111ab, 112ab, 113a, 114ab, 115, 122abc, 134b, 135b, 139b, 140c, 141ab, 143c, 144ab, 145b, 151d, 154a, 158abc, 159c, 160b, 162bc, 173a, 176b, 179abcd, 181, 182c, 183d, 184a, 187a, 188d, 189a, 191, 192c, 193ab, 194b, 195d, 196b.

Uses: defs 131b, Scrap_Node 157b, sector 131b, uses 131b.

The variable `pending_lang` acts as a one-shot channel between the @l command in pass1 (which fills it with a language tag like "js" or "python") and `collect_file_name` or `collect_macro_name` (which consume it to set the flags and the `lang` field on the next @o or @d name).

⟨ *Global variable declarations* 158a ⟩ ≡

```
extern Name *file_names;
extern Name *macro_names;
extern Name *user_names;
extern int scrap_name_has_parameters;
extern int scrap_ended_with;
extern char pending_lang[64];
```

◇

Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.

Fragment referenced in 21a.

Defines: file_names 40b, 43c, 53e, 80a, 99, 112a, 113a, 115, 122b, 158b, 173a, macro_names 40b, 43c, 53e, 81a, 100d, 113a, 115, 122b, 158b, 178c, pending_lang 44c, 158b, 175ab, user_names 43c, 53e, 84c, 103a, 140ab, 158b, 188d.

Uses: Name 157c.

⟨ *Global variable definitions* 158b ⟩ ≡

```
Name *file_names = NULL;
Name *macro_names = NULL;
Name *user_names = NULL;
int scrap_name_has_parameters;
int scrap_ended_with;
char pending_lang[64] = "";
```

◇
 Fragment defined by 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b.
 Fragment referenced in 25e.
 Uses: file_names 158a, macro_names 158a, Name 157c, pending_lang 158a, user_names 158a.

⟨ *Function prototypes 158c* ⟩ ≡

```
extern Name *collect_file_name(void);
extern Name *collect_macro_name(void);
extern Arglist *collect_scrap_name(int current_scrap);
extern Name *name_add(Name **rt, char *spelling, unsigned char sector);
extern Name *prefix_add(Name **rt, char *spelling, unsigned char sector);
extern char *save_string(char *s);
extern void reverse_lists(Name *names);
extern void collect_lang_def(void);
extern void collect_weave_format(void);
```

◇
 Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
 Fragment referenced in 21a.
 Uses: Arglist 179c, collect_file_name 173a, collect_lang_def 170, collect_macro_name 176b, collect_scrap_name 180, collect_weave_format 172, Name 157c, name_add 162b, prefix_add 159c, reverse_lists 183d, save_string 159b, sector 131b.

"names.c" 159a≡

```
enum { LESS, GREATER, EQUAL, PREFIX, EXTENSION };

static int compare(char *x, char *y)
{
    int len, result;
    int xl = strlen(x);
    int yl = strlen(y);
    int xp = x[xl - 1] == ' ';
    int yp = y[yl - 1] == ' ';
    if (xp) xl--;
    if (yp) yl--;
    len = xl < yl ? xl : yl;
    result = strncmp(x, y, len);
    if (result < 0) return GREATER;
    else if (result > 0) return LESS;
    else if (xl < yl) {
        if (xp) return EXTENSION;
        else return LESS;
    }
    else if (xl > yl) {
        if (yp) return PREFIX;
        else return GREATER;
    }
    else return EQUAL;
}
```

◇
 File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.
 Defines: compare 159c, 160b, EQUAL 159c, 160b, EXTENSION 159c, 160b, GREATER 159c, 160b, LESS 159c, 160b, PREFIX 159c, 160b.
 Uses: strlen 21b.

"names.c" 159b≡

```

char *save_string(char *s)
{
    char *new_str = (char *) arena_getmem((strlen(s) + 1) * sizeof(char));
    strcpy(new_str, s);
    return new_str;
}

```

◇

File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.

Defines: save_string 36a, 40a, 44b, 129, 136d, 158c, 159c, 162c, 179b, 183c.

Uses: arena_getmem 200b, strlen 21b.

"names.c" 159c≡

```

static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector);

static char * found_name = NULL;

Name *prefix_add(Name **rt, char *spelling, unsigned char sector)
{
    Name *node = *rt;
    int cmp;

    while (node) {
        switch ((cmp = compare(node->spelling, spelling))) {
            case GREATER:    rt = &node->rlink;
                            break;
            case LESS:      rt = &node->llink;
                            break;
            case EQUAL:
                            found_name = node->spelling;
                            FALLTHROUGH;
            case EXTENSION: if (node->sector > sector) {
                            rt = &node->rlink;
                            break;
                        }
                            else if (node->sector < sector) {
                            rt = &node->llink;
                            break;
                        }
                            if (cmp == EXTENSION)
                                node->spelling = save_string(spelling);
                                return node;
            case PREFIX:    < Check for ambiguous prefix 160a >
                            return node;
        }
        node = *rt;
    }
    < Create new name entry 162c >
}

```

◇

File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.

Defines: prefix_add 158c, 178c.

Uses: compare 159a, EQUAL 159a, EXTENSION 159a, FALLTHROUGH 23a, GREATER 159a, LESS 159a, Name 157c, PREFIX 159a, save_string 159b, sector 131b.

Since a very short prefix might match more than one fragment name, I need to check for other matches to avoid mistakes. Basically, I simply continue the search down *both* branches of the tree.

⟨ *Check for ambiguous prefix 160a* ⟩ ≡

```
{
    if (ambiguous_prefix(node->llink, spelling, sector) ||
        ambiguous_prefix(node->rlink, spelling, sector))
        fprintf(stderr,
            "%s: ambiguous prefix %c<%s...%c> (%s, line %d)\n",
            command_name, nw_char, spelling, nw_char, source_name, source_line);
}
```

◇

Fragment referenced in 159c.

Uses: `command_name` 29c, `fprintf` 21b, `nw_char` 29ab, `sector` 131b, `source_line` 126b, `source_name` 126b, `stderr` 21b.

"names.c" 160b≡

```
static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector)
{
    while (node) {
        switch (compare(node->spelling, spelling)) {
            case GREATER:    node = node->rlink;
                            break;
            case LESS:      node = node->llink;
                            break;
            case EXTENSION:
            case PREFIX:
            case EQUAL:     if (node->sector > sector) {
                            node = node->rlink;
                            break;
                        }
                        else if (node->sector < sector) {
                            node = node->llink;
                            break;
                        }
                        return TRUE;
        }
    }
    return FALSE;
}
```

◇

File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.

Uses: `compare` 159a, `EQUAL` 159a, `EXTENSION` 159a, `FALSE` 23a, `GREATER` 159a, `LESS` 159a, `Name` 157c, `PREFIX` 159a, `sector` 131b, `TRUE` 23a.

Rob Shillingsburg suggested that I organize the index of user-specified identifiers more traditionally; that is, not relying on strict ASCII comparisons via `strcmp`. Ideally, we'd like to see the index ordered like this:

```
aardvark
Adam
atom
Atomic
atoms
```

The function `robs_strcmp` implements the desired predicate. It returns -2 for alphabetically less-than, -1 for less-than but only differing in case, zero for equal, 1 for greater-than but only in case and 2 for alphabetically greater-than.

⟨ *Function prototypes 161a* ⟩ ≡

```
extern int robs_strcmp(char*, char*);
```

◇
 Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
 Fragment referenced in 21a.
 Uses: `robs_strcmp` 161b.

"names.c" 161b≡

```
int robs_strcmp(char* x, char* y)
{
    int cmp = 0;

    for (; *x && *y; x++, y++)
    {
        < Skip invisibles on x 162a >
        < Skip invisibles on y 162a >
        if (*x == *y)
            continue;
        if (islower(*x) && toupper(*x) == *y)
        {
            if (!cmp) cmp = 1;
            continue;
        }
        if (islower(*y) && *x == toupper(*y))
        {
            if (!cmp) cmp = -1;
            continue;
        }
        return 2*(toupper(*x) - toupper(*y));
    }
    if (*x)
        return 2;
    if (*y)
        return -2;
    return cmp;
}
```

◇
 File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.
 Defines: `robs_strcmp` 82b, 110a, 161a, 162bc, 193a.
 Uses: `islower` 21b, `toupper` 21b.

Certain character sequences are invisible when printed. We don't want them to be considered for the alphabetical ordering.

< Skip invisibles on p 162a > ≡

```
if (*p == '|')
    p++;
```

◇
 Fragment referenced in 161b.

"names.c" 162b≡

```
Name *name_add(Name **rt, char *spelling, unsigned char sector)
{
    Name *node = *rt;
    while (node) {
        int result = robs_strcmp(node->spelling, spelling);
```

```

    if (result > 0)
        rt = &node->llink;
    else if (result < 0)
        rt = &node->rlink;
    else
    {
        found_name = node->spelling;
        if (node->sector > sector)
            rt = &node->llink;
        else if (node->sector < sector)
            rt = &node->rlink;
        else
            return node;
    }
    node = *rt;
}
⟨ Create new name entry 162c ⟩
}

```

◇
File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.
Defines: name_add 140ab, 158c, 173a.
Uses: Name 157c, robs_strcmp 161b, sector 131b.

⟨ Create new name entry 162c ⟩ ≡

```

{
    node = (Name *) arena_getmem(sizeof(Name));
    if (found_name && robs_strcmp(found_name, spelling) == 0)
        node->spelling = found_name;
    else
        node->spelling = save_string(spelling);
    node->mark = FALSE;
    node->llink = NULL;
    node->rlink = NULL;
    node->uses = NULL;
    node->defs = NULL;
    node->arg[0] =
    node->arg[1] =
    node->arg[2] =
    node->arg[3] =
    node->arg[4] =
    node->arg[5] =
    node->arg[6] =
    node->arg[7] =
    node->arg[8] = NULL;
    node->tab_flag = TRUE;
    node->indent_flag = TRUE;
    node->debug_flag = FALSE;
    node->location_flag = FALSE;
    node->suppress_markers = FALSE;
    node->comment_flag = 0;
    node->lang[0] = '\0';
    node->sector = sector;
    *rt = node;
    return node;
}

```

◇
Fragment referenced in 159c, 162b.

Uses: arena_getmem 200b, defs 131b, FALSE 23a, Name 157c, robs_strcmp 161b, save_string 159b, sector 131b, TRUE 23a, uses 131b.

3.3.1 Language Auto-Detection

When a Markdown fence (‘‘‘c, ‘‘‘python) or a L^AT_EX marker (%%c) precedes an @o directive, weft auto-detects the language’s comment style and, for C/C++, enables #line directives. This is driven by a data table that maps fence tags to one of the six comment styles defined in Section 3.3.5.

Each entry records:

- **name** — the fence tag (lowercase, e.g. "js", "python", "lua").
- **comment** — index into the comment_begin/comment_end arrays (1–5).
- **line_directive** — TRUE only for languages whose toolchains honour #line (C family).

The built-in table covers over 250 entries (languages, file extensions, and aliases) organised by comment style. For any language not listed, the @L command (Section 3.3.2) lets users add entries at the document level. User-defined entries are searched first, so they can also override built-in defaults. If a fence tag matches neither table, weft defaults to C++ line comments (//) with no #line.

"names.c" 163≡

```
typedef struct {
    const char *name;
    unsigned char comment;
    char line_directive;
} LangEntry;

static const LangEntry lang_table[] = {
    /* ---- Style 1: C block comments WITH #line ---- */
    {"c",          1, TRUE},
    {"h",          1, TRUE},
    {"hpp",        2, TRUE},
    {"hxx",        2, TRUE},
    {"hh",         2, TRUE},
    {"cpp",        2, TRUE},
    {"c++",        2, TRUE},
    {"cxx",        2, TRUE},
    {"cc",         2, TRUE},
    {"objc",       1, TRUE},
    {"objcpp",     2, TRUE},
    {"m",          1, TRUE},
    {"mm",         2, TRUE},
    {"ino",        2, TRUE},    /* Arduino */
    {"pde",       2, TRUE},    /* Processing (C-like) */
    /* ---- Style 1: C block comments WITHOUT #line ---- */
    {"css",        1, FALSE},
    {"scss",       1, FALSE},
    {"sass",       1, FALSE},
    {"less",       1, FALSE},
    {"stylus",     1, FALSE},
    {"styl",       1, FALSE},
    /* ---- Style 2: C++ line comments (//) ---- */
    /* --- JavaScript / TypeScript --- */
    {"js",         2, FALSE},
    {"javascript", 2, FALSE},
    {"jsx",        2, FALSE},
```

```

{"mjs",          2, FALSE},
{"cjs",          2, FALSE},
{"ts",           2, FALSE},
{"typescript",   2, FALSE},
{"tsx",          2, FALSE},
{"mts",          2, FALSE},
{"cts",          2, FALSE},
{"svelte",       2, FALSE},
{"vue",          2, FALSE},
/* --- JVM languages --- */
{"java",         2, FALSE},
{"scala",        2, FALSE},
{"sc",           2, FALSE},
{"kotlin",       2, FALSE},
{"kt",           2, FALSE},
{"kts",          2, FALSE},
{"groovy",       2, FALSE},
{"gradle",       2, FALSE},
{"clj",          2, FALSE}, /* Clojure (;; but // is close enough) */
{"cljs",         2, FALSE},
{"cljc",         2, FALSE},
/* --- .NET --- */
{"cs",           2, FALSE},
{"csharp",       2, FALSE},
{"fs",           2, FALSE},
{"fsharp",       2, FALSE},
{"fsx",          2, FALSE},
{"vb",           2, FALSE}, /* VB.NET uses ' but // is a safe fallback */
/* --- Systems languages --- */
{"go",           2, FALSE},
{"rust",         2, FALSE},
{"rs",           2, FALSE},
{"zig",          2, FALSE},
{"d",            2, FALSE},
{"v",            2, FALSE},
{"vlang",        2, FALSE},
{"swift",        2, FALSE},
{"dart",         2, FALSE},
{"odin",         2, FALSE},
{"jai",          2, FALSE},
{"carbon",       2, FALSE},
{"mojo",         2, FALSE},
/* --- Web / general --- */
{"php",          2, FALSE},
{"hack",         2, FALSE},
{"jsonc",        2, FALSE},
{"json5",        2, FALSE},
{"graphql",      2, FALSE},
{"gql",          2, FALSE},
{"prisma",       2, FALSE},
{"wgs1",         2, FALSE}, /* WebGPU shading language */
/* --- GPU / shaders --- */
{"glsl",         2, FALSE},
{"hlsl",         2, FALSE},
{"vert",         2, FALSE},
{"frag",         2, FALSE},
{"comp",         2, FALSE},
{"geom",         2, FALSE},

```

```

{"tesc",      2, FALSE},
{"tese",      2, FALSE},
{"metal",     2, FALSE},
{"cu",        2, TRUE},  /* CUDA: supports #line */
{"cuh",       2, TRUE},
{"cl",        1, FALSE}, /* OpenCL: C-style block comments */
/* --- Protocol / IDL / blockchain --- */
{"proto",     2, FALSE},
{"protobuf",  2, FALSE},
{"thrift",    2, FALSE},
{"flatbuffers", 2, FALSE},
{"fbs",       2, FALSE},
{"capnp",     2, FALSE},
{"avdl",      2, FALSE}, /* Avro IDL */
{"solidity",  2, FALSE},
{"sol",       2, FALSE},
{"vyper",     3, FALSE},
{"vy",        3, FALSE},
{"move",      2, FALSE},
/* --- Functional (// style) --- */
{"reason",    2, FALSE},
{"re",        2, FALSE},
{"res",       2, FALSE}, /* ReScript */
{"ml",        2, FALSE}, /* OCaml also uses (**) but // in comments */
{"mli",       2, FALSE},
{"sml",       2, FALSE},
{"sig",       2, FALSE},
{"fut",       2, FALSE}, /* Futhark */
/* --- Other // languages --- */
{"actionscript", 2, FALSE},
{"as",          2, FALSE},
{"apex",        2, FALSE},
{"bicep",       2, FALSE},
{"ceu",         2, FALSE},
{"chapel",      2, FALSE},
{"chpl",       2, FALSE},
{"cilk",        2, TRUE},
{"jsonnet",    2, FALSE},
{"libsonnet",  2, FALSE},
{"p4",         2, FALSE},
{"qml",        2, FALSE},
{"slang",      2, FALSE},
/* ---- Style 3: Shell / hash comments (#) ---- */
/* --- Python --- */
{"py",         3, FALSE},
{"python",     3, FALSE},
{"pyx",        3, FALSE}, /* Cython */
{"pxd",        3, FALSE},
{"pyi",        3, FALSE}, /* type stubs */
{"sage",       3, FALSE}, /* SageMath */
/* --- Shell --- */
{"sh",         3, FALSE},
{"bash",       3, FALSE},
{"zsh",        3, FALSE},
{"fish",       3, FALSE},
{"ksh",        3, FALSE},
{"csh",        3, FALSE},
{"tcsh",       3, FALSE},

```

```

{"dash",          3, FALSE},
{"nu",            3, FALSE}, /* Nushell */
/* --- Classic scripting --- */
{"perl",          3, FALSE},
{"pl",            3, FALSE},
{"pm",            3, FALSE},
{"ruby",          3, FALSE},
{"rb",            3, FALSE},
{"rake",          3, FALSE},
{"gemspec",       3, FALSE},
{"r",             3, FALSE},
{"rmd",           3, FALSE},
{"tcl",           3, FALSE},
{"awk",           3, FALSE},
{"sed",           3, FALSE},
/* --- Build / make --- */
{"makefile",      3, FALSE},
{"make",          3, FALSE},
{"mk",            3, FALSE},
{"cmake",         3, FALSE},
{"just",          3, FALSE},
{"justfile",      3, FALSE},
{"meson",         3, FALSE},
{"scons",         3, FALSE},
{"bazel",         3, FALSE},
{"bzl",           3, FALSE},
{"buck",          3, FALSE},
{"pants",         3, FALSE},
{"gnumakefile",   3, FALSE},
/* --- Scientific / modern --- */
{"julia",         3, FALSE},
{"jl",            3, FALSE},
{"nim",           3, FALSE},
{"nimble",        3, FALSE},
{"crystal",       3, FALSE},
{"cr",            3, FALSE},
{"elixir",        3, FALSE},
{"ex",            3, FALSE},
{"exs",           3, FALSE},
{"coco",          3, FALSE}, /* Coconut (Python superset) */
{"coffee",       3, FALSE}, /* CoffeeScript */
{"litcoffee",     3, FALSE},
/* --- Config / DevOps --- */
{"yaml",          3, FALSE},
{"yml",           3, FALSE},
{"toml",          3, FALSE},
{"dockerfile",    3, FALSE},
{"docker",        3, FALSE},
{"containerfile", 3, FALSE},
{"terraform",     3, FALSE},
{"tf",            3, FALSE},
{"tfvars",        3, FALSE},
{"hcl",           3, FALSE},
{"nix",           3, FALSE},
{"conf",          3, FALSE},
{"ini",           3, FALSE},
{"properties",    3, FALSE},
{"env",           3, FALSE},

```

```

{"editorconfig", 3, FALSE},
{"gitignore",    3, FALSE},
{"dockerignore", 3, FALSE},
{"flake8",       3, FALSE},
{"pylintrc",     3, FALSE},
{"powershell",   3, FALSE},
{"ps1",          3, FALSE},
{"psm1",         3, FALSE},
{"psd1",         3, FALSE},
/* --- Hardware description (# style) --- */
{"tcl",          3, FALSE},
{"xdc",          3, FALSE}, /* Xilinx constraints */
{"sdc",          3, FALSE}, /* Synopsys constraints */
/* --- Data / query (# style) --- */
{"gn",           3, FALSE}, /* GN build */
{"cfg",          3, FALSE},
{"pip",          3, FALSE},
{"cask",         3, FALSE}, /* Homebrew */
{"gemfile",      3, FALSE},
{"vagrantfile",  3, FALSE},
{"procfile",     3, FALSE},
/* ---- Style 4: double-dash comments (--) ---- */
{"lua",          4, FALSE},
{"luau",         4, FALSE}, /* Roblox Lua */
{"sql",          4, FALSE},
{"psql",         4, FALSE},
{"plsql",        4, FALSE},
{"plpgsql",      4, FALSE},
{"mysql",        4, FALSE},
{"sqlite",       4, FALSE},
{"hive",         4, FALSE},
{"presto",       4, FALSE},
{"sparksql",     4, FALSE},
{"hs",           4, FALSE},
{"haskell",     4, FALSE},
{"lhs",          4, FALSE},
{"elm",          4, FALSE},
{"purescript",   4, FALSE},
{"purs",         4, FALSE},
{"ada",          4, FALSE},
{"adb",          4, FALSE},
{"ads",          4, FALSE},
{"vhdl",         4, FALSE},
{"vhd",          4, FALSE},
{"lean",         4, FALSE},
{"lean4",        4, FALSE},
{"agda",         4, FALSE},
{"idr",          4, FALSE}, /* Idris */
{"idris",        4, FALSE},
{"erl",          4, FALSE}, /* Erlang uses % but -- is common in embedded SQL */
{"erlang",       4, FALSE},
/* ---- Style 5: HTML comments (<!-- -->) ---- */
{"html",         5, FALSE},
{"htm",          5, FALSE},
{"xml",          5, FALSE},
{"svg",          5, FALSE},
{"xhtml",        5, FALSE},
{"xsl",          5, FALSE},

```

```

{"xslt",      5, FALSE},
{"xsd",      5, FALSE},
{"dtd",      5, FALSE},
{"rss",      5, FALSE},
{"atom",     5, FALSE},
{"mathml",   5, FALSE},
{"plist",    5, FALSE},
{"xaml",     5, FALSE},
{"cspj",     5, FALSE},
{"fsproj",   5, FALSE},
{"vbproj",   5, FALSE},
{"vcxproj",  5, FALSE},
{"sln",      5, FALSE},
{"props",    5, FALSE},
{"targets",  5, FALSE},
{"nuspec",   5, FALSE},
{"resx",     5, FALSE},
{"wxml",     5, FALSE},
{"axml",     5, FALSE},
{"jspx",     5, FALSE},
{"jsp",      5, FALSE},
{"asp",      5, FALSE},
{"ascx",     5, FALSE},
{"aspx",     5, FALSE},
{"cshtml",   5, FALSE}, /* Razor */
{"vbhtml",   5, FALSE},
{"erb",      5, FALSE},
{"ejs",      5, FALSE},
{"njk",      5, FALSE}, /* Nunjucks */
{"hbs",      5, FALSE}, /* Handlebars */
{"mustache", 5, FALSE},
{"liquid",   5, FALSE},
{"twig",     5, FALSE},
{"blade",    5, FALSE}, /* Laravel Blade */
{"pug",      5, FALSE}, /* Pug/Jade */
{"slim",     5, FALSE},
{"haml",     5, FALSE},
{"md",       5, FALSE}, /* Markdown: HTML comments are valid */
{"markdown", 5, FALSE},
{"mdx",      5, FALSE},
{"rst",      5, FALSE}, /* reStructuredText (... is native but <!-- works) */
{"tex",      5, FALSE}, /* LaTeX: % is native but no style for it */
{"latex",    5, FALSE},
/* ---- Style 0: no comments (data formats) ---- */
{"json",     0, FALSE},
{"json5",    0, FALSE},
{"csv",      0, FALSE},
{"tsv",      0, FALSE},
{"txt",      0, FALSE},
{"text",     0, FALSE},
{"log",      0, FALSE},
{"diff",     0, FALSE},
{"patch",    0, FALSE},
/* Sentinel */
{NULL,      0, FALSE}
};

/* User-defined languages via @L command */

```

```

#define MAX_USER_LANGS 32
static LangEntry user_langs[MAX_USER_LANGS];
static char user_lang_names[MAX_USER_LANGS][64];
static int user_lang_count = 0;

static const LangEntry *find_lang(const char *name)
{
    int i;
    const LangEntry *p;
    /* User-defined languages override built-in */
    for (i = 0; i < user_lang_count; i++)
        if (strcmp(user_langs[i].name, name) == 0) return &user_langs[i];
    for (p = lang_table; p->name; p++)
        if (strcmp(p->name, name) == 0) return p;
    return NULL;
}

static void lang_to_flags(const char *lang,
                        unsigned char *comment_flag,
                        char *debug_flag,
                        char *location_flag)
{
    const LangEntry *e = find_lang(lang);
    if (e) {
        *comment_flag = e->comment;
        *debug_flag = e->line_directive;
        *location_flag = (e->comment != 0);
    } else {
        /* Unknown language: default to // comments, no #line */
        *comment_flag = 2;
        *debug_flag = FALSE;
        *location_flag = TRUE;
    }
}

```

◇ File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.
 Defines: find_lang 175a, LangEntry 175a, lang_table Never used, lang_to_flags 175ab, user_langs 170, user_lang_count 170, user_lang_names 170.
 Uses: FALSE 23a, TRUE 23a, uses 131b.

3.3.2 User-Defined Languages

The @L command lets users register new language mappings (or override built-in ones) directly in the document preamble. This is useful for languages that are not in the built-in table, or when the default comment style for a language is not what the user wants.

Syntax: @L *name style* [+d]

- *name* — the fence tag that will trigger this mapping (e.g. `nim`, `glsl`, `wgsl`).
- *style* — one of `//`, `#`, `--`, `/*`, or `<!--`, selecting the comment delimiters.
- +d — optional; if present, enables `#line` directive generation for this language.

Examples:

```

@L nim #           % Nim uses # comments
@L glsl // +d      % GLSL uses // and supports #line
@L c //           % override: use // instead of /* */ for C

```

Up to 32 user-defined languages may be registered. They are searched before the built-in table, so a definition like `@L c //` effectively overrides the built-in mapping for C.

The `@L` command is processed during pass1 and silently skipped during weaving. It should appear in the document preamble (before any `@o` or `@d` directives that depend on it).

"names.c" 170≡

```
void collect_lang_def(void)
{
    char name[64];
    char *p = name;
    char style[16];
    char *sp;
    int c;
    unsigned char comment = 0;
    char line_dir = FALSE;

    /* Skip whitespace after @L */
    c = source_get();
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read language name */
    while (c != EOF && c != ' ' && c != '\t' && c != '\n'
           && (p - name) < 62) {
        *p++ = c;
        c = source_get();
    }
    *p = '\0';

    if (name[0] == '\0') {
        fprintf(stderr, "%s: expected language name after %cL (%s, %d)\n",
                command_name, nw_char, source_name, source_line);
        while (c != EOF && c != '\n') c = source_get();
        return;
    }

    /* Skip whitespace */
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read comment style token */
    sp = style;
    while (c != EOF && c != ' ' && c != '\t' && c != '\n'
           && (sp - style) < 14) {
        *sp++ = c;
        c = source_get();
    }
    *sp = '\0';

    /* Map style string to comment index */
    if (strcmp(style, "//") == 0)        comment = 2;
    else if (strcmp(style, "#") == 0)    comment = 3;
    else if (strcmp(style, "--") == 0)   comment = 4;
    else if (strcmp(style, "/*") == 0)   comment = 1;
    else if (strcmp(style, "<!--") == 0)  comment = 5;
    else {
        fprintf(stderr,
                "%s: unknown comment style '%s' for %cL (%s, %d)\n",
```

```

        command_name, style, nw_char, source_name, source_line);
fprintf(stderr, " Valid styles: // # -- /* <!--\n");
while (c != EOF && c != '\n') c = source_get();
return;
}

/* Check for optional +d flag (#line directives) */
while (c == ' ' || c == '\t')
    c = source_get();
if (c == '+') {
    c = source_get();
    if (c == 'd') line_dir = TRUE;
}

/* Consume rest of line */
while (c != EOF && c != '\n') c = source_get();

/* Register the user language */
if (user_lang_count < MAX_USER_LANGS) {
    snprintf(user_lang_names[user_lang_count],
             sizeof(user_lang_names[user_lang_count]), "%s", name);
    user_langs[user_lang_count].name = user_lang_names[user_lang_count];
    user_langs[user_lang_count].comment = comment;
    user_langs[user_lang_count].line_directive = line_dir;
    user_lang_count++;
} else {
    fprintf(stderr, "%s: too many %cL definitions (max %d) (%s, %d)\n",
            command_name, nw_char, MAX_USER_LANGS,
            source_name, source_line);
}
}
}

```

◇

File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.

Defines: collect_lang_def 45a, 158c.

Uses: command_name 29c, FALSE 23a, fprintf 21b, nw_char 29ab, source_get 127c, source_line 126b, source_name 126b, stderr 21b, TRUE 23a, user_langs 163, user_lang_count 163, user_lang_names 163.

3.3.3 Weave Format Declaration

The `@W` command declares the weave output format in the source file. Valid tokens are `md` (Markdown) and `tex` (`LATEX`). If the user has already specified a format on the command line (`-w md` or `-w tex`), the declaration is silently ignored.

"names.c" 172≡

```

void collect_weave_format(void)
{
    char token[64];
    char *p = token;
    int c;

    /* Skip whitespace after @W */
    c = source_get();
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read format token */
    while (c != EOF && c != ' ' && c != '\t' && c != '\n')

```

```

        && (p - token) < 62) {
    *p++ = c;
    c = source_get();
}
*p = '\0';

/* Consume rest of line */
while (c != EOF && c != '\n') c = source_get();

if (token[0] == '\0') {
    fprintf(stderr, "%s: expected format after %cW (md or tex) (%s, %d)\n",
        command_name, nw_char, source_name, source_line);
    return;
}

/* If CLI already set the format, skip silently */
if (weave_format_from_cli)
    return;

if (strcmp(token, "md") == 0)
    weave_format = 2;
else if (strcmp(token, "tex") == 0)
    weave_format = 1;
else {
    fprintf(stderr, "%s: unknown weave format '%s' for %cW (%s, %d)\n",
        command_name, token, nw_char, source_name, source_line);
    fprintf(stderr, "  Valid formats: md tex\n");
}
}
}

```

◇

File defined by [25c](#), [159abc](#), [160b](#), [161b](#), [162b](#), [163](#), [170](#), [172](#), [173a](#), [176b](#), [179bd](#), [180](#), [183de](#).

Defines: [collect_weave_format](#) [45b](#), [158c](#).

Uses: [command_name](#) [29c](#), [fprintf](#) [21b](#), [nw_char](#) [29ab](#), [source_get](#) [127c](#), [source_line](#) [126b](#), [source_name](#) [126b](#), [stderr](#) [21b](#), [weave_format](#) [27](#), [weave_format_from_cli](#) [27](#).

3.3.4 Collecting File and Fragment Names

An output file name (@o) is terminated by whitespace. After reading the name we check for optional per-file flags (-d, -t, -i, -c) and then, if no explicit comment or debug flags were given, apply any pending language defaults from a preceding fence, %%% marker, or @L definition.

"names.c" 173a≡

```

Name *collect_file_name(void)
{
    Name *new_name;
    char name[MAX_NAME_LEN];
    char *p = name;
    int start_line = source_line;
    int c = source_get(), c2;
    while (isspace(c))
        c = source_get();
    while (isgraph(c)) {
        *p++ = c;
        c = source_get();
    }
    if (p == name) {
        fprintf(stderr, "%s: expected file name (%s, %d)\n",

```

```

        command_name, source_name, start_line);
    exit(-1);
}
*p = '\0';
/* File names are always global. */
new_name = name_add(&file_names, name, 0);
⟨ Handle optional per-file flags 173b ⟩
⟨ Apply pending language if no explicit flags 175a ⟩
c2 = source_get();
if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
    fprintf(stderr, "%s: expected %c{, %c[, or %c( after file name (%s, %d)\n",
        command_name, nw_char, nw_char, nw_char, source_name, start_line);
    exit(-1);
}
return new_name;
}

```

◇

File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.

Defines: collect_file_name 47d, 62, 90a, 158c.

Uses: command_name 29c, exit 21b, file_names 158a, fprintf 21b, isgraph 21b, isspace 21b, MAX_NAME_LEN 23b, Name 157c, name_add 162b, nw_char 29ab, source_get 127c, source_line 126b, source_name 126b, stderr 21b.

⟨ Handle optional per-file flags 173b ⟩ ≡

```

{
    while (1) {
        while (isspace(c))
            c = source_get();
        if (c == '-') {
            c = source_get();
            do {
                switch (c) {
                    case 't': new_name->tab_flag = FALSE;
                        break;
                    case 'd': new_name->debug_flag = TRUE;
                        break;
                    case 'i': new_name->indent_flag = FALSE;
                        break;
                    case 's': new_name->suppress_markers = TRUE;
                        break;
                    case 'c': ⟨ Get comment delimiters 174 ⟩
                        break;
                    default : fprintf(stderr, "%s: unexpected per-file flag (%s, %d)\n",
                        command_name, source_name, source_line);
                        break;
                }
                c = source_get();
            } while (!isspace(c));
        }
        else break;
    }
}

```

◇

Fragment referenced in 173a.

Uses: command_name 29c, FALSE 23a, fprintf 21b, isspace 21b, source_get 127c, source_line 126b, source_name 126b, stderr 21b, TRUE 23a.

The explicit per-file comment flags (-cc, -c+, -cp) select among the first three comment styles. These

flags predate the auto-detection system; they still work and take precedence over any fence-based or @L-based setting.

The `-s` flag suppresses `#line` directives and section markers in the tangled output for that file, while still allowing language auto-detection to set the comment style and the `lang` tag (used by the Markdown weaver for syntax highlighting classes). This is useful for bootstrap source files or any output where tangle markers are unwanted.

⟨ *Get comment delimiters* 174 ⟩ ≡

```
c = source_get();
if (c == 'c')
    new_name->comment_flag = 1;
else if (c == '+')
    new_name->comment_flag = 2;
else if (c == 'p')
    new_name->comment_flag = 3;
else
    fprintf(stderr, "%s: Unrecognised comment flag (%s, %d)\n",
            command_name, source_name, source_line);
```

◇
Fragment referenced in 173b.

Uses: `command_name` 29c, `fprintf` 21b, `source_get` 127c, `source_line` 126b, `source_name` 126b, `stderr` 21b.

If there is a pending language tag from a fence or `%%` marker, and the user has not specified explicit comment or debug flags, we apply the language defaults automatically.

⟨ *Apply pending language if no explicit flags* 175a ⟩ ≡

```
if (pending_lang[0] != '\0') {
    strncpy(new_name->lang, pending_lang, 63);
    new_name->lang[63] = '\0';
    if (new_name->comment_flag == 0 && !new_name->debug_flag) {
        lang_to_flags(pending_lang,
                      &new_name->comment_flag,
                      &new_name->debug_flag,
                      &new_name->location_flag);
    }
    pending_lang[0] = '\0';
} else if (new_name->lang[0] == '\0') {
    const char *dot = strrchr(new_name->spelling, '.');
    if (dot && dot[1]) {
        const LangEntry *e = find_lang(dot + 1);
        if (e) {
            strncpy(new_name->lang, dot + 1, 63);
            new_name->lang[63] = '\0';
            if (new_name->comment_flag == 0 && !new_name->debug_flag) {
                lang_to_flags(new_name->lang,
                              &new_name->comment_flag,
                              &new_name->debug_flag,
                              &new_name->location_flag);
            }
        }
    }
}
if (new_name->suppress_markers) {
    new_name->debug_flag = FALSE;
    new_name->location_flag = FALSE;
}
```

◇
 Fragment referenced in 173a.
 Uses: FALSE 23a, find_lang 163, LangEntry 163, lang_to_flags 163, pending_lang 158a.

For @d (macro) definitions, the language tag is stored and flags are inferred from @l, just as for output files. At expansion time, if a fragment carries an explicit @l tag its own flags override the parent's—so @l txt before @d suppresses markers and newlines, enabling true inline expansion.

⟨ Apply pending language to macro 175b ⟩ ≡

```

if (pending_lang[0] != '\0') {
    strncpy(node->lang, pending_lang, 63);
    node->lang[63] = '\0';
    if (node->comment_flag == 0 && !node->debug_flag) {
        lang_to_flags(pending_lang,
                     &node->comment_flag,
                     &node->debug_flag,
                     &node->location_flag);
    }
    pending_lang[0] = '\0';
}

```

◇
 Fragment referenced in 177a, 178d.
 Uses: lang_to_flags 163, pending_lang 158a.

3.3.5 Comment Styles

weft supports six comment styles, indexed 0–5. The arrays below provide the opening, continuation, and closing delimiters for each. Style 0 means “no comments”; the remaining five cover the vast majority of programming languages:

Index	Style	Begin	Mid	End
0	none	(empty)	(empty)	(empty)
1	C	/*	*	*/
2	C++	//	//	(empty)
3	Shell	#	#	(empty)
4	Lua/SQL	--	--	(empty)
5	HTML	<!--	(spaces)	-->

These are used both for block comments (@c) and for the Knuth-style section markers generated by the language auto-detection system.

⟨ Forward declarations for scraps.c 176a ⟩ ≡

```

char * comment_begin[6] = { "", "/* ", "// ", "# ", "-- ", "<!-- "};
char * comment_mid[6]   = { "", " * ", "// ", "# ", "-- ", "    "};
char * comment_end[6]   = { "", " */", "", "", "", " -->"};

```

◇
 Fragment defined by 147ab, 154a, 176a, 192c.
 Fragment referenced in 132a.
 Defines: comment_begin 52c, 149bc, 153a, comment_end 52c, 149bc, 153a, comment_mid 52c.

Name terminated by \n or @{; but keep skipping until @{

"names.c" 176b≡

```

Name *collect_macro_name(void)
{
    char name[MAX_NAME_LEN];
    char args[1000];
    char * arg[9];
    char * argp = args;
    int argc = 0;
    char *p = name;
    int start_line = source_line;
    int c = source_get(), c2;
    unsigned char sector = current_sector;

    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    while (isspace(c))
        c = source_get();
    while (c != EOF) {
        Name * node;
        switch (c) {
            case '\t':
            case ' ': *p++ = ' ';
                        do
                            c = source_get();
                        while (c == ' ' || c == '\t');
                        break;
            case '\n': < Skip until scrap begins, then return name 178d >
            default:
                if (c==nw_char)
                {
                    < Check for terminating at-sequence and return name 177a >
                    break;
                }
                *p++ = c;
                c = source_get();
                break;
        }
    }
    fprintf(stderr, "%s: expected fragment name (%s, %d)\n",
            command_name, source_name, start_line);
    exit(-1);
    return NULL; /* unreachable return to avoid warnings on some compilers */
}

```

◇ File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.
 Defines: collect_macro_name 48a, 63a, 90b, 158c.
 Uses: command_name 29c, current_sector 47b, exit 21b, fprintf 21b, isspace 21b, MAX_NAME_LEN 23b, Name 157c,
 nw_char 29ab, sector 131b, source_get 127c, source_line 126b, source_name 126b, stderr 21b.

< Check for terminating at-sequence and return name 177a > ≡

```

{
    c = source_get();
    switch (c) {
        case '(':
        case '[':
        case '{': < Cleanup and install name 178c >

```

```

        < Apply pending language to macro 175b >
        return install_args(node, argc, arg);
    case '\\': < Enter the next argument 177b >
        break;
    default:
        if (c==nw_char)
        {
            *p++ = c;
            break;
        }
        fprintf(stderr,
            "%s: unexpected %c%c in fragment definition name (%s, %d)\n",
            command_name, nw_char, c, source_name, start_line);
        exit(-1);
    }
}

```

◇

Fragment referenced in 176b.

Uses: command_name 29c, exit 21b, fprintf 21b, nw_char 29ab, source_get 127c, source_name 126b, stderr 21b.

< Enter the next argument 177b > ≡

```

    arg[argc] = argp;
    while ((c = source_get()) != EOF) {
        if (c==nw_char) {
            c2 = source_get();
            if (c2=='\\') {
                < Make this argument 178b >
                c = source_get();
                break;
            }
            else
                *argp++ = c2;
        }
        else
            *argp++ = c;
    }
    *p++ = ARG_CHR;

```

◇

Fragment referenced in 177a.

Uses: ARG_CHR 178a, nw_char 29ab, source_get 127c.

< Type declarations 178a > ≡

```

#define ARG_CHR '\\001'

```

◇

Fragment defined by 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b.

Fragment referenced in 21a.

Defines: ARG_CHR 63b, 77b, 79, 91a, 97b, 98b, 145b, 153d, 154c, 177b, 181, 182b.

< Make this argument 178b > ≡

```

    if (argc < 9) {
        *argp++ = '\\000';
        argc += 1;
    }

```

◇

Fragment referenced in 177b.

⟨ Cleanup and install name 178c ⟩ ≡

```
{
  if (p > name && p[-1] == ' ')
    p--;
  if (p - name > 3 && p[-1] == '.' && p[-2] == '.' && p[-3] == '.') {
    p[-3] = ' ';
    p -= 2;
  }
  if (p == name || name[0] == ' ') {
    fprintf(stderr, "%s: empty name (%s, %d)\n",
             command_name, source_name, source_line);
    exit(-1);
  }
  *p = '\0';
  node = prefix_add(&macro_names, name, sector);
}
```

◇ Fragment referenced in 177a, 178d, 181.

Uses: command_name 29c, exit 21b, fprintf 21b, macro_names 158a, prefix_add 159c, sector 131b, source_line 126b, source_name 126b, stderr 21b.

⟨ Skip until scrap begins, then return name 178d ⟩ ≡

```
{
  do
    c = source_get();
  while (isspace(c));
  c2 = source_get();
  if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
    fprintf(stderr, "%s: expected %c{ after fragment name (%s, %d)\n",
             command_name, nw_char, source_name, start_line);
    exit(-1);
  }
  ⟨ Cleanup and install name 178c ⟩
  ⟨ Apply pending language to macro 175b ⟩
  return install_args(node, argc, arg);
}
```

◇ Fragment referenced in 176b.

Uses: command_name 29c, exit 21b, fprintf 21b, isspace 21b, nw_char 29ab, source_get 127c, source_name 126b, stderr 21b.

⟨ Function prototypes 179a ⟩ ≡

```
extern Name *install_args(Name *name, int argc, char *arg[9]);
```

◇ Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: Name 157c.

"names.c" 179b ≡

```
Name *install_args(Name * name, int argc, char *arg[9])
{
  int i;
```

```

    for (i = 0; i < argc; i++) {
        if (name->arg[i] == NULL)
            name->arg[i] = save_string(arg[i]);
    }
    return name;
}

```

◇
File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.
Uses: Name 157c, save_string 159b.

⟨ *Type declarations* 179c ⟩ ≡

```

typedef struct arglist
{Name * name;
struct arglist * args;
struct arglist * next;
} Arglist;

```

◇
Fragment defined by 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b.
Fragment referenced in 21a.
Defines: Arglist 54b, 57b, 61c, 77b, 79, 86b, 89c, 97b, 98b, 132b, 140c, 141d, 144ab, 145abcd, 146b, 147b, 151d, 153c, 154a, 158c, 179d, 180, 182c, 183ac, 185c, 186a, 187a, 192b.
Uses: Name 157c, next 131a.

"names.c" 179d≡

```

Arglist * buildArglist(Name * name, Arglist * a)
{
    Arglist * args = (Arglist *)arena_getmem(sizeof(Arglist));

    args->args = a;
    args->next = NULL;
    args->name = name;
    return args;
}

```

◇
File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.
Defines: buildArglist 181, 182c, 183c.
Uses: arena_getmem 200b, Arglist 179c, Name 157c, next 131a.

Terminated by @>

"names.c" 180≡

```

Arglist * collect_scrap_name(int current_scrap)
{
    char name[MAX_NAME_LEN];
    char *p = name;
    int c = source_get();
    unsigned char sector = current_sector;
    Arglist * head = NULL;
    Arglist ** tail = &head;

    if (c == '+')
    {
        sector = 0;
        c = source_get();
    }
}

```

```

while (c == ' ' || c == '\t')
    c = source_get();
while (c != EOF) {
    switch (c) {
        case '\t':
        case ' ': *p++ = ' ';
                    do
                        c = source_get();
                        while (c == ' ' || c == '\t');
                        break;
        default:
            if (c==nw_char)
            {
                < Look for end of scrap name and return 181>
                break;
            }
            if (!isgraph(c)) {
                fprintf(stderr,
                    "%s: unexpected character in fragment name (%s, %d)\n",
                    command_name, source_name, source_line);
                exit(-1);
            }
            *p++ = c;
            c = source_get();
            break;
    }
}
fprintf(stderr, "%s: unexpected end of file (%s, %d)\n",
    command_name, source_name, source_line);
exit(-1);
return NULL; /* unreachable return to avoid warnings on some compilers */
}

```

◇ File defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.
 Defines: collect_scrap_name 61c, 77b, 89c, 97b, 140c, 158c, 183b.
 Uses: Arglist 179c, command_name 29c, current_sector 47b, exit 21b, fprintf 21b, isgraph 21b, MAX_NAME_LEN 23b,
 nw_char 29ab, sector 131b, source_get 127c, source_line 126b, source_name 126b, stderr 21b.

< Look for end of scrap name and return 181> ≡

```

{
    Name * node;

    c = source_get();
    switch (c) {

        case '\': {
            < Add plain string argument 182a>
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
        case '1': case '2': case '3':
        case '4': case '5': case '6':
        case '7': case '8': case '9': {
            < Add a propagated argument 182b>
        }
        *p++ = ARG_CHR;
    }
}

```

```

        c = source_get();
        break;
    case '{': {
        < Add an inline scrap argument 182c >
    }
    *p++ = ARG_CHR;
    c = source_get();
    break;
    case '<':
        < Add macro call argument 183b >
    *p++ = ARG_CHR;
    c = source_get();
    break;
    case '(':
        scrap_name_has_parameters = 1;
        < Cleanup and install name 178c >
        return buildArglist(node, head);
    case '>':
        scrap_name_has_parameters = 0;
        < Cleanup and install name 178c >
        return buildArglist(node, head);

    default:
        if (c==nw_char)
        {
            *p++ = c;
            c = source_get();
            break;
        }
        fprintf(stderr,
            "%s: unexpected %c%c in fragment invocation name (%s, %d)\n",
            command_name, nw_char, c, source_name, source_line);
        exit(-1);
    }
}

```

◇ Fragment referenced in 180.

Uses: ARG_CHR 178a, buildArglist 179d, command_name 29c, exit 21b, fprintf 21b, Name 157c, nw_char 29ab, source_get 127c, source_line 126b, source_name 126b, stderr 21b.

A plain string argument has no name and a string as a value.

< Add plain string argument 182a > ≡

```

char buff[MAX_NAME_LEN];
char * s = buff;
int c, c2;

while ((c = source_get()) != EOF) {
    if (c==nw_char) {
        c2 = source_get();
        if (c2=='\''')
            break;
        *s++ = c2;
    }
    else
        *s++ = c;
}

```

```
*s = '\000';
< Add buff to current arg list 183c >
```

◇

Fragment referenced in 181.

Uses: MAX_NAME_LEN 23b, nw_char 29ab, source_get 127c.

A parameter argument propagated into an interior fragment has no name and a string of ARG_CHAR followed by a digit as value.

< Add a propagated argument 182b > ≡

```
char buff[3];
buff[0] = ARG_CHAR;
buff[1] = c;
buff[2] = '\000';
< Add buff to current arg list 183c >
```

◇

Fragment referenced in 181.

Uses: ARG_CHAR 178a.

< Add an inline scrap argument 182c > ≡

```
int s = collect_scrap();
Scrap_Node * d = (Scrap_Node *)arena_getmem(sizeof(Scrap_Node));
d->scrap = s;
d->quoted = 0;
d->next = NULL;
*tail = buildArglist((Name *)1, (Arglist *)d);
tail = &(*tail)->next;
```

◇

Fragment referenced in 181.

Uses: arena_getmem 200b, Arglist 179c, buildArglist 179d, collect_scrap 136c, Name 157c, next 131a, Scrap_Node 157b.

< Type declarations 183a > ≡

```
typedef struct embed {
    Scrap_Node * defs;
    Arglist * args;
} Embed_Node;
```

◇

Fragment defined by 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b.

Fragment referenced in 21a.

Defines: Embed_Node 54b, 145b, 154b.

Uses: Arglist 179c, defs 131b, Scrap_Node 157b.

< Add macro call argument 183b > ≡

```
*tail = collect_scrap_name(current_scrap);
if (current_scrap >= 0)
    add_to_use((*tail)->name, current_scrap);
tail = &(*tail)->next;
```

◇

Fragment referenced in 181.

Uses: add_to_use 141a, collect_scrap_name 180, next 131a.

< Add buff to current arg list 183c > ≡

```

    *tail = buildArglist(NULL, (Arglist *)save_string(buff));
    tail = &(*tail)->next;

```

◇

Fragment referenced in [182ab](#).

Uses: [Arglist 179c](#), [buildArglist 179d](#), [next 131a](#), [save_string 159b](#).

"names.c" 183d≡

```

static Scrap_Node *reverse(Scrap_Node *a); /* a forward declaration */

void reverse_lists(Name *names)
{
    while (names) {
        reverse_lists(names->llink);
        names->defs = reverse(names->defs);
        names->uses = reverse(names->uses);
        names = names->rlink;
    }
}

```

◇

File defined by [25c](#), [159abc](#), [160b](#), [161b](#), [162b](#), [163](#), [170](#), [172](#), [173a](#), [176b](#), [179bd](#), [180](#), [183de](#).

Defines: [reverse_lists 53e](#), [158c](#).

Uses: [defs 131b](#), [Name 157c](#), [reverse 183e](#), [Scrap_Node 157b](#), [uses 131b](#).

Just for fun, here's a non-recursive version of the traditional list reversal code. Note that it reverses the list in place; that is, it does no new allocations.

"names.c" 183e≡

```

static Scrap_Node *reverse(Scrap_Node *a)
{
    if (a) {
        Scrap_Node *b = a->next;
        a->next = NULL;
        while (b) {
            Scrap_Node *c = b->next;
            b->next = a;
            a = b;
            b = c;
        }
    }
    return a;
}

```

◇

File defined by [25c](#), [159abc](#), [160b](#), [161b](#), [162b](#), [163](#), [170](#), [172](#), [173a](#), [176b](#), [179bd](#), [180](#), [183de](#).

Defines: [reverse 27](#), [31](#), [183d](#).

Uses: [next 131a](#), [Scrap_Node 157b](#).

3.4 Searching for Index Entries

Given the array of scraps and a set of index entries, we need to search all the scraps for occurrences of each entry. The obvious approach to this problem would be quite expensive for large documents; however, there is an interesting paper describing an efficient solution [1].

"scraps.c" 184a≡

```

typedef struct name_node {
    struct name_node *next;
    Name *name;
} Name_Node;

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: Name_Node 184b, 189b, 190, 191.
 Uses: Name 157c, next 131a.

"scraps.c" 184b≡

```

typedef struct goto_node {
    Name_Node *output;           /* list of words ending in this state */
    struct move_node *moves;     /* list of possible moves */
    struct goto_node *fail;      /* and where to go when no move fits */
    struct goto_node *next;      /* next goto node with same depth */
} Goto_Node;

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: Goto_Node 184c, 185ab, 188d, 189b, 190, 191.
 Uses: Name_Node 184a, next 131a.

"scraps.c" 184c≡

```

typedef struct move_node {
    struct move_node *next;
    Goto_Node *state;
    char c;
} Move_Node;

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: Move_Node 185b, 189b, 190.
 Uses: Goto_Node 184b, next 131a.

"scraps.c" 185a≡

```

static Goto_Node *root[256];
static int max_depth;
static Goto_Node **depths;

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: depths 188d, 189b, 190, max_depth 188d, 189b, 190, root 111b, 188d, 189b, 190, 191, 192c, 193a.
 Uses: Goto_Node 184b.

"scraps.c" 185b≡

```

static Goto_Node *goto_lookup(char c, Goto_Node *g)
{
    Move_Node *m = g->moves;
    while (m && m->c != c)
        m = m->next;
    if (m)
        return m->state;
    else
        return NULL;
}

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: goto_lookup 189b, 190, 191.
 Uses: Goto_Node 184b, Move_Node 184c, next 131a.

3.4.1 Retrieving scrap uses

"scraps.c" 185c≡

```
typedef struct ArgMgr_s
{
    char * pv;
    char * bgn;
    Arglist * arg;
    struct ArgMgr_s * old;
} ArgMgr;
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: ArgMgr 185d, 186ab, 187c.
 Uses: Arglist 179c.

"scraps.c" 185d≡

```
typedef struct ArgManager_s
{
    Manager * m;
    ArgMgr * a;
} ArgManager;
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: ArgManager 186ab, 187c, 188d, 191, 196b.
 Uses: ArgMgr 185c, Manager 135f.

"scraps.c" 186a≡

```
static void
pushArglist(ArgManager * mgr, Arglist * a)
{
    ArgMgr * b = malloc(sizeof(ArgMgr));

    if (b == NULL)
    {
        fprintf(stderr, "Can't allocate space for an argument manager\n");
        exit(EXIT_FAILURE);
    }
    b->pv = b->bgn = NULL;
    b->arg = a;
    b->old = mgr->a;
    mgr->a = b;
}
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: pushArglist 187a, 192b.
 Uses: Arglist 179c, ArgManager 185d, ArgMgr 185c, exit 21b, fprintf 21b, malloc 21b, stderr 21b.

"scraps.c" 186b≡

```
static char argpop(ArgManager * mgr)
{
    while (mgr->a != NULL)
    {
        ArgMgr * a = mgr->a;

        ⟨ Perhaps —return— a character from the current arg 186c ⟩
        ⟨ Perhaps start a new arg 187a ⟩
        ⟨ Otherwise pop the current arg 187b ⟩
    }

    return (pop(mgr->m));
}
```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: argpop 191, 192ab.

Uses: ArgManager 185d, ArgMgr 185c, pop 141c.

We separate individual arguments using spaces.

⟨ Perhaps —return— a character from the current arg 186c ⟩ ≡

```
if (a->pv != NULL)
{
    char c = *a->pv++;

    if (c != '\0')
        return c;
    a->pv = NULL;
    return ' ';
}
```

◇
Fragment referenced in 186b.

I'm pretty sure that only the first case is ever used. I don't think the others can occur in this context, which makes the whole —ArgManager— thing doubtful.

⟨ Perhaps start a new arg 187a ⟩ ≡

```
if (a->arg) {
    Arglist * b = a->arg;

    a->arg = b->next;
    if (b->name == NULL) {
        a->bgn = a->pv = (char *)b->args;
    } else if (b->name == (Name *)1) {
        a->bgn = a->pv = "{Embedded Scrap}";
    } else {
        pushArglist(mgr, b->args);
    }
}
```

◇
Fragment referenced in 186b.
Uses: Arglist 179c, Name 157c, next 131a, pushArglist 186a.

⟨ Otherwise pop the current arg 187b ⟩ ≡

```

    } else {
        mgr->a = a->old;
        free(a);
    }

```

◇
Fragment referenced in [186b](#).

"scraps.c" 187c≡

```

static char
prev_char(ArgManager * mgr, int n)
{
    char c = '\0';
    ArgMgr * a = mgr->a;
    Manager * m = mgr->m;

    if (a != NULL) {
        ⟨ Get the nth previous character from an argument 188a ⟩
    } else {
        ⟨ Get the nth previous character from a scrap 188b ⟩
    }

    return c;
}

```

◇
File defined by [25b](#), [131ab](#), [132ac](#), [133abc](#), [134acde](#), [135abf](#), [136abc](#), [141acd](#), [143abcd](#), [144ab](#), [145d](#), [146b](#), [155b](#), [184abc](#), [185abcd](#), [186ab](#), [187c](#), [188d](#), [189a](#), [192d](#), [193ad](#), [195b](#), [196ab](#), [197b](#).

Defines: `prev_char` [196b](#).

Uses: `ArgManager` [185d](#), `ArgMgr` [185c](#), `Manager` [135f](#).

⟨ Get the nth previous character from an argument 188a ⟩ ≡

```

if (a->pv && a->pv - n >= a->bgn)
    c = *a->pv;
else if (a->bgn) {
    int j = strlen(a->bgn) + 1;

    if (n >= j)
        c = a->bgn[j - n];
    else
        c = ' ';
}

```

◇
Fragment referenced in [187c](#).
Uses: `strlen` [21b](#).

When the offset k falls within the current slab we index it directly. When k is negative we must reach into the previous slab: since k is already negative, `SLAB_SIZE + k` yields the correct index (e.g. $k = -1$ gives `chars[1023]`, the last element).

⟨ Get the nth previous character from a scrap 188b ⟩ ≡

```

int k = m->index - n - 2;

if (k >= 0)
    c = m->scrap->chars[k];
else if (m->prev)
    c = m->prev->chars[SLAB_SIZE + k];

```

◇
 Fragment referenced in 187c.
 Uses: SLAB_SIZE 131a.

3.4.2 Building the Automata

⟨ *Function prototypes* 188c ⟩ ≡

```
extern void search(void);
```

◇
 Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
 Fragment referenced in 21a.
 Uses: search 188d.

"scraps.c" 188d≡

```
static void build_gotos(Name *tree);
static int reject_match(Name *name, char post, ArgManager *reader);

void search(void)
{
  int i;
  for (i=0; i<256; i++)
    root[i] = NULL;
  max_depth = 10;
  depths = (Goto_Node **) arena_getmem(max_depth * sizeof(Goto_Node *));
  for (i=0; i<max_depth; i++)
    depths[i] = NULL;
  build_gotos(user_names);
  ⟨ Build failure functions 190 ⟩
  ⟨ Search scraps 191 ⟩
}
```

◇
 File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: search 31, 43c, 188c, 202.
 Uses: arena_getmem 200b, ArgManager 185d, build_gotos 189a, depths 185a, Goto_Node 184b, max_depth 185a, Name 157c, reject_match 196b, root 185a, user_names 158a.

"scraps.c" 189a≡

```
static void build_gotos(Name *tree)
{
  while (tree) {
    ⟨ Extend goto graph with tree->spelling 189b ⟩
    build_gotos(tree->rlink);
    tree = tree->llink;
  }
}
```

◇
 File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: build_gotos 188d.
 Uses: Name 157c.

⟨ *Extend goto graph with tree->spelling* 189b ⟩ ≡

```

{
    int depth = 2;
    char *p = tree->spelling;
    char c = *p++;
    Goto_Node *q = root[(unsigned char)c];
    Name_Node *last;
    if (!q) {
        q = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
        root[(unsigned char)c] = q;
        q->moves = NULL;
        q->fail = NULL;
        q->output = NULL;
        q->next = depths[1];
        depths[1] = q;
    }
    while ((c = *p++)) {
        Goto_Node *new_node = goto_lookup(c, q);
        if (!new_node) {
            Move_Node *new_move = (Move_Node *) arena_getmem(sizeof(Move_Node));
            new_node = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
            new_node->moves = NULL;
            new_node->fail = NULL;
            new_node->output = NULL;
            new_move->state = new_node;
            new_move->c = c;
            new_move->next = q->moves;
            q->moves = new_move;
            if (depth == max_depth) {
                int i;
                Goto_Node **new_depths =
                    (Goto_Node **) arena_getmem(2*depth*sizeof(Goto_Node *));
                max_depth = 2 * depth;
                for (i=0; i<depth; i++)
                    new_depths[i] = depths[i];
                depths = new_depths;
                for (i=depth; i<max_depth; i++)
                    depths[i] = NULL;
            }
            new_node->next = depths[depth];
            depths[depth] = new_node;
        }
        q = new_node;
        depth++;
    }
    last = q->output;
    q->output = (Name_Node *) arena_getmem(sizeof(Name_Node));
    q->output->next = last;
    q->output->name = tree;
}

```

◇

Fragment referenced in 189a.

Uses: arena_getmem 200b, depths 185a, goto_lookup 185b, Goto_Node 184b, max_depth 185a, Move_Node 184c, Name_Node 184a, next 131a, root 185a.

⟨ Build failure functions 190 ⟩ ≡

```

{
    int depth;

```

```

for (depth=1; depth<max_depth; depth++) {
    Goto_Node *r = depths[depth];
    while (r) {
        Move_Node *m = r->moves;
        while (m) {
            char a = m->c;
            Goto_Node *s = m->state;
            Goto_Node *state = r->fail;
            while (state && !goto_lookup(a, state))
                state = state->fail;
            if (state)
                s->fail = goto_lookup(a, state);
            else
                s->fail = root[(unsigned char)a];
            if (s->fail) {
                Name_Node *p = s->fail->output;
                while (p) {
                    Name_Node *q = (Name_Node *) arena_getmem(sizeof(Name_Node));
                    q->name = p->name;
                    q->next = s->output;
                    s->output = q;
                    p = p->next;
                }
            }
            m = m->next;
        }
        r = r->next;
    }
}

```

◇

Fragment referenced in 188d.

Uses: arena_getmem 200b, depths 185a, goto_lookup 185b, Goto_Node 184b, max_depth 185a, Move_Node 184c, Name_Node 184a, next 131a, root 185a.

3.4.3 Searching the Scraps

⟨ Search scraps 191 ⟩ ≡

```

{
    for (i=1; i<scraps; i++) {
        char c, last = '\0';
        Manager rd;
        ArgManager reader;
        Goto_Node *state = NULL;
        rd.prev = NULL;
        rd.scrap = scrap_array(i).slab;
        rd.index = 0;
        reader.m = &rd;
        reader.a = NULL;
        c = argpop(&reader);
        while (c) {
            while (state && !goto_lookup(c, state))
                state = state->fail;
            if (state)
                state = goto_lookup(c, state);
            else
                state = root[(unsigned char)c];
        }
    }
}

```

```

    < Skip over at at 192a >
    < Skip over a scrap use 192b >
    < Skip over a block comment 53c >
    last = c;
    c = argpop(&reader);
    if (state && state->output) {
        Name_Node *p = state->output;
        do {
            Name *name = p->name;
            if (!reject_match(name, c, &reader) &&
                scrap_array(i).sector == name->sector &&
                (!name->uses || name->uses->scrap != i)) {
                Scrap_Node *new_use =
                    (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
                new_use->scrap = i;
                new_use->next = name->uses;
                name->uses = new_use;
                if (!scrap_is_in(name->defs, i))
                    add_uses(&(scrap_array(i).uses), name);
            }
            p = p->next;
        } while (p);
    }
}
}
}

```

◇

Fragment referenced in 188d.

Uses: add_uses 193a, arena_getmem 200b, ArgManager 185d, argpop 186b, defs 131b, goto_lookup 185b, Goto_Node 184b, Manager 135f, Name 157c, Name_Node 184a, next 131a, reject_match 196b, root 185a, scraps 132a, scrap_array 132a, scrap_is_in 192d, Scrap_Node 157b, sector 131b, slab 131b, uses 131b.

< Skip over at at 192a > ≡

```

    if (last == nw_char && c == nw_char)
    {
        last = '\0';
        c = argpop(&reader);
    }

```

◇

Fragment referenced in 191.

Uses: argpop 186b, nw_char 29ab.

< Skip over a scrap use 192b > ≡

```

    if (last == nw_char && c == '<')
    {
        char buf[MAX_NAME_LEN];
        char * p = buf;
        Arglist * args;
        int idx;

        c = argpop(&reader);
        while ((c = argpop(&reader)) != nw_char)
            *p++ = c;
        c = argpop(&reader);
        *p = '\0';
        if (sscanf(buf, "%d", &idx) != 1) {

```

```

        fprintf(stderr, "%s: found an internal problem (3)\n", command_name);
        exit(-1);
    }
    args = arglist_lookup(idx);
    pushArglist(&reader, args);
}

```

◇

Fragment referenced in 191.

Uses: Arglist 179c, arglist_lookup 147b, argpop 186b, command_name 29c, exit 21b, fprintf 21b, MAX_NAME_LEN 23b, nw_char 29ab, pushArglist 186a, stderr 21b.

⟨ Forward declarations for scraps.c 192c ⟩ ≡

```

static void add_uses(Uses **root, Name *name);
static int scrap_is_in(Scrap_Node *list, int i);

```

◇

Fragment defined by 147ab, 154a, 176a, 192c.

Fragment referenced in 132a.

Uses: add_uses 193a, Name 157c, root 185a, scrap_is_in 192d, Scrap_Node 157b, Uses 193b.

"scraps.c" 192d≡

```

static int scrap_is_in(Scrap_Node * list, int i)
{
    while (list != NULL) {
        if (list->scrap == i)
            return TRUE;
        list = list->next;
    }
    return FALSE;
}

```

◇

File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: scrap_is_in 191, 192c.

Uses: FALSE 23a, next 131a, Scrap_Node 157b, TRUE 23a.

"scraps.c" 193a≡

```

static void add_uses(Uses * * root, Name *name)
{
    int cmp;
    Uses *p, **q = root;

    while ((p = *q, p != NULL)
        && (cmp = robs_strcmp(p->defn->spelling, name->spelling)) < 0)
        q = &(p->next);
    if (p == NULL || cmp > 0)
    {
        Uses *new_use = arena_getmem(sizeof(Uses));
        new_use->next = p;
        new_use->defn = name;
        *q = new_use;
    }
}

```

◇

File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: add_uses 140ab, 191, 192c.
 Uses: arena_getmem 200b, Name 157c, next 131a, robs_strcmp 161b, root 185a, Uses 193b.

⟨ *Type declarations* 193b ⟩ ≡

```
typedef struct uses {
    struct uses *next;
    Name *defn;
} Uses;
```

◇
 Fragment defined by 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b.
 Fragment referenced in 21a.
 Defines: Uses 59a, 104, 105a, 106a, 108, 110b, 132b, 133c, 134a, 192c, 193ad, 195b.
 Uses: Name 157c, next 131a, uses 131b.

⟨ *Function prototypes* 193c ⟩ ≡

```
extern void format_uses_refs(FILE *, int);
```

◇
 Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
 Fragment referenced in 21a.
 Uses: FILE 21b, format_uses_refs 193d.

"scraps.c" 193d≡

```
void
format_uses_refs(FILE * tex_file, int scrap)
{
    Uses * p = scrap_array(scrap).uses;
    if (p != NULL)
        ⟨ Write uses references 194a ⟩
}
```

◇
 File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
 Defines: format_uses_refs 62, 63a, 193c.
 Uses: FILE 21b, scrap_array 132a, Uses 193b, uses 131b.

⟨ *Write uses references* 194a ⟩ ≡

```
{
    char join = ' ';
    fputs("\\item \\WEFTtxtIdentsUsed\\nobreak\\", tex_file);
    do {
        ⟨ Write one use reference 194b ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".", tex_file);
}
```

◇
 Fragment referenced in 193d.
 Uses: fputs 21b, next 131a.

⟨ *Write one use reference 194b* ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c \\verb%c%s%c\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
if (defs)
{
    do {
        ⟨ Write one referenced scrap 194c ⟩
        first = FALSE;
        defs = defs->next;
    }while (defs!= NULL);
}
else
{
    fputs("\\WEFTnotglobal", tex_file);
}
```

◇ Fragment referenced in 194a.

Uses: defs 131b, FALSE 23a, first 200a, fprintf 21b, fputs 21b, Name 157c, next 131a, nw_char 29ab, page 131b, Scrap_Node 157b, TRUE 23a.

⟨ *Write one referenced scrap 194c* ⟩ ≡

```
fputs("\\WEFTlink{weft", tex_file);
write_scrap_ref(tex_file, defs->scrap, -1, &page);
fputs("}{", tex_file);
write_scrap_ref(tex_file, defs->scrap, first, &page);
fputs("}", tex_file);
```

◇ Fragment referenced in 194b, 195d.

Uses: defs 131b, first 200a, fputs 21b, page 131b, write_scrap_ref 133a.

⟨ *Function prototypes 195a* ⟩ ≡

```
extern void format_defs_refs(FILE *, int);
```

◇ Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.

Fragment referenced in 21a.

Uses: FILE 21b, format_defs_refs 195b.

"scraps.c" 195b≡

```
void
format_defs_refs(FILE * tex_file, int scrap)
{
    Uses * p = scrap_array(scrap).defs;
    if (p != NULL)
        ⟨ Write defs references 195c ⟩
}
```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: format_defs_refs 62, 63a, 195a.

Uses: defs 131b, FILE 21b, scrap_array 132a, Uses 193b.

⟨ Write defs references 195c ⟩ ≡

```
{
    char join = ' ';
    fputs("\\item \\WEFTtxtIdentsDefed\\nobreak\\", tex_file);
    do {
        ⟨ Write one def reference 195d ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".", tex_file);
}
```

◇
Fragment referenced in 195b.
Uses: fputs 21b, next 131a.

⟨ Write one def reference 195d ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c \\verb%c%s%c\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
    fputs("\\WEFTtxtIdentsNotUsed", tex_file);
}
else {
    do {
        if (defs->scrap != scrap) {
            ⟨ Write one referenced scrap 194c ⟩
            first = FALSE;
        }
        defs = defs->next;
    }while (defs != NULL);
}
```

◇
Fragment referenced in 195c.
Uses: defs 131b, FALSE 23a, first 200a, fprintf 21b, fputs 21b, Name 157c, next 131a, nw_char 29ab, page 131b, Scrap_Node 157b, TRUE 23a, uses 131b.

Rejecting Matches

A problem with simple substring matching is that the string “he” would match longer strings like “she” and “her.” Norman Ramsey suggested examining the characters occurring immediately before and after a match and rejecting the match if it appears to be part of a longer token. Of course, the concept of *token* is language-dependent, so we may be occasionally mistaken. For the present, we’ll consider the mechanism an experiment.

"scraps.c" 196a≡

```
#define sym_char(c) (isalnum(c) || (c) == '_')

static int op_char(char c)
```

```

{
    switch (c) {
        case '!':          case '#': case '%': case '$': case '^':
        case '&': case '*': case '-': case '+': case '=': case '/':
        case '|': case '~': case '<': case '>':
            return TRUE;
        default:
            return c==nw_char ? TRUE : FALSE;
    }
}

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: op_char 196b, sym_char 196b.

Uses: FALSE 23a, nw_char 29ab, TRUE 23a.

"scraps.c" 196b≡

```

static int reject_match(Name *name, char post, ArgManager *reader)
{
    int len = strlen(name->spelling);
    char first = name->spelling[0];
    char last = name->spelling[len - 1];
    char prev = prev_char(reader, len);
    if (sym_char(last) && sym_char(post)) return TRUE;
    if (sym_char(first) && sym_char(prev)) return TRUE;
    if (op_char(last) && op_char(post)) return TRUE;
    if (op_char(first) && op_char(prev)) return TRUE;
    return FALSE; /* Here is 196b-01 */
}

```

◇ File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

Defines: reject_match 188d, 191.

Uses: ArgManager 185d, FALSE 23a, first 200a, Name 157c, op_char 196a, prev_char 187c, strlen 21b, sym_char 196a, TRUE 23a.

3.5 Labels

Refer to 197a-01. And another one 196b-01.

⟨ Get label from 197a ⟩ ≡

```

char label_name[MAX_NAME_LEN];
char * p = label_name;
while (c = @1, c != nw_char) /* Here is 197a-01 */
    *p++ = c;
*p = '\0';
c = @1;

```

◇ Fragment referenced in 76b, 138, 151a.

Uses: MAX_NAME_LEN 23b, nw_char 29ab.

"scraps.c" 197b≡

```

void
write_label(char label_name[], FILE * file)
⟨ Search for label(⟨ Write the label to file 197d ⟩,⟨ Complain about missing label 197e ⟩) 198d ⟩

```

◇
File defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.
Defines: `write_label` 76b, 151a, 197c.
Uses: FILE 21b.

⟨ *Function prototypes* 197c ⟩ ≡

```
void write_label(char label_name[], FILE * file);
```

◇
Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
Fragment referenced in 21a.
Uses: FILE 21b, `write_label` 197b.

⟨ *Write the label to file* 197d ⟩ ≡

```
write_single_scrap_ref(file, lbl->scrap);  
fprintf(file, "-%02d", lbl->seq);
```

◇
Fragment referenced in 197b.
Uses: `fprintf` 21b, `write_single_scrap_ref` 133b.

⟨ *Complain about missing label* 197e ⟩ ≡

```
fprintf(stderr, "Can't find label %s.\n", label_name);
```

◇
Fragment referenced in 197b.
Uses: `fprintf` 21b, `stderr` 21b.

⟨ *Save label to label store* 197f ⟩ ≡

```
if (label_name[0])  
    ⟨ Search for label(⟨ Complain about duplicate labels 198b ⟩,⟨ Create a new label entry 198a ⟩) 198d  
else  
{  
    ⟨ Complain about empty label 198c ⟩  
}
```

◇
Fragment referenced in 138.

⟨ *Create a new label entry* 198a ⟩ ≡

```
lbl = (label_node *)arena_getmem(sizeof(label_node) + (p - label_name));  
lbl->left = lbl->right = NULL;  
strcpy(lbl->name, label_name);  
lbl->scrap = current_scrap;  
lbl->seq = ++lblseq;  
*plbl = lbl;
```

◇
Fragment referenced in 197f.
Uses: `arena_getmem` 200b, `label_node` 199b.

⟨ *Complain about duplicate labels* 198b ⟩ ≡

```
fprintf(stderr, "Duplicate label %s.\n", label_name);
```

◇
Fragment referenced in 197f.

Uses: [fprintf 21b](#), [stderr 21b](#).

⟨ *Complain about empty label* 198c ⟩ ≡

```
fprintf(stderr, "Empty label.\n");
```

◇

Fragment referenced in [197f](#).

Uses: [fprintf 21b](#), [stderr 21b](#).

⟨ *Search for label(Found,Notfound)* 198d ⟩ ≡

```
{
    label_node * * plbl = &label_tab;
    for (;;)
    {
        label_node * lbl = *plbl;

        if (lbl)
        {
            int cmp = label_name[0] - lbl->name[0];

            if (cmp == 0)
                cmp = strcmp(label_name + 1, lbl->name + 1);
            if (cmp < 0)
                plbl = &lbl->left;
            else if (cmp > 0)
                plbl = &lbl->right;
            else
            {
                Found
                break;
            }
        }
        else
        {
            Notfound
            break;
        }
    }
}
```

◇

Fragment referenced in [197bf](#).

Uses: [label_node 199b](#), [label_tab 199a](#).

"global.c" 199a≡

```
label_node * label_tab = NULL;
```

◇

File defined by [25e](#), [199a](#).

Defines: [label_tab 198d](#), [199c](#).

Uses: [label_node 199b](#).

⟨ *Type declarations* 199b ⟩ ≡

```
typedef struct l_node
{
    struct l_node * left, * right;
    int scrap, seq;
```

```

    char name[1];
} label_node;

```

◇
 Fragment defined by 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b.
 Fragment referenced in 21a.
 Defines: `label_node` 198ad, 199ac.

⟨ *Global variable declarations* 199c ⟩ ≡

```

extern label_node * label_tab;

```

◇
 Fragment defined by 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c.
 Fragment referenced in 21a.
 Uses: `label_node` 199b, `label_tab` 199a.

3.6 Memory Management

I manage memory using a simple scheme inspired by Hanson’s idea of *arenas* [2]. Basically, I allocate all the storage required when processing a source file (primarily for names and scraps) using calls to `arena_getmem(n)`, where `n` specifies the number of bytes to be allocated. When the storage is no longer required, the entire arena is freed with a single call to `arena_free()`. Both operations are quite fast.

⟨ *Function prototypes* 199d ⟩ ≡

```

extern void *arena_getmem(size_t n);
extern void arena_free(void);
extern void arglist_reset(void);

```

◇
 Fragment defined by 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d.
 Fragment referenced in 21a.
 Uses: `arena_free` 201b, `arena_getmem` 200b, `arglist_reset` 147b, `size_t` 21b.

"arena.c" 199e≡

```

typedef struct chunk {
    struct chunk *next;
    char *limit;
    char *avail;
} Chunk;

```

◇
 File defined by 25d, 199e, 200ab, 201b.
 Defines: `Chunk` 200ac, 201a.
 Uses: `next` 131a.

We define an empty chunk called `first`. The variable `arena` points at the current chunk of memory; it’s initially pointed at `first`. As soon as some storage is required, a “real” chunk of memory will be allocated and attached to `first->next`; storage will be allocated from the new chunk (and later chunks if necessary).

"arena.c" 200a≡

```

static Chunk first = { NULL, NULL, NULL };
static Chunk *arena = &first;

```

◇
 File defined by 25d, 199e, 200ab, 201b.
 Defines: `arena` 200bc, 201ab, `first` 31, 105bc, 106c, 108, 109, 113a, 132b, 133a, 194bc, 195d, 196b, 201b, 202.
 Uses: `Chunk` 199e.

3.6.1 Allocating Memory

The routine `arena_getmem(n)` returns a pointer to (at least) `n` bytes of memory. Note that `n` is rounded up to ensure that returned pointers are always aligned. We align to the nearest 8-byte segment, since that'll satisfy the more common 2-byte and 4-byte alignment restrictions too.

"arena.c" 200b≡

```
void *arena_getmem(size_t n)
{
    char *q;
    char *p = arena->avail;
    n = (n + 7) & ~7;           /* ensuring alignment to 8 bytes */
    q = p + n;
    if (q <= arena->limit) {
        arena->avail = q;
        return p;
    }
    ⟨ Find a new chunk of memory 200c ⟩
}
```

◇

File defined by 25d, 199e, 200ab, 201b.

Defines: `arena_getmem` 36a, 40a, 48b, 56a, 132c, 136ad, 140ab, 141a, 145bc, 159b, 162c, 179d, 182c, 188d, 189b, 190, 191, 193a, 198a, 199d.

Uses: `arena` 200a, `size_t` 21b.

If the current chunk doesn't have adequate space (at least `n` bytes) we examine the rest of the list of chunks (starting at `arena->next`) looking for a chunk with adequate space. If `n` is very large, we may not find it right away or we may not find a suitable chunk at all.

⟨ Find a new chunk of memory 200c ⟩ ≡

```
{
    Chunk *ap = arena;
    Chunk *np = ap->next;
    while (np) {
        char *v = sizeof(Chunk) + (char *) np;
        if (v + n <= np->limit) {
            np->avail = v + n;
            arena = np;
            return v;
        }
        ap = np;
        np = ap->next;
    }
    ⟨ Allocate a new chunk of memory 201a ⟩
}
```

◇

Fragment referenced in 200b.

Uses: `arena` 200a, `Chunk` 199e, `next` 131a.

If there isn't a suitable chunk of memory on the free list, then we need to allocate a new one.

⟨ Allocate a new chunk of memory 201a ⟩ ≡

```
{
    size_t m = n + 10000;
    np = (Chunk *) malloc(m);
    np->limit = m + (char *) np;
```

```

    np->avail = n + sizeof(Chunk) + (char *) np;
    np->next = NULL;
    ap->next = np;
    arena = np;
    return sizeof(Chunk) + (char *) np;
}

```

◇
 Fragment referenced in [200c](#).
 Uses: [arena 200a](#), [Chunk 199e](#), [malloc 21b](#), [next 131a](#), [size_t 21b](#).

3.6.2 Freeing Memory

To free all the memory in the arena, we need only point **arena** back to the first empty chunk.

"arena.c" 201b≡

```

void arena_free(void)
{
    arena = &first;
    arglist_reset();
}

```

◇
 File defined by [25d](#), [199e](#), [200ab](#), [201b](#).
 Defines: [arena_free 40b](#), [199d](#).
 Uses: [arena 200a](#), [arglist_reset 147b](#), [first 200a](#).

Chapter 4

Man page

Here is the UNIX man page for weft:

```
"weft.1" 202≡

.TH WEFT 1 "March 2026"
.SH NAME
weft, a literate programming tool
.SH SYNOPSIS
.B weft
.br
\fBweft\fP [options] [file] ...
.SH DESCRIPTION
.I weft
is a literate programming tool like Knuth's
.I WEB,
only simpler.
A
.I weft
file contains program source code interleaved with documentation.
When
.I weft
is given a
.I weft
file, it writes the program file(s).
By default only tangling is performed. Weaving (documentation
generation) is requested with \fB-w\fP.
.SH COMMAND LINE OPTIONS
.br
\fB-t\fP Suppresses weave output (even if \fB-w\fP is given).
.br
\fB-o\fP Suppresses generation of the output files.
.br
\fB-d\fP List dangling identifier references in indexes.
.br
\fB-c\fP Forces output files to overwrite old files of the same
name without comparing for equality first.
.br
\fB-v\fP The verbose flag. Forces output of progress reports.
.br
\fB-n\fP Forces sequential numbering of scraps (instead of page
numbers).
.br
\fB-s\fP Doesn't print list of scraps making up file at end of
```

each scrap.

`\fB-p path`\fP Prepend path to the filenames for all the output files.

`\fB-V string`\fP Provide the string for replacement of the @v operation. This is intended as a means for including version information in generated output.

`\fB-x`\fP Include cross-references in comments in output files.

`\fB-h options`\fP Turn on hyperlinks using the hyperref package of LaTeX and provide the options to the package.

`\fB-r`\fP Turn on hyperlinks using the hyperref package of LaTeX, with the package options being in the text.

`\fB-I path`\fP Provide a directory to search for included files. This may appear several times.

.br

`\fB-m`\fP Emit a JSON map of all output files, fragments, their definitions, and dependency relationships to stdout. No weave or tangle is performed. Useful for AI tooling and project navigation.

.br

`\fB-e name`\fP Extract the named fragment and all its transitive dependencies as Markdown to stdout. Fragment names support prefix matching. No weave or tangle is performed.

.br

`\fB-R file[:line]`\fP Read section markers from a tangled output file and emit JSON mapping to stdout. If a line number is given, emit the single innermost scrap covering that line. Without a line number, emit all scrap regions. No .weft source files are needed.

.br

`\fB-w [format]`\fP Request weave output. Without a format token, the format is taken from the `\fB@W`\fP directive in the source file. With `\fBmd`\fP, a `\fB.md`\fP file is generated; with `\fBtex`\fP, a `\fB.tex`\fP file is generated. A CLI format overrides `\fB@W`\fP. If `\fB-w`\fP is given without a format and no `\fB@W`\fP exists, weft exits with an error.

.br

`\fB--bodies`\fP Include raw scrap body text in `\fB-m`\fP JSON output. Each definition entry gets a `\fB"body"`\fP field with the JSON-escaped source code of the scrap.

.br

`\fB--prose`\fP Include the documentation paragraph preceding each `\fB@d`\fP/`\fB@o`\fP directive as a `\fB"prose"`\fP field in `\fB-m`\fP JSON output.

.br

`\fB--diff`\fP Dry-run tangle: show a unified diff of what would change in each output file without writing anything. If a file does not yet exist, all lines are shown with a `\fB+`\fP prefix.

.br

`\fB--lint`\fP Static analysis of the literate source. Reports unused fragments (defined but never referenced), undefined references (used but never defined), and fuzzy-match suggestions for likely typos (Levenshtein distance ≤ 3).

.br

`\fB--callers`\fP When combined with `\fB-e name`\fP, show the upward call chain from `\fB@o`\fP roots down to the named fragment as an indented tree.

.br

`\fB--errors`\fP Read compiler or linter output from stdin and annotate `\fBfile:line:`\fP patterns with `\fB.weft`\fP source locations by parsing section markers embedded in the tangled output. No `\fB.weft`\fP source files are needed. Example: `\fBmake 2>&1 | weft --errors`\fP.

.br

`\fB--help\fP` Print a comprehensive help message including all options, usage examples, and an AI-assisted workflow guide, then exit.

`.SH` FORMAT OF WEFT FILES

A

`.I` weft

file contains mostly ordinary

`.I` LaTeX.

The file is read and copied to output (`.tex` file) unless a

`.I` weft

command is encountered. All

`.I` weft

commands start with an ‘at-sign’ (`@`).

Files and fragments are defined with the following commands:

`.PP`

`@o \fIfile-name flags scrap\fP` where scrap is smaller than one page.

`.br`

`@O \fIfile-name flags scrap\fP` where scrap is bigger than one page.

`.br`

`@d \fIfragment-name scrap\fP`. Where scrap is smaller than one page.

`.br`

`@D \fIfragment-name scrap\fP`. Where scrap is bigger than one page.

`.PP`

`@q \fIfragment-name scrap\fP`. Where scrap is smaller than one page.

The scrap is not expanded in the output, allowing you to construct output files which can, perhaps after further processing, be input to

`.I` weft.

`.br`

`@Q \fIfragment-name scrap\fP`. Where scrap is bigger than one page.

Likewise.

`.PP`

Scraps have specific begin and end

markers;

which begin and end marker you use determines how the scrap will be typeset in the `.tex` file:

`.br`

`\fB@{\fP...\fB@}\fP` for code, typeset with the LaTeX `\fBlistings\fP` package (load `\fB\usepackage{listings}\fP` in your preamble).

`.br`

`\fB@[\fP...\fB@]\fP` for LaTeX paragraph mode formatting, and

`.br`

`\fB@(\fP...\fB@)\fP` for LaTeX math mode formatting.

`.br`

Any amount of whitespace

(including carriage returns) may appear between a name and the beginning of a scrap.

`.PP`

Several code/file scraps may have the same name;

`.I` weft

concatenates their definitions to produce a single scrap.

Code scrap definitions are like macro definitions;

`.I` weft

extracts a program by expanding one scrap.

The definition of that scrap contains references to other scraps, which are themselves expanded, and so on.

`\fIweft\fP`'s output is readable; it preserves the indentation of expanded scraps with respect to the scraps in which they appear.

`.PP`

.SH PER FILE OPTIONS

When defining an output file, the programmer has the option of using flags to control the output.

.PP

\fB-d\fR option,

.I weft

will emit line number indications at scrap boundaries.

.br

\fB-i\fR option,

.I weft

suppresses the indentation of fragments (useful for \fBFortran\fR).

.br

\fB-t\fP option makes \fIweft\fP

copy tabs untouched from input to output.

.br

\fB-c\fIx\fP Include comments in the output file.

\fIx\fP may be \fBc\fP for C-style comments, \fB+\fP for C++ and

\fBp\fP for perl and similar.

.br

\fB-s\fP Suppress section markers and \fB#line\fP directives in the tangled output for this file. Language auto-detection still sets the comment style and the \fBlang\fP tag (used for Markdown syntax highlighting), but no markers appear in the output.

.PP

.SH MINOR COMMANDS

.br

@ Causes a single ‘at-sign’ to be copied into the output.

.br

@_ Causes the text between it and the next {\tt @_} to be made bold (for keywords, etc.) in the formatted document

.br

%% Comments out a line so that it doesn’t appear in the output.

.br

@i \fBfilename\fR causes the file named to be included.

.br

@f Creates an index of output files.

.br

@m Creates an index of fragments.

.br

@u Creates an index of user-specified identifiers.

.br

@W \fIformat\fP Declares the weave output format (\fBmd\fP or \fBtex\fP).

Used by \fB-w\fP when no explicit format is given on the command line.

.br

@l \fItag\fP Sets the language for the next @o or @d directive.

The tag is looked up in the language table to determine the comment style for section markers and the \fBclass\fP attribute for Markdown syntax highlighting. Place it on the line before the directive it applies to.

.br

@L \fIname style\fP [+d] Defines a custom language mapping.

\fIname\fP is the tag (e.g. \fBnim\fP, \fBgls\fP),

\fIstyle\fP is one of \fB/\fP, \fB#\fP, \fB--\fP, \fB/*\fP, or \fB<!--\fP.

The optional \fB+d\fP flag enables \fB#line\fP directive generation.

User-defined languages override built-in entries. Must appear in the document preamble before any @o or @d that depends on it.

.PP

.SH SECTION MARKERS

By default, when a language is detected (via @l, @L, or file extension of @o), weft wraps each scrap in the tangled output with opening and closing section markers using the appropriate comment syntax. For example, C output includes:

.nf

```
/* {1: source.w:10} */
int main() { return 0; }
/* {:1} */
```

.fi

The opening marker shows the scrap number and source location; the closing marker shows only the scrap number. Nested fragment expansions produce nested markers.

.PP

Languages with \fB#line\fP support (C, C++, Objective-C, etc.) also emit \fB#line\fP directives at scrap boundaries.

.PP

The \fB-s\fP per-file flag suppresses both section markers and \fB#line\fP directives.

.PP

.SH LANGUAGE INFERENCE

When no @l tag precedes an @o directive, weft infers the language from the output file's extension. For example, \fB@o app.js\fP automatically detects JavaScript (// comments, no #line). The inferred language is used for section markers in the tangle and for the \fBclass="language-X"\fP attribute in the Markdown weave.

.PP

Over 250 file extensions are recognised, covering C, C++, Python, JavaScript, Go, Rust, Ruby, Lua, SQL, HTML, and many more.

Extensions not in the table produce no markers.

.PP

To mark an identifier for inclusion in the index, it must be mentioned at the end of the scrap it was defined in. The line starts with @| and ends with the \fBend of scrap\fP mark \fB@}\fP.

.PP

.SH ERROR MESSAGES

.PP

.SH BUGS

.PP

.SH AUTHOR

Preston Briggs.

Internet address \fBpreston@cs.rice.edu\fP.

.SH MAINTAINER

Simon Wright

Internet address \fBsimon@pushface.org\fP

.br

Keith Harwood

Internet address \fBKeith.Harwood@vitalmis.com\fP

◇
Uses: exit 21b, FILE 21b, first 200a, main 25f, next 131a, page 131b, prose 131b, scraps 132a, search 188d.

Chapter 5

Indices

Three sets of indices can be created automatically: an index of file names, an index of fragment names, and an index of user-specified identifiers. An index entry includes the name of the entry, where it was defined, and where it was referenced.

5.1 Files

"arena.c" Defined by 25d, 199e, 200ab, 201b.

"global.c" Defined by 25e, 199a.

"global.h" Defined by 21a.

"input.c" Defined by 25a, 127abc, 128b, 130c.

"latex.c" Defined by 24b, 57b, 58, 64a, 69ab, 70ab, 71a, 72abc, 73, 74a, 79, 80b, 81b, 82a, 84bd.

"main.c" Defined by 23c, 25f.

"markdown.c" Defined by 24c, 86b, 87a, 91b, 94ab, 98ab, 100a, 101ab, 103b, 104, 106a.

"names.c" Defined by 25c, 159abc, 160b, 161b, 162b, 163, 170, 172, 173a, 176b, 179bd, 180, 183de.

"output.c" Defined by 24d, 107, 108, 109, 110ab, 111ab, 112ab, 113ab, 114ab, 115, 116ab, 117ab, 120, 121, 122c.

"pass1.c" Defined by 24a, 43bc.

"scraps.c" Defined by 25b, 131ab, 132ac, 133abc, 134acde, 135abf, 136abc, 141acd, 143abcd, 144ab, 145d, 146b, 155b, 184abc, 185abcd, 186ab, 187c, 188d, 189a, 192d, 193ad, 195b, 196ab, 197b.

"weft.1" Defined by 202.

5.2 Fragments

⟨ Accumulate prose character 44a ⟩ Referenced in 43d.

⟨ Accumulate scrap and return scraps++ 137a ⟩ Referenced in 136c.

⟨ Add a propagated argument 182b ⟩ Referenced in 181.

⟨ Add an at character to the block or break 50b ⟩ Referenced in 50a.

⟨ Add an inline scrap argument 182c ⟩ Referenced in 181.

⟨ Add any other character to the block 51a ⟩ Referenced in 50a.

⟨ Add buff to current arg list 183c ⟩ Referenced in 182ab.

⟨ Add letters to scraps with duplicate page numbers 156a ⟩ Referenced in 155b.

⟨ Add macro call argument 183b ⟩ Referenced in 181.

⟨ Add more indentation char 148 ⟩ Referenced in 147c, 150cd.

⟨ Add one char to the block buffer 50a ⟩ Referenced in 49a.

⟨ Add plain string argument 182a ⟩ Referenced in 181.

⟨ Add the source path to the include path list 40a ⟩ Referenced in 39b.

⟨ Add user identifier definition 140b ⟩ Referenced in 139b.

⟨ Add user identifier use 140a ⟩ Referenced in 139b.

⟨ Add scrap to name's definition list 48b ⟩ Referenced in 47d, 48a.

⟨ Allocate a new chunk of memory 201a ⟩ Referenced in 200c.

⟨ Apply pending language if no explicit flags 175a ⟩ Referenced in 173a.

<Apply pending language to macro 175b> Referenced in 177a, 178d.
 <Begin or end a block comment 53d> Referenced in 60c.
 <Begin the cross-reference environment 67e> Referenced in 62, 63a.
 <Begin the scrap environment 64b> Referenced in 62, 63a.
 <Bold Keyword 76f> Referenced in 75a.
 <Bold md Keyword 96b> Referenced in 95c.
 <Build a new arglist 145c> Referenced in 144b.
 <Build failure functions 190> Referenced in 188d.
 <Build fragment definition 48a> Referenced in 45d.
 <Build output file definition 47d> Referenced in 45d.
 <Build `source_name` and `tex_name` 39b> Referenced in 39a.
 <Check at-sequence for end-of-scrap 75a> Referenced in 73.
 <Check for ambiguous prefix 160a> Referenced in 159c.
 <Check for cosmetic fence line 88a> Referenced in 87b.
 <Check for end of scrap name 146a> Referenced in 145d.
 <Check for long options 30> Referenced in 29f.
 <Check for macro invocation in scrap 150d> Referenced in 147c.
 <Check for macro parameters 56a> Referenced in 146a.
 <Check for terminating at-sequence and return name 177a> Referenced in 176b.
 <Check md at-sequence for end-of-scrap 95c> Referenced in 94b.
 <Cleanup and install name 178c> Referenced in 177a, 178d, 181.
 <Close the current sector 47a> Referenced in 45d, 60c, 88b.
 <Collect a block comment 49a> Referenced in 45d.
 <Collect include-file name 130a> Referenced in 129.
 <Collect language apply in pass1 44c> Referenced in 45d.
 <Collect user language definition in pass1 45a> Referenced in 45d.
 <Collect user-specified index entries 139b> Referenced in 137b.
 <Collect weave format declaration in pass1 45b> Referenced in 45d.
 <Comment this macro use 153d> Referenced in 151d, 153ac.
 <Compare the temp file and the old file 124c> Referenced in 124a.
 <Complain about duplicate labels 198b> Referenced in 197f.
 <Complain about empty label 198c> Referenced in 197f.
 <Complain about missing label 197e> Referenced in 197b.
 <Copy block comment from scrap 52c> Referenced in 150d.
 <Copy file name into file 151c> Referenced in 150d.
 <Copy fragment title into file 153c> Referenced in 150d.
 <Copy label from scrap into file 151a> Referenced in 150d.
 <Copy label from source into 76b> Referenced in 60c, 75a, 88b, 95c.
 <Copy macro into file 151d> Referenced in 150d.
 <Copy version info into file 76a> Referenced in 75a, 95c, 150d.
 <Copy version info into md file 89b> Referenced in 88b.
 <Copy version info into tex file 61b> Referenced in 60c.
 <Copy `defs->scrap` to file 147c> Referenced in 146b.
 <Copy `source_file` into `md_file` 87b> Referenced in 87a.
 <Copy `source_file` into `tex_file` 60b> Referenced in 58.
 <Create a new label entry 198a> Referenced in 197f.
 <Create new name entry 162c> Referenced in 159c, 162b.
 <Create new scrap, managed by `writer` 136d> Referenced in 136c.
 <Emit a quoted scrap argument inline 78b> Referenced in 79.
 <End block 65c> Referenced in 53d, 64b, 67a.
 <Enter the next argument 177b> Referenced in 177a.
 <Expand macro into file 61c> Referenced in 60c.
 <Expand md macro into file 89c> Referenced in 88b.
 <Expand md tab into spaces 95b> Referenced in 94b.
 <Expand tab into spaces 74c> Referenced in 73, 150c.
 <Extend goto graph with `tree->spelling` 189b> Referenced in 189a.
 <Fill in the md scrap 91c> Referenced in 90ab.
 <Fill in the middle of the scrap environment 65d> Referenced in 62, 63a.

<Find a free temporary file 123b> Referenced in 123a.
 <Find a new chunk of memory 200c> Referenced in 200b.
 <Finish md scrap 92a> Referenced in 90ab.
 <Finish the cross-reference environment 68a> Referenced in 62, 63a.
 <Finish the scrap environment 67a> Referenced in 62, 63a.
 <For all remaining scraps 156c> Referenced in 156a.
 <Format a file index entry 80c> Referenced in 80b.
 <Format a md file index entry 100b> Referenced in 100a.
 <Format a md index entry 101c> Referenced in 101b.
 <Format a md user index entry 103c> Referenced in 103b.
 <Format a user index entry 85> Referenced in 84d.
 <Format an index entry 82d> Referenced in 82a.
 <Format macro name 77b> Referenced in 75a.
 <Format macro parameters 56b> Referenced in 77b, 79.
 <Format md macro name 97b> Referenced in 95c.
 <Forward declarations for scraps.c 147ab, 154a, 176a, 192c> Referenced in 132a.
 <Function prototypes 43a, 57a, 71b, 74b, 84a, 86a, 122ab, 126a, 128a, 132b, 134b, 141b, 145a, 155a, 158c, 161a, 179a, 188c, 193c, 195a, 197c, 199d> Referenced in 21a.
 <Get comment delimiters 174> Referenced in 173b.
 <Get label from 197a> Referenced in 76b, 138, 151a.
 <Get label while collecting scrap 138> Referenced in 137b.
 <Get the nth previous character from a scrap 188b> Referenced in 187c.
 <Get the nth previous character from an argument 188a> Referenced in 187c.
 <Global variable declarations 27, 29ac, 47b, 51d, 67b, 126b, 135d, 158a, 199c> Referenced in 21a.
 <Global variable definitions 28, 29bd, 47c, 51e, 67c, 126c, 135e, 158b> Referenced in 25e.
 <Handle an “at” character 128c> Referenced in 127c.
 <Handle at-sign during scrap accumulation 137b> Referenced in 137a.
 <Handle macro invocation in scrap 140c> Referenced in 137b.
 <Handle macro parameter substitution 54b> Referenced in 150d.
 <Handle optional per-file flags 173b> Referenced in 173a.
 <Handle reverse map mode 37b> Referenced in 25f.
 <Handle tab characters on output 150c> Referenced in 147c.
 <Handle the file name in `argv[arg]` 39a> Referenced in 38.
 <Handle EOF 130b> Referenced in 127c.
 <HTML-escape character c 95a> Referenced in 94b.
 <If we break the line at this word 53b> Referenced in 52c.
 <Include a fragment use in comment 154c> Referenced in 154a.
 <Include an embedded scrap in comment 154b> Referenced in 154a.
 <Include block comment in a scrap 52b> Referenced in 137b.
 <Include files 21b> Referenced in 21a.
 <Indent suppressed 150b> Referenced in 149d.
 <Insert appropriate indentation 149d> Referenced in 147c, 149a.
 <Insert debugging information if required 149a> Referenced in 147c, 150d.
 <Insert section closing marker 149c> Referenced in 147c.
 <Insert section opening marker 149b> Referenced in 147c.
 <Interpret at-sequence 60c> Referenced in 60b.
 <Interpret command-line arguments 29ef> Referenced in 25f.
 <Interpret md at-sequence 88b> Referenced in 87b.
 <Interpret the argument string `s` 34> Referenced in 29f.
 <Italic “*whatever*” 77a> Referenced in 75a.
 <Italic md “*whatever*” 97a> Referenced in 95c.
 <Limits 23b> Referenced in 21a.
 <Look for end of scrap name and return 181> Referenced in 180.
 <Make this argument 178b> Referenced in 177b.
 <Move a word to the file 53a> Referenced in 52c.
 <Move the temporary file to the target, if required 124a> Referenced in 123a.
 <Note a delimiter collision 74d> Referenced in 73.
 <Open an include file 129> Referenced in 128c.

< Operating System Dependencies 26 > Referenced in 21a, 25e.
 < Otherwise pop the current arg 187b > Referenced in 186b.
 < Perhaps add an include path 36a > Referenced in 29f.
 < Perhaps add letters to the page numbers 157a > Referenced in 156a.
 < Perhaps comment this macro 153a > Referenced in 54b, 151d.
 < Perhaps get the extract name 36d > Referenced in 29f.
 < Perhaps get the hyperref options 36c > Referenced in 29f.
 < Perhaps get the prepend path 35 > Referenced in 29f.
 < Perhaps get the reverse map argument 37a > Referenced in 29f.
 < Perhaps get the version info string 36b > Referenced in 29f.
 < Perhaps put a delayed indent 153b > Referenced in 52c, 153a.
 < Perhaps —return— a character from the current arg 186c > Referenced in 186b.
 < Perhaps skip white-space 51b > Referenced in 51a.
 < Perhaps start a new arg 187a > Referenced in 186b.
 < Print help message 31 > Referenced in 30.
 < Process a file 40b > Referenced in 39a.
 < Process the remaining arguments (file names) 38 > Referenced in 25f.
 < Put out the indent 150a > Referenced in 54b, 149d.
 < Read line in .aux file 155c > Referenced in 155b.
 < Rename the temporary file to the target 125 > Referenced in 124ac.
 < Reverse cross-reference lists 53e > Referenced in 43c.
 < Rob’s ordering 82b > Referenced in 82a, 101b.
 < Save label to label store 197f > Referenced in 138.
 < Save macro name 140d > Referenced in 140c.
 < Save macro parameters 55 > Referenced in 140c.
 < Save pending prose 44b > Referenced in 47d, 48a.
 < Scan at-sequence 45d > Referenced in 43d.
 < Scan the source file, looking for at-sequences 43d > Referenced in 43c.
 < Search for label(*Found,Notfound*) 198d > Referenced in 197bf.
 < Search scraps 191 > Referenced in 188d.
 < Set locale information 37c > Referenced in 25f.
 < Set up name, args and next 145b > Referenced in 144b.
 < Show diff of temp file vs real file 124b > Referenced in 124a.
 < Show md presence of a block comment 96a > Referenced in 95c.
 < Show presence of a block comment 52a > Referenced in 75a.
 < Skip at-L language definition line 45c > Referenced in 60c, 88b.
 < Skip commented-out code 76e > Referenced in 75a, 95c, 137b.
 < Skip invisibles on *p* 162a > Referenced in 161b.
 < Skip over a block comment 53c > Referenced in 191.
 < Skip over a scrap use 192b > Referenced in 191.
 < Skip over an in-text scrap 48c > Referenced in 45d.
 < Skip over at at 192a > Referenced in 191.
 < Skip over at-sign or go to skipped 48d > Referenced in 48c.
 < Skip over index entries 76d > Referenced in 75a, 95c.
 < Skip to the next nw-char 51c > Referenced in 49a.
 < Skip until scrap begins, then return name 178d > Referenced in 176b.
 < Skip whitespace 49b > Referenced in 49a.
 < Sort *key* of size *n* for *ordering* 82c > Referenced in 82a, 101b.
 < Start block 65a > Referenced in 53d, 64b.
 < Step *i* to the next valid scrap 156b > Referenced in 156a.
 < Step to next sector 46 > Referenced in 45d, 60c, 88b.
 < Switch block 65b > Referenced in 64b.
 < Type declarations 23a, 54a, 122d, 157bc, 178a, 179c, 183a, 193b, 199b > Referenced in 21a.
 < Warn (only once) about needing to rerun after Latex 135c > Referenced in 133a, 155c.
 < Write abbreviated definition list 78a > Referenced in 77b, 79.
 < Write defining scrap numbers 83a > Referenced in 82d.
 < Write defs references 195c > Referenced in 195b.
 < Write file defs 68b > Referenced in 62.

< Write file's defining scrap numbers [80d](#) > Referenced in [80c](#).
 < Write in-text scrap [67d](#) > Referenced in [60c](#).
 < Write index of file names [80a](#) > Referenced in [60c](#).
 < Write index of macro names [81a](#) > Referenced in [60c](#).
 < Write index of user-specified names [84c](#) > Referenced in [60c](#).
 < Write LaTeX limbo definitions [59a](#) > Referenced in [58](#).
 < Write macro definition [63a](#) > Referenced in [60c](#).
 < Write macro defs [68c](#) > Referenced in [63a](#).
 < Write macro refs [68d](#) > Referenced in [63a](#).
 < Write md defining scrap numbers [102a](#) > Referenced in [101c](#).
 < Write md defs references [106b](#) > Referenced in [106a](#).
 < Write md file cross-refs [92c](#) > Referenced in [90a](#).
 < Write md file defs [92d](#) > Referenced in [92c](#).
 < Write md file's defining scrap numbers [100c](#) > Referenced in [100b](#).
 < Write md in-text scrap [92b](#) > Referenced in [88b](#).
 < Write md index of file names [99](#) > Referenced in [88b](#).
 < Write md index of macro names [100d](#) > Referenced in [88b](#).
 < Write md index of user-specified names [103a](#) > Referenced in [88b](#).
 < Write md macro cross-refs [93a](#) > Referenced in [90b](#).
 < Write md macro definition [90b](#) > Referenced in [88b](#).
 < Write md macro defs [93b](#) > Referenced in [93a](#).
 < Write md macro refs [93c](#) > Referenced in [93a](#).
 < Write md output file definition [90a](#) > Referenced in [88b](#).
 < Write md referencing scrap numbers [102b](#) > Referenced in [101c](#).
 < Write md uses references [105a](#) > Referenced in [104](#).
 < Write one def reference [195d](#) > Referenced in [195c](#).
 < Write one md def reference [106c](#) > Referenced in [106b](#).
 < Write one md referenced scrap [105c](#) > Referenced in [105b](#), [106c](#).
 < Write one md use reference [105b](#) > Referenced in [105a](#).
 < Write one referenced scrap [194c](#) > Referenced in [194b](#), [195d](#).
 < Write one use reference [194b](#) > Referenced in [194a](#).
 < Write out files->spelling [123a](#) > Referenced in [122c](#).
 < Write output file definition [62](#) > Referenced in [60c](#).
 < Write referencing scrap numbers [83b](#) > Referenced in [82d](#).
 < Write the hyperlink usage macro [60a](#) > Referenced in [59a](#).
 < Write the label to file [197d](#) > Referenced in [197b](#).
 < Write the macro's name [63b](#) > Referenced in [63a](#), [82d](#).
 < Write the md macro's name [91a](#) > Referenced in [90b](#), [101c](#).
 < Write the scrap formatting macros [59b](#) > Referenced in [59a](#).
 < Write uses references [194a](#) > Referenced in [193d](#).

5.3 Identifiers

Knuth prints his index of identifiers in a two-column format. I could force this automatically by emitting the `\twocolumn` command; but this has the side effect of forcing a new page. Therefore, it seems better to leave it this up to the user.

`add_to_use`: [55](#), [140c](#), [141a](#), [141b](#), [183b](#).
`add_uses`: [140ab](#), [191](#), [192c](#), [193a](#).
`already_warned`: [135c](#), [135d](#), [135e](#), [155b](#).
`arena`: [200a](#), [200bc](#), [201ab](#).
`arena_free`: [40b](#), [199d](#), [201b](#).
`arena_getmem`: [36a](#), [40a](#), [48b](#), [56a](#), [132c](#), [136ad](#), [140ab](#), [141a](#), [145bc](#), [159b](#), [162c](#), [179d](#), [182c](#), [188d](#), [189b](#), [190](#), [191](#), [193a](#), [198a](#), [199d](#), [200b](#).
`Arglist`: [54b](#), [57b](#), [61c](#), [77b](#), [79](#), [86b](#), [89c](#), [97b](#), [98b](#), [132b](#), [140c](#), [141d](#), [144ab](#), [145abcd](#), [146b](#), [147b](#), [151d](#), [153c](#), [154a](#), [158c](#), [179c](#), [179d](#), [180](#), [182c](#), [183ac](#), [185c](#), [186a](#), [187a](#), [192b](#).
`arglist_count`: [147b](#).
`arglist_lookup`: [141d](#), [145d](#), [147b](#), [192b](#).

arglist_register: 140d, [147b](#).
 arglist_reset: [147b](#), 199d, 201b.
 arglist_table: [147b](#).
 ArgManager: [185d](#), 186ab, 187c, 188d, 191, 196b.
 ArgMgr: [185c](#), 185d, 186ab, 187c.
 argpop: [186b](#), 191, 192ab.
 ARG_CHR: 63b, 77b, 79, 91a, 97b, 98b, 145b, 153d, 154c, 177b, [178a](#), 181, 182b.
 backup: [143d](#), 149d.
 begin_tex_escape: 52a, [72c](#), 75a, 76f, 77ab.
 blockBuff: 49a, [51d](#), 51e, 52b.
 bodies_flag: [27](#), 28, 30, 108.
 buildArglist: [179d](#), 181, 182c, 183c.
 build_gotos: 188d, [189a](#).
 callers_flag: [27](#), 28, 30, 40b.
 Chunk: [199e](#), 200ac, 201a.
 clear_marks: [111a](#), 112b, 113a.
 code_inline: 56b, [72c](#), 73.
 collect_file_name: 47d, 62, 90a, 158c, [173a](#).
 collect_lang_def: 45a, 158c, [170](#).
 collect_lint_names: [114a](#), 115.
 collect_macro_name: 48a, 63a, 90b, 158c, [176b](#).
 collect_numbers: 40b, 155a, [155b](#).
 collect_scrap: 47d, 48a, 55, 132b, [136c](#), 182c.
 collect_scrap_name: 61c, 77b, 89c, 97b, 140c, 158c, [180](#), 183b.
 collect_weave_format: 45b, 158c, [172](#).
 command_name: [29c](#), 29de, 31, 34, 38, 40b, 45d, 48cd, 58, 68d, 74d, 77b, 79, 87a, 93c, 97b, 98b, 112b, 113a, 123b, 125, 128c, 129, 130ac, 135c, 137ab, 139b, 145d, 147b, 151d, 160a, 170, 172, 173ab, 174, 176b, 177a, 178cd, 180, 181, 192b.
 comment_ArglistElement: 153d, [154a](#), 154c.
 comment_begin: 52c, 149bc, 153a, [176a](#).
 comment_end: 52c, 149bc, 153a, [176a](#).
 comment_mid: 52c, [176a](#).
 compare: [159a](#), 159c, 160b.
 compare_flag: [27](#), 28, 34, 124a.
 copy_scrap: 56b, 57b, 65d, 67d, [73](#).
 current_sector: 40b, 46, 47a, [47b](#), 47c, 81a, 84c, 100d, 103a, 136d, 139b, 176b, 180.
 dangling_flag: [27](#), 28, 34, 85, 103c.
 defs: 48b, 54b, 61c, 68bc, 77b, 78a, 79, 80d, 83a, 85, 89c, 92d, 93b, 97b, 98b, 100c, 102a, 103c, 105bc, 106c, 108, 110b, 114ab, 123a, [131b](#), 132b, 134a, 136d, 140b, 145b, 146b, 147c, 149abc, 151cd, 153acd, 154b, 157c, 162c, 183ad, 191, 194bc, 195bd.
 delayed_indent: 54b, [147a](#), 147c, 150b, 151d, 153ab.
 depths: [185a](#), 188d, 189b, 190.
 diff_flag: [27](#), 28, 30, 124a.
 double_at: [127a](#), 128c, 130c.
 dump_scrap_json: 108, 134b, [143a](#).
 dump_scrap_text: 110b, 134b, [141d](#), 143a.
 Embed_Node: 54b, 145b, 154b, [183a](#).
 emit_name_json: [108](#), 109.
 end_line: 108, [131b](#), 134e, 136d, 137b.
 end_tex_escape: 52a, [72c](#), 75a, 76f, 77ab.
 EQUAL: [159a](#), 159c, 160b.
 errors_flag: [27](#), 28, 30, 37b.
 exit: [21b](#), 25f, 30, 37b, 38, 40b, 48cd, 70a, 112b, 113a, 117b, 123b, 128c, 129, 130ac, 137ab, 139b, 145d, 147b, 151d, 173a, 176b, 177a, 178cd, 180, 181, 186a, 192b, 202.
 EXTENSION: [159a](#), 159c, 160b.
 extract_flag: [27](#), 28, 34, 36d.
 extract_fragment: [110b](#), 113a.
 extract_name: [27](#), 28, 36d, 40b.

extra_scraps: 65d, 67a, [67b](#), 67c, 78b, 91c, 92a, 98b.
FALLTHROUGH: [23a](#), 45d, 60c, 75a, 88b, 127c, 159c.
FALSE: [23a](#), 27, 28, 34, 35, 36abcd, 37a, 40b, 60bc, 61c, 65c, 67d, 69a, 72c, 73, 78b, 83a, 84b, 85, 87b, 88ab, 89c, 92b, 94a, 98b, 102a, 103c, 105b, 106c, 111a, 128c, 130c, 151d, 155b, 160b, 162c, 163, 170, 173b, 175a, 192d, 194b, 195d, 196ab.
fclose: [21b](#), 58, 78b, 87a, 117b, 120, 123ab, 124bc, 130b, 143a, 155b.
FILE: [21b](#), 31, 57b, 58, 64a, 69a, 70b, 72bc, 73, 78b, 79, 80b, 82a, 84d, 86b, 87a, 91b, 94ab, 98ab, 100a, 101b, 103b, 104, 106a, 107, 108, 110b, 117b, 120, 123a, 124bc, 127ab, 132b, 133ab, 134b, 141d, 143a, 146b, 154a, 155b, 193cd, 195ab, 197bc, 202.
file_line: [131b](#), 134d, 136d, 137a, 147c.
file_name: 43ac, 57a, 58, 86a, 87a, [131b](#), 134c, 136d, 137a, 149ab.
file_names: 40b, 43c, 53e, 80a, 99, 112a, 113a, 115, 122b, [158a](#), 158b, 173a.
find_closest: [114a](#), 114b.
find_lang: [163](#), 175a.
find_name: [110a](#), 112b, 113a.
first: 31, 105bc, 106c, 108, 109, 113a, 132b, 133a, 194bc, 195d, 196b, [200a](#), 201b, 202.
fopen: [21b](#), 58, 87a, 117b, 120, 123b, 124bc, 129, 130c, 155b.
format_defs_refs: 62, 63a, 195a, [195b](#).
format_entry: 57b, 81a, [82a](#).
format_file_entry: 57b, 80a, [80b](#).
format_user_entry: 57b, 84c, [84d](#).
format_uses_refs: 62, 63a, 193c, [193d](#).
fprintf: [21b](#), 34, 37c, 38, 40b, 43c, 45d, 48cd, 52a, 56b, 58, 60a, 62, 64b, 65d, 68bd, 70a, 74d, 76f, 77ab, 79, 80c, 85, 87a, 90a, 91c, 92d, 93c, 96b, 97ab, 98b, 100b, 103c, 105b, 106c, 107, 108, 110b, 112b, 113a, 114b, 115, 117b, 123ab, 125, 128c, 129, 130ac, 133a, 135c, 137ab, 139b, 141d, 143a, 145d, 147b, 149abc, 151cd, 153d, 154a, 160a, 170, 172, 173ab, 174, 176b, 177a, 178cd, 180, 181, 186a, 192b, 194b, 195d, 197de, 198bc.
fputs: [21b](#), 52c, 54b, 56b, 59ab, 61b, 62, 63a, 64ab, 65abcd, 67ae, 68abcd, 69a, 70b, 72bc, 73, 75a, 76a, 77b, 78ab, 79, 80acd, 81a, 82d, 83ab, 84c, 85, 89b, 90ab, 91bc, 92d, 93bc, 94ab, 95ac, 96ab, 97b, 98ab, 99, 100bcd, 101c, 102ab, 103ac, 105abc, 106bc, 107, 108, 109, 110b, 112a, 117b, 121, 124b, 133a, 143a, 147c, 151c, 153a, 194abc, 195cd.
getc: [21b](#), 124c, 127c, 128c, 129, 130abc.
getenv: [21b](#), 37c.
get_scrap_defs: 106a, 132b, [134a](#).
get_scrap_uses: 104, 108, 110b, 132b, [133c](#).
goto_lookup: [185b](#), 189b, 190, 191.
Goto_Node: [184b](#), 184c, 185ab, 188d, 189b, 190, 191.
GREATER: [159a](#), 159c, 160b.
has_sector: 81a, 84a, [84b](#), 84c, 100d, 103a.
hyperoptions: 27, 28, 36c, 59a, 60a.
hyperopt_flag: 27, 28, 34, 36c.
hyperref_flag: 27, 28, 34, 36c, 59a.
incl: 27, 28, 36a, 40a, 129.
includepath_flag: 27, 28, 34, 36a.
include_depth: [127a](#), 129, 130bc.
init_scraps: 43c, 132b, [132c](#).
instance: 61c, 89c, [144b](#), 145ab, 151d.
isgraph: [21b](#), 130a, 173a, 180.
islower: [21b](#), 161b.
isspace: [21b](#), 67a, 92a, 139b, 173ab, 176b, 178d.
is_file_name: [111b](#), 112a.
json_puts: 107, 108, 117b.
label_node: 198ad, 199a, [199b](#), 199c.
label_tab: 198d, [199a](#), 199c.
LangEntry: [163](#), 175a.
lang_table: [163](#).
lang_to_flags: [163](#), 175ab.
latex_code_putc: 57b, [70b](#), 72b, 73, 78b.
latex_language: 57b, 65d, [71a](#).

LESS: [159a](#), [159c](#), [160b](#).
 letter: [121](#), [131b](#), [133a](#), [136d](#), [157a](#).
 levenshtein: [113b](#), [114a](#).
 lint_flag: [27](#), [28](#), [30](#), [40b](#).
 lint_name: [114b](#), [115](#).
 lint_names: [114a](#).
 lint_name_count: [114a](#), [115](#).
 load_entry: [81b](#), [82a](#).
 lookup: [54b](#), [144a](#), [145b](#).
 macro_names: [40b](#), [43c](#), [53e](#), [81a](#), [100d](#), [113a](#), [115](#), [122b](#), [158a](#), [158b](#), [178c](#).
 main: [25f](#), [202](#).
 malloc: [21b](#), [82a](#), [101b](#), [186a](#), [201a](#).
 Manager: [135f](#), [136abc](#), [141cd](#), [143d](#), [145d](#), [147c](#), [185d](#), [187c](#), [191](#).
 map_flag: [27](#), [28](#), [34](#), [40b](#).
 markdown_flag: [27](#), [28](#), [34](#), [40b](#).
 MarkerFrame: [116a](#), [116b](#), [117b](#), [120](#), [121](#).
 MAX_ARGLISTS: [147b](#).
 max_depth: [185a](#), [188d](#), [189b](#), [190](#).
 MAX_INDENT: [61c](#), [89c](#), [122d](#), [123a](#), [148](#), [150a](#).
 MAX_LINT_NAMES: [114a](#).
 MAX_NAME_LEN: [23b](#), [139b](#), [141d](#), [145d](#), [173a](#), [176b](#), [180](#), [182a](#), [192b](#), [197a](#).
 md_copy_scrap: [86b](#), [91c](#), [92b](#), [94b](#).
 md_format_defs_refs: [86b](#), [92c](#), [93a](#), [106a](#).
 md_format_entry: [86b](#), [100d](#), [101b](#).
 md_format_file_entry: [86b](#), [99](#), [100a](#).
 md_format_user_entry: [86b](#), [103a](#), [103b](#).
 md_format_uses_refs: [86b](#), [92c](#), [93a](#), [104](#).
 md_load_entry: [101a](#), [101b](#).
 md_print_scrap_numbers: [86b](#), [92d](#), [93bc](#), [94a](#), [100c](#), [102b](#).
 md_scrap_type: [94b](#), [95c](#).
 md_write_arg: [86b](#), [91a](#), [91b](#), [95c](#), [98b](#).
 md_write_ArglistElement: [86b](#), [97b](#), [98b](#).
 md_write_literal: [86b](#), [97b](#), [98a](#), [98b](#).
 Move_Node: [184c](#), [185b](#), [189b](#), [190](#).
 Name: [31](#), [47d](#), [48a](#), [54b](#), [57b](#), [61c](#), [62](#), [63a](#), [73](#), [77b](#), [79](#), [80b](#), [81b](#), [82ac](#), [84abd](#), [86b](#), [89c](#), [90ab](#), [94b](#), [97b](#), [98b](#), [100a](#),
 [101ab](#), [103b](#), [105b](#), [106c](#), [108](#), [109](#), [110ab](#), [111ab](#), [112ab](#), [113a](#), [114ab](#), [115](#), [122abc](#), [134b](#), [135b](#), [139b](#), [140c](#),
 [141ab](#), [143c](#), [144ab](#), [145b](#), [151d](#), [154a](#), [157c](#), [158abc](#), [159c](#), [160b](#), [162bc](#), [173a](#), [176b](#), [179abcd](#), [181](#), [182c](#), [183d](#),
 [184a](#), [187a](#), [188d](#), [189a](#), [191](#), [192c](#), [193ab](#), [194b](#), [195d](#), [196b](#).
 name_add: [140ab](#), [158c](#), [162b](#), [173a](#).
 Name_Node: [184a](#), [184b](#), [189b](#), [190](#), [191](#).
 next: [27](#), [31](#), [34](#), [36a](#), [40a](#), [48b](#), [54b](#), [68bcd](#), [69a](#), [77b](#), [78a](#), [79](#), [80d](#), [83ab](#), [85](#), [92d](#), [93bc](#), [94a](#), [97b](#), [98b](#), [100c](#), [102ab](#),
 [103c](#), [105ab](#), [106bc](#), [108](#), [110b](#), [112ab](#), [126a](#), [129](#), [131a](#), [136a](#), [140ab](#), [141ac](#), [144ab](#), [145bc](#), [146b](#), [153d](#), [154c](#),
 [157b](#), [179cd](#), [182c](#), [183bce](#), [184abc](#), [185b](#), [187a](#), [189b](#), [190](#), [191](#), [192d](#), [193ab](#), [194ab](#), [195cd](#), [199e](#), [200c](#), [201a](#),
 [202](#).
 number_flag: [27](#), [28](#), [34](#), [40b](#), [155b](#).
 num_scrap: [82a](#), [101b](#), [132a](#), [132b](#).
 nw_char: [29a](#), [29b](#), [31](#), [40b](#), [43d](#), [45d](#), [48cd](#), [50a](#), [51c](#), [52b](#), [53c](#), [55](#), [56ab](#), [60bc](#), [62](#), [69b](#), [72b](#), [73](#), [74a](#), [75a](#), [76df](#), [80c](#),
 [85](#), [87b](#), [88b](#), [94b](#), [95c](#), [96b](#), [127c](#), [128c](#), [130c](#), [137ab](#), [138](#), [139b](#), [140cd](#), [141d](#), [145d](#), [147c](#), [150bd](#), [151cd](#), [153d](#),
 [154ac](#), [160a](#), [170](#), [172](#), [173a](#), [176b](#), [177ab](#), [178d](#), [180](#), [181](#), [182a](#), [192ab](#), [194b](#), [195d](#), [196a](#), [197a](#).
 op_char: [196a](#), [196b](#).
 output_flag: [27](#), [28](#), [34](#), [40b](#).
 owner: [112ab](#), [131b](#), [135b](#), [136d](#), [143c](#).
 page: [69a](#), [83a](#), [85](#), [94a](#), [102a](#), [103c](#), [105bc](#), [106c](#), [131b](#), [132b](#), [133ab](#), [136d](#), [155bc](#), [156b](#), [157a](#), [194bc](#), [195d](#), [202](#).
 Parameters: [54a](#), [56a](#), [61c](#), [89c](#), [132b](#), [145d](#), [146b](#), [147c](#).
 parse_close_marker: [117a](#), [117b](#), [120](#).
 parse_open_marker: [116b](#), [117b](#), [120](#).
 pass1: [40b](#), [43a](#), [43c](#), [75a](#), [95c](#).
 pending_lang: [44c](#), [158a](#), [158b](#), [175ab](#).

pending_prose: [43b](#), [43c](#), [44b](#), [47d](#), [48a](#).
 pop: [52c](#), [53ac](#), [56a](#), [141c](#), [141d](#), [145d](#), [146a](#), [147c](#), [149d](#), [150d](#), [151b](#), [186b](#).
 pop_scrap_name: [145d](#), [151d](#).
 PREFIX: [159a](#), [159c](#), [160b](#).
 prefix_add: [158c](#), [159c](#), [178c](#).
 prev_char: [187c](#), [196b](#).
 prev_sector: [40b](#), [46](#), [47b](#), [47c](#).
 print_callers: [112a](#), [112b](#).
 print_scrap_numbers: [57b](#), [68bcd](#), [69a](#), [80d](#), [83b](#).
 prose: [27](#), [30](#), [31](#), [108](#), [131b](#), [135a](#), [136d](#), [143b](#), [202](#).
 prose_buf: [43b](#), [44ab](#).
 prose_flag: [27](#), [28](#), [30](#), [43d](#), [44b](#), [108](#).
 prose_len: [43b](#), [43c](#), [44ab](#).
 push: [52b](#), [55](#), [136a](#), [136b](#), [137ab](#), [138](#), [140cd](#).
 pushArglist: [186a](#), [187a](#), [192b](#).
 pushes: [55](#), [136b](#), [138](#), [140d](#).
 putc: [21b](#), [52c](#), [53a](#), [60bc](#), [70b](#), [73](#), [74c](#), [77b](#), [78b](#), [79](#), [80cd](#), [82d](#), [83ab](#), [85](#), [87b](#), [88ab](#), [94b](#), [95abc](#), [97b](#), [98b](#), [100b](#), [101c](#), [102a](#), [133a](#), [147c](#), [150acd](#), [151d](#), [153abc](#).
 reject_match: [188d](#), [191](#), [196b](#).
 remove: [21b](#), [124abc](#).
 reverse: [27](#), [31](#), [183d](#), [183e](#).
 reverse_lists: [53e](#), [158c](#), [183d](#).
 reverse_lookup_in_file: [120](#), [121](#).
 reverse_map_arg: [27](#), [28](#), [37ab](#).
 reverse_map_flag: [27](#), [28](#), [34](#), [37a](#).
 robs_strcmp: [82b](#), [110a](#), [161a](#), [161b](#), [162bc](#), [193a](#).
 root: [111b](#), [185a](#), [188d](#), [189b](#), [190](#), [191](#), [192c](#), [193a](#).
 save_string: [36a](#), [40a](#), [44b](#), [129](#), [136d](#), [158c](#), [159b](#), [159c](#), [162c](#), [179b](#), [183c](#).
 SCRAP: [132a](#), [132c](#), [136d](#).
 ScrapEntry: [131b](#), [132ac](#), [136d](#).
 scraps: [24bc](#), [27](#), [31](#), [56b](#), [57b](#), [62](#), [63a](#), [64b](#), [67a](#), [69a](#), [86b](#), [90ab](#), [92ac](#), [93a](#), [94a](#), [108](#), [132a](#), [132c](#), [136d](#), [155bc](#), [156bc](#), [191](#), [202](#).
 scrap_array: [132a](#), [133ac](#), [134acde](#), [135ab](#), [136d](#), [137ab](#), [140ab](#), [141d](#), [143bc](#), [147c](#), [149ab](#), [155bc](#), [156b](#), [157a](#), [191](#), [193d](#), [195b](#).
 SCRAP_BITS: [132a](#).
 scrap_end_line: [108](#), [134b](#), [134e](#).
 scrap_file_line: [108](#), [110b](#), [114b](#), [134b](#), [134d](#).
 scrap_file_name: [108](#), [110b](#), [114b](#), [134b](#), [134c](#).
 scrap_flag: [27](#), [28](#), [34](#), [62](#), [92c](#).
 scrap_is_in: [191](#), [192c](#), [192d](#).
 SCRAP_MASK: [132a](#), [136d](#).
 Scrap_Node: [48b](#), [54b](#), [57b](#), [69a](#), [78ab](#), [80d](#), [83ab](#), [85](#), [86b](#), [94a](#), [97b](#), [98b](#), [100c](#), [102ab](#), [103c](#), [105b](#), [106c](#), [108](#), [110b](#), [112ab](#), [132b](#), [140ab](#), [141a](#), [145b](#), [146b](#), [157b](#), [157c](#), [182c](#), [183ade](#), [191](#), [192cd](#), [194b](#), [195d](#).
 scrap_owner: [112ab](#), [134b](#), [135b](#).
 scrap_prose: [108](#), [134b](#), [135a](#).
 SCRAP_SHIFT: [132a](#), [136d](#).
 SCRAP_SIZE: [132a](#), [132c](#), [136d](#).
 scrap_type: [72a](#), [72c](#), [73](#), [74d](#), [75a](#), [77b](#).
 search: [31](#), [43c](#), [188c](#), [188d](#), [202](#).
 sector: [31](#), [57ab](#), [58](#), [81a](#), [82ad](#), [84bcd](#), [85](#), [86ab](#), [87a](#), [100d](#), [101bc](#), [103abc](#), [108](#), [131b](#), [136d](#), [139b](#), [140abd](#), [141d](#), [151d](#), [154c](#), [157c](#), [158c](#), [159c](#), [160ab](#), [162bc](#), [176b](#), [178c](#), [180](#), [191](#).
 setlocale: [21b](#), [37c](#).
 set_scrap_owner: [48b](#), [134b](#), [143c](#).
 set_scrap_prose: [47d](#), [48a](#), [134b](#), [143b](#).
 size_t: [21b](#), [78b](#), [116b](#), [121](#), [199d](#), [200b](#), [201a](#).
 Slab: [131a](#), [131b](#), [135f](#), [136ad](#), [141c](#).
 slab: [131a](#), [131b](#), [136d](#), [141d](#), [147c](#), [191](#).
 SLAB_SIZE: [131a](#), [136a](#), [141c](#), [143d](#), [188b](#).

source_file: [127a](#), [127c](#), [128bc](#), [129](#), [130abc](#).
 source_get: [43d](#), [44c](#), [45cd](#), [46](#), [47a](#), [48cd](#), [49b](#), [50ab](#), [51bc](#), [55](#), [56b](#), [60bc](#), [61bc](#), [67ad](#), [73](#), [75a](#), [76cdef](#), [80a](#), [81a](#), [84c](#), [87b](#), [88ab](#), [89bc](#), [92ab](#), [94b](#), [95c](#), [96b](#), [99](#), [100d](#), [103a](#), [126a](#), [127c](#), [129](#), [130b](#), [137ab](#), [139ab](#), [140c](#), [170](#), [172](#), [173ab](#), [174](#), [176b](#), [177ab](#), [178d](#), [180](#), [181](#), [182a](#).
 source_last: [56b](#), [65d](#), [73](#), [94b](#), [126a](#), [127c](#).
 source_line: [45d](#), [48cd](#), [126b](#), [126c](#), [127c](#), [128bc](#), [129](#), [130abc](#), [136d](#), [137b](#), [139b](#), [160a](#), [170](#), [172](#), [173ab](#), [174](#), [176b](#), [178c](#), [180](#), [181](#).
 source_name: [39ab](#), [40ab](#), [45d](#), [48cd](#), [126b](#), [126c](#), [128c](#), [129](#), [130abc](#), [136d](#), [137b](#), [139b](#), [160a](#), [170](#), [172](#), [173ab](#), [174](#), [176b](#), [177a](#), [178cd](#), [180](#), [181](#).
 source_open: [43c](#), [58](#), [87a](#), [126a](#), [130c](#).
 source_peek: [49b](#), [50b](#), [51b](#), [126a](#), [127c](#), [128bc](#), [129](#), [130bc](#).
 source_ungetc: [50ab](#), [51c](#), [81a](#), [100d](#), [128a](#), [128b](#).
 stack: [117b](#), [120](#), [127b](#), [129](#), [130b](#).
 stderr: [21b](#), [34](#), [37c](#), [38](#), [40b](#), [43c](#), [45d](#), [48cd](#), [58](#), [68bd](#), [70a](#), [74d](#), [77b](#), [79](#), [87a](#), [92d](#), [93c](#), [97b](#), [98b](#), [112b](#), [113a](#), [114b](#), [115](#), [117b](#), [123ab](#), [125](#), [128c](#), [129](#), [130ac](#), [135c](#), [137ab](#), [139b](#), [145d](#), [147b](#), [151d](#), [160a](#), [170](#), [172](#), [173ab](#), [174](#), [176b](#), [177a](#), [178cd](#), [180](#), [181](#), [186a](#), [192b](#), [197e](#), [198bc](#).
 strlen: [21b](#), [52c](#), [113b](#), [117b](#), [155c](#), [159ab](#), [188a](#), [196b](#).
 sym_char: [196a](#), [196b](#).
 tex_flag: [27](#), [28](#), [34](#), [40b](#), [43c](#), [151d](#).
 toupper: [21b](#), [161b](#).
 TRUE: [23a](#), [27](#), [28](#), [30](#), [34](#), [36c](#), [40b](#), [56b](#), [60c](#), [61c](#), [65ad](#), [69a](#), [74d](#), [78b](#), [83a](#), [84b](#), [85](#), [87b](#), [88ab](#), [89c](#), [91c](#), [92a](#), [94a](#), [98b](#), [102a](#), [103c](#), [105b](#), [106c](#), [110b](#), [112a](#), [128c](#), [133b](#), [135c](#), [151d](#), [160b](#), [162c](#), [163](#), [170](#), [173b](#), [192d](#), [194b](#), [195d](#), [196ab](#).
 update_delimit_scrap: [45d](#), [60c](#), [74a](#), [74b](#), [88b](#).
 user_langs: [163](#), [170](#).
 user_lang_count: [163](#), [170](#).
 user_lang_names: [163](#), [170](#).
 user_names: [43c](#), [53e](#), [84c](#), [103a](#), [140ab](#), [158a](#), [158b](#), [188d](#).
 Uses: [59a](#), [104](#), [105a](#), [106a](#), [108](#), [110b](#), [132b](#), [133c](#), [134a](#), [192c](#), [193a](#), [193b](#), [193d](#), [195b](#).
 uses: [31](#), [68d](#), [83b](#), [85](#), [93c](#), [102b](#), [103c](#), [106c](#), [108](#), [110b](#), [112ab](#), [114b](#), [131b](#), [133c](#), [136d](#), [140a](#), [141a](#), [157c](#), [162c](#), [163](#), [183d](#), [191](#), [193bd](#), [195d](#).
 verbose_flag: [27](#), [28](#), [34](#), [43c](#), [58](#), [87a](#), [123a](#).
 version_info_flag: [27](#), [28](#), [34](#), [36b](#).
 version_string: [27](#), [28](#), [36b](#), [61b](#), [76a](#), [89b](#).
 weave_flag: [27](#), [28](#), [34](#), [40b](#).
 weave_format: [27](#), [28](#), [34](#), [40b](#), [172](#).
 weave_format_from_cli: [27](#), [28](#), [34](#), [40b](#), [172](#).
 WEFT_VERSION: [23b](#), [30](#), [31](#).
 write_arg: [57b](#), [63b](#), [64a](#), [75a](#), [79](#).
 write_ArglistElement: [57b](#), [77b](#), [79](#).
 write_callers: [40b](#), [112b](#), [122b](#).
 write_error_annotator: [37b](#), [121](#), [122b](#).
 write_extract: [40b](#), [113a](#), [122b](#).
 write_files: [40b](#), [122a](#), [122c](#).
 write_label: [76b](#), [151a](#), [197b](#), [197c](#).
 write_lint: [40b](#), [115](#), [122b](#).
 write_literal: [57b](#), [72b](#), [77b](#), [79](#).
 write_map: [40b](#), [109](#), [122b](#).
 write_md: [40b](#), [86a](#), [87a](#).
 write_reverse_map: [37b](#), [117b](#), [122b](#).
 write_scrap: [54b](#), [61c](#), [78b](#), [89c](#), [98b](#), [123a](#), [132b](#), [146b](#), [151d](#), [154b](#).
 write_scrap_ref: [69a](#), [83a](#), [85](#), [94a](#), [102a](#), [103c](#), [105c](#), [132b](#), [133a](#), [133b](#), [194c](#).
 write_single_scrap_ref: [56b](#), [62](#), [63a](#), [68d](#), [78a](#), [80d](#), [83b](#), [85](#), [90ab](#), [93c](#), [97b](#), [98b](#), [100c](#), [102b](#), [103c](#), [132b](#), [133b](#), [153ac](#), [197d](#).
 write_tex: [40b](#), [57a](#), [58](#).
 xref_flag: [27](#), [28](#), [34](#), [153ac](#).

Bibliography

- [1] A[lfred] V. Aho and M[argaret] J. Corasick, *Efficient string matching: An aid to bibliographic search.*, CACM **18** (1975), no. 6, 333–340. [184](#)
- [2] C[hris] Hanson, *Efficient stack allocation for tail-recursive languages.*, (1990). [200](#)
- [3] Donald E. Knuth, *Literate programming*, The Computer Journal **27** (1984), no. 2, 97–111. [1](#)
- [4] ———, *The T_EXbook*, Computers and Typesetting, vol. 1986aA, Addison-Wesley, Reading, MA, USA, 1986. [1](#)
- [5] ———, *T_EX: The program*, Computers and Typesetting, vol. B, Addison-Wesley, Reading, MA, USA, 1986b1986. [1](#)
- [6] ———, *METAFont: THE PROGRAM*, Computers and Typesetting, vol. D, Addison-Wesley, Reading, MA, USA, 1986d1986. [1](#)
- [7] Leslie Lamport, *L^AT_EXa document preparation system user's guide and reference manual*, Addison-Wesley, Reading, MA, USA, 1985. [1](#)
- [8] Silvio Levy, *WEB adapted to C, another approach*, TUB **8** (1987), no. 1, 12–13. [1](#)
- [9] Norman Ramsey, *Literate-programming tools need not be complex*, Report at <ftp.cs.princeton.edu> in `/reports/1991/351.ps.Z`. Software at <ftp.cs.princeton.edu> in `/pub/noweb.shar.Z` and at bellcore.com in `/pub/norman/noweb.shar.Z`. CS-TR-351-91, Department of Computer Science, Princeton University, August 1992, Submitted to *IEEE Software*. [1](#)
- [10] Ross Williams, *Funnelweb user's manual*, <ftp.adelaide.edu.au> in `/pub/compression` and `/pub/funnelweb`, University of Adelaide, Adelaide, South Australia, Australia, 1992. [1](#)